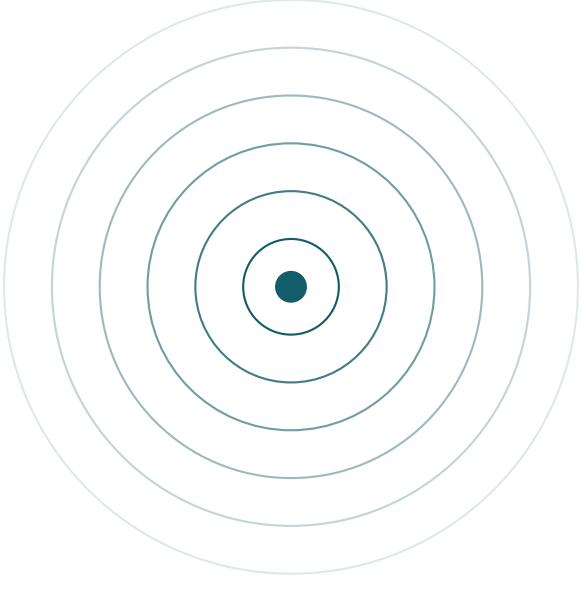
From the first model call to a system you can defend



Modern AI Engineering from Zero

Understand, Use, Evaluate, Build

FIRST EDITION

Modern AI Engineering from Zero

What an AI Engineer Needs to Understand, Use, Evaluate and Build Today

FIRST EDITION

Modern AI Engineering from Zero: Understand, Use, Evaluate, Build

Copyright © 2026 The Author. All rights reserved.

Revision History for the First Edition

2026-10-19 First draft

The programs in this book are included for their instructional value. They have been tested with Python 3.14 on macOS 26, but no warranty, express or implied, is offered with respect to them, and no liability is accepted for any damages arising from their use. Third-party services and tools described here change over time; check their current documentation.

Typeset in Crimson Pro, Source Sans 3, and Source Code Pro with XeLaTeX. Source: Markdown, converted with pandoc.

Contents

Preface . . . . . **xi**

Part I. Understand **1**

Mental Model **2**

1. What a Model Is, and What You Get From It **3**

The Short Answer 3
Using Is Not Training 4
The Exchange 5
What It Knows 6
What You Choose, and What You Do Not 7
What It Costs 7
What You Wait For 8
Why "It Is a Service" Is the Useful Picture 9
Lab: What Is This Thing Made Of? 9
Summary, Key Terms, and Review Questions 10

2. The Parts Around the Model **13**

The Cast 13
Context: Everything You Send 14
Retrieval: Finding the Material 15
Tools: Letting It Act 15
Checks: Finding Out Before the User Does 16
People: Who Actually Decides 17
The Whole Thing Has a Name 18
The Same Small Cast, Arranged Differently 18
What This Chapter Has Not Told You 19
Lab: Name the Parts 20
Summary, Key Terms, and Review Questions 21

3. Why It Is Assembled This Way **23**

The Shape of the Argument 23

It Does Not Know Your Data . . . 23
It Cannot Act in the World . . . 25
It Cannot Reliably Check Itself . . . 26
It Is Not Accountable . . . 27
The Pattern . . . 28
Lab: What Does It Cost Them? . . . 29
Summary, Key Terms, and Review Questions . . . 30

4. The Agent Loop, and How Much Freedom It Actually Gets . . . 33

One Change, and It Is Not a Small One . . . 33
The Loop, Step by Step . . . 34
Where the Control Flow Went . . . 35
Compounding Error . . . 35
Stopping . . . 37
How Much Freedom Real Systems Give . . . 37
Freedom Is a Dial, Not a Switch . . . 38
Approval Points . . . 38
Lab: Watch a Loop . . . 39
Summary, Key Terms, and Review Questions . . . 40

5. Where the Work Actually Goes . . . 43

Why the Prompt Looks Like the Job . . . 43
Choosing What Reaches the Model . . . 44
Deciding What It May Do . . . 44
Finding Out Whether It Works . . . 45
Controlling What It Costs . . . 46
Telling Which Part Failed . . . 46
Why Finding Out Takes So Long . . . 46
What This Does to a Project . . . 48
Lab: Sort an Account of Real Work . . . 48
Summary, Key Terms, and Review Questions . . . 51

6. What Changes Fast, and What Changes Slowly . . . 54

Three Kinds of Claim . . . 54
The Test: What Would Have to Change? . . . 55
The Hard Case: A Convention That Looks Like a Principle . . . 56
How Strong Is the Evidence? . . . 56
Saying It Out Loud . . . 58
How to Read a Release Announcement . . . 59
What to Memorise, and What to Look Up . . . 60
Lab: Sort Nine Claims . . . 61
Summary, Key Terms, and Review Questions . . . 62

7. A Map of This Book . . . 65

The Four Parts . . . 65
Why Part I Has Two Halves . . . 66

The One System the Labs Build . . . 66
Which Chapters Carry the Most Weight . . . 67
What This Book Leaves Out, On Purpose . . . 69
How to Read It. . . . 70
Lab: Place Eight Questions . . . 70
Summary, Key Terms, and Review Questions . . . 71

Inside the Model **73**

8. The Maths This Book Uses. . . . 74
How to Read a Formula at All . . . 74
Vectors, and What a Dimension Is . . . 75
The Dot Product, and Why It Measures Agreement . . . 75
Cosine Similarity. . . . 76
Sums and Products, Written Down. . . . 77
Matrices, and Multiplying Them. . . . 78
Exponentials and Logarithms. . . . 79
Softmax, Read as a Formula . . . 80
Big-O: How Cost Grows. . . . 81
Lab: Nine Lines of Arithmetic . . . 82
Summary, Key Terms, and Review Questions . . . 83

9. What the Model Is Made Of . . . 86
The Short Answer . . . 86
From Text to Numbers . . . 87
The Embedding Table: Coordinates Nobody Chose . . . 87
A Stack of Identical Layers. . . . 88
What Comes Out Is a Distribution, Not a Word . . . 89
The Whole Thing, in Five Tokens and Four Dimensions . . . 90
Parameters: the Numbers That Were Fitted . . . 91
One Pass, One Token . . . 92
Lab: Run a Forward Pass by Hand . . . 93
Summary, Key Terms, and Review Questions . . . 95

10. Tokens. . . . 98
Why Not Words, and Why Not Letters . . . 98
Building the Vocabulary: Merge the Commonest Pair. . . . 99
What the Merges Bought . . . 102
The Space Belongs to the Word. . . . 103
Input Tokens and Output Tokens . . . 103
Why Code and Other Languages Cost More . . . 104
Why It Cannot Count the Letters in a Word . . . 105
Counting Before You Send. . . . 105
Lab: Build a Vocabulary by Hand. . . . 106
Summary, Key Terms, and Review Questions . . . 110

11. Attention, Worked by Hand . . . 114
What Each Position Needs . . . 114
Three Views of the Same Vector: Query, Key, Value . . . 115
The Scores: Every Query Against Every Key . . . 116
Why Divide by the Square Root . . . 117
Masking: No Looking Ahead . . . 119
Softmax, and the Weighted Blend . . . 119
The Formula, All at Once . . . 120
This Formula Cannot Tell the Order . . . 121
Several Heads . . . 123
What It Costs: Big-O at Last . . . 125
Lab: Two More Rows . . . 126
Summary, Key Terms, and Review Questions . . . 128

12. The Stack: Layers and the Residual Stream . . . 132
A Layer Does Not Replace, It Adds . . . 132
What Is in a Layer . . . 133
The Feed-Forward Block, Worked . . . 134
Normalisation . . . 136
Ninety-Six of Them . . . 137
Where Is the Fact Stored? . . . 137
Where the Parameters Actually Sit . . . 138
Lab: One Layer, By Hand . . . 139
Summary, Key Terms, and Review Questions . . . 141

13. From Scores to a Word . . . 145
Where We Are . . . 145
Greedy: Take the Biggest . . . 146
Temperature . . . 146
Top-k and Top-p . . . 148
How a Reply Ends . . . 149
Why Temperature Zero Is Still Not a Guarantee . . . 150
Why One Good Output Proves Nothing . . . 151
Lab: Draw From a Distribution . . . 151
Summary, Key Terms, and Review Questions . . . 154

14. How It Learned to Do This . . . 158
The Objective, and Why It Needs No Labels . . . 158
Cross-Entropy: How Wrong Was That Guess? . . . 159
Perplexity . . . 160
The Loop, From Outside . . . 161
Three Stages, Three Kinds of Data . . . 162
The Preference Objective . . . 164
What Each Stage Cannot Do . . . 165
Lab: Compute a Loss . . . 166

Summary, Key Terms, and Review Questions 168

15. What It Costs From the Inside 172

Two Different Computations 172
Why the First Token Is Slow 173
The Key–Value Cache 174
Why Doubling the Prompt More Than Doubles the Work 175
Batching: Your Request Runs With Others 176
Quantisation: Bits Per Parameter 177
Mixture of Experts. 178
Lab: Size a Machine. 179
Summary, Key Terms, and Review Questions 180

16. Why It Fails the Way It Does 184

There Is No Step Where Truth Is Checked. 184
The Failures That Come From Tokens. 185
The Context Window, and What Falls Out of It 186
Long Inputs Are Not Used Evenly 187
Why Thinking Out Loud Works 187
What It Costs, and What the Trace Is Not. 188
Which of These You Can Fix 190
Lab: Diagnose Six Failures. 190
Summary, Key Terms, and Review Questions 192

17. Where the Theory You're Skipping Begins 195

The List. 196
What They Have in Common 197
What to Say When Somebody Pushes. 198
What a Boundary Is Not 199
Lab: Write Your Own Boundary 199
Summary, Key Terms, and Review Questions 200

Part II. Use 203

18. The Prompt as a Specification 204

Four Separable Parts 204
The Message List. 206
Every Turn Re-Sends Everything. 207
When the History Will Not Fit. 208
Change One Thing at a Time. 209
What Current Practice Actually Weights. 210
Lab: Take a Prompt Apart 210
Summary, Key Terms, and Review Questions 212

19. Calling a Model Reliably . . . 216
Streaming . . . 216
Rate Limits and 429s . . . 218
Retries: Backoff and Jitter . . . 219
Timeouts . . . 220
Concurrency, From Zero . . . 221
Batch Endpoints . . . 223
Idempotency . . . 223
When It Dies Half-Way . . . 224
When the Pipeline Has Stages . . . 226
Lab: Make a Fragile Script Robust . . . 228
Summary, Key Terms, and Review Questions . . . 230

20. Is the Difference Real? . . . 235
The Set You Measure On . . . 235
One Run Is Not a Score . . . 236
An Interval You Can Justify . . . 238
Overlapping Intervals Are Not the Test . . . 240
Compare on the Same Cases . . . 240
How Many Cases, and How Many Trials . . . 242
Every Change Is an Experiment . . . 244
Lab: Measure a Difference You Cannot See . . . 245
Summary, Key Terms, and Review Questions . . . 248

21. Structured Output and Schemas . . . 253
Why a Program Must Never Trust Prose . . . 253
Asking for a Fixed Format . . . 254
A Schema Is a Contract Your Code Enforces . . . 255
The Four Layers, and Which Two You Get for Free . . . 257
Measuring How Often It Fails . . . 258
What to Do With a Failure . . . 260
Lab: Nine Ways to Fail, and What to Do About Them . . . 262
Summary, Key Terms, and Review Questions . . . 264

22. Constrained Decoding and Strict Modes . . . 268
What a Constraint Actually Does . . . 268
The Mask, Step by Step . . . 270
Why the Support Varies Between Engines . . . 272
When a Strict Format Makes the Answer Worse . . . 274
What You Still Have to Test . . . 275
Lab: Build a Constrained Decoder and Break It . . . 276
Summary, Key Terms, and Review Questions . . . 278

23. How a Tool Call Works, and Who Decides . . . 283
Nothing Called Anything . . . 283
Describing a Function to a Model . . . 284

The Proposal . . . 286
Your Code Decides . . . 286
Why Validation Cannot Live in the Instructions . . . 289
What the Model Sees Next . . . 290
What the Descriptions Cost . . . 292
Lab: Run the Five Steps by Hand . . . 293
Summary, Key Terms, and Review Questions . . . 295

24. Embeddings, and Choosing One for Your Data . . . 299
What an Embedding Model Produces . . . 299
Closeness, Applied . . . 300
Ranking . . . 302
Why the Best Model Is a Property of Your Data . . . 302
What a Public Benchmark Score Is . . . 305
Lab: Measure Two Models on Your Own Terms . . . 306
Summary, Key Terms, and Review Questions . . . 308

25. Where Dense Search Fails . . . 312
A Part Number Is Almost No Signal . . . 312
Similarity Is Symmetric. Relevance Is Not. . . . 315
Why You Cannot Judge This by Eye. . . . 317
What You Can Conclude Without an Answer Key . . . 319
And What to Do About It. . . . 320
Lab: Find the Failures Yourself . . . 320
Summary, Key Terms, and Review Questions . . . 322

26. Lexical, Dense and Hybrid Retrieval . . . 326
Matching the Words As Written . . . 326
BM25, and the Two Knobs In It . . . 327
Why the Two Scores Cannot Simply Be Added . . . 330
Writing Down How Much a Document Answers . . . 333
Three Measures, and What Each One Misses . . . 334
Two Encoders, or One Model That Sees Both . . . 339
What a Reranker Cannot Do, and Why That Is a Derivation . . . 341
Measuring Hybrid Against Dense on Your Own Data . . . 342
Lab: Build Both Searches and Score Them. . . . 347
Summary, Key Terms, and Review Questions . . . 349

Part III. Apply **356**

Part IV. What Next **357**

Appendices 358

A. Cheat Sheet. 358
Part I · Mental Model. 358
Part I · Inside the Model 358
Part II · Use 361

B. Interview Bank 366
Part I · Mental Model. 367
Part I · Inside the Model 367
Part II · Use 368
Part III · Apply 370
Part IV · What Next 371

C. Glossary 372

Index 386

Preface

Placeholder preface, used only to prove the build. Replaced before any chapter is written.

PART I

Understand



What to ignore for now: every setting, price and product name. This part is what the thing is, not how you drive it.

PART I

Mental Model



What to ignore for now: what is inside the model — the second half of this part opens it.

CHAPTER 1

What a Model Is, and What You Get From It

This chapter answers the question underneath every other question in this book: when an engineer says they are "using a model", what exactly is in their hands? By the end you will know what you get, what you choose, what you are charged for, what you wait for, and the one thing you do not control at all.

The problem. Everyone talks about "using AI" and I picture training something. I have no idea what you actually get, or how.

The question. What is the thing an engineer actually holds, and what can it do?

The Short Answer

A **model** is a service. Your program sends it some text. It sends some text back. You are charged for the amount of text that moves in both directions, and you wait: a noticeable pause before the first word, and longer still for a long reply.

That is the whole of it. Everything else in this book is about what you send, what you do with what comes back, and how you find out whether any of it was any good.

The sentence is worth reading twice, because it contradicts the picture most people start with. You do not download a brain. You do not teach it your business. You do not own it. You make a request over a network to a program running on somebody else's machine, in the same way your application might request a currency rate or a map tile.



"A service you call" describes how most applications use a model, not all of them. You can run a model on hardware you control, and some teams do. That changes who pays the electricity bill and who is woken at night; it does not change the shape of the exchange, which is still text in, text out. This book teaches the common case and says where the other one differs.

Using Is Not Training

Two completely different activities get called "doing AI", and confusing them is the most common beginner's error in this field.

Training is the process that produced the model. It consumed an enormous amount of text and an enormous amount of computation, it happened once, it was done by a large organisation, and it finished before you ever heard of the result. What it produced is a very large collection of numbers.

Inference is what you do. You send text to the finished model and it computes a reply. Nothing is learned: the numbers do not change because you called it, so the model itself carries nothing from one call to the next. Products that appear to remember you, store your earlier messages and send them again with the next request. The memory is in the product. It is never in the model.

	Training	Inference
Who does it	A handful of large organisations	You, on every request
How often	Once, over weeks or months	As often as your application runs
What changes	The model's numbers	Nothing
What you need	Thousands of specialised processors	A network connection

The model you call was trained once and its numbers are now fixed. Two things are true of it, and the second surprises people.

The first: it will not get better at your task overnight. A weakness you find today is still there tomorrow — unless the provider swaps the model underneath you, and more on that in a moment.

The second: it will not repeat itself exactly either. Two identical requests usually come back worded differently. That has nothing to do with the numbers being fixed. It comes from the way a reply is produced — one piece at a time, with an element of chance at each step. Chapter 13 shows where the chance enters and what it means for testing anything.

Providers do replace models underneath you, and a replacement can change behaviour without warning. Pinning a version is how teams stop that happening silently, and Chapter 54 shows how.



When you read a claim about "AI", ask which activity it describes. "We trained a model on our documents" and "we sent our documents to a model" are different sentences with different costs, different risks and different failure modes. A surprising amount of confused writing comes from using the first to describe the second.

A model trained once on general material, then pointed at many tasks it was never specifically built for, is called a **foundation model**. The name tells you its intended use: it is a foundation you build on, not a finished product.

There is a third activity — continuing a finished model's training on examples of your own — and it is a real option with real costs. It is the last lever to reach for rather than the first, and Chapter 39 is where it is taught. Until then, assume the model is fixed.

The Exchange

Strip away the libraries and a request looks like this.

You send:

- **Instructions.** What you want done, in words.
- **The input.** The thing you want it done to.
- **Settings.** A small number of switches that change how the reply is produced — how adventurous the wording is allowed to be, how long the reply may run. Chapter 13 teaches them.

You get back:

- **Generated text.** Which may be prose, or a list, or something structured that your code can read.
- **An accounting of how much text moved**, which is what you are billed on.

Here is one exchange with the three parts of the request marked. The settings are not shown, because they travel beside the text rather than in it, and the accounting comes back with the reply. Nothing here is special to any provider.

```
WHAT YOUR PROGRAM SENDS
  instructions
    You are a support assistant. Answer in two sentences.
    If the answer is not in the notes, say you do not know.
  added material
    Order 4471 shipped on Tuesday by courier. Tracking KP-9930.
  the request
    Has my order gone out yet?

WHAT COMES BACK
    Yes — order 4471 shipped on Tuesday by courier.
    You can track it with the reference KP-9930.
```

Two things in that exchange are worth marking, because the rest of Part I grows out of them. The model had never heard of order 4471. It answered because the order's details were put there by somebody's code, in the request itself. And the instruction *"say you do not know"* is in there because, left alone, the model would have answered anyway.

That is the whole interface. There is no query language to learn, nothing to configure, no index to build. The absence is disorienting at first. Most components you call accept only the requests they were built for and refuse everything else, so the limit on what you can ask is obvious, and the answer, when it comes, is the right kind of thing. A model's interface sets no such limit: you can ask it anything, and it will almost always produce something that looks like an answer. Your difficulty moves somewhere else entirely — to finding out whether what came back is right.



The amount of text is not measured in words or characters. It has its own unit with its own rules, and those rules explain several things that look like bugs. Chapter 10 defines it properly. Until then, "the amount of text" is accurate enough.

What It Knows

A model's knowledge came from the text it was trained on. That gives you three facts with real consequences.

It knows a great deal, generally. Common knowledge, how language works, how code is usually written, how documents are usually shaped. This breadth is what makes a foundation model useful on a task nobody prepared it for.

Its knowledge stops at a date. Training consumed text gathered up to some point and no further. That point is the model's **training cut-off**. Anything after that point is absent — not wrong, absent. The model has no way of knowing that it is missing something, which is why it will often answer a question about last week with the confidence it uses for a question about last century.

It has not seen your private material. Your internal documents, your database, your customer's order history were not in the training text. Anything of yours that was published openly may well have been — but you cannot build on that, because the model may hold a distorted memory of an old version and has no way to fetch the current one.

And the model itself cannot look anything up. It computes a reply from what you sent and what it absorbed in training, and from nothing else. Products that search the web or read your files do it with machinery around the model, which is what Chapter 2 is about.



A model will answer a question about your data as readily as any other question. It does not say "I have no access to that." It produces something plausible, because producing something plausible is what it does. Chapter 16 explains why this is the design working as intended rather than a defect.

This has a practical corollary that will shape most of your work. **If you want the model to use information, you have to put that information in the request.** Not in a configuration file, not in a database it connects to — in the text you send, on that call. Chapter 2 is about the machinery that gets built to do this well.

What You Choose, and What You Do Not

You choose	You do not choose
Which model to call	What that model knows
What text you send it	How it was trained
The settings on the call	What it will say to a given request
What your code does with the reply	Whether the provider changes it next month

The right-hand column is uncomfortable and it is worth sitting with. A model is a component whose behaviour you influence but do not specify. You cannot read its rules, because it has none written down. You cannot patch it. When it does something you do not want, your options are to change what you send, change which model you call, or put a check around the result and handle the bad cases.

Nearly all of the engineering in this book lives in those three options, and most of it lives in the third.

What It Costs

You are charged for the text you send *and* the text you get back, usually at different rates, with the reply costing more. Providers quote the price per large block of text, so a single request costs a tiny fraction of a currency unit and the number looks like nothing at all.

It stops looking like nothing when you multiply it. Two habits make the bill, and both are invisible on a single call:

- **A long request, sent many times.** If you attach a page of instructions to every request, you pay for that page on every request, for the lifetime of the feature.
- **A long reply.** Replies cost more than requests, and a model asked an open question will happily produce six paragraphs where one would have done.

Here is the shape of the arithmetic, with made-up rates to show the mechanism rather than any real price. Say text you send costs 1 unit per million words, and text you get back costs 4 — the reply is dearer, as it usually is.

A feature sends 500 words of instructions and a 500-word document, and gets back a 200-word summary. That is 1,000 words in and 200 out:

```
in    1,000 words  x 1 unit per million  =  0.0010 units
out     200 words  x 4 units per million =  0.0008 units
                                   per call  0.0018 units
x 10,000 calls a day                          18 units a day
x 30 days                                    540 units a month
```

Now add half a page of standing instructions — 250 more words — to every request. The input goes from 1,000 words to 1,250, so the call costs 0.00125 + 0.0008 = 0.00205 units: 20.5 a day, about 615 a month.

The feature has not changed and no user has noticed. The bill is up about 14 per cent, for as long as the feature exists. Note what did *not* happen: the input grew by a quarter and the bill grew by a seventh, because the dearer half of the exchange is the reply. **Which half of the exchange you are growing decides how much a change costs you.**



Every price in this field changes, and the cheap model of this year is the expensive one of next. The arithmetic above is durable. The numbers are not, which is why this book gives you the first and almost never the second.

Chapter 53 takes cost seriously, with the levers that actually move it. For now, carry one idea: **cost scales with the text you move, so the length of what you send is an engineering decision and not a matter of taste.**

What You Wait For

A model does not reply instantly. That delay is its **latency**. It is long enough to feel, and it behaves differently from the delays you are used to.

A database query is mostly over when it is over. A model's reply arrives in two distinct phases, and you experience them separately:

1. **The wait before anything appears.** The model has to read everything you sent before it can produce a single word. Send more, wait longer.
2. **The pace of the words after that.** The reply is produced a piece at a time, in order, and arrives as a stream. A long reply therefore takes longer than a short one — not because it is harder, but because there is more of it to produce.

This is why interfaces built on models show text appearing progressively. It is not a visual effect. It is the shape of the underlying process, exposed to the user because hiding it would mean waiting for the whole reply before showing anything.



Two systems with the same total response time can feel completely different. One that shows its first word quickly and then types steadily feels responsive. One that shows nothing and then delivers everything at once feels broken, even when it finishes first. Which of the two phases you optimise is a product decision before it is a technical one.

Chapter 15 explains why the two phases exist and what they cost; Chapter 52 is about measuring and reducing them. Here, the point is smaller: **latency is a property you design around, not a number you can ignore.**

Why “It Is a Service” Is the Useful Picture

Hold the model as a component with four properties: it takes text, it returns text, it charges by the amount, and it takes a moment. Everything in the rest of Part I follows from treating it that way.

A component that cannot see your data needs something to fetch that data. A component that cannot act in the world needs something to act for it. A component that cannot reliably check itself needs something to check it. And a component that is not accountable for what it does needs somebody who is, wherever the consequences are serious.

Four gaps, and the next chapter is the five parts that get built around them.

Lab: What Is This Thing Made Of?

No code. This lab trains the habit the rest of the book depends on: looking at an AI product and seeing its parts instead of its demonstration.

Step 1. Find three AI features you can actually use. A coding assistant, a search box that answers in sentences, a support chat, a summarising button in an application you already have. Use each one twice.

Expected result: three products in front of you, used, not read about.

Step 2. For each, write one line saying what text you think was sent on your behalf. Your typed request is part of it. What else must have been included for the answer to come out as it did — a page of yours, a document, an earlier message in the conversation?

Expected result: at least one product where the answer clearly used information you never typed. That information was put into the request by somebody’s code.

Step 3. For each, note which of the two waits you felt: a pause before anything appeared, a steady stream of words, or both.

Expected result: you can name the phases in a product you used today.

Step 4. For each, write down what evidence you were given that the feature works. Count a measured claim and a demonstration as different things.

Expected result: mostly demonstrations. This is the normal answer, and it is the reason a later part of this book spends so long on measurement.

Step 5. Ask one of them a question about something that happened in the last few days, and one about your own private information.

Expected result: one of three things, and all three are worth having. A confident answer that cannot possibly be grounded in anything the model knows — keep it, it is the most useful thing in this lab. Or a correct answer, which means it did not come from the model at all: something fetched it and put it in the request, and that is Chapter 2. Or a refusal — "I cannot browse the web", "I have no access to your account" — which is the product telling you where its boundary is.

Summary, Key Terms, and Review Questions

Summary

A model is a service: text in, text out, charged by the amount, answered in a noticeable pause before the first word and longer for a long reply. It was trained once by somebody else and is fixed while you use it. It knows a great deal in general, knows nothing after its training cut-off, and knows nothing of yours — and it will not tell you when it is out of its depth. You choose the model, the text you send, the settings and what your code does with the reply. You do not choose what it knows or how it behaves. Cost grows with the text you move; the wait has two phases, one before the first word and one that paces the rest.

Key Terms

Term	Meaning
Model	A service that takes text and returns generated text, charged by the amount of text
Training	The one-time process that produced the model's numbers. Not something you do on a request
Inference	Running a finished model to get a reply. This is what "using a model" means
Foundation model	A model trained once on general material, then used for many tasks it was not specifically built for
Training cut-off	The point in time after which no text reached the model's training. It knows nothing later, and cannot tell that it does not
Latency	The delay between sending a request and receiving the reply, felt as a wait before the first word and a pace after it

Worth Memorising

Item	Value or formula
What cost scales with	The amount of text moved, in and out. Not the difficulty of the question
The two rates are not equal	Text you get back costs several times text you send — about 4× in the book's worked shape
Cost of a feature, per call	(words in × rate in) + (words out × rate out), then × calls × days

Review Questions

1. A colleague says "we trained the model on our support tickets". What two different things could they mean, and how would the costs differ?
2. You ask a model about an event from last week and get a confident, detailed answer. Give two reasons the answer might be worthless.
3. Your feature sends half a page of standing instructions with every request. The feature is unchanged for six months. What happens to its cost, and what decides how much it costs you?
4. Why does a long reply take longer to arrive than a short one, even on the same model?
5. Name the one thing in the exchange you do not control, and the three options you have when the model does something you do not want.

Review Answers

1. They may mean **inference** — sending ticket text in each request, which costs per call and changes nothing permanently. Or they may mean continuing the model's **training** on those tickets, which is a one-off project with its own costs and leaves them owning a model they must maintain. Chapter 39 covers the second.
2. The event may fall after the model's training cut-off, so it cannot know about it; and the model has no way to signal that it is missing information, so it produces something plausible instead of refusing.
3. You pay for that page on every single call, for six months. How much it adds depends on which half of the exchange it grows: input is the cheaper half, so a page that grows the input by a quarter raises the bill by less than a quarter — in the worked example, a quarter more input gave about a seventh more bill.
4. The reply is produced a piece at a time, in order. More pieces take more time. The length of the reply, not its difficulty, sets that part of the wait.
5. You do not control what the model knows or how it behaves. Your three options are: change what you send, change which model you call, or check the reply and handle the bad cases.

> **You understand this when** you can explain to someone why a model cannot answer a question about your own documents unless you put those documents in the request.

CHAPTER 2

The Parts Around the Model

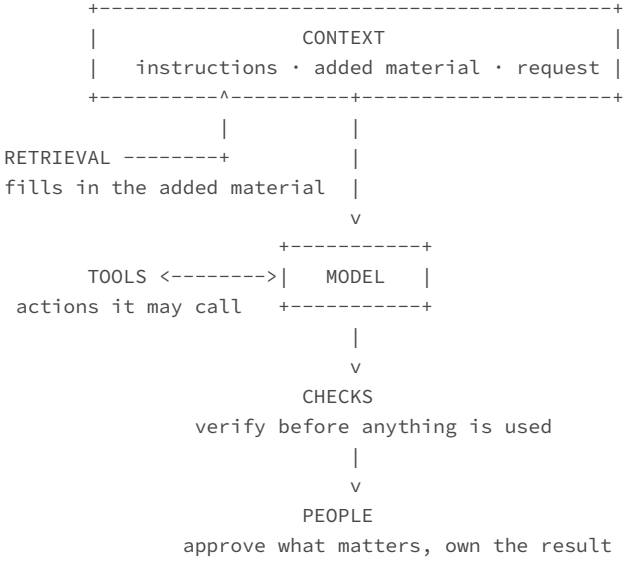
Chapter 1 left the model as a component: text in, text out, charged by the amount. This chapter puts the five things that get built around it on the table and says what each one is for. By the end you will be able to look at an AI feature and name its parts, which is the habit the whole of Part I is training.

The problem. The model is one box. Everything I read draws five more boxes around it and I can't see why any of them are there.

The question. What are the other parts, and what does each one do?

The Cast

Here is the whole cast. Five parts, and the model in the middle of them.



Chapter 1 ended on four gaps: the model does not know your data, cannot act in the world, cannot reliably check itself, and is not accountable. Four of the parts above answer one gap each — retrieval, tools, checks, people, in that order. The fifth, the context, answers no gap. It is the thing you hand over, and the other four exist to fill it, to act on what comes back, or to decide whether any of it may be used.

Chapter 3 is the argument for why those four gaps are real, and what each part costs you in return. This chapter is the tour: what each one is, what it is for, and what it looks like when it is there.

Context: Everything You Send

Chapter 1 showed one request going out, with three labelled pieces: the instructions, the added material and the request itself. Those three pieces together have a name. They are the **context** — the settings from Chapter 1 are not part of it, because they travel beside the text rather than in it — everything the model gets on this call, assembled by your code and sent together.

Piece	What it is	Who writes it
Instructions	What to do, how to behave, what to refuse	You, usually once, and they ship with every call
Added material	Facts this particular answer needs	Your code, freshly, for this call
The request	What the user actually asked	The user

Three things about the context are worth fixing in your mind now, because almost every later decision in this book is a decision about one of them.

The context is assembled fresh on every call. There is no session the model keeps for you. If a conversation is to continue, the earlier turns are part of what gets sent again. If a document is to be discussed, the document goes in. Chapter 1 put it as a corollary; here it is as a design rule: *what is not in the context does not exist.*

The context is where the money is. Chapter 1 established that cost grows with the text you move. What this chapter adds is *where that text lives*: almost all of it is the context. That turns the size of the context into an engineering decision rather than a detail, and Chapter 53 is where it becomes a budget.

The context has a ceiling. There is a limit on how much text a model will accept in one call. It is large, and it is not infinite, and real systems hit it. What to do when more material is relevant than will fit is a genuine engineering problem, and Chapters 28 and 35 are two different answers to it.



You will see the word *prompt* used for all of this, for the instructions alone, and for what the user typed. The word is used three ways and the ambiguity is genuine, so this book says **context** when it means everything sent, and says *instructions* or *the request* when it means one piece.

Retrieval: Finding the Material

The added material row above says "your code, freshly, for this call". **Retrieval** is the machinery that does the finding: given a request, search your own material and return the passages that bear on it, so they can go into the context.

The shape is usually this, whatever the implementation.

1. Something arrives — a question, a ticket, a document to process.
2. Your code searches a collection you control: documents, records, past conversations.
3. The best few passages come back.
4. Those passages are put into the context, with the request.
5. The model answers from what it was given.

Not every system runs those steps in that order — some let the model ask for the search itself, once it has seen the question — but the job being done is the same one.

Chapter 1's support example is a retrieval system with the retrieval taken out. The line `Order 4471 shipped on Tuesday by courier` did not come from the model and did not come from the user. Something looked up order 4471 and put that line in. That something is retrieval, even when it is a single database query.



"Retrieval" in this field usually brings to mind searching by meaning rather than by keyword, and Chapters 24 to 26 are about that. But a plain database lookup that puts a row into the context is doing the same job in the same place. The part is defined by where it sits, not by how clever it is.

What retrieval is *for* is narrow and worth stating exactly: it supplies facts the model does not hold. It does not make the model cleverer, and it cannot fix an answer that was badly reasoned. What it changes is what the model knows at the moment it answers — not how well it reasons about it.

Tools: Letting It Act

A **tool** is an action you allow the model to ask for. Looking up an order. Checking stock. Sending a message. Running a search. Creating a ticket.

The sentence that matters is this one, and it is the whole of tools in Part I:

The model proposes. Your code decides.

The model itself never reaches out and does anything. It produces a request — *call `refund` with order 4471 and amount 30* — and that request is text like any other text it produces. Your code receives it, decides whether to honour it, runs the actual function if it does, and puts the result back into the context so the model can carry on.

	What it does	What it does not do
The model	Asks for a tool, with arguments	Run anything
Your code	Defines which tools exist, validates the request, runs it, returns the result	Write the request itself

Both of them decide something, and the two decisions are different. The model decides *that* a tool is wanted. Your code decides *whether that happens*. Your code can take the first decision away — insisting the model use a particular tool, or offering it none at all — but the model can never take the second.

Three consequences follow immediately, and all three come back later with force.

You decide what exists. The model can only use tools you described to it. It may well ask for something you never offered — a plausible-sounding function that does not exist — and your code refuses it, because there is nothing there to run.

You decide what each tool may reach. A tool that cannot touch a system cannot damage that system, so what each one is allowed to do is a decision you make once, in your code.

You decide what happens on the day. A proposed call is a suggestion arriving in text. It can be wrong, it can have nonsense arguments, and it can be there because something the model read told it to ask — which is Chapter 51, and is as alarming as it sounds. Validating it is your code's job, not the model's.

How the proposal is expressed and handled in practice is Chapter 23. What makes a tool one the model uses well is Chapter 36.

Checks: Finding Out Before the User Does

A **check** is anything that verifies the output before it is used. Chapter 1 established that you cannot predict what the model will produce. A check is what you do about that.

What *shape* means is easiest to see concretely: your code expects an answer and a price it can do arithmetic with, and what came back is a friendly paragraph with the price written out in words. A shape check is what notices. Chapter 21 is how you ask for one properly.

Checks come in a range, from cheap and certain to expensive and approximate:

Check	Catches	Cost
Does it fit the required shape?	Output your code cannot read	Nothing, and it is certain
Does it pass a rule you can write?	A price below zero, a date in the past, a missing citation	Almost nothing
Does a second model judge it acceptable?	Rudeness, irrelevance, an answer that ignored the material	A whole extra call, and it is approximate

Below the bottom of that table there is a person — and a person is not a fourth rung of checking. They are the next section.

Work down that table in order. Anything a rule can decide should be decided by a rule, because a rule costs nothing and gives the same verdict every time. Judgement is kept for what actually needs judgement.



A check that is itself a model call is not a referee standing outside the system. It is another component with the same properties as the first: you cannot predict what it will say, it can be wrong, and it has to be checked before it is trusted. Chapter 31 is entirely about how you earn the right to believe one.

There is a second half to checking that is easy to miss on the first pass. A check tells you about *one* answer, now. Finding out whether the system is any good **in general** is a different activity, with different machinery, and the book spends Chapters 30 to 33 on it.

People: Who Actually Decides

The fifth part is **people**: the one engineers forget when they draw the diagram, and the one every serious system has.

In most systems that do anything consequential, a person still reviews or approves the cases that matter. Not every case — that would remove the point of building it — but the ones where being wrong is expensive.



That "most" is current practice rather than a law, and the evidence behind it is thinner than it sounds: the figures people quote come from a small number of surveys of deployed systems. Treat it as what teams are doing now. The reason underneath it is the durable part, and it is next.

The usual arrangement is a threshold. Refunds under a small amount go through; larger ones reach a queue. Routine replies send themselves; anything touching a complaint waits for a human. The system does the volume, and a person owns the consequential tail.

The durable reason is not that models are not good enough yet. It is that somebody has to be accountable for a decision that affects a customer, and accountability does not sit in a component. How much a person is asked to approve will move as systems get better. That there is a person who answers for the consequential cases is a different kind of claim. Chapter 3 makes that argument properly, and Chapter 51 shows what goes wrong when the approval step is missing from a place that needed one.

The Whole Thing Has a Name

A model with some arrangement of those parts around it is a **compound system**: several components, each doing a job, with the model as one of them rather than as the product.

The name earns its place because it changes what questions you ask. “Is the model any good?” is not answerable about a compound system, and it is not the question anybody should be asking. The useful questions are about parts and joins:

- Did retrieval find the right material?
- Was the context assembled well?
- Did the model use what it was given?
- Did the checks catch what they should have?
- Did the right cases reach a person?

A system can fail at any one of those while every other part works perfectly. Telling them apart is its own skill, and Chapter 5 is about how much of the real work it turns out to be.

The Same Small Cast, Arranged Differently

Look at AI products for a week and the five parts keep reappearing in different proportions. The arrangement is the product; the parts barely change.

Every product here has a context, because a context is what you send — so the interesting columns are the four that vary.

Product	Retrieval	Tools	Checks	People
Answers questions about a company's documents	The whole point of it	Usually none	Is the answer supported by what was found?	Reviewed when a customer sees it
Writes a first draft inside an editor	The parts of your document near the cursor	Formatting, inserting	Mostly the human reading it	The author, always, by design
Triages incoming tickets	Similar past tickets	Tagging, routing, replying	A rule about when it may reply unaided	Anything it is unsure about
Writes and runs code	The repository	Read a file, write a file, run the tests	**The tests themselves**	A review before it merges

The last row is worth a second look. Running the tests is both a tool and a check: the model asks for it like any other action, and the result is the most trustworthy *automatic* verdict here, because it is a rule and not an opinion — though only about what the tests actually cover, and not at all if the model wrote them. Where a real, cheap, automatic verdict exists, the system around the model gets simpler and much more reliable. Where none exists — "is this summary good?" — everything gets harder. That difference will come up again and again.



When you meet a new AI product, do not ask which model it uses. Ask what it retrieves, what it is allowed to do, what it checks, and who approves. The answers tell you what it actually is. The model name tells you almost nothing.

What This Chapter Has Not Told You

Deliberately, three things:

- **Why each part is necessary.** You have been told what each part is for. The argument that the model cannot do these jobs itself, and what each part costs you in return, is Chapter 3.
- **What happens when the model runs the loop itself.** Everything here has your code in charge: your code retrieves, your code decides whether a tool runs, your code applies the checks. Hand that decision to the model, repeatedly, and the arrangement gets a different name and a different set of problems. That is Chapter 4.

- **How any of the parts actually work.** Retrieval is Chapters 24 to 29. Tools are 23 and 36. Checks are 21, 30 to 33. This was the tour.

Lab: Name the Parts

No code. You did a version of this in Chapter 1's lab; this time you have the vocabulary.

Step 1. Take the three AI features you used for Chapter 1's lab. For each, fill in one row: retrieval, tools, checks, people. Write *none* where you believe there is none, and *unknown* where you cannot tell from outside.

Expected result: three rows, several *unknown* entries. Users cannot see checks, and that is normal.

Step 2. For one of them, write down what you think the instructions say. You will not be right, but try: what must the product have told the model about how to behave?

Expected result: at least one instruction you are confident about, usually a refusal — something the product consistently will not do.

Step 3. Look for the ceiling. Give one of them far more material than it expects — a long document, or a very long conversation — then ask it about something from well inside that material.

Expected result: one of three. A refusal, which is the limit stated out loud. An answer that has quietly lost part of what you gave it. Or no visible effect at all, which means you have not reached the ceiling yet — modern limits are large. Which part gets lost, and why long inputs are used unevenly, is Chapter 16.

Step 4. Find a place where a person must be in the loop. Ask one of them to do something consequential and irreversible.

Expected result: a refusal, a confirmation step, or a handover to a human. If a product does something irreversible with no confirmation, you have found a design decision worth remembering.

Step 5. For each of the three, write the single sentence: *this product is a model plus* ___. Fill the blank with parts, not adjectives.

Expected result: three sentences that would tell another engineer how the product is built.

Summary, Key Terms, and Review Questions

Summary

Five parts get built around the model. The **context** is everything you send on a call — instructions, added material and the request — assembled fresh every time, limited in size, and responsible for most of the cost. **Retrieval** finds the added material in your own collection and supplies facts the model does not hold. **Tools** are actions the model may ask for: it proposes, your code decides, runs and returns. **Checks** verify the output before anybody uses it, from a shape it must fit through to a second model's opinion. **People** approve the consequential cases and own the result. The whole arrangement is a **compound system**, and once you see it as parts, "is the model any good?" stops being the question.

Key Terms

Term	Meaning
Context	Everything sent to the model on one call: the instructions, any added material, and the request
Retrieval	Searching your own collection for the passages that bear on a request, so they can be put into the context
Tool	An action the model may ask for. The model proposes it; your code decides whether to run it
Check	Anything that verifies the output before it is used, from a required shape through to a second model's opinion
People	The reviewers and approvers who handle the consequential cases and carry responsibility for them
Compound system	A system of several parts with the model as one of them, rather than as the whole product

Review Questions

1. Your assistant answered correctly yesterday and cannot answer the same question today. Name two parts that could be responsible and say how you would tell them apart.
2. A colleague says their system "lets the model update the database". What is actually happening, and where is the decision made?
3. Why should a rule-based check run before a model-based one, when the model-based one catches more?
4. A product summarises documents and a product writes code. Which has the easier time checking its own output, and why?
5. You are shown a diagram with the model, the context, retrieval, tools and checks. Which

part is missing, and what job does it do?

Review Answers

1. **Retrieval** may have failed to find the material, or the **context** may have been assembled differently — a longer conversation pushing the useful part out, for instance. You tell them apart by looking at what was actually sent: if the right passage was in the context and the answer is still wrong, retrieval did its job and the problem is elsewhere.
2. The model is proposing a tool call, and their code is deciding whether to run it and then running it. The model updates nothing. The decision sits in their code, which is also where the validation has to be.
3. A rule costs nothing and gives the same verdict every time. A model-based check costs a whole extra call and is only approximate — it is a component with the same unpredictability as the first one. You spend judgement only on what needs judgement.
4. The code product. Running the tests is a real, cheap, automatic verdict, and it is a rule rather than an opinion. "Is this summary good?" has no equivalent, which is why everything around it is harder.
5. **People.** They approve the consequential cases and carry the responsibility for them, which is why the threshold for what reaches them is a design decision and not an afterthought.

> **You understand this when** you can look at any AI feature and say what it retrieves, what it is allowed to do, what it checks, and who approves.

CHAPTER 3

Why It Is Assembled This Way

Chapter 2 named the five parts and said what each one does. This chapter makes the argument underneath them: why each gap is real rather than a limitation somebody will fix, and what you pay for closing it. Every part you add buys something and costs something, and knowing the price is the difference between designing a system and copying a diagram.

The problem. Why not just ask the model? Why does every real system wrap it in so much machinery?

The question. Which jobs can the model genuinely not do, and what does each fix cost me?

The Shape of the Argument

Four gaps, in the order Chapter 2 gave them. For each one, three questions:

1. **Is it real?** Or is it a limitation of today's models that will close on its own?
2. **What closes it?**
3. **What does closing it cost?**

The third question is the one this chapter exists for. If a part had no cost there would be no decision to make, and the interesting design decisions all live in what you give up.

It Does Not Know Your Data

Is the gap real?

Yes, and it is permanent, which is not obvious.

The easy version of this gap is the one Chapter 1 gave: the model's training stopped at a date, so it has not seen anything since. That half does shrink — models get retrained, and newer ones tend to know more recent things.

The half that does not shrink is everything else:

- **Your data is private.** It is not in any training set and you do not want it to be.
- **Your data changes constantly.** An order's status is different this afternoon. No amount of training reaches that.
- **Your data is different per reader.** Two users ask the same question and the correct answer differs, because they are allowed to see different things. A model's knowledge is one body of knowledge, identical for everyone who calls it.

That last point is the one to keep. Even a model trained this morning on everything your company owns would answer every user from the same store, and could not be told that this user may not see that record. It would be wrong wherever permissions differ or the data has moved since — which, in most companies, is most of the time.



You may have heard that continuing a model's training on your own documents solves this. It does not. Training is a snapshot too, so the data still goes stale; and weights carry no notion of who is allowed to see what, so per-reader scope is still impossible. It also puts your private material into a set of weights, which is the thing the first point says you would rather avoid. It is one of several reasons Chapter 39 treats tuning as the last lever rather than the first.

What closes it, and what it costs

Retrieval closes it, and the price is a second system that can be wrong on its own.

You gain	You pay
Answers about material the model never saw	A search system to build, fill and keep current
Material that is current as of this request	Latency: a search has to happen somewhere in the request
Per-user answers, because you choose what to fetch	Context space, which is finite and not free
A citation — a pointer back to the passage an answer came from — when you keep track of what you fetched	**A new failure mode: retrieving the wrong thing**

The last row is the important one. Before retrieval, a wrong answer meant the model was wrong. After retrieval, a wrong answer might mean the search found nothing useful, or found something misleading, and the model faithfully used it. **You have not removed the failure. You have split it in two**, and telling the halves apart becomes work you now have to do.

It Cannot Act in the World

Is the gap real?

Yes, and this one is not even a limitation. It is what the component is.

A model produces text. That is the whole of its output. Text that says `refund(order=4471)` is still text; it refunds nothing. Something else has to read that text and make a refund happen, and that something is a program somebody wrote.

This will not change with better models, because it is not about capability. A model that could act directly would be a model wired to a system that executes what it says — which is exactly the arrangement we are describing, with the wire hidden.



This is worth being precise about in conversation, because the loose version — "the model called the API" — hides the part that matters. Something decided to honour the request. If you cannot say what decided, you cannot say what would have stopped it.

What closes it, and what it costs

Tools close it, and the price is everything a tool can reach.

You gain	You pay
The system can do things, not only say them	Every tool is a way in, and it has to be secured
The model can fetch what it needs mid-answer	A tool that is slow makes the whole answer slow
Actions can be logged, and some can be compensated — if you build that	A tool that is down takes a working feature with it
—	**Actions can happen that nobody intended**

That last cost deserves its own sentence, because it is different in kind from the others. A wrong answer is a wrong answer. A wrong **action** has already happened by the time you read it. You can sometimes issue a second action that compensates — re-charge the card, send a correction — but that is not an undo, and for a deleted record or a message somebody has already read there is often nothing to issue.

An **irreversible action** is one of these: something a tool does that no later action undoes. This is the whole reason Chapter 2 insisted that your code decides, and it is why the cheapest safety measure in this book is declining to give a tool an irreversible action at all. Chapter 51 spends a long time on what a tool may touch.

It Cannot Reliably Check Itself

Is the gap real?

Yes, and the reason is structural rather than a matter of effort.

Ask a model whether its last answer was correct and it produces another answer. That second answer comes from the same process as the first and works from the same information, so it stands in the same relation to the truth as the first did: usually right, sometimes confidently wrong, and not consulting any record either way. **A confident answer and a confident check are the same kind of object.**

You can often improve things by asking in a way that makes the model look harder, and Chapter 31 is about where that does and does not work. But the structural point stands: a check that shares a component's failure modes is not an independent check. It will tend to be wrong in the same places, for the same reasons.



This is why "I asked it to double-check and it said it was sure" is worth very little as evidence. What the model says about its own confidence is produced the same way as everything else it says. It is not reading off a measurement of whether the answer is right, because it has none to read.

What closes it, and what it costs

Checks close it, partially, and the price depends entirely on which kind you can afford.

Kind of check	You gain	You pay
A rule your code applies	The same verdict every time, for nothing	It only catches what you thought to write down
A second model asked to judge	Catches what rules cannot describe	Another call, and another component that can be wrong
A person	Real judgement	Time, money, and a queue

The deeper cost is one the table cannot show: **every check is built against somebody's idea of what might go wrong.** A rule is built narrowly, against one named failure. A model judge is built broadly — "is this rude?" covers rudeness nobody listed — but it is still built against a question somebody thought to ask. Either way, a system is blind to the failures nobody imagined. That is not an argument against checking. It is the reason Chapters 30 to 33 start by reading real failures rather than by writing checks.

There is also a cost in the other direction, and it is the one teams discover late. **A check that is too strict rejects good answers.** Say your rule is that an answer must cite a source. A reply that answers the question perfectly from a single obvious fact, without citing anything, now fails — and your users see a refusal where a good answer was waiting.

Every check that has a setting has this shape. Tighten it and you block more bad output and more good output; loosen it and you let more of both through. You can escape the trade by building a *better* check, which is real work; you cannot escape it by choosing a cleverer threshold on the check you have.

It Is Not Accountable

Is the gap real?

Yes — but not in the way it is usually told, and the usual telling gets the design wrong. It is also the only one of the four where adding the part that closes it leaves the cause exactly where it was.

Here is the version to discard first: *without a person in the loop, nobody is answerable.* That is false. If your company deploys a system that refuses a loan, loses a customer's money or publishes something defamatory, **your company is answerable**, whether or not anybody looked at the decision. Automating something does not move responsibility onto the thing you automated it with. In many fields that is written into law.

So **accountability** — somebody who can be asked to answer for a decision, and who bears a consequence for it — **never actually moves.** It sits with the operator — you, your team, your company — from the first line of code. What the model cannot do is **carry any of it for you**:

- It cannot be asked to explain a decision in terms a person can defend.
- It cannot be sanctioned, struck off, called in front of a regulator, or sued.
- It cannot notice that this particular case is the one that should not be decided by rule.

That is the real gap, and it is sharper than the usual one. **You are answerable for decisions you did not see and could not have defended.** The system produced them; the responsibility was yours the whole time.



This is usually filed under "ethics" and set aside. It is better understood as a design constraint, like a rate limit: there is a set of decisions your organisation cannot defend making unseen, and the system has to be built so those reach somebody who can. Where that set is drawn is partly law and partly judgement — the legal floor is not yours to move, and everything above it is a design decision you own.

Does a better model close this? It can shrink the part of the set you draw yourself — a system you trust more is one you choose to supervise less closely. It cannot touch the part the law draws, and it does not change who is answerable. That is why this gap is unlike the first three: progress moves its cost, never its cause.

What closes it, and what it costs

People close it — not by becoming accountable, but by being how an accountable organisation discharges what it already owes.

You gain	You pay
A decision somebody can defend, on the cases that need defending	The system is now as fast as its queue
Judgement on the cases nobody anticipated	Reviewers are expensive, and good ones are scarce
A record of who decided what, and on what evidence	Approval is a bottleneck that grows with volume
—	**Rubber-stamping, which looks like review and is not**

The last row is the failure that catches people out. That is **rubber-stamping**: approval given without real attention. Show a reviewer a hundred confident approvals an hour and most will approve nearly all of them: almost every one is fine, and attention does not survive that. The effect has two halves, and the second is less obvious than the first — volume wears attention down, and a confident recommendation pulls a reviewer toward agreeing with it even when there is plenty of time. You then pay the full cost of review and receive very little of its value.



That is a well-studied effect in people supervising automated systems rather than something peculiar to this field, and it is one of the sturdier findings behind any claim in this chapter. Expect it; design against it.

So the question is never *should a person review this?* It is **which cases, how many an hour, and with what put in front of them** — and above the legal floor, that is a design decision like any other.

The Pattern

Read those four together and the shape is the same every time.

> ***Every part you add removes one failure and introduces another.***

The part	Removes	Introduces
Retrieval	Answering from nothing	Retrieving the wrong thing
Tools	Being unable to act	Acting wrongly
Checks	Shipping bad output unnoticed	Blocking good output, and a false sense of safety
People	Decisions made unseen that nobody could defend	Delay, cost, and rubber-stamping

This is not pessimism, and it is not an argument for building less. It is the reason compound systems are **designed** rather than assembled: you are choosing which failures you would rather have.

Two consequences follow, and they run through the rest of the book.

The simplest arrangement that solves the problem is usually right. Not because simple is virtuous, but because every part is a failure you are buying. A part that is not earning its cost should not be there, and the way you find out whether it earns its cost is to measure — which is why so much of this book is about measurement.

Where a cheap, automatic verdict exists, take it. Chapter 2 noted that a system which can run its own tests has an advantage over one that can only ask whether a summary is good. Now you can see why: its check is the one kind whose cost stays small. Not nothing — a test suite is a rule, so it still only catches what somebody wrote down, and it tells you nothing at all if the model wrote the tests. Small is rare enough.

Lab: What Does It Cost Them?

No code. Chapter 2's lab asked what parts a product has. This one asks what it paid.

Step 1. Take one product from Chapter 2's lab that clearly retrieves — one that answers from documents or from your own data. Ask it something whose answer you can verify yourself.

Expected result: an answer you can check. Keep it.

Step 2. Now ask it something that sounds like it should be in its material but is not — a plausible neighbouring question about something that does not exist.

Expected result: one of three. It says it does not know — which usually means an instruction told it to, as in Chapter 1's exchange, rather than that anything checked. It answers from general knowledge without saying so, which is the retrieval cost in action. Or it invents something, which is the same cost, further along.

Step 3. For a product with tools, find the most irreversible thing it can do. Do not do it. Work out what stands between you and it: a confirmation, a limit, a person, or nothing.

Expected result: either a step you can describe — a confirmation, a limit, a person — which is somebody's answer to the cost in the tools table. Or nothing at all between you and it, which is also an answer, and a more interesting one.

Step 4. Find a queue. Use a product where something you submit is reviewed, and notice how long it takes and what you are told while waiting.

Expected result: a wait you can measure, and usually a message designed to make it bearable. Both are the price of the fourth gap.

Step 5. Write one sentence for a product you use: *it pays ___ so that it can ___*.

Expected result: a sentence about trade-offs rather than features. If you cannot write one, you do not yet know what the product chose, which is itself worth knowing.

Summary, Key Terms, and Review Questions

Summary

Four gaps, four parts, four prices. The model does not know your data — permanently, because your data is private, changing and different per reader — so retrieval closes it and you pay with a second system that can be wrong on its own. It cannot act, because it only produces text, so tools close it and you pay with every surface a tool can reach, including actions that cannot be undone. It cannot reliably check itself, because a check from the same component shares its failures, so checks close it and you pay for what they were not built to catch, and for the good answers a strict check rejects. It is not accountable — accountability stays with you whatever you build — so people are how you discharge it on the cases that need defending, and you pay in delay, cost and the risk of rubber-stamping. The pattern is the lesson: **every part removes one failure and introduces another**, which is why the simplest arrangement that works is usually right.

Key Terms

Term	Meaning
Accountability	Somebody who can be asked to answer for a decision, and who bears a consequence for it. A property of people and organisations, never of a component
Irreversible action	Something a tool does that cannot be undone once it has happened — a refund, a message, a deletion. The reason a tool's reach is decided before it is offered
Rubber-stamping	Approval given without real attention, usually because the volume is too high for any. It costs what review costs and delivers none of its value

Review Questions

1. A colleague says the "does not know your data" gap will disappear once models can be retrained cheaply every week. Give the strongest reason they are wrong.
2. What is the difference in kind between a wrong answer and a wrong action, and what does that difference imply about which tools you offer?
3. Why is asking a model to check its own answer weaker evidence than it feels?
4. Your system sends 100 cases an hour to a human reviewer who approves 99 of them. What have you bought, and what have you paid?
5. A teammate proposes adding a second model to judge every answer. Name two costs, and name the question you would ask before agreeing.

Review Answers

1. Per-reader scope. Even a model retrained this morning on everything the company owns answers every user from the same body of knowledge, so it cannot be told that this user may not see that record. Freshness is the half of the gap that shrinks; privacy and per-reader scope are not.
2. A wrong answer can be read and discarded. A wrong action has already happened — the refund is paid, the message is sent. So the decision about which tools to offer is made before anything runs, and the cheapest protection is not offering an irreversible one.
3. The check comes from the same component and works from the same information, so it tends to fail in the same places. It is not consulting a record. And what it says about its own confidence is produced the same way as everything else it says, so it is not a measurement of whether the answer is right.
4. You have bought defensibility on those cases: somebody looked, and could account for the decision afterwards. You were accountable either way — that never moved. You are paying the full cost of review, and at 100 an hour you are probably also paying for

rubber-stamping, so the value may be much lower than the cost. The fix is fewer, better-chosen cases, not a faster reviewer.

5. It costs a whole extra call per answer, in money and latency, and it adds a component with the same kind of unpredictability as the first — a different model will not fail in exactly the same places, but it is not an independent referee either. The question to ask first: what failure are we trying to catch, and could a rule catch it instead?

> **You understand this when** you can say, for any part of an AI system, what failure it removes and what failure it introduces.

CHAPTER 4

The Agent Loop, and How Much Freedom It Actually Gets

Chapter 2 put five parts around the model and left your code in charge of the sequence: your code retrieved, your code decided whether a tool ran, your code applied the checks. This chapter hands one of those decisions to the model, repeatedly, and looks at what that changes. It is the arrangement everybody means by *agent*, and the gap between how it sounds and how it is actually deployed is the most useful thing in the chapter.

The problem. "Agent" sounds like it does the whole job by itself. That can't be how anyone ships this.

The question. What is an agent, really, and how much is it actually allowed to do?

One Change, and It Is Not a Small One

An **agent** is the arrangement you already know, given a loop: the model decides what to do next, something happens, the model sees the result, and it decides again — until the job is done or you stop it.

No new component has been added. None of Chapter 2's five parts is new here, and the model is the same model — the tools are the tools of Chapter 2. What is new is code you write: a loop, and the rules that end it. What *changed* is **who decides the order**.

	Chapter 2's arrangement	An agent
What happens first	Your code decides	Your code decides
What happens next	**Your code decides**	**The model decides**
How many times	Once, or a number you wrote	Until something stops it
What the sequence looks like	Fixed in your code, whatever it branches on	Decided as it goes

That second row is the whole chapter.

The Loop, Step by Step

Here is one run. The task is a support request: *"Where is my order, and if it is late, refund the delivery fee."*

```
TURN 1  model : I need the order. Call lookup_order(4471).
        code  : checks the call, runs it, returns the result
        result: shipped 2 Nov, promised 1 Nov, fee 4.99

TURN 2  model : It is late. Call refund(4471, fee).
        code  : checks it -- under the limit, so it runs
        result: refunded

TURN 3  model : Done. Reply to the customer.
        code  : no tool asked for, so the loop ends
```

Three turns. Each one is a complete exchange of the kind Chapter 1 showed: text out, text back. Two things are new. The result of each turn is added to the context — Chapter 2's word for everything you send — so the model decides turn 3 with turns 1 and 2 in front of it. And something has to decide when to stop, which is its own section below.

Each pass through that cycle is a **step**. The number of steps is not fixed, and that is the point: the model takes as many as it judges it needs.



Your code still decides whether each tool call runs, exactly as in Chapter 2. Handing over the loop does not hand over that. The model chooses what to *ask for*; your code is still the thing that says yes.

Where the Control Flow Went

Every program you have written so far decides its own order. The order might depend on data — an `if`, a `while` — but the *rules* for choosing are in the code, and you can read them.

In an agent, the rules for choosing are not in the code. They are in a model's judgement, produced fresh on each step, and Chapter 1 established that you cannot predict what a model will produce.

So **your program's control flow is now generated at run time by something that is not determined**. Three things follow:

You cannot read the program and know what it does. You can read what it is *allowed* to do — the tools you offered. The sequence it actually takes is not written anywhere until it happens.

Two identical runs can take different paths. Same request, same data, different number of steps and a different order. Chapter 13 explains where that variation comes from; here it is enough to know that the same request can take a different path.

A mistake on step 2 is still being acted on at step 9. This one deserves its own section.

Compounding Error

A single model call is usually right. A sequence of them is a different question, and the arithmetic is unkind. The name for it is **compounding error**, and it is the governing fact of this chapter.

Suppose each step is right nine times out of ten — which would be a good model on a well-chosen task. Then a run of several steps succeeds only if *every* step was right:

```
steps     chance the whole run is right
    1     0.900
    3     0.729
    5     0.590
    7     0.478
   10     0.349
   20     0.122
```

Worked once: three steps at nine-in-ten is 0.9 × 0.9 × 0.9 = 0.729. Three simplifications are buried in it. Two are worth knowing now: it treats the steps as independent, as though one going well told you nothing about the next, and it uses the same figure for every step, when in practice some are far riskier than others. The third is in the note below.

Seven steps is where nine-in-ten per step becomes worse than a coin toss. At twenty steps it is one run in eight.

Now the same table for a step that is right ninety-nine times in a hundred:

```
steps     chance the whole run is right
   10     0.904
   20     0.818
```

A per-step improvement that looks marginal — 0.9 to 0.99 — is the difference between a system that fails most long runs and one that mostly completes them. That is why so much of this book is about measuring per-step quality: at the scale of a single call the gap looks like a rounding error, and over a run it is the whole outcome.



The third simplification is that the table assumes a wrong step sinks the run. Real agents sometimes notice and recover — a failed search retried with better words, a tool error read and worked around — so the true numbers are better than the table. The *shape* is not: whatever the per-step figure is, multiplying it by itself enough times takes it to the floor.



Do not lean on that recovery. A wrong step does not simply fail: it produces a *result*, and that result goes into the next step as though it were true. A model working from one bad fact is not choosing badly — it is choosing well from the wrong starting point, which is much harder to spot. So recovery is rarer than it sounds, and real runs sit closer to the table than the note above might suggest.

Practitioners also report that a model which has committed to a plan tends to carry on rather than reconsider; that is a widely shared impression rather than a settled measurement, so weigh it accordingly.

This is also why "it worked when I tried it" is such weak evidence for an agent. One successful run out of a distribution that succeeds a third of the time is not surprising; it is expected, one time in three.

Stopping

A loop needs a reason to end. An agent has three, and you need all of them:

Ends when	Why you need it
The model says it is done	The normal case
A step limit is reached	The model may never say it is done
Something your code refuses	A loop heading somewhere it should not go

The **stopping condition** is the set of rules that end the run. The step limit is the one beginners leave out, and leaving it out is how you discover that a model asked the same question forty times while the bill ran.



Set the step limit at something you have a reason for, and log every run that hits it. A run that hits the limit is not a failure to be retried. It is a task the system did not know how to finish, and the record of what it tried is worth more than the answer would have been.

How Much Freedom Real Systems Give

Here is where the word *agent* and the practice part company.

One study of agents deployed in production — twenty case studies and a survey of eighty-six practitioners — found that **68% ran at most ten steps before a person intervened**. Most used ordinary models with instructions rather than anything trained for the job, most were judged by people looking at the output, and the difficulty the practitioners named most often was **reliability**.



That is one study, and one study is not a settled fact. Treat the shape as informative and the exact figures as indicative. The shape is what matters here: deployed agents are kept on short runs with a person close by, which is the opposite of the picture the word suggests.

The study says what teams do, not why. But the table above offers a reason that fits: ten steps at nine-in-ten is a run that works a third of the time. On those numbers, shortening the leash is not timidity — it is arithmetic.

Freedom Is a Dial, Not a Switch

"Is it an agent?" is the wrong question, because the interesting systems sit between the two ends.

How much the model decides	What it looks like
Nothing	Your code calls the model once, in a place you chose
Which of a few things to do next	Your code offers a short menu; the model picks; your code runs the step
How many steps, and in what order	The model runs the loop until it stops

Real systems mix these. A fixed path for the part you understand, a loop only for the part that genuinely varies. Chapter 34 names the arrangements in the middle of that table and says when each is worth it; the point here is that the dial exists.

And the principle from Chapter 3 applies with full force. Every part you add removes one failure and introduces another — and the loop is a part. It removes *having to know the sequence in advance*, which is a real and sometimes impossible requirement. It introduces a path you cannot predict, errors that compound, and a cost per task that is decided at run time rather than written down — a step limit caps it, but only by cutting the work off.

So: the least freedom that solves the problem. Not because autonomy is dangerous in the abstract, but because each extra step is multiplied into the figure at the bottom of the compounding table.

Approval Points

An **approval point** is a step the loop cannot pass without a person. It is how Chapter 3's fourth gap actually gets closed, and it is usually one line in the code that decides whether a tool call runs.

Chapter 2 gave the usual shape: a threshold, with the routine cases going through and the consequential ones reaching a person. In a loop there is a second axis, which is *what kind of action* rather than *how large*: a file can be read freely all day, and writing one needs a confirmation every time.

What makes this work is that the agent **stops and waits** rather than asking and continuing. A loop that proposes an action, is refused, and tries a slightly different route to the same action has not been controlled — it has been inconvenienced.



The approval point has to be in your code, not in the instructions. Telling the model *"always ask before refunding more than a set amount"* is a request, and Chapter 1 established that you cannot rely on what comes back. Checking the amount before you run the refund is a guarantee.

Lab: Watch a Loop

No code. You are looking for the loop from outside, and for where it is bounded.

Step 1. Use an AI feature that clearly does several things for one request — a coding assistant that edits files and runs commands, a research tool that searches and then summarises, an assistant that can use more than one app. Give it a task needing at least three distinct actions.

Expected result: visible intermediate steps. Most such products show them, because watching is the only way a user can tell it is still working.

Step 2. Count the steps. Then run a *similar but not identical* task and count again.

Expected result: different counts. If they are identical every time, you are probably looking at a fixed path rather than a loop — which is a finding, not a failure.

Step 3. Find the stopping condition. Give it something open-ended enough that it should run a long time, and watch for the limit — a message about running out of steps, a pause asking whether to continue, or a stop with the task unfinished.

Expected result: some ceiling, stated or unstated. If you cannot find one, note what you tried; "I could not reach it" is different from "there is none".

Step 4. Find an approval point. Ask for something consequential — and as in Chapter 3, do not let it complete. Watch whether it stops and waits, asks and carries on regardless, or begins without asking at all. Cancel as soon as you have seen which.

Expected result: one of those three. The second is the interesting one: a product that asks and proceeds regardless has an approval point in its instructions and not in its code.

Step 5. Make it fail early. Give it a task whose first step you know will go wrong — a file that does not exist, a search with no results. Watch what it does on the next step.

Expected result: either a recovery, or a plan carried on as though the failed step had worked. The second is compounding, visible.

Summary, Key Terms, and Review Questions

Summary

An **agent** is the same arrangement as before with one change: the model decides what happens next, repeatedly, instead of your code deciding. No new component is added; control flow moves, and you write a loop and the rules that end it. That means you cannot read the program and know its sequence, two runs can differ, and a mistake early is still being built on later. Compounding error is the governing fact: at nine-in-ten per step, seven steps is worse than a coin toss and twenty is one run in eight. A loop needs a **stopping condition** — the model saying it is done, a step limit, and your code refusing. Deployed systems are kept short and supervised: in one study, most agents ran ten steps or fewer before a person stepped in. Freedom is a dial, and the right setting is the least that solves the problem.

Key Terms

Term	Meaning
Agent	A system in which the model decides what to do next, repeatedly, rather than your code deciding the sequence
Step	One pass through the loop: the model decides, something runs, the result comes back
Compounding error	Two things at once across a run: each step's chance of being right multiplies against the others, so a high per-step figure becomes a low whole-run one; and a wrong step's result is carried into the next step as though it were true
Stopping condition	The set of rules that end a run — the model finishing, a step limit, or your code refusing
Approval point	A step the loop cannot pass without a person, enforced in your code rather than requested in the instructions

Worth Memorising

Item	Value or formula
Whole-run success over *n* steps	per-step reliability to the power *n*
Ten steps at nine-in-ten	0.9^10 = 0.349 — about a third of runs finish
Ten steps at ninety-nine in a hundred	0.99^10 = 0.904 — the same loop, now usable
What that means for a demonstration	A system succeeding a third of the time shows you a good run roughly every third try

Review Questions

1. What exactly is added to a system to turn it into an agent, and what is taken away?
2. Each step of your agent is right 90% of the time. Your task needs ten steps. Roughly how often does a whole run succeed, and what does that tell you about demonstrations?
3. Give two reasons a step limit is not optional.
4. A colleague says their agent is safe because the instructions tell it to ask before deleting anything. What is wrong with that, and what would you do instead?
5. Why can two identical requests to the same agent take different numbers of steps?

Review Answers

1. No new component — none of Chapter 2's five parts, and the same model. What is added is code: the loop itself and the rules that end it. What is taken away is your code's control of the sequence: the model decides what happens next, repeatedly, until a stopping condition is met.
2. About a third of the time — 0.9 to the tenth is 0.349. A demonstration that worked tells you almost nothing, because a system that succeeds a third of the time produces a successful run roughly every third attempt.
3. The model may never decide it is done, so the loop has no guaranteed exit without one. And each step costs money and time, so an unbounded loop is an unbounded bill on a task that may never finish.
4. The instruction is a request, not a guarantee — you cannot rely on what comes back, and nothing stops the model proposing the delete anyway. The approval point belongs in the code that decides whether the call runs, where refusing it is certain.
5. The model decides each step fresh, and two identical requests do not have to come back the same. The same request can produce a different plan, a different order, and a different number of steps.

> **You understand this when** you can explain why a system that is right nine times out of ten per step is a bad bet over twenty steps.

CHAPTER 5

Where the Work Actually Goes

Chapters 1 and 2 built a system, Chapter 3 argued about its parts, and Chapter 4 handed the model the loop. This chapter is about your time: which activities you will actually spend your days on, and why it is almost never the one you expect.

The problem. I assumed the hard part was writing the prompt. People who do this for a living say it isn't.

The question. If not the prompt, then what?

Why the Prompt Looks Like the Job

The instructions are the part of an AI system you can read straight through and change in a sentence.

You can see them. You can alter one word and watch the answer change. They look like the place where the behaviour is configured, and for the first hour of building something they genuinely are the thing you are working on. Most writing aimed at beginners shows you a prompt, because a prompt fits on a page.

Everything else is either invisible from outside or does not exist until you build it. So this chapter is about **four activities** rather than about Chapter 2's five parts — the two lists are not the same, and the activities cut across the parts:

The activity	What it is
Choosing what reaches the model	Deciding, per request, what goes into the context
Deciding what it may do	Fixing the boundary around the tools, before anything runs
Finding out whether it works	Building the thing that tells you, and then using it
Controlling what it costs	Noticing when a change makes the bill grow faster than the use

Writing the instructions is part of the first. It is not one of the four.

Choosing What Reaches the Model

Chapter 1 established that if you want the model to use information, it has to be in the request. Chapter 2 named the machinery that puts it there. This is the work of deciding **what to put in**, every time, for every request.

It is a continuous series of judgements, not a setup task:

- Which documents bear on this question, out of a collection too large to send?
- How much of each one — the whole thing, a section, a sentence?
- How many earlier turns of the conversation still matter?
- What gets dropped when it does not all fit?

Take the support request from Chapter 1. The answer needed one line: order 4471's shipping record. Choosing it meant passing over the rest of that order's history, the returns policy, the courier's tracking feed and the customer's three earlier messages — and deciding how much of whatever was kept to send. None of that is in the instructions. All of it is a judgement somebody made, for that one request.

Every one of those has a cost on both sides. Send too little and the answer is poorly informed. Send too much and you pay for it on every call, and the answer gets worse in ways Chapter 16 explains.

Deciding What It May Do

This is Chapter 2's tools and Chapter 3's **second** gap — the model cannot act — as a job rather than as a principle.

The work is not writing the tool. A tool is usually a function that already exists in your codebase with a description attached. The work is the boundary around it, and it is made of decisions that are cheap now and expensive later:

The decision	What it fixes
Which tools exist at all	What the model can actually use. It may still ask for one you never described, and your code has nothing to run
What each one may touch	Whether a mistake is an inconvenience or an incident
What must be checked before it runs	Which bad calls are refused by your code rather than by luck
Which cases stop and wait	Where a person is in the loop, from Chapter 4

In a running project this is not a one-off. Every new tool is a small design review: what can this reach, what happens if the model calls it with wrong arguments, and what does the worst plausible call do? A team that answers those three questions, and decides which cases should stop and wait, is doing this job. A team that adds tools because they are useful and finds out afterwards is doing it too, just later and at a worse price.



The cheapest version of this work is subtraction. A tool that only reads cannot change anything. A tool whose scope is one customer's records cannot touch another's. Much of the safety in a well-built system comes from what its tools were never allowed to do, not from checks catching them doing it.

Finding Out Whether It Works

Here is the activity that does not exist until you build it.

You can tell whether *this* answer looks reasonable by reading it. You cannot establish that a feature works by using it, and the reason is in the earlier chapters: Chapter 1 showed the same request can come back differently, and Chapter 4 showed that an arrangement which succeeds a third of the time still hands you a good first try about a third of the time. Using it can reveal a failure. It cannot establish that there isn't one.

So knowing whether the thing works means building something that measures it — a set of cases, a way to decide whether each answer passed, and a number you can compare with last week's. The field's word for that activity is **evaluation**, and Chapters 30 to 33 are where you learn to do it.

One part of evaluation has its own name and its own reputation for consuming time: **error analysis**, which means reading real outputs, sorting the failures into kinds, and working on the largest kind. Chapter 30 teaches the method.



Practitioners report that evaluation and error analysis together take a substantial part of building an application. That is a widely repeated account from people doing the work, not a measurement — nobody has timed it across a representative set of projects, and the share plainly differs between a prototype and a system with users. Take the direction seriously and hold the size loosely. Chapter 6 is about saying exactly this kind of thing out loud.

Controlling What It Costs

Chapter 1 gave the arithmetic and Chapter 2 said where the text lives. The ongoing work is noticing when a change makes the bill grow faster than the usage.

The things that do it are all easy to miss in a single call:

- An extra paragraph of standing instructions, paid for on every request.
- A reply that got longer because the instructions got vaguer.
- A retrieval step fetching ten passages where three would have done.
- A loop taking six steps instead of two.

Every one of those shows in a monthly total. None of them looks like a problem in the call you are looking at, and the total on its own does not tell you which of them did it — so the activity is really two: watching a number, and being able to attribute a change in it to a change you made. Chapter 53 is where that becomes a method.

Telling Which Part Failed

Chapter 2 promised this chapter would say how much of the real work telling failures apart turns out to be. There is no number for it. What there is: it is an activity of its own, nobody gets to skip it, and it is part of why reading outputs is slow.

A wrong answer is one symptom with five common causes. Retrieval may have found nothing useful. The context may have been assembled badly, or been too full. The model may have ignored material that was right there. A check may have passed something it should have caught. A case that needed a person may never have reached one.

Nothing in the output tells you which. The answer looks the same either way, and the natural response — change the instructions and try again — is the move that tells you least about which.

Separating them is its own activity, and it is why two things later in this book exist: recording what actually happened on each request, so a failure can be traced rather than argued about (Chapter 33), and measuring retrieval and generation separately, so you can say which half failed (Chapter 32). Both are work you have to do deliberately. Neither happens because the system is well written.

Why Finding Out Takes So Long

Two reasons, and they are different in kind.

It is the activity that decides the other three. Which of two retrieval arrangements is better? Is the tool boundary too tight, blocking useful work? Can the instructions be shortened without making the answers worse? Each of those is a measurement question, and without a measurement you are changing things and hoping — and because the system is not repeatable, trying it twice does not settle it either. Chapter 20 is where you learn how many times is enough.

Not every question is like this. The tool boundary has a part that measurement cannot decide: some things are refused because of what they would cost if they went wrong, not because of how often they do. Chapter 3 made that argument. But the questions that *are* empirical are most of the day-to-day ones.

And the failures have to be hunted. Ordinary software has a loud failure class: something crashes, and the crash announces itself with a stack trace and a timestamp. A model-based system has that class too — a crash, a timeout, output your code cannot read. What it has in addition is a failure that announces nothing: a fluent, confident, wrong answer. The system does not know it is wrong, because knowing would need the very check you are trying to build.

Some of those failures do reach you. A check you wrote catches the ones you anticipated. Users report the ones that affect them, slowly and unevenly. What is missing is the class nobody anticipated and nobody complained about. That class reaches you only if somebody goes looking for it: reading output nobody has complained about, or measuring over cases nobody has reported on.



When you join a team building on models, the most useful question is "how do you know it is working?" The quality of the answer tells you more than any description of the architecture. "Our users would tell us" is an answer — it describes a real channel — but it is the slowest one available, and it only ever returns the failures somebody noticed and minded enough to mention.

What This Does to a Project

The shape of the work is the opposite of what the first week suggests.

	The first week	The months after
What you do	Write instructions, get something working	Read outputs, find what is wrong, fix the largest thing
What feels productive	Everything. It works almost immediately	Slow. Much of it produces no visible change
What you learn	That the idea is possible	Whether it is reliable

A working demonstration arriving quickly is real, and it is a large part of why these projects get approved. The trouble is what it suggests about the rest. It does not mean the work is nearly done, because making something work once and establishing that it works are not the same activity, and the second has barely started when the first one ends.



The gap between *a demonstration exists* and *this can be relied on* is where the measuring happens, and a plan that has allowed time for the demonstration has allowed none for it. That is the practical thing to carry out of this chapter: when you estimate work on a system built around a model, put the measuring in the estimate, because it is the part that will surprise you.

None of this makes the instructions unimportant — Chapter 18 is a whole chapter on writing them properly, and a badly specified request wastes everything downstream of it. The claim is narrower: they are the part you will finish first and think about least.

Lab: Sort an Account of Real Work

No code, nothing to install, fifteen minutes. The account below is **invented**, which matters and is said again in step 3: because it was written for this exercise, it cannot tell you anything about where real engineering time goes. What it can do is show you whether you can tell the four activities apart in somebody's prose, which is a skill you need before any real account is worth reading. It has a key, so you can be wrong and find out.

Step 1. Before reading it, write one line: of the four activities, which do you expect an account like this to spend the most paragraphs on?

Expected result: a written prediction. Writing it first is the whole point — you cannot quietly adjust it afterwards.

The account

1. *We had three weeks before the demo and two engineers, one of whom was new.*

2. *The feature was a support assistant. A customer asks about an order and it answers from our own records rather than from anything the model was trained on.*

3. *The first version pasted the customer's entire order history into every request. It worked. It also cost about nine cents a question, and the projected monthly figure was the first thing anyone outside the team asked about.*

4. *So we cut it down to the three most recent orders plus the returns policy, which meant deciding what gets dropped when somebody has forty orders and only three fit.*

5. *We spent two days on the wording of the instruction that tells it to refuse anything it cannot answer from the records.*

6. *We gave it one tool, a lookup by order number. We spent longer agreeing that it must not issue refunds than we spent writing the lookup itself.*

7. *In week two the platform team moved us onto the new queue, which cost us a day and was nobody's fault.*

8. *We had no way of telling whether any of it was working, so we wrote down forty real customer questions with the answer each one should have, and started scoring them by hand.*

9. *Reading forty scored answers in one sitting, we noticed the bad ones were mostly a single kind of mistake: when a customer mentioned two orders, it answered about the wrong one.*

10. *We added a sentence to the instruction about which order to prefer, and the score went from 31 of 40 to 34 of 40.*

11. *The demo went well. We shipped it to a hundred users the week after.*

12. *Three weeks later a customer was shown the correct details of an order that was not theirs. The lookup took an order number and never checked whose order it was.*

Step 2. Label each of the twelve paragraphs with one of six things: the four activities, *the instructions*, or **none of these**. *The instructions* is on the list but is still not one of the four — it is part of the first, as this chapter said at the start. It is separated out here because telling the two apart in somebody's prose is the whole exercise.

Expected result: the key is below. Six labels, twelve paragraphs.

Label	Paragraphs	Count
None of these	1, 2, 7, 11	4
The instructions	5, 10	2
Deciding what it may do	6, 12	2
Finding out whether it works	8, 9	2
Choosing what reaches the model	4	1
Controlling what it costs	3	1

Three of those are worth arguing with, so argue with them:

- **Paragraph 3** is cost, not *choosing what reaches the model*. Pasting the whole history is a choice about what reaches the model, but what the paragraph is *about* is the bill and who asked about it. Paragraph 4 is where that choice gets made.
- **Paragraph 10** is *the instructions*, even though it ends in a number. What they did was change some wording; the score is how they checked it. Keep the change and the check apart — conflating them is how teams come to believe a reworded prompt fixed something.
- **Paragraph 12** is *deciding what it may do*. The failure happened in production, but it was decided in paragraph 6, when a tool was given an order number and no notion of whose order it was. That is the point of the Tip in *Deciding What It May Do*: "a tool whose scope is one customer's records cannot touch another's".

Step 3. Compare the counts with your prediction from step 1.

Expected result: **none of these** is the largest label, at four — team, product description, platform work and shipping are all real paragraphs and none of the four activities. Among the four activities, *finding out whether it works* and *deciding what it may do* tie at two, while *choosing what reaches the model* and *controlling what it costs* have one each. *The instructions* also has two, and is not one of the four.

Now notice what that distribution is worth: **nothing.** I wrote these paragraphs, so I chose those counts, and I could have made any activity win — including the one this chapter argues for, which is sitting at joint first. If your prediction looks confirmed here, it is a coincidence I manufactured. I wrote the ending too, so the account cannot tell you how often a project goes this way either. The share claim earlier in this chapter rests on what practitioners report; one invented account is evidence neither for it nor against it.

Step 4. Find the sentence where the account says how it knew something had improved. Write down what it would take to believe it.

Expected result: paragraph 10 — "the score went from 31 of 40 to 34 of 40". That is a measurement, taken on a written-down set of cases, and it is more than most accounts offer. It

is also three answers out of forty, from one run, with no indication of how much the score moves when nothing changes. Chapter 20 is where you learn whether a difference like that is real. Noticing that the question is open is this chapter's job; answering it is not.

Step 5. Paragraph 12 describes a failure reaching a customer. Work through the five causes this chapter listed and say which one it was.

Expected result: **none of the five** — and the first reason is that the answer was not even wrong. The details were correct; they went to the wrong person. The chapter's five are causes of a *wrong answer*, so none of them is in the running. Nor would they be if the answer had been wrong: retrieval found what it was asked for, the context was assembled correctly, the model used what it was given, no check was wrong about what it checked (the authorisation check never existed, so nothing judged badly), and no person was needed. The tool did exactly what it was built to do, for the wrong person. That is why this chapter says *five common causes* rather than five causes — a tool that acts with the service's rights instead of the caller's is a sixth, and Chapter 51 owns it. A list of causes you trust too far is its own failure mode.

Step 6, optional, and the only step that can tell you anything about the world. Find two real written accounts of somebody building an AI feature — an engineering post, a conference talk, a retrospective. Not a tutorial and not an announcement. Label their paragraphs the same way, and compare the two with each other. There is no key. If they disagree with each other, that is the honest finding and two accounts is too small a sample. Whatever you get, remember you counted words rather than hours: a word count says what somebody chose to write about.

Summary, Key Terms, and Review Questions

Summary

The instructions are the visible part of the job and the part you finish first. The work is four activities: choosing what reaches the model, deciding what it may do, finding out whether it works, and controlling what it costs — plus the thing that cuts across all of them, telling which part failed when an answer is wrong. **Evaluation** is the name for finding out whether it works, and **error analysis** is the part of it that means reading real outputs and sorting failures into kinds; practitioners report that the two together take a substantial part of building an application, which is an account from people doing the work rather than a measurement. Finding out takes long for two reasons: it decides most of the other questions, and its failures do not announce themselves the way a crash does, so the ones nobody anticipated reach you only if somebody reads output nobody complained about. The practical consequence is an estimate: put the measuring in the plan.

Key Terms

Term	Meaning
Evaluation	Building and using something that tells you whether a system works: a set of cases, a way to judge each answer, and a number you can compare over time
Error analysis	The part of evaluation that means reading real outputs, sorting the failures into kinds, and working on the largest kind

Review Questions

1. Why do beginners consistently overestimate how much of the work is writing instructions?
2. Your colleague says the feature is working because they have tried it several times and it was good. Give two reasons that is weak evidence.
3. Give one question about a system that measurement answers, and one that it cannot, and say what settles the second.
4. A user reports a wrong answer. Name three different parts that could have caused it, and say what you would need in order to tell them apart.
5. A project plan allows two weeks for "build the AI feature" and nothing else. What is missing?

Review Answers

1. Because the instructions are the only part you can read straight through and change in a sentence, and the only part that fits on a page of an article. The four activities are either invisible from outside or do not exist until somebody builds them.
2. The same request can come back differently, so a handful of good answers is a small sample of a distribution they have not measured. And by Chapter 4's ten-step arithmetic, an arrangement that succeeds a third of the time still hands you a good first try about a third of the time, so "I tried it and it worked" is also what an unreliable system looks like from close up.
3. Measurement answers which of two retrieval arrangements is better — run both over the same cases and compare. It cannot settle whether a tool should be allowed to issue refunds unsupervised, because that turns on what a rare failure would cost rather than on how often it happens; that is a judgement about consequences, and above the legal floor it is a design decision.
4. Any three of: retrieval found nothing useful; the context was assembled badly or was too full; the model ignored material it was given; a check passed something it should have caught; a case needing a person never reached one. To tell them apart you need a record of what actually happened on that request — what was retrieved, what was sent, what came back — and retrieval and generation measured separately.

5. Everything that decides whether it can be relied on: a set of cases, a way to judge answers, the reading of real outputs, and the cycle of fixing the largest failure and measuring again. The two weeks are for making it work once, which is a different activity from establishing that it works.

> **You understand this when** you can name the four activities, say which one decides most of the others, and explain why nothing in a wrong answer tells you which part produced it.

CHAPTER 6

What Changes Fast, and What Changes Slowly

Chapters 1 to 5 built a system and said where the work goes. This chapter is about something else entirely: how to read everything else you are going to read.

The problem. Half of what I read is already out of date and I can't tell which half.

The question. Is there a way to sort a claim as I read it, rather than finding out later which parts were worth learning?

Three Kinds of Claim

Almost everything written about this field is one of three things. They look identical on the page and they keep company in the same paragraph, which is why the mix is hard to see.

Kind	What it is	How long it is good for
Lasting principle	A reason, or a shape that does not move: why you would do the thing at all, or what the parts are and how they fit	Years and years. It outlives the thing it was first said about
Current convention	How competent people work at the moment. True, load-bearing and replaceable	Months to a few years
Dated detail	A fact with a date on it. A name, a number, a version	Months, sometimes less

Three examples of the same subject, searching your own documents, one of each kind:

- *Lasting principle.* If you want an answer grounded in your own material, that material has to reach the model, and you have to be able to say afterwards which part of it the answer came from.

- *Current convention.* Combine keyword search with meaning-based search and then re-order the results, rather than using meaning-based search alone. Chapters 26 and 28 are where that gets built.
- *Dated detail.* The ceiling on how much text the model you are using will accept, which Chapter 2 said every model has.

All three are worth knowing. They are worth knowing differently.

The Test: What Would Have to Change?

Ask, of any claim: **what would have to happen in the world for this to stop being true?**

If the answer is	The claim is a	So
A company ships something, or changes a price	**Dated detail**	Do not memorise it. Know that it exists and where to look
The field works out a better way of doing this	**Current convention**	Learn it properly, and hold it loosely
The underlying reason turns out to be wrong	**Lasting principle**	Learn it so you can derive the rest

Worked on three claims:

1. *"Cost scales with the amount of text that moves."* What would have to change? Serving the model would have to stop getting more expensive as the text gets longer — a fact about the work being done, not about anybody's price list. **Lasting principle**, and it is the form Chapter 1 settled on: *cost scales with the text you move.* Note what is *not* the principle: that providers bill you per unit of text is how they happen to charge today, and you can run a model on hardware you own and be billed nothing per unit at all. **How you pay is the convention; that longer text costs more is the principle.** This pair is the whole chapter in one example, so it is worth re-reading.
2. *"Prefer a fixed sequence of steps to a free-running loop unless you have measured that the loop pays."* What would have to change? Loops would have to get reliable enough that the arithmetic in Chapter 4 stops biting. That is possible and it has not happened. **Current convention** — and one whose reason you can derive yourself, from Chapter 4's arithmetic, rather than having to take on trust.
3. *"This model accepts two hundred thousand units of text."* A product decision, announced on a Tuesday. **Dated detail.**



The test is quick enough to run while reading. You do not need to be right every time. You need to stop treating all three as the same kind of thing, because that is what makes a field feel like it is moving too fast to learn — you are trying to memorise the dated details and derive the principles, which is exactly backwards.

The Hard Case: A Convention That Looks Like a Principle

The honest difficulty is the middle row. A convention that has been true for a few years, that everyone around you follows, and that has a good reason attached, is almost indistinguishable from a principle.

Two signs that you are looking at a convention:

- **It names a technique rather than a reason.** "Re-order the search results" — the step named above — is a technique. "The thing that retrieves cheaply is not the thing that judges best" is the reason underneath it, and the reason is what survives the technique.
- **It has a predecessor you can remember.** If people did something else five years ago for the same reason, the reason is the principle and the something is the convention.

When you cannot tell, write down the reason you think is underneath. If you can state it and it is not the technique restated, you have found a principle. If you cannot, you have a convention and you should treat it as one.

How Strong Is the Evidence?

Sorting a claim by how long it will last is one axis. The other is how much you should believe it now.

The rungs below rise in **how much they tell you about settled practice** — not uniformly in how well they show that a thing works. Keep those apart, because the table's last column mixes them on purpose and that is where it is easy to go wrong.

Strength	What it is	What it supports
Announcement	Somebody says a thing exists, or works	That the claim was made
Demonstration	It worked, on an example chosen by the person showing you	That it can work once
One study	Measured, in one setting, with one set of choices	That it worked there
Practitioner report	A survey, or a set of accounts: what people say they do	What people do. Only weakly whether it works
Settled practice	Many independent teams, over years, with the failure modes written down	Relying on it, and explaining it to someone else

There is one more kind of evidence, and it is **not on that ladder at all**:

Outside the ladder	What it is	What it supports
A derivation	It follows from arithmetic, or from a mechanism you can check yourself	Whatever the arithmetic covers — and it does not date

Chapter 4's multiplication is one: nobody had to survey anybody to establish that ten steps at nine-in-ten succeed about a third of the time. It is off the ladder because the ladder measures how much somebody else's reporting can be trusted, and a derivation needs nobody's reporting. When a claim can be derived, derive it.

Three things this table is for.

First, a vendor's own material is an announcement until you can see what it was measured on. It may contain numbers, and the numbers may be real. But it was written by the party that benefits, measured on a problem it chose, and the comparison it draws is the one that flattered it. A score on a named public test that you could re-run yourself is more than an announcement. A score on a test only they can run is not, however precise it looks. This is not an accusation of dishonesty; it is what the genre is.

Second, evidence that something works and evidence that people use it are different claims, and neither implies the other on its own. A practice can be widespread and not work — much of our received wisdom is exactly that. A practice can be well supported and barely adopted. So when somebody tells you "everyone is doing X", you have learned about adoption. That becomes evidence about whether X is a good idea only when the practice has survived a few years and its failure modes are written down somewhere you can read them — which is what separates the bottom of this table from the top.

Third, a survey of practitioners is a practitioner report even when it is published as a study. Chapter 4's twenty case studies and eighty-six practitioners is the example: it is careful work, and what it establishes is what teams *do*, which is why Chapter 4 says the study tells you the shape rather than the reason. Published form does not set the rung; what was

measured does.



Chapter 5's claim about **how much** of the work is evaluation and error analysis is a **practitioner report** — an account from people doing the job, not a measurement. That is the strongest evidence available for that particular claim, which is why the chapter says so rather than rounding it up. Chapter 5's argument for *what* the four activities are does not rest on that report; only the share does. Where the evidence is thin, saying so is part of the claim.

When Two Rungs Are Close

The three-way sort has a hard case and so does this. Take one claim down two candidate rungs: *"Reordering your search results improves answers."*

- **Is it a demonstration or a study?** If somebody shows you two answers, before and after, on a question they picked — demonstration. If they ran a fixed set of questions they wrote down in advance and reported how many improved — a study. The question that separates them is *who chose the examples, and when.*
- **Is it a study or settled practice?** One team measuring once is a study. What *would* make it settled is many teams measuring it, the result holding, and the cases where it *does not* help becoming known and written down. **The written-down failure modes are the tell** — settled practice comes with them, and a single study rarely does. On this claim we are not there: re-ordering results is a current convention, and whether it helps on a given set of documents is still something you measure. Chapters 26 and 28 are where you measure it.

Notice what just happened: the same claim appeared on both axes. It is a **current convention** on the first axis and somewhere around **one study** on the second, and those are two different questions about it — how long it will last, and how much to believe it now. A claim always has a place on both, and the places need not agree.

Both boundaries turn on the same thing: not how confident the claim sounds, but how much the claimant could have hidden without lying.

Saying It Out Loud

Knowing the strength of a claim is only half of it. The other half is being able to say it in one short sentence, without sounding like you are attacking anybody.

What you mean	A sentence that says it
One study, one setting	"That's one study, and I don't know how much it transfers."
A survey, not a measurement	"That's what people report doing, which isn't the same as it working."
A vendor's number	"That's their own test. Do we know what it was compared with?"
Settled, rely on it	"That one's well established — I'd start from it."
It can be derived	"We can work that one out ourselves — we don't have to take their word for it."
I would need to check	"I'd want to measure that on our own data before building on it."

That last one is a promise, so it is worth knowing what keeping it involves. Chapter 20 is how you tell a real difference from noise, and Chapters 30 to 33 are how you build the measurement in the first place. Saying "I'd want to measure that" without knowing either is how a reasonable instinct turns into a stalled project.

These sentences do real work. They end the argument where the argument should end — at what is known — instead of at whoever is most confident. And the habit protects you from the specific failure this field produces in large quantities: repeating a plausible claim confidently because you read it three times, in three articles, all of which were drawing on the same announcement.



The common failure is not believing something false. It is **losing track of where a belief came from.** You read a claim, it becomes familiar, and six weeks later you are asserting it as something you know. Nothing in your memory marks it as a thing you read once in a blog post. That is why the strength belongs in the sentence you say out loud: saying it is how you keep the label attached.

How to Read a Release Announcement

An announcement is a genre with a fairly fixed shape, and once you know the shape you can read one in a minute and extract the two or three facts in it.

What it usually contains, roughly in this order: a claim of improvement, a number, the name of the test it was measured on, a comparison, a list of capabilities, and a price. Any of them can be missing — a model you can download and run yourself often has no price at all, and plenty of announcements name no public test, which is itself the most informative thing on the page.

Four questions get you what is actually there:

1. **What is the claim, precisely?** "Better at reasoning" is not a claim. "Scores *n* on *this named test*" is.

2. **Measured on what?** A named public test tells you something. *Their* internal evaluation tells you almost nothing, because you cannot see it and neither can anybody else.
3. **Compared with what?** Against their own previous model is a weaker claim than against the best competitor, and the choice of comparison is the most informative thing on the page.
4. **What changed about the price and the limits?** This is usually the part that affects your work, and it is usually the part written in the smallest type.

Then sort what you found. The capability claim is a **dated detail** — it will be true of a different model in six months. The price is a **dated detail**. If the announcement describes a change in how you are expected to *work* — a new way of giving the model tools, say — that is a candidate **current convention**, and it stays a candidate until you see evidence that engineers outside the company are working that way.



The useful question after reading one is not "is this better?" It is "does this change anything I am building?" Most of the time the honest answer is no, and the announcement was interesting rather than relevant. Interesting and relevant are allowed to be different.

What to Memorise, and What to Look Up

The distinction follows from the sort, with one addition. You memorise what lasts — the reasons, the shapes, and the failure modes that come with settled practice — and what you can derive, because a derivation you have to rebuild from scratch every time is not much use to you. You look up everything with a date on it.

Memorise	Look up
The reasons. Why ground an answer, why limit a tool, why measure before believing	Prices, text ceilings, model names, rate limits
The shapes. What the parts of a system are and how they fit	Exact parameter names and interface details
The arithmetic you cannot look up quickly. What multiplying per-step reliability does over ten steps	Scores on particular named tests
The failure modes, and which part causes which	Which library version fixed which bug

The reason to put it this way round is not tidiness. **A principle lets you derive the detail or at least recognise a wrong one, and a list of current prices and limits will never give you back the principle.** Somebody who has memorised current prices and text ceilings and cannot say why retrieval exists knows strictly less than somebody who can derive what retrieval is for and has to look the prices up, and the gap between them grows every month.

This also tells you what to do when something you learned goes out of date. If it was a dated detail, replace the number and carry on. If it was a convention, learn the new one

and ask what reason it serves — usually the same reason, served better. If a *principle* goes, that is a real event, and it is rare enough that it is worth noticing properly rather than quietly updating.

Lab: Sort Nine Claims

No code, no account, nothing to install. Fifteen minutes, and it has a right answer, which is the point: you can check yourself before trying this on real writing.

Below is a made-up announcement. It is invented, so nothing in it dates, and it is built to contain every case in this chapter.

```
Introducing Northwind-3
-----------------------
Northwind-3 is our strongest model yet. It scores 71.4 on
CodeTask-V, up from 58.2 for Northwind-2, and it
accepts 400,000 units of text in a single request.

Our internal evaluation shows a 30% reduction in tool-calling
errors. Teams building agents tell us they now leave runs
unsupervised for longer.

Because quality depends on what the model is given, we
recommend retrieving supporting passages rather than pasting
whole documents, and keeping any single tool narrow in what
it can reach.

Northwind-3 is available today at $2.50 per million units in
and $10.00 per million out.
```

Step 1. Mark each of these nine claims **P** (lasting principle), **C** (current convention) or **D** (dated detail).

1. Northwind-3 scores 71.4 on CodeTask-V.
2. It accepts 400,000 units of text per request.
3. It costs $2.50 per million units in.
4. Quality depends on what the model is given.
5. Retrieve supporting passages rather than pasting whole documents.
6. Keep any single tool narrow in what it can reach.
7. Northwind-3 is stronger than Northwind-2.
8. Teams leave agent runs unsupervised for longer.
9. Tool-calling errors fell by 30%.

Expected result: **D** for 1, 2, 3, 7 and 9; **P** for 4 and 6; **C** for 5; and 8 is the interesting one — it is a **C** reported as a trend, and it is the only claim here whose kind depends on whether you believe it. Six of the nine are dated details, which is normal: that is what an announcement is for.

Step 2. For 4 and 6, write the reason in one sentence without naming a technique.

Expected result: for 4, something like *the model answers from what is in front of it, so what you put in front of it decides the answer* — Chapter 1. For 6, something like *a tool that cannot reach a thing cannot damage it* — Chapter 3. If your sentence names a technique instead of giving a reason, you have written a **C**.

Step 3. Now label the evidence offered for claims 7, 8 and 9: announcement, demonstration, one study, practitioner report, settled practice, a derivation, or *none*.

Expected result: 7 is an **announcement** — the two scores are theirs. 8 is a **practitioner report**, and a weak one: it is the vendor relaying what teams tell them, with no number and no count. 9 is an **announcement** too, and this is the trap — it looks like a study because it has a percentage, but "our internal evaluation" is a test nobody else can run.

Step 4. Answer the four questions from the section above about claim 9, the 30%.

Expected result: the claim is a reduction in tool-calling errors, which is precise enough. It was measured on their own evaluation, which you cannot see. It was compared with Northwind-2 — their own previous model, not a competitor, which is the weaker of the two comparisons. And the price went up or down by an amount you cannot tell, because no previous price is given. That last gap is the one worth noticing: the announcement states a price without stating the change.

Step 5. Write the one sentence you would say in a meeting about claim 9.

Expected result: one sentence, under twenty words, naming the evidence without dismissing the claim — for example *"That's their own internal evaluation against their own last model, so I'd want to try it on our cases."*

Step 6, optional. Now do steps 1 and 3 on a real piece of writing you find yourself. There is no key for this one, and that is the exercise: the counts you get are yours, and the only wrong answer is not noticing which kind you were reading.

Summary, Key Terms, and Review Questions

Summary

Almost every claim in this field is a **lasting principle** — a reason, or a shape that does not move — a **current convention**, or a **dated detail**, and you tell them apart by asking what would have to change in the world for the claim to stop being true: a company shipping something or changing a price, the field finding a better way, or the reason itself being

wrong. The hard case is a convention that has been true long enough to look like a principle; the tell is that it names a technique and has a predecessor you can remember. Separately, ask how strong the evidence is — announcement, demonstration, one study, practitioner report, settled practice — or a derivation you can check yourself, which stands outside that ladder because it needs nobody's reporting. Remember that a vendor's own material is an announcement until you can see what it was measured on, that a survey of practitioners is a practitioner report however it was published, and that evidence a thing works and evidence people use it are different claims. Read an announcement for four things: the precise claim, what it was measured on, what it was compared with, and what changed about the price and the limits. Say the strength out loud when you repeat a claim, because the real failure is not believing something false but losing track of where a belief came from. Memorise the reasons, the shapes, the arithmetic and the failure modes; look up everything with a date on it.

Key Terms

Term	Meaning
Lasting principle	A reason for doing something, or a shape that does not move, which outlives the technique it was first attached to. It stops being true only if the reason itself is wrong
Current convention	How competent engineers work at the moment: true, load-bearing and replaceable. It usually names a technique rather than a reason, and usually has a predecessor you can remember
Dated detail	A fact with a date on it — a name, a number, a version. It stops being true when somebody ships something or changes a price
Practitioner report	A survey or a set of accounts of what people say they do. It is a practitioner report however it was published, and it supports what people *do* far better than whether it works
Evidence strength	How much a claim is supported: announcement, demonstration, one study, practitioner report, settled practice — or a derivation, which stands outside the ladder because you can check it yourself

Review Questions

1. Give the one question that sorts a claim into the three kinds, and say what each of the three answers looks like.
2. A colleague says "everyone re-orders their search results now." What have you learned, and what have you not?
3. A provider's announcement says their new model is 30% better at coding. List the things you would want to know before repeating that.

4. Why is "losing track of where a belief came from" a worse problem than believing something false?
5. You learn that a convention you relied on has been replaced. What is the first question to ask about the replacement?

Review Answers

1. *What would have to happen in the world for this to stop being true?* If the answer is that a company ships something or changes a price, it is a dated detail. If it is that the field works out a better way of doing this, it is a current convention. If it is that the underlying reason turns out to be wrong, it is a lasting principle.
2. You have learned something about adoption: the practice is widespread. You have learned nothing yet about whether it would help on your data, because adoption and evidence are separate claims and neither implies the other on its own. Adoption starts to count as evidence once a practice has survived a few years and its failure modes are written down. You also do not know whether "everyone" is a measurement or an impression.
3. What test the 30% was measured on, and whether anybody outside the company can run it; what it was compared against, which is the most informative choice on the page; and what changed about the price and the limits, which is the part most likely to affect your work. The capability claim itself is a dated detail and will attach to a different model within months.
4. Because a false belief can be corrected by meeting the evidence, while a belief with no provenance is rarely put to the test at all: there is nothing in your memory marking it as something you read once, so you assert it with the confidence of something you checked. It also spreads, because you sound sure. Saying the evidence strength out loud is what keeps the label attached.
5. What reason does the new one serve? Usually it is the same reason, served better — which tells you the principle underneath was right and only the technique moved. If it serves a genuinely different reason, something larger has changed and it is worth the time to work out what.

> **You understand this when** you can take a claim you already believe, say which of the three kinds it is, name the evidence behind it, and notice if you cannot remember where you got it.

CHAPTER 7

A Map of This Book

Six chapters in, you have the shape of a system and a way to read claims about it. This chapter is the map: what the rest of the book contains, in what order, and what it leaves out on purpose.

The problem. There's clearly a lot here. I don't know what order to learn it in, or what I can safely leave until later.

The question. What is the shape of the whole thing, and which parts can wait?

The Four Parts

Part	Chapters	The question it answers	What to ignore while you are in it
I · Understand	1–17	What is this thing, and what is it made of?	Every setting, price and product name. This part is what the thing is, not how you drive it
II · Use	18–39	How do I make each part work, and how do I know it does?	Serving your own models, and the bill
III · Apply	40–50	Can I build the whole thing and measure it?	Scale and deployment. Everything runs on one machine
IV · What Next	51–57	What can go wrong, what makes it slow, what breaks, what it costs — and what I say out loud	Nothing. Chapter 57 is where the book says what to skip

Part IV's first four chapters are *four threads* asked of the whole system: what can go wrong and what limits the damage (51), what makes it slow (52), what it costs (53), and how it breaks and how you diagnose that (54). Chapter 55 then designs a system out loud, and 56 and 57 say what to learn next and what to leave.

The order is not arbitrary. Part II is unreadable without Part I's vocabulary; Part III is a waste of time before Part II, because a lab you cannot measure teaches you nothing; and Part IV

asks questions — *what would you do about this failure?* — that only make sense once you have built something.

Why Part I Has Two Halves

Part I is in two halves, and both of them are *Understand*.

- **Mental Model**, chapters 1–7, is the half you have just read. It is the system *around* the model: what the model is as a component, what the other parts are for, why they are assembled that way, what an agent loop actually is, where the work goes, and how to read what you read. While you are in it, ignore what is *inside* the model — the second half opens it.
- **Inside the Model**, chapters 8–17, is the model itself: what it is made of, how text becomes numbers, how attention works with the arithmetic done by hand, what a stack of layers accumulates, how scores become a word, how it learned to do this, what it costs from the inside, and why it fails in the particular ways it does. While you are in it, ignore how the model was trained beyond the shape of that last loop.

Neither is a part of its own, because they answer the same question. The first says what the model is *for*; the second says what it *does* when it answers. You can read Part II after the first half alone, and you will understand every instruction in it. You will not understand *why* any of them work until you have read the second.



The second half uses real notation and does the arithmetic. It does *not* teach machine-learning or deep-learning theory — no loss surfaces, no optimisers, no proofs. Chapter 8 gives you exactly the mathematics the book uses and nothing more, and Chapter 17 marks the line where the theory you are skipping begins, so you know what you have skipped rather than not knowing it is there.

The One System the Labs Build

Part III is eleven chapters that are each a single lab, and they are not eleven separate exercises. They build **one small system**, each lab starting where the last one stopped.

Labs	What gets built
40–42	A model call you can measure, an evaluation set with two prompts compared properly, and a fixed output shape your code can check, with one tool
43–45	Two ways of turning text into numbers, tested against each other; retrieval with a re-ordering step, measured; then the pipeline joined end to end
46–48	Failure modes named and a judge you can trust, a fixed path against a loop, and context and tools measured
49–50	Cost and latency measured and reduced, then an attack on the thing you just built

Two design decisions are worth knowing before you get there.

The book ships the data. About forty short public-domain documents, eight queries with a hand-made judgement of how well each document answers each one, and one deliberately awkward PDF with a table and a scanned page. Every lab runs on those. No lab says "use your own documents", for the reason Chapter 6 would give you: an expected result that depends on the reader's files holds for nobody, so you could never tell whether your run was right.

Every step has an expected result. Not exact output — Chapter 1 established that the same request can come back differently — but the shape the output takes, or which assertion must pass, or a range your number should fall in. Where a real run is shown, the book says plainly that your numbers will differ.



Labs are easy to skip, and reading about measuring something is not the same state of knowledge as having measured it. If you are short of time, do fewer labs properly rather than all of them quickly.

Which Chapters Carry the Most Weight

This book is written for a particular situation: the one where you have to explain this material out loud, to somebody who will notice a wrong answer. That is why some chapters matter more than others, and it is worth saying plainly before the table below.

And here I have to be careful, because Chapter 6 just taught you to ask where a claim comes from, and this one is a **judgement**, not a measurement. Nobody has surveyed what questions get asked, and if they had, the answer would vary by company, role and year. What follows is reasoning about which chapters the rest of the book leans on hardest — treat it as a suggestion about reading order and not as a prediction.

Weight	Chapters	Why
Heaviest	1–7, 18, 20, 30–33	Almost everything else refers back to them. The mental model, the prompt as a specification, and the ability to say how you would know something works — including whether a difference is real
Heavy	23, 26, 28, 34, 51, 53, 54	Tool calls, retrieval and its assembly, when a loop is warranted, security, cost, diagnosing a failure. The questions with a wrong answer somebody will notice
Depends on the role	8–17, 19, 21–22, 29, 36–39, 52	The model's internals, reliability engineering, output shapes, scale, tool design and agents, adaptation, speed. Which of these matters depends entirely on what the work is
Last, and worth it	55	One whole system designed out loud, which is the only chapter that rehearses saying the rest

One related sentence is worth adding, because it answers a different question from the table: **chapters 18 to 29 are the half of Part II a conversation reaches first** — calling a model, and retrieval. They are where the questions start, because they are the part of the system somebody can ask you to describe without first agreeing what the system is.

Appendix B is a hundred questions with answers of 150 to 180 words each, which is about what sixty seconds of speech holds. That length is the point: it is **interview depth** — the mechanism, the trade-off, and how you would check it — and it is the depth this whole book aims at. Not exhaustive, and not a summary either.

What This Book Leaves Out, On Purpose

A book that admits what it skipped is more useful than one that implies it covered everything.

Left out	Why, and where it is
Machine-learning and deep-learning theory	It is a different subject and a longer book. Chapter 17 marks where it starts; Chapter 56 names the standard books
Training a model from scratch	It is outside what this book is for — building applications on models somebody else trained. Chapter 14 gives the shape of the loop and stops
Vendor comparisons and product tours	They are **dated detail** in Chapter 6's sense — wrong by spring. The book organises by capability and names no leader, so there is nowhere for it to date
Serving models at scale	Chapter 52 covers it as a lever you might pull, not as a thing you will operate
Everything an interview does not reach	Chapter 57 is the whole argument, including where to stop

That last row is the important one, and it is the hardest thing in the book to act on. Every prerequisite has a prerequisite. You can want to understand retrieval, so you read about how text becomes numbers, which sends you to linear algebra, which sends you to proofs, and a month later you have not built anything. **The regress has no natural bottom**, which means stopping is a decision somebody has to make rather than a place you arrive at. Chapter 57 argues for where to stop. Until then, the working rule is the one this book is built on: learn the thing you are about to use, and let the layer underneath wait until something you cannot explain makes you go down a level.

How to Read It

If you have	Read
Months	Straight through, labs included. The order is the argument
Weeks	Part I's first half, then 18 and 20, then 30–33, then labs 40 and 41, then 55. Add labs 43–45 if you can — lab 46 works on the outputs they produce, so it is not one you can jump straight to. Then the "heavy" row as time allows
Days	Chapters 1–7, Chapter 55, Appendix A (the cheat sheet) and Appendix B (the question bank). Accept that you will be answering from a mental model rather than from experience, and say so if asked
Already building something	Whichever chapter names the problem you have now. The book is written to be entered in the middle; every chapter says which chapter owns the concepts it leans on

That last row needs one piece of machinery explained. Every concept in this book has one **concept owner** — a single chapter that teaches it, with every other chapter referring to it by number instead of re-teaching it. So when a chapter says "Chapter 28 owns this", that is not the book being coy — it is telling you exactly where the full treatment is, and promising that you are not going to meet three partial explanations of the same thing.

Lab: Place Eight Questions

No code. Ten minutes. This one has a key, so you can check yourself.

Below are eight questions of the kind this book is preparing you for. For each, name the part — **I**, **II**, **III** or **IV** — whose chapters answer it. You are not answering the questions; you are placing them.

1. "Why does your system need a retrieval step at all?"
2. "Your answers got worse after a provider updated the model. What do you do?"
3. "How would you tell whether prompt A is better than prompt B?"
4. "What is actually inside the thing you are calling?"
5. "Walk me through building this and show me the numbers you got."
6. "What does one request cost, and what makes the bill grow faster than usage?"
7. "Why can't the model just check its own answer?"
8. "A web page tells your agent to email its data somewhere. What happens?"

Expected result: I for 1, 4 and 7 — each is a question about what the thing is and why it is

assembled that way. II for 3, because comparing two prompts properly is a method you learn and not a thing you build once. III for 5, which is the only question that cannot be answered without having done the labs. IV for 2, 6 and 8 — failure, cost and security are all Part IV, and all three assume you have built something to break.

If you placed 7 in Part II, you were thinking of Chapters 31 and 32, which build the checks — and that is a reasonable reading. Part I — Chapter 3 — is where the *reason* a self-check is not independent lives, and the question as asked is a *why*.

If you placed 3 in Part III, look again: Part III *does* the comparison, but Chapter 20 is where you learn whether a difference is real. The habit of separating *the method* from *the exercise that uses it* is much of what separates Part II from Part III.

Summary, Key Terms, and Review Questions

Summary

The book is four parts: **Understand** (1–17, what this is and what it is made of), **Use** (18–39, how to make each part work and know that it does), **Apply** (40–50, eleven labs that build one small system in order, on data the book ships), and **What Next** (51–57, what can go wrong, what makes it slow, what breaks, what it costs, and saying it out loud). Part I has two halves — *Mental Model*, the system around the model, and *Inside the Model*, the model itself — which are one part because they answer the same question in two directions. Which chapters carry most weight is a judgement and not a measurement, and it is offered as reading order rather than prediction. The book deliberately omits machine-learning theory, training from scratch, vendor comparisons and operating models at scale, and for each of those but the vendor comparisons it marks where the subject begins, because the regress of prerequisites has no natural bottom and stopping is a decision rather than a destination. Every concept has one **concept owner**, so the book can be entered in the middle.

Key Terms

Term	Meaning
Interview depth	Knowing something well enough to give the mechanism, the trade-off, and how you would check it, in about a minute of speech. Not exhaustive, and more than a summary
Concept owner	The single chapter that teaches a given concept. Every other chapter refers to it by number rather than re-teaching it, so the reader never meets three partial explanations of one thing

Review Questions

1. Why are *Mental Model* and *Inside the Model* two halves of one part rather than two parts?
2. Why does the book ship the data its labs use, instead of telling you to use your own documents?
3. The chapter ranks some chapters as carrying more weight than others. What kind of claim is that, and how should you treat it?
4. What does "the regress of prerequisites has no natural bottom" mean in practice, and what is the working rule it implies?
5. You have one week. What is the argument for reading Chapter 55 even though you will not have done most of the labs?

Review Answers

1. Because they answer the same question from two directions: the first says what the model is *for* as a component in a system, the second says what it *does* when it answers. Part II is readable after the first half alone; the second half is what makes Part II's instructions make sense rather than merely usable.
2. Because an expected result that depends on the reader's files holds for nobody — you could run a lab correctly and have no way to tell. Shipped data means every step can state what you should see, so the lab can be checked rather than merely performed.
3. It is a judgement, not a measurement: nobody has surveyed what gets asked, and the answer would vary by company, role and year anyway. It is reasoning rather than reporting, so the honest label is the one the chapter gives it: a judgement. Treat it as a suggested reading order and not as a prediction about any particular conversation.
4. Every prerequisite has a prerequisite, so following them downwards never terminates on its own — you can go from retrieval to how text becomes numbers to linear algebra to proofs and build nothing. The working rule is to learn the thing you are about to use, and go down a level only when something you cannot explain forces you to.
5. Because Chapter 55 is the only chapter that rehearses *saying* the material, and that is a separate skill from knowing it. Having a structure for talking through a system — what you would clarify, what you would measure, what you would not build yet — is worth more in a short conversation than one more chapter of content you cannot yet organise.

> **You understand this when** you can say what each of the four parts answers, name which half of Part I you have just finished, and say what the book leaves out and where it says so.

PART I

Inside the Model



What to ignore for now: how the model was trained, beyond the shape of the loop.

CHAPTER 8

The Maths This Book Uses

This is the first chapter of the second half, and it is the shortest kind of chapter there is: a vocabulary list. Nearly all of it is used by the nine chapters that follow, the rest by Part II, and nothing in it is harder than arithmetic.

The problem. I can follow an argument in words, but the moment someone writes a formula I stop reading.

The question. What is the smallest set of notation that lets me read the rest, and can I learn it by doing the arithmetic rather than by taking anybody's word for it?

How to Read a Formula at All

Before any particular notation, the habit that makes all of it readable.

A formula is a sentence. It has a subject on the left, a verb somewhere, and objects on the right. What makes it feel impenetrable is not the ideas but three things that are easy to fix: unfamiliar letters, operations written as symbols rather than words, and the fact that nobody reads it aloud to you.

So read it in this order, every time:

1. **Name every symbol.** Not "what does it do" — just "what is this one". A letter you cannot name is a letter you cannot think with.
2. **Say what shape each one is.** A single number? A list of numbers? A grid?
3. **Read the operations outward from the innermost brackets**, exactly as you would evaluate them.
4. **Substitute the smallest numbers that could possibly work**, and do it by hand.

Step 4 is the one people skip and the one that does the work. A formula you have evaluated once with two-digit numbers is no longer notation; it is a procedure you happen to know. Nearly every section below has one.



If you get stuck, the problem is almost always step 1. Go back and find the symbol you have been silently skipping. In this field it is usually a subscript — the small letter below the line, which is nearly always "which one of them" and nearly never anything harder.

Vectors, and What a Dimension Is

A **vector** is an ordered list of numbers. That is the whole definition.

```
a = (2, 0, 1, 3)
```

That is a vector with four **dimensions**, which is to say four slots. "Dimension" sounds spatial and the word comes from geometry, but here it means a position in a list and nothing more. A model that works in 4,096 dimensions is handling lists of 4,096 numbers.

Two things about those slots matter for everything later.

The order is fixed and it carries all the meaning. Slot 3 of one vector is comparable with slot 3 of another, and with nothing else. Two vectors can only be compared at all if they have the same number of slots.

A single slot, on its own, usually means nothing you could name. You cannot look at slot 3 and say "that one is about cooking". This is not a gap in the explanation — it is a real property of the vectors this book deals with, and Chapter 9 says why.



The word *vector* gets used for two different things that happen to share a notation: a list of numbers you chose (the three numbers describing a product, say) and a list of numbers that came out of a model. The arithmetic is identical. What differs is whether anybody can tell you what a slot means.

The Dot Product, and Why It Measures Agreement

Take two vectors with the same number of slots. Multiply them slot by slot, and add up the results. That is the **dot product**, written $a \cdot b$.

With $a = (2, 0, 1, 3)$ and $b = (1, 4, 0, 1)$:

```
slot    a     b     a x b
  1     2     1       2
  2     0     4       0
  3     1     0       0
  4     3     1       3
                     ----
                       5
```

So $a \cdot b = 5$. In notation, with n slots:

$$a \cdot b = \sum_{i=1}^{n} a_i b_i$$

Read that with the four steps. a_i is slot i of a. The $\sum$ says: for each i from 1 to n, work out $a_i b_i$, and add all of those together. It is the table above, written in one line.

Why it behaves like agreement. Look at what each slot contributes. Two large numbers in the same slot contribute a large positive amount. A large number paired with zero contributes nothing. A positive paired with a negative subtracts. So the total is large when the two vectors are large *in the same slots*, and small when they put their weight in different places. That is what "agreement" means here, and it is why this one operation turns up wherever two lists of numbers have to be compared.

And now the trade-off, which matters more than the definition. The dot product grows when the vectors grow, whatever their direction. Take $c = 10a = (20, 0, 10, 30)$ — the same direction as a, ten times longer:

```
a . b  =   5
c . b  =  50     ten times larger, same direction
```

Nothing about the agreement changed. Only the length did. So a dot product on its own cannot tell you whether two things are alike — a long vector will out-score a short one that points the same way, whenever it agrees with that other vector at all. That is the problem the next section exists to fix.

Cosine Similarity

The **length** of a vector, written $\|a\|$, is what you would measure with a ruler if the slots were coordinates: square every slot, add them, take the square root.

```
||a|| = sqrt(2^2 + 0^2 + 1^2 + 3^2) = sqrt(14) = 3.742
||b|| = sqrt(1^2 + 4^2 + 0^2 + 1^2) = sqrt(18) = 4.243
```

Cosine similarity is the dot product with the lengths divided out:

$$\cos(a, b) = \frac{a \cdot b}{\|a\| \, \|b\|}$$

```
cos(a, b) = 5 / (3.742 x 4.243) = 5 / 15.877 = 0.315
```

Now do the same for $c = 10a$. The dot product was ten times bigger, but so is the length, and the tens cancel:

```
cos(c, b) = 50 / (37.42 x 4.243) = 50 / 158.77 = 0.315
```

Identical. Dividing by the lengths throws away magnitude and keeps direction alone, which is usually what you wanted. The result always falls between -1 and 1: 1 means the same direction, 0 means no agreement, and -1 means opposite.

It is called cosine similarity because that fraction *is* the cosine of the angle between the two vectors. You never need the angle, and nothing in this chapter computes one.



Cosine similarity measures direction. It does not measure whether two things mean the same, whether one answers the other, or whether either is any good. A high score says the two lists of numbers point the same way. What follows from that depends entirely on where the numbers came from, and Chapter 25 is about the cases where the answer is “nothing useful”.

Sums and Products, Written Down

Two symbols, and they work the same way.

$\sum$ (capital sigma) means **add these up**. $\prod$ (capital pi) means **multiply these together**. Both are written the same way, and all four parts are instructions:

```
 n
---
\        symbol:  add up (or multiply, for a product)
/__  x   below:   start the counter i at 1
i=1  i   above:   stop when i reaches n
         beside:  what to do with each i
```

So $\sum_{i=1}^{4} a_i$ with $a = (2, 0, 1, 3)$ is $2 + 0 + 1 + 3 = 6$, and $\prod_{i=1}^{4} a_i$ is $2 \times 0 \times 1 \times 3 = 0$.

That is all there is to it. The reason these look harder than they are is that the interesting part is usually the expression beside the symbol, not the symbol — and that expression is often itself a formula you already know. The dot product above is just $\sum a_i b_i$: sigma, with a multiplication beside it.



You met $\prod$ in Chapter 4 without the notation. "Ten steps, each right nine times in ten" is a product: $0.9^{10} = 0.349$. Compounding error is a $\prod$, and that is why it falls so fast — multiplying numbers between 0 and 1 always shrinks them.

Matrices, and Multiplying Them

A **matrix** is a grid of numbers: rows and columns. You can read it as a stack of vectors, and which way you read it depends on what you are doing.

```
A =  [ 1  2  0 ]     two rows, three columns:
     [ 0  1  3 ]     "a 2 x 3 matrix"
```

Matrix multiplication is the dot product, done for every pair. To multiply A by B, take each *row* of A and each *column* of B, and put their dot product where that row meets that column.

With A above and

```
B =  [ 2  1 ]     three rows, two columns:
     [ 0  4 ]     "a 3 x 2 matrix"
     [ 1  0 ]
```

the top-left entry of the answer is row 1 of A dotted with column 1 of B:

```
(1, 2, 0) . (2, 0, 1) = 1x2 + 2x0 + 0x1 = 2
```

All four entries, worked the same way:

```
          col 1                col 2
row 1   1x2+2x0+0x1 = 2     1x1+2x4+0x0 = 9
row 2   0x2+1x0+3x1 = 3     0x1+1x4+3x0 = 4

AB =  [ 2  9 ]
      [ 3  4 ]
```

The shape rule, which is the only part worth memorising. A row of A and a column of B must have the same number of slots, or the dot product is not defined. So the columns of A must equal the rows of B, and the answer takes the rows of A and the columns of B:

```
(2 x 3) times (3 x 2)  gives  (2 x 2)
      ^         ^
      these must match
```

When you read later that something "has shape 4096 × 11008", that is a grid of about 45 million numbers, and the shape rule is what tells you what it can be multiplied by.

Exponentials and Logarithms

One number turns up constantly: e, about 2.718. Raising it to a power is written e^x or $\exp(x)$, and the two mean the same.

```
e^0 = 1       e^1 = 2.718      e^2 = 7.389      e^3 = 20.086
```

What matters is how fast that grows, and that it is always positive. Each step of 1 in x multiplies the answer by e: divide the figures above and $7.389/2.718 = 2.719$, which is e to the rounding — the two figures were each rounded on the way, which is the same effect step 9 of the lab makes explicit. Equal steps in, multiplication out. And e^x is always positive, for every x including negative ones: $e^{-2} = 0.135$. That property is what lets softmax produce probabilities at all, two sections from now — nothing else in it keeps a result from coming out negative.

A **logarithm** undoes it. $\ln(y)$ answers "what power of e gives y?", so $\ln(7.389) = 2$. The one property to carry:

$$\ln(xy) = \ln x + \ln y$$

A logarithm turns multiplication into addition. Check it:

```
ln(2 x 3) = ln(6)  = 1.792
ln 2 + ln 3 = 0.693 + 1.099 = 1.792
```

That is why logarithms appear wherever many probabilities get multiplied together. Multiply enough numbers below one and you get something too small for a computer to hold: a thousand factors of 0.1 is already far past the limit, where a thousand factors of 0.99 is only 4.3×10^{-5} and perfectly fine. Add their logarithms instead and nothing underflows. Chapter 14 uses exactly this.

Softmax, Read as a Formula

Softmax turns a list of scores into a list of probabilities. It comes back in two later chapters by name, so it is worth doing by hand once here.

$$p_i = \frac{\exp(z_i)}{\sum_{j=1}^{V} \exp(z_j)}$$

Read it with the four steps from the start of this chapter. z is the list of scores and z_i is score number i; V is how many there are. The top is "e to the power of this score". The bottom is "add up e to the power of *every* score". So each answer is one score's share of the total.

Worked on three scores, $z = (2.0,\ 1.0,\ 0.1)$:

```
step 1   exponentiate each score
         e^2.0 = 7.389     e^1.0 = 2.718     e^0.1 = 1.105

step 2   add them up
         7.389 + 2.718 + 1.105 = 11.212

step 3   divide each by that total
         7.389 / 11.212 = 0.659
         2.718 / 11.212 = 0.242
         1.105 / 11.212 = 0.099
                              -----
                              1.000
```

Four properties follow straight from the arithmetic, and all four get used later:

Property	Why
Every result is positive	e^x is always positive, whatever the score was
They add to exactly 1	Every share is divided by the same total
The order is preserved	A bigger score always gets a bigger share
Gaps get exaggerated	Two shares differ by e to the power of their score gap: a gap of 1 becomes a factor of 2.7

That last one is the interesting one. These scores went in about a point apart, and the probabilities came out heavily favouring the top. So softmax does not just normalise — it turns a *gap* in the scores into a *multiple* in the shares, and how much it sharpens depends entirely on how far apart the scores were. Put in $(0.2, 0.1, 0.01)$, which is just as evenly spread, and the shares come out $(0.366, 0.331, 0.303)$ — barely sharpened at all. Chapter 13 is about controlling that.



Nothing about softmax knows what the scores mean. It is arithmetic on a list. When you see it inside the attention formula in Chapter 11 and again on the way to a word in Chapter 13, it is doing the same three steps both times.

Big-O: How Cost Grows

Big-O notation says how a cost grows as an input gets bigger. It is not a measurement of time and it carries no units.

$O(n)$ — read "order n" — means: double the input, double the work. $O(n^2)$ means: double the input, quadruple the work.

```
     n          n^2
 1,000      1,000,000
 2,000      4,000,000      n doubled,  work x 4
 4,000     16,000,000      n doubled again,  work x 4 again
```

What it deliberately throws away. Constants and smaller terms. Something taking $3n^2 + 500n + 9000$ operations is simply $O(n^2)$, because once n is large the squared term is all that matters. So big-O cannot tell you which of two programs is faster today — one $O(n^2)$ program can beat another by a factor of fifty. It tells you which one gets worse faster, which is the question that decides whether something survives a bigger input.

A cost can grow in more than one thing at once. You will meet $O(n^2 d)$ in Chapter 11: quadratic in one quantity n and linear in another d. Read it as "double n and the work goes up four times; double d and it doubles."



When somebody says an approach "does not scale", the useful reply is "in what?" A cost that is quadratic in the number of documents and linear in their length is a different engineering problem from the other way round, and the fix is different too.

Lab: Nine Lines of Arithmetic

No code and nothing to install, though you may use a calculator. Twenty minutes, and there is a key, so you can be wrong and find out. Use these two vectors throughout:

```
u = (3, 1, 0, 2)
v = (1, 0, 3, 2)
```

Step 1. Compute $u \cdot v$, slot by slot, writing the table out.

Expected result: $3{\times}1 + 1{\times}0 + 0{\times}3 + 2{\times}2 = 3 + 0 + 0 + 4 = 7$.

Step 2. Compute $\|u\|$ and $\|v\|$.

Expected result: both are $\sqrt{14} = 3.742$ — $9 + 1 + 0 + 4$ and $1 + 0 + 9 + 4$ both come to 14. Two different vectors of the same length; that is exactly the case a dot product cannot distinguish by magnitude.

Step 3. Compute $\cos(u, v)$.

Expected result: $7/(3.742 \times 3.742) = 7/14 = 0.5$. Note what happened: when both lengths are the same, the division is by the square of that length, and the arithmetic came out clean.

Step 4. Now double u to $u' = (6, 2, 0, 4)$. Compute $u' \cdot v$ and $\cos(u', v)$.

Expected result: the dot product is 14, twice what it was. The cosine is still 0.5. If your cosine moved, you have divided by the old length.

Step 5. Write $u \cdot v$ in sigma notation, then say in words what each of the four parts of the symbol is telling you to do.

Expected result: $\sum_{i=1}^{4} u_i v_i$, and four instructions — add up; start the counter at 1; stop at 4; the thing to add is slot i of u times slot i of v.

Step 6. Multiply these two matrices, or say why you cannot:

```
P =  [ 1  0  2 ]        Q =  [ 1  1 ]
     [ 3  1  0 ]             [ 0  2 ]
```

Expected result: **you cannot.** P is 2×3 and Q is 2×2; a row of P has three slots and a column of Q has two, so the dot product is undefined. The shape rule is the whole answer. If you got a grid of numbers, you multiplied something by something it does not fit.

Step 7. Apply softmax to $z = (1.0,\ 1.0,\ 3.0)$. The exponentials you need are $e^1 = 2.718$ and $e^3 = 20.086$.

Expected result: total $2.718 + 2.718 + 20.086 = 25.522$, and dividing each exponential by it gives $(0.106,\ 0.106,\ 0.787)$. Two equal scores give two equal shares, and a score two higher takes about four fifths of the whole. That is the sharpening at gaps of about a point, in one line.

Your three figures add to 0.999 rather than 1, and that is not a mistake: the two exponentials you were handed had already been rounded to three places before you divided by them. Carried to four places the shares are $(0.1065,\ 0.1065,\ 0.7870)$ and they do sum to exactly 1. Step 9 makes the same point on purpose.

Step 8. Without computing anything: a process is $O(n^2)$ and takes 3 seconds at $n = 1{,}000$. Roughly what does it take at $n = 10{,}000$?

Expected result: about 300 seconds. Ten times the input is a hundred times the work. Note the "roughly": big-O throws away constants, so this is the growth and not a promise.

Step 9. Check $\ln(4 \times 5) = \ln 4 + \ln 5$, using $\ln 4 = 1.386$, $\ln 5 = 1.609$ and $\ln 20 = 2.996$.

Expected result: $1.386 + 1.609 = 2.995$ against $\ln 20 = 2.996$. They agree to the rounding, which is the point — the last digit differs because the three logarithms were each rounded, not because the rule is approximate.

Summary, Key Terms, and Review Questions

Summary

A **vector** is an ordered list of numbers and a **dimension** is one slot in it; the order carries the meaning and a single slot usually names nothing. The **dot product** multiplies two vectors slot by slot and adds the results, which makes it large when both put their weight in the same slots — but it also grows with length, so **cosine similarity** divides the lengths out and keeps direction alone, landing between −1 and 1. $\sum$ means add these up and $\prod$ means multiply them, and both are read as four instructions: what to do, where to start, where to stop, and what to do it to. A **matrix** is a grid, and multiplying two matrices is the dot product of every row with every column, which is why the columns of the first must match the rows of the second. e^x is always positive and turns equal steps into multiplication; a **logarithm** undoes it and turns multiplication into addition. **Softmax** exponentiates every score and divides by the total, giving positive numbers that sum to 1, in the same order, turning each gap in the scores into a multiple in the shares. **Big-O** says how a cost grows rather than how long it takes, and throws away constants on purpose.

Key Terms

Term	Meaning
Vector	An ordered list of numbers. Two vectors can be compared only if they have the same number of slots
Dimension	One slot in a vector. A position in a list, not a direction in space, and usually not nameable on its own
Dot product	Multiply two vectors slot by slot and add the results. Large when both are large in the same slots — and also larger when either is simply longer
Cosine similarity	The dot product divided by both lengths, which removes magnitude and keeps direction. Always between –1 and 1
Length	What a ruler would measure if the slots were coordinates: square every slot, add them, take the square root
Logarithm	The inverse of raising e to a power. It turns multiplication into addition, which is why it appears wherever many probabilities are combined
Matrix	A grid of numbers, read as rows or as columns depending on what you are doing
Softmax	Turns a list of scores into positive numbers that sum to 1, preserving their order and turning each gap between scores into a multiple between shares
Big-O	How a cost grows as an input grows, with constants and smaller terms deliberately discarded. Not a measurement of time

Worth Memorising

Item	Value or formula
Dot product	$a \cdot b = \sum_i a_i b_i$
Cosine similarity	$a \cdot b / (\|a\|\|b\|)$, between –1 and 1
Softmax	$p_i = \exp(z_i) / \sum_j \exp(z_j)$
Matrix shape rule	(m × k) times (k × n) gives (m × n); the inner numbers must match
Logarithms turn products into sums	$\ln(xy) = \ln x + \ln y$
e	About 2.718. e^x is positive for every x
Quadratic growth	$O(n^2)$: ten times the input, a hundred times the work

Review Questions

1. You meet a formula containing a symbol you cannot name. What is the first thing you do, and why that first?
2. Two *pairs* of vectors have the same cosine similarity as each other, but very different dot products. What does that tell you about them?
3. You are asked to multiply a 3 × 5 matrix by a 3 × 5 matrix. What do you say?
4. Name two properties of softmax that follow directly from the fact that e^x is always positive.
5. A colleague says their retrieval step "does not scale". What do you ask them?

Review Answers

1. Name it — just name it, before asking what it does. A letter you cannot name is a letter you cannot think with, so every later step is guesswork built on it. Then say what shape it is: a single number, a list, or a grid. In this field the symbol being silently skipped is usually a subscript, and a subscript is almost always "which one of them" rather than anything harder. Only once every symbol has a name and a shape is it worth reading the operations, and only then is it worth substituting small numbers — which is the step that turns the formula from notation into a procedure you know.
2. That within each pair the vectors sit at the same angle, and that one pair is simply longer than the other. Cosine ignores length by construction, so two pairs with identical cosines can have dot products differing by any factor you like, purely because one pair is longer. If the number you care about is "are these alike", the cosine is the one to look at.
3. That it is not defined. A row of the first has five slots and a column of the second has three, so there is no dot product to take. The inner numbers of (3 × 5) and (3 × 5) do not match. Either the second needs transposing — making it 5 × 3, which would give a 3 × 3 answer — or one of the shapes is wrong.
4. Every output is positive, because every power of e is positive; and the outputs can be read as probabilities at all, because positive numbers each divided by their own total necessarily land between 0 and 1 and sum to exactly 1. Order preservation follows too, since e^x is increasing.
5. In what? A cost can grow in the number of documents, the length of each one, the number of users, or several at once, and the fix differs for each. Ask which quantity is growing, and what the growth is in that quantity — linear, quadratic, or something else. Also ask what the constant is, because big-O discards it and a quadratic process on small inputs is often perfectly fine.

> **You understand this when** you can take a formula you have not seen before, name every symbol in it, say what shape each one is, and work it out with two-digit numbers.

CHAPTER 9

What the Model Is Made Of

Chapter 1 held the model as a black box: text in, text out, charged by the amount. This chapter opens the box and names the three things inside it. By the end you will be able to say what shape the input takes, what shape the output takes, and what sits between them — and you will have run one whole pass of a very small model with a pen.

The problem. I know text goes in and text comes out. I don't know what is in between, or what shape it has.

The question. What are the actual pieces, what shape is each one, and what does a single call make the model compute?

The Short Answer

Three stages, in order, every time.

```
text  -->  a list of whole numbers
           one number per piece of text, from a fixed list

numbers  -->  a list of vectors
              each number is a row looked up in a table

vectors  -->  a stack of layers  -->  one vector
              then one score per entry in the fixed list,
              turned into probabilities that add to 1
```

So the thing you get back from a single call is not a word. It is a **probability distribution**: one number for every entry the model is able to produce, saying how likely that one is to come next. Something outside the model then picks one. That pick, and the settings that steer it, belong to Chapter 13.

Everything else in this chapter is detail on those three stages, plus one fact that follows from

them and surprises nearly everybody: a single pass produces exactly one of those distributions, so a hundred-token reply is a hundred passes.

From Text to Numbers

A model does not work on letters or on words. Before anything else happens, your text is cut into pieces, and each piece is replaced by its position in a fixed list.

That fixed list is the model's **vocabulary**: every entry the model can read or produce, numbered. Nearly all of them are pieces of text, and a handful are not — Chapter 13 meets one of those. A vocabulary is built once, before training, and does not change while you are using the model. Each piece is called a token, and tokens are what you are billed for.

```
"the cat sat"  -->  ["the", " cat", " sat"]  -->  [0, 1, 2]
```

Three facts about the vocabulary are worth having now, because the rest of the chapter leans on them.

It is finite and fixed. A model with a vocabulary of 128,000 entries can produce those 128,000 things and nothing else. Any text it writes is a sequence drawn from that list.

The pieces are not words. They are usually fragments — part of a word, a word with its leading space, a punctuation mark. This is why asking a model to count the letters in a word asks about something the piece does not carry: a piece arrives as a seat number, and a seat number is not spelled. Chapter 10 is where that is worked out properly.

Every token is just an index. The number is a seat number, not a quantity: nothing in the model treats token 1 as one more than token 0. It is used to find a row and for nothing else.



Why the pieces are fragments rather than words, how the list is built, why your short question can cost more than somebody else's long one, and how to count tokens before you send them — all of that is Chapter 10, which is where tokens are taught properly. For this chapter you need only the shape: a fixed numbered list, and your text arriving as a list of indexes into it.

The Embedding Table: Coordinates Nobody Chose

An index is useless for arithmetic, so the first thing the model does with it is look it up.

The **embedding table** is a matrix with one row per vocabulary entry. Each row is a vector, and looking up that row is how a token enters the model: from here on the arithmetic works on the vector and never on the token. The rows are called embeddings. A vocabulary of 128,000 entries and vectors of 4,096 slots makes a table of 128,000 × 4,096 numbers:

```
128,000 x 4,096 = 524,288,000 numbers
```

Half a billion numbers, and that is before a single layer. The number of slots in each row — 4,096 here — is the model's **width**, and it is the same number all the way through: the vectors going into and out of every layer have that many slots. Width is usually written d in formulas. Other shapes do appear *inside* a layer — Chapter 8's "shape 4096 × 11008" was one of them — and Chapter 12 is where those live.

Looking up three tokens gives three rows, which is a 3 × 4,096 matrix: one vector per position, in the order you sent them.

Position is supplied separately, and this is worth knowing before anybody asks. A row says *which* token this is. It says nothing about *where* in the input the token sits, and the two rows for a word used twice in a sentence are the same row. So the model is given the position as well. Chapter 11 is where it arrives, and why leaving it out would make word order invisible.

Now the part that matters, and it is the honest answer to a question interviewers ask. Nobody chose those coordinates. They are not a description of the token written down by a linguist; no committee decided that slot 57 would be about cooking. They are **whatever made next-token prediction work** — the training process adjusted them until the model got better at guessing what comes next, and these are the numbers it stopped at. And no dimension means anything. You cannot read slot 57 of any row and say what it is for.

That is not a simplification for beginners. It is the state of the art, and it is why Chapter 8 told you a single slot usually names nothing.



You will meet confident explanations of what the dimensions of an embedding "represent". Treat any such claim as something to check rather than something to repeat. The rows are certainly not random — they were fitted, and how they sit relative to each other is useful. Chapter 24 covers the separate models built to produce vectors you can search with, which is where that usefulness is put to work. But *why* the arrangement carries meaning at all is a question this book does not answer, and Chapter 17 lists it among the things you will not be able to explain when you finish.

A Stack of Identical Layers

The list of vectors now goes through a stack.

A **layer** is a block of arithmetic that takes one vector per position and returns one vector per position, with the same width in as out. The layers are **identical in shape and different in content**: every one is built the same way, and every one holds its own numbers, fitted separately. A model with 32 layers runs 32 blocks of the same shape, in order, and each of them has its own set of numbers. That number of layers is the model's **depth**.

```
rows  -->  layer 1  -->  layer 2  -->  ...  -->  layer 32
           |             |                       |
           same width in and out, every time
```

Two consequences of "same width in and out".

The stack can be any depth. Because the shape coming out of a layer is the shape going into one, you can put thirty-two of them in a row, or ninety-six, without changing anything else. That is the whole reason the design *can be* scaled by stacking. What more depth buys is Chapter 12's, and why it buys it is on Chapter 17's list.

There is no loop. The vectors go through layer 1, then layer 2, and so on to the end, once. Nothing goes back. The model does not re-read or reconsider, and one pass over a hard question costs exactly what one pass over an easy one of the same length costs: the same amount of arithmetic runs either way. Chapter 16 is where that fact turns into something you can use.

What is *inside* a layer is Chapter 11 (the attention block, worked by hand) and Chapter 12 (the other block, what the running vector accumulates, and what nobody can tell you about depth). For this chapter a layer is a box of the right shape, and that is enough to see the pass end to end.



When you read that a model "has 32 layers and a width of 4,096", you now know exactly what both numbers mean: a depth of thirty-two blocks in a row, each moving lists of 4,096-slot vectors. Depth, width and the vocabulary size describe most of the model's shape.

What Comes Out Is a Distribution, Not a Word

The stack gives back one vector per position, same as it took. For the next token, only the last position's vector matters — it is the one that has seen the whole input, and Chapter 11 shows how.

That vector has d slots, and you need one number per vocabulary entry. So the final step multiplies it by a matrix with one column per entry. Shape rule from Chapter 8:

```
(1 x 4,096) times (4,096 x 128,000)  gives  (1 x 128,000)
            ^              ^
            these must match
```

One raw score for every token in the vocabulary. Then softmax — the formula from Chapter 8 — turns those scores into positive numbers that add to exactly 1. That is the **next-token distribution**, and it is what one call to the model computes.



In many models the matrix that produces those scores is the embedding table used a second time and read the other way round — the same table that turned a token into a vector now turns a vector into a score for that token. Some models keep a separate matrix for it. Nothing in this chapter depends on which. Chapter 13 gives those raw scores their name and shows how one of them becomes a word.

The Whole Thing, in Five Tokens and Four Dimensions

Now do it. A vocabulary of five entries and a width of four, so every number fits on the page.

The vocabulary and its embedding table:

```
id  token   embedding
 0  the     (1, 0, 2, 0)
 1  cat     (0, 2, 1, 1)
 2  sat     (2, 1, 0, 1)
 3  mat     (0, 1, 3, 0)
 4  on      (1, 1, 1, 2)
```

Stage 1. The input is "the cat sat", which is ids $(0, 1, 2)$.

Stage 2. Look up those three rows. The result is a 3×4 matrix — one vector per position:

```
position 1   the   (1, 0, 2, 0)
position 2   cat   (0, 2, 1, 1)
position 3   sat   (2, 1, 0, 1)
```

Stage 3. Those go through the stack. We are not opening the stack here, so take its answer for the last position as given:

```
h = (0, 1, 1, 2)
```

Stage 4. One score per vocabulary entry. Taking the first case from the note above, that is the dot product of h with every row of the table. Slot by slot, all five:

```
token   row           row . (0, 1, 1, 2)          score
the     (1, 0, 2, 0)  0 + 0 + 2 + 0                 2
cat     (0, 2, 1, 1)  0 + 2 + 1 + 2                 5
sat     (2, 1, 0, 1)  0 + 1 + 0 + 2                 3
mat     (0, 1, 3, 0)  0 + 1 + 3 + 0                 4
on      (1, 1, 1, 2)  0 + 1 + 1 + 4                 6
```

Stage 5. Softmax those five scores. Exponentiate each, add them up, divide:

```
e^2 = 7.389     e^5 = 148.413    e^3 = 20.086
e^4 = 54.598    e^6 = 403.429

total = 7.389 + 148.413 + 20.086 + 54.598 + 403.429
      = 633.915

the     7.389 / 633.915 = 0.012
cat   148.413 / 633.915 = 0.234
sat    20.086 / 633.915 = 0.032
mat    54.598 / 633.915 = 0.086
on    403.429 / 633.915 = 0.636
                         -----
                         1.000
```

That list of five numbers is the model's entire output for this call. "on" at 0.636, "cat" at 0.234, and three long shots. The model has not produced a word. It has produced a ranking with weights, and it is now finished until somebody calls it again.

Notice what the arithmetic shows that prose cannot. The scores came from dot products against the embedding rows, so **a token scores highly when its row agrees with h slot by slot** — the same "agreement" Chapter 8 built. And softmax sharpened the gaps, exactly as it did there: scores spread over 2 to 6 came out as probabilities spread over 0.012 to 0.636.



The numbers in that table are invented, and a real table's numbers would be fitted from training. The *arithmetic* is the real thing, at the real shape — change 5 to 128,000 and 4 to 4,096 and nothing about the procedure differs.

Parameters: the Numbers That Were Fitted

A **parameter** is one of the numbers inside the model: training set it, and inference never changes it. The embedding table is parameters. Every layer's numbers are parameters. The final matrix is parameters.

The count is what a model's name usually reports. "A 7B model" means about seven billion parameters. That one number tells you roughly what hardware it needs and roughly what it costs to run, which is why it is the first thing anybody says about a model.

Our five-token toy has 20 parameters in its embedding table, and a real model of the shape above has 524,288,000 in that table alone. Where the rest of them sit — how many are in each layer and in which part of it — is Chapter 12, and what one parameter occupies in memory is Chapter 15.



Vocabulary sizes around 100,000 to 200,000 entries, widths of a few thousand, and depths of a few dozen to a hundred describe the models of the moment. The figures will move. The three quantities — vocabulary, width, depth — and the fact that the embedding table is the first of them multiplied by the second will not.

Two things parameters are *not*, both of which get muddled:

- **Not your data.** Nothing you send on a call becomes a parameter. The numbers were fixed before you arrived, which is Chapter 1's point about training and inference, now with a name for the thing that stays fixed.
- **Not the settings.** Temperature and the rest are dials outside the model's own numbers: they act on the scores and on the picking, never on the parameters. Chapter 13 owns them.

One Pass, One Token

One **forward pass** is the whole journey above: look up, through the stack once, one distribution out. And it yields exactly one distribution — one guess about one position.

So how does a model write a paragraph? By being called again. Something picks a token from the distribution, appends it to the input, and runs another forward pass over the longer input. Then again. A reply of a hundred tokens is a hundred forward passes, each one working on an input one token longer than the last. A model used this way is called **autoregressive**: its own output becomes part of its next input.

```
pass 1   the cat sat            -->  on
pass 2   the cat sat on         -->  the
pass 3   the cat sat on the     -->  mat
```

Three things fall out of this, and all three are easy to get wrong.

The loop is not in the model. The model has no memory between passes and no notion that it is in the middle of a sentence. Appending the token and calling again is done by the program around it. This is the same observation as Chapter 1's "the memory is in the product", one level further down.

The reply arrives a piece at a time because it is produced a piece at a time. The streaming you see in products is not a visual effect; it is one token per pass, shown as each pass finishes.

Re-reading the whole input on every pass sounds ruinously wasteful, and it would be. A naive implementation would redo all the work for every earlier position on every pass. Real systems do not, and the mechanism that saves them — along with the memory bill it brings — is Chapter 15.



This is the fact to reach for when somebody asks why long outputs cost more than long inputs. A long input is read once. A long output is one pass per token, over an input that keeps growing.

Lab: Run a Forward Pass by Hand

No installation and nothing to set up beyond Python, which you have. Twenty-five minutes, and there is a key for every step, so you can be wrong and find out.

A different five-token model. Its vocabulary and embedding table:

```
id  token   embedding
 0  i       (2, 0, 1, 0)
 1  like    (0, 2, 0, 1)
 2  cold    (1, 1, 1, 0)
 3  tea     (0, 2, 1, 2)
 4  .       (1, 0, 0, 1)
```

Step 1. The input is "i like cold". Write down the list of ids, then the matrix of rows the lookup produces, and say its shape.

Expected result: ids $(0, 1, 2)$; rows $(2, 0, 1, 0)$, $(0, 2, 0, 1)$, $(1, 1, 1, 0)$; shape 3×4 — three positions, width four. If you wrote 4×3 you have the shape rule the wrong way round.

Step 2. The stack returns $h = (1, 2, 1, 1)$ for the last position. Compute the five scores by hand, writing each dot product out slot by slot.

Expected result:

```
i      (2,0,1,0) . h = 2 + 0 + 1 + 0 = 3
like   (0,2,0,1) . h = 0 + 4 + 0 + 1 = 5
cold   (1,1,1,0) . h = 1 + 2 + 1 + 0 = 4
tea    (0,2,1,2) . h = 0 + 4 + 1 + 2 = 7
.      (1,0,0,1) . h = 1 + 0 + 0 + 1 = 2
```

Step 3. Without computing the softmax: which token is most likely, and which is least? Why can you answer that from the scores alone?

Expected result: "tea" is most likely and "." is least. Softmax preserves order — Chapter 8's third property — so the ranking is already decided by the scores. Only the *sizes* of the probabilities need the arithmetic.

Step 4. Say precisely what the model has *not* done at this point, and who does it.

Expected result: it has not chosen a token. It has produced five numbers. Choosing is done outside the model, by the code that called it, under settings that Chapter 13 teaches. A

model that always produced its top-scoring token would be one particular choice among several, not the only possibility.

Step 5. Now run it. In a new empty folder, save this as `forward.py`:

```
import math

vocab = ["i", "like", "cold", "tea", "."]
table = [(2, 0, 1, 0),        # i
         (0, 2, 0, 1),        # like
         (1, 1, 1, 0),        # cold
         (0, 2, 1, 2),        # tea
         (1, 0, 0, 1)]        # .

ids = [0, 1, 2]                      # "i like cold"
rows = [table[i] for i in ids]       # the lookup
h = (1, 2, 1, 1)                     # what the stack returned

scores = [sum(a * b for a, b in zip(row, h)) for row in table]
total = sum(math.exp(s) for s in scores)
probs = [math.exp(s) / total for s in scores]

for word, s, p in zip(vocab, scores, probs):
    print(f"{word:>5}   score {s:>2}   p {p:.3f}")
```

Expected result: exactly this, and your five scores from step 2 in the middle column.

```
    i   score  3   p 0.015
 like   score  5   p 0.112
 cold   score  4   p 0.041
  tea   score  7   p 0.826
    .   score  2   p 0.006
```

Step 6. Count the parameters in this toy's embedding table. Then count them for a model with a vocabulary of 50,000 and a width of 2,048.

Expected result: 5 × 4 = 20 for the toy. 50,000 × 2,048 = 102,400,000 for the other — a hundred million parameters in the lookup table alone, in a model far smaller than the ones people call large.

Step 7. Change one number: set `h` to `(1, 0, 1, 1)`, so the second slot is zero. Run it again. Which scores moved, which did not, and why?

Expected result:

```
     i   score  3   p 0.348
  like   score  1   p 0.047
  cold   score  2   p 0.128
   tea   score  3   p 0.348
     .   score  2   p 0.128
```

"i" and "." did not move, because their rows have 0 in slot 2 and that slot now contributes nothing either way. The other three fell, each by twice its own slot-2 value. And the prediction is destroyed: "tea" has gone from 0.826 to a tie with "i" at 0.348. One slot out of four, in one vector, did that. Be exact about what it shows: these numbers are invented and four slots is not 4,096, so it says nothing about how far a real model moves when one slot changes. What it shows is that nothing in this arithmetic protects any one score from any one slot.

Step 8. The model must produce the reply "tea ." — two tokens. How many forward passes, and what is the input to each?

Expected result: two passes. The first sees "i like cold" and yields a distribution from which "tea" is taken; the second sees "i like cold tea" and yields one from which "." is taken. Each pass reads everything before it. If you answered one, you have the central point of the last section still to re-read.

Summary, Key Terms, and Review Questions

Summary

Text is cut into tokens and each token is replaced by its index into the model's **vocabulary**, a numbered list fixed before training and unchanged while you use the model. Each index is looked up in the **embedding table**, a matrix with one row per vocabulary entry, giving one vector per position; the number of slots in those vectors is the model's **width**, the same from one layer to the next all the way through. Those coordinates were chosen by nobody — they are whatever made next-token prediction work, and no dimension means anything. The vectors pass through a stack of **layers** — the count of them is the model's **depth** — identical in shape and each holding its own numbers, with the same width in as out, once, with no loop. The last position's vector is then multiplied by a matrix with one column per vocabulary entry to give one score per entry, and softmax turns those scores into the **next-token distribution** — one probability for every token the model can produce, adding to 1. That distribution is the whole output of one **forward pass**, so a long reply is many passes, each over a slightly longer input, which is what **autoregressive** means. A **parameter** is one of the fitted numbers inside all of this, and the count is what a model's name reports.

Key Terms

Term	Meaning
Vocabulary	The fixed, numbered list of every entry a model can read or produce — nearly all pieces of text, and a few that are not. Built once, before training, and fixed while you use the model
Embedding table	A matrix with one row per vocabulary entry. Looking a token up in it is how that token enters the model; the rows are called embeddings
Width	The number of slots in the vectors moving from layer to layer, the same all the way through. Written d in formulas
Layer	A block of arithmetic taking one vector per position and returning one per position, same width in as out. Every layer has the same shape and its own numbers
Depth	The number of layers in the stack. The last of the three numbers — vocabulary, width, depth — that describe most of a model's shape
Next-token distribution	One probability for every entry in the vocabulary, adding to 1, saying how likely each is to come next. The entire output of one call
Forward pass	One journey through the model — lookup, the stack once, one distribution out. It produces exactly one distribution
Autoregressive	Producing output one token at a time, with each token appended to the input before the next pass
Parameter	One number inside the model, set by training and unchanged by inference. The count is what "a 7B model" reports

Worth Memorising

Item	Value or formula
Embedding table size	vocabulary × width numbers. 128,000 × 4,096 = 524,288,000
The output step	One score per vocabulary entry, so the final shape is $(1 \times d)$ times $(d \times \text{vocabulary})$
What one forward pass yields	Exactly one next-token distribution
Passes for a reply	One per token produced, each over an input one token longer
The three top-level shape numbers	Vocabulary, width, depth. There are more inside a layer, which is Chapter 12's

Review Questions

1. What does one call to a model actually compute, and what has it not done?

2. Somebody tells you that dimension 412 of a model's embeddings encodes formality. What do you say?
3. A model has 32 layers. What is identical between layer 3 and layer 29, and what is not?
4. Why is a 500-token reply so much dearer than a 500-token prompt?
5. Your vocabulary has 200,000 entries and your width is 4,096. How many numbers are in the embedding table, and why might that matter for a small model?
6. What does "autoregressive" mean, and where does the loop live?

Review Answers

1. It computes one next-token distribution: one probability for every entry in the vocabulary, adding to 1, for the single position after everything you sent. It has not chosen a token, produced a word, or decided anything about the rest of the reply. Choosing is done by the code around the model, and a long reply is that whole computation repeated once per token.
2. That the claim needs checking rather than repeating, and that the honest default is the opposite: the coordinates are whatever made next-token prediction work, nobody chose them, and no dimension means anything on its own. The rows are not random — they were fitted, and useful things follow from how they sit relative to each other — but naming what one slot does is a different and much stronger claim. Why the arrangement carries meaning at all is outside what this book explains.
3. Their shape is identical: both take one vector per position and return one per position, with the same width in as out. That is what lets them be stacked at all. Their numbers are not identical — each layer holds its own parameters, fitted separately. "Identical layers" means identical in structure, never in content.
4. The prompt is read in one pass. The reply is one pass per token, and each of those passes works on an input that is one token longer than the last. So the reply is both more passes and more work with each successive pass, which is also why providers charge more for output than for input.
5. 200,000 × 4,096 = 819,200,000 numbers — over eight hundred million parameters before a single layer. In a small model that can be a large share of the total, so the vocabulary size is a real design decision rather than a detail, and it sets the size of the output step too.
6. It means the model's own output becomes part of its next input: pick a token from the distribution, append it, run another forward pass, repeat. The loop lives in the program around the model. The model itself has no memory between passes and no idea that it is part-way through a sentence.

> **You understand this when** you can say what a single call computes without using the word "answer", and explain why writing a hundred tokens means running the model a hundred times.

CHAPTER 10

Tokens

Chapter 9 said your text arrives at the model as a list of indexes into a fixed vocabulary, and left four questions open: why the pieces are fragments rather than words, how the list is built, why your short question can cost more than somebody else's long one, and how to count before you send. This chapter answers all four, and the way it answers the second is by building a vocabulary by hand, from a corpus of five words, so that nothing here has to be taken on trust.

The problem. I was billed for "tokens". I don't know what one is, or why my short question cost more than someone else's long one.

The question. What is a token, where does the list of them come from, and what does knowing that let me predict about my bill and my waiting?

Why Not Words, and Why Not Letters

A **token** is one entry in the model's vocabulary — one of the pieces text gets cut into before anything else happens. The obvious question is why the pieces are not simply words, and the answer is worth working through, because it explains nearly everything odd about tokens afterwards.

Words fail for two reasons, and the first is arithmetic. Chapter 9 showed that the embedding table holds one row per vocabulary entry, and that the last step of a forward pass produces one score per vocabulary entry. Both of those grow with the vocabulary. English has hundreds of thousands of word forms before you count names, product codes, typos and hyphenations, so suppose you wanted 500,000 entries at a width of 4,096:

```
500,000 x 4,096 = 2,048,000,000 numbers
```

Two billion parameters in the lookup table alone — and, where the output matrix is a separate one rather than the embedding table read backwards, the same again. That is more than many whole models.

The second reason is worse, because no size of vocabulary fixes it. A word that is not in the list has no row. Not a poor row — no row at all. Every name you have never heard of, every new product, every misspelling, every word in a language that was not planned for arrives as nothing. The usual patch is one shared "a word I do not have" entry that all of them map to, which keeps the program running and throws the word itself away: every unknown name becomes the same unknown name. That is not a quality problem, it is a hole.

Letters fix the hole and create a different one. An alphabet, digits and punctuation make a vocabulary of a hundred-odd entries. Every *word* is representable — no word arrives as nothing — and the lookup table is tiny. But an English word runs about five characters, so cutting at letters makes the list of pieces several times longer than cutting at words — and Chapter 9 showed that every piece of a reply costs one whole forward pass. Several times the pieces is several times the passes, and the input grows the same way.

So the two obvious answers fail in opposite directions: words give you a vocabulary too large to hold and still cannot represent everything, letters leave no unknown words but make the sequences too long.

The working answer is to let the corpus decide where to cut. Keep single characters in the vocabulary, so nothing is ever unrepresentable. Then add, as extra entries, the *combinations* that turn up often enough to be worth a seat of their own. A piece that is part of a word rather than a whole one is called a **subword**, and the result is a compromise that leans exactly where the text leans:

- Common words end up as one token each.
- Rare words are spelled out of several pieces.
- Nothing is ever unrepresentable, because the single pieces are still in there.

And "single pieces" means bytes, not letters — which is where the *byte* in the usual name comes from. A vocabulary built from the letters of one alphabet would still have no answer for a script nobody planned for. A computer stores text as bytes, there are exactly 256 of them, and every one of them is in the vocabulary from the start. Any text whatever can therefore be represented, in the worst case one token per byte. That is the convention for the models this book is about; a tokeniser built without it falls back to the shared unknown entry instead, and loses the text. Keep that worst case in mind: it is the ceiling that the section on other languages lands against.

The rest of this chapter is what that does to you.

Building the Vocabulary: Merge the Commonest Pair

The usual way to pick those combinations is **byte-pair encoding**. The name sounds forbidding and the procedure is four lines:

1. Start with a vocabulary of every single character in the corpus — in a real run, every one of the 256 bytes.
2. Count every adjacent pair of symbols across the whole corpus.
3. Merge the commonest pair into one new symbol, and add it to the vocabulary.
4. Go back to step 2. Stop when the vocabulary is the size you wanted.

That is the whole idea. It is greedy compression: at each step, take the saving that is worth most right now, and never reconsider. A real run starts from the 256 bytes; the worked example below starts from letters instead, because a corpus of five English words contains nothing else and letters are what you can read. The list of merges, in the order they were made, is the **merge list**, and it is kept — the same merges are replayed, in the same order, every time text is tokenised afterwards.

Now do it. Here is a corpus of five words, with how often each occurs:

```
cat        7
cats       3
hat        5
hats       2
catalyst   1
```

The alphabet is the seven letters that occur: a, c, h, l, s, t, y. Every word starts as its letters, kept apart:

```
c a t         7
c a t s       3
h a t         5
h a t s       2
c a t a l y s t  1
```

Merge 1. Count every adjacent pair, weighting each by how often its word occurs. `at` occurs once in every one of the five words, so it scores 7 + 3 + 5 + 2 + 1 = 18:

```
pair   where it occurs                      count
at     cat, cats, hat, hats, catalyst       18
ca     cat, cats, catalyst                  11
ha     hat, hats                             7
ts     cats, hats                            5
ta     catalyst                              1
al     catalyst                              1
ly     catalyst                              1
ys     catalyst                              1
st     catalyst                              1
```

`at` wins. Merge it, and the corpus becomes:

```
c at           7
c at s         3
h at           5
h at s         2
c at a l y s t  1
```

That is what "compression" means here, and it is countable. The corpus held 64 symbols before the merge and holds 46 after. The saving is 18 — the pair's count exactly, because each of those 18 occurrences became one symbol instead of two. **Merge the commonest pair** is just another way of saying *take the biggest saving available this step*.

Merge 2. Count again — and notice that the counts have changed, because the symbols have.

```
pair    count
c+at    11        (cat 7, cats 3, catalyst 1)
h+at     7        (hat 5, hats 2)
at+s     5        (cats 3, hats 2)
at+a     1
a+l      1
l+y      1
y+s      1
s+t      1
```

`cat` wins at 11. The vocabulary now holds a piece that is a whole word, and nobody put it there — it arrived because the letters kept turning up together.

Merges 3, 4 and 5. The same procedure, three more times:

```
merge 3   h + at    = hat     at 7
merge 4   cat + s   = cats    at 3
merge 5   hat + s   = hats    at 2
```

Five merges. Stop there, and the vocabulary is the seven letters plus the five merged symbols — twelve entries:

```
a  c  h  l  s  t  y  at  cat  hat  cats  hats
```

What the Merges Bought

Tokenise the five words with that vocabulary by replaying the merges in order:

```
word       tokens                    how many
cat        cat                            1
cats       cats                           1
hat        hat                            1
hats       hats                           1
catalyst   cat a l y s t                  6
```

Four of the five words are now one token each, and the fifth is six. Nothing about "catalyst" is harder than "cats". It is longer, and more to the point it is *rarer*, so none of the five merges was spent on it. The merges went where the corpus was dense.

This is the single most useful fact in the chapter, and it is now a thing you have watched happen rather than a thing you have been told:

> ***A token is not a unit of meaning or a unit of length. It is a unit of frequency in somebody else's corpus.***

Most surprises about token counts for the rest of your career follow from that sentence. Your short question cost more than somebody else's long one because theirs was made of pieces the corpus had seen a great deal and yours was not.



The merge list is part of the model, fixed when the model was built, exactly like the embedding table of Chapter 9. As a caller you do not choose it and cannot change it, and two models with different vocabularies can cut the same sentence into different numbers of pieces — can, not must: step 8 of the lab builds a different vocabulary that happens to give the same count. A token count is therefore a fact about a *particular* model, never about your text on its own.

The Space Belongs to the Word

One convention catches everybody once, so meet it now.

Real tokenisers attach the space *before* a word to the word, rather than treating it as a separate piece. So in the sentence "the cat sat", the pieces are "the", " cat", " sat" — the two that follow a space carry it, and the one at the start does not. That is why Chapter 9's worked example listed the vocabulary as `the, cat, sat, mat, on`: those stand for the pieces as a tokeniser would hold them, printed without their leading spaces so the table would read.

Two consequences, and the second one costs people real money.

"cat" and " cat" are different tokens. They have different rows, different indexes, and there is nothing that tells the model they are related other than the fact that they usually turn up in similar places. A word at the start of a line and the same word mid-sentence are genuinely different inputs.

A trailing space changes everything after it. If your instructions end with a space where the model expects to continue, the next piece can no longer carry its own leading space, and the model has to produce a different token than the one it would otherwise have produced. This is a real source of "why did the output format change when I only edited whitespace".



When a prompt behaves strangely after an edit that "changed nothing", check the whitespace at the end of it first. It is the cheapest thing to check and it is wrong surprisingly often.

Input Tokens and Output Tokens

Your bill and your waiting both have two halves, and the halves are counted separately.

Input tokens are everything you send on the call: the instructions, any material your code attached, and the request itself — all of Chapter 2's context, counted as pieces. **Output tokens** are what comes back.

They are priced differently, and output costs several times input, which Chapter 1 worked through with made-up rates. What Chapter 1 could not say, because the word had not been given to you, is what the unit is. It is this one. Not words, not characters, not requests.

They also behave differently in time, and Chapter 9 is why:

	Input tokens	Output tokens
What they are	Everything you send	Everything that comes back
What they cost	The cheaper rate	Several times more
What they do to the wait	Set how long before the first word appears	Set how long the rest takes, one pass each
Who controls them	You, exactly	You, by capping it or asking for less

The last row is the one worth sitting with. You can count your input before you send it, to the token. You cannot count your output in advance — you can cap it, and you can ask for brevity, but within that cap the model decides when it stops. Chapter 13 is where stop conditions and length limits live, Chapter 15 is why the two halves cost such different amounts of time from the inside, and Chapter 53 is what to do about the bill.



A long reply is the expensive half and the slow half at once. An instruction that produces six paragraphs where one would do is not a style problem; it is a cost and latency problem with a style-shaped cause.

Why Code and Other Languages Cost More

Both follow from the merge procedure, and once you have watched it run neither is surprising.

Code. The merges were bought with a corpus that is mostly prose, so the pieces that got seats are mostly prose. Code then pays in three ways at once. Its punctuation is dense, and each mark that did not earn a merge is a token on its own. Its indentation is runs of spaces, which are often several tokens of nothing. And its identifiers are made-up compounds — `get_user_by_id`, `maxRetryCount` — that no corpus has seen often enough to merge, so each one is cut into several pieces.

Other languages. English dominated the corpora these vocabularies were built from, so the merges were spent on English. A language written in another script often has no merged pieces at all, and falls back towards that ceiling of one token per byte — a piece per character where merges joined that character's bytes, several per character where they did not, because a character outside the common range is itself stored as more than one byte. The same meaning, in the same number of characters, can cost several times more than it does in English.



How much more is a dated detail and a vendor-specific one. It moves whenever a model ships a new vocabulary, and newer vocabularies have generally narrowed the gap. The *reason* for the gap — merges are spent where the corpus is dense — does not move, and neither does the advice: if you serve a language other than English, measure its token cost yourself before you price anything.

Both cases are the same case. You are not being charged for difficulty. You are being charged for how well your text matches the corpus somebody else compressed.

Why It Cannot Count the Letters in a Word

Here is the one everybody meets. Ask a model how many times a letter appears in a word and it will often get it wrong, with complete confidence, while being able to write you a working program that gets it right.

Chapter 9 has already given you the answer, so put the two halves together. The word arrives as one index, or two, or three. That index is looked up in the embedding table, and from there the arithmetic works on the vector and never on the token. **A token's spelling is not in the input.** Nothing in the model's input says that this piece is made of ten characters of which three are the same, because the piece is a seat number and the seat number is not spelled. Some of the pieces may be single letters — `catalyst` above came out as five of them after `cat` — but even then it is the seat number that arrives, not the letter.

So the model is not counting badly. It is answering a question about something it was not given, from whatever it absorbed about that word's spelling during training — which is real, but is recall rather than counting, and recall of a thing nobody was being tested on.

Two things follow that are worth saying out loud in an interview.

It is not a bug and a better model does not necessarily fix it. It is a consequence of the input representation. Models have got better at these questions without gaining the ability to see letters; the representation has not changed.

The fix is not to ask harder. If the answer has to be right, do not ask the model to count — have it write the code that counts, or do the counting in your own code. This is one instance of a pattern that recurs through this book: a check that code can do belongs in code.

Chapter 16 is where this sits alongside the rest of the model's characteristic failures, including what tokenisation does to arithmetic and to digits.

Counting Before You Send

Two ways, and you need both.

The rough one, for deciding things. English prose runs about four characters to the token, or about three-quarters of a word. So:

```
1,000 tokens  ~  750 words  ~  4,000 characters of prose
```

That is enough to answer "will this document fit", "roughly what will this feature cost", and "is this instruction block worth trimming". It is also enough to spot an error of the kind that matters — a number ten times too big.

The exact one, for anything you are billed on. Count with the tokeniser belonging to the model you are calling. That program — the one that cuts text into tokens and maps each to its index, using one model's vocabulary and merge list — is its **tokeniser**. Providers publish theirs, or an endpoint that counts for you, and the count is a function of the vocabulary and the merge list, so a count from the wrong model does not carry. Your reply also reports what was actually used, which is the figure to trust and the figure to log.



Do not estimate the input to a feature that runs ten thousand times a day. The rule of thumb is for sizing a decision, not for sizing a bill. The gap between four characters per token and the truth for *your* text — which may be full of code, names, or another language — is exactly the gap that makes a feature cost three times what you budgeted.



"About four characters" is a current convention and a dated one. It is a property of today's English-heavy vocabularies, not a law, and a model with a larger vocabulary cuts the same prose into fewer pieces. Measure it for your own text and your own model rather than carrying the number around.

Lab: Build a Vocabulary by Hand

Pen and paper for the first half, then about thirty lines of Python. Thirty minutes, and there is a key for every step.

The corpus, five words with their counts:

```
run      6
runs     4
sun      5
suns     1
runner   2
```

Step 1. Write down the alphabet — every distinct character that occurs — and say how many entries the vocabulary starts with.

Expected result: e, n, r, s, u. Five entries. Note that `t`, `a` and the rest of the alphabet are *not* in it: the vocabulary is built from this corpus and nothing else.

Step 2. Split every word into characters and count every adjacent pair, weighting each pair by how often its word occurs. Which pair wins, and with what count?

Expected result: `un` at 18. It occurs once in each of the five words, so it scores $6+4+5+1+2 = 18$. The full table, which is worth writing out because step 3 depends on it:

```
un  18     ru  12     su  6     ns  5
er   2     ne   2     nn  2
```

Step 3. Merge `un` into one symbol, rewrite the corpus, and count the pairs again. Which pair wins now, and why is its count not 18?

Expected result: `run` at 12, from `r + un`. The count is 12 and not 18 because `ru` only occurred in the three words beginning with r — run (6), runs (4) and runner (2). Merging changes which pairs exist, so every count has to be taken again from the new corpus. The new table:

```
run  12     sun  6     uns  5     er  2     ne  2     unn  2
```

Step 4. Carry on to five merges in total. Write down the five merges in order.

Expected result:

```
merge 1   u + n     = un      at 18
merge 2   r + un    = run     at 12
merge 3   s + un    = sun     at 6
merge 4   run + s   = runs    at 4
merge 5   ???
```

Merge 5 is the interesting one, and it is step 5.

Step 5. At merge 5 the remaining pairs are `er` at 2, `ne` at 2 and `runn` at 2, with `suns` at 1. Three pairs are tied at the top. What does the procedure as stated tell you to do?

Expected result: **nothing — the procedure does not say.** "Merge the commonest pair" has no answer when three are equally common, and this is not a hole in the explanation; it is a hole in the idea, which each implementation settles in its own way — often by accident rather than by design, taking whatever order a sort or a dictionary happens to give. Different settlements give different vocabularies from the same corpus. For the rest of this lab, break ties alphabetically, so merge 5 is `e + r = er`.

Step 6. With those five merges, tokenise all five words. How many tokens is each?

Expected result:

```
run      run            1
runs     runs           1
sun      sun            1
suns     sun s          2
runner   run n er       3
```

suns and runs are the same shape and differ only in how often they occurred — four times against once — and that alone is why one is a single token and the other is two.

Step 7. Now the program. In a new empty folder, save this as `merges.py`:

```
from collections import Counter

corpus = {"run": 6, "runs": 4, "sun": 5, "suns": 1, "runner": 2}
words = {tuple(w): n for w, n in corpus.items()}

def pair_counts(words):
    counts = Counter()
    for word, n in words.items():
        for pair in zip(word, word[1:]):
            counts[pair] += n
    return counts

def apply_merge(words, a, b):
    out = {}
    for word, n in words.items():
        new, i = [], 0
        while i < len(word):
            if i + 1 < len(word) and word[i] == a and word[i + 1] == b:
                new.append(a + b)
                i += 2
            else:
                new.append(word[i])
                i += 1
        out[tuple(new)] = n
    return out

for step in range(1, 6):
    counts = pair_counts(words)
    # highest count first, then alphabetical: the tie-break of step 5
    (a, b), n = sorted(counts.items(), key=lambda kv: (-kv[1], kv[0]))[0]
    print(f"merge {step}: {a} + {b} = {a + b}  at {n}")
    words = apply_merge(words, a, b)
```

```
for word, n in words.items():
    print(f"  {''.join(word):<8} {' '.join(word):<12} {len(word)} token(s)")
```

Expected result: exactly this, matching your steps 4, 5 and 6 line for line.

```
merge 1: u + n = un  at 18
merge 2: r + un = run  at 12
merge 3: s + un = sun  at 6
merge 4: run + s = runs  at 4
merge 5: e + r = er  at 2
  run      run          1 token(s)
  runs     runs         1 token(s)
  sun      sun          1 token(s)
  suns     sun s        2 token(s)
  runner   run n er     3 token(s)
```

Step 8. Change the tie-break and watch the vocabulary change. Replace the sort key with `key=lambda kv: (kv[1], kv[0]), reverse=True`, which still takes the commonest pair but settles ties the other way round. Re-run. Which merge differs, and what happens to `runner`?

Expected result: merges 1 to 4 are identical, because nothing was tied. Merge 5 becomes `run + n = runn` instead of `e + r = er`, and `runner` comes out as `runn e r` rather than `run n er` — still 3 tokens, but different pieces, from the same corpus and the same rule. **The tie-break is not part of the idea and it changes the vocabulary**, which is step 5's point made by a program rather than by an argument.

Step 9. Now make a word commoner and watch a different word get dearer. Change `suns` from 1 to 7, leave everything else alone, re-run with the original tie-break. What is `runner` now, and why?

Expected result: **four tokens**, up from three, in a word you did not touch.

```
merge 1: u + n = un  at 24
merge 2: r + un = run  at 12
merge 3: s + un = sun  at 12
merge 4: sun + s = suns  at 7
merge 5: run + s = runs  at 4
  run      run          1 token(s)
  runs     runs         1 token(s)
  sun      sun          1 token(s)
  suns     suns         1 token(s)
  runner   run n e r    4 token(s)
```

`suns` is now common enough to win merge 4, which pushes `runs` to merge 5 and leaves no

merge at all for `er`. There are still five merges; they were simply spent elsewhere. **At a fixed vocabulary size the merges are zero-sum**, so every piece of text that is well compressed is well compressed at the expense of some other text — which is "Why Code and Other Languages Cost More" in one experiment, and the reason your language, your code and your identifiers cost what they do.

Step 10. Estimate, with the rule of thumb, what it would cost to send the program in step 7 to a model as input.

Expected result: the program is about 1,100 characters, so about 275 tokens at four characters each. Then the part that matters: it is **code**, so expect the real count to be well above that — dense punctuation, runs of indentation and made-up identifiers like `pair_counts` and `apply_merge` are exactly what the merges were not spent on. The rule of thumb sized the decision; only the tokeniser of the model you are calling can size the bill.

Summary, Key Terms, and Review Questions

Summary

A **token** is one entry in the model's vocabulary, and the pieces are subwords rather than words because words make the embedding table and the output step enormous while still leaving text they cannot represent, and single characters make the sequence several times longer, at one forward pass each. The base vocabulary is the 256 bytes, which is why nothing is ever unrepresentable and why the worst case is one token per byte. **Byte-pair encoding** builds the vocabulary by starting from the 256 single bytes, counting every adjacent pair across a corpus, merging the commonest, and repeating; the ordered **merge list** is kept and replayed on every piece of text afterwards. The result is that common words become one token and rare ones are spelled out of several, so **a token is a unit of frequency in somebody else's corpus**, not a unit of meaning or of length. The leading space belongs to the word, so "cat" and " cat" are different tokens. **Input tokens** and **output tokens** are counted and priced separately, output costs several times more and sets how long the rest of the reply takes, and only the input can be counted before you send. Code and non-English text cost more because the merges were spent on English prose. A model has no way to count the letters in a word, because a token's spelling is not in its input; a right answer is recall of the spelling rather than counting. Count roughly at four characters to the token, and exactly with the tokeniser of the model you are calling.

Key Terms

Term	Meaning
Token	One entry in a model's vocabulary — usually a piece text is cut into. Beyond the single bytes, which are always there, which pieces earned an entry is a matter of frequency in the corpus the vocabulary was built from — not of meaning or length
Subword	A token that is part of a word rather than a whole one. What makes it possible to represent a word the vocabulary has never seen
Tokeniser	The program that cuts text into tokens and maps them to indexes, using one model's vocabulary and merge list. A count does not carry from one model's tokeniser to another's
Byte-pair encoding	Building a vocabulary by starting from the 256 single bytes and repeatedly merging the commonest adjacent pair in a corpus until the vocabulary is the size wanted. Starting from bytes is what makes any text at all representable
Merge list	The merges, in the order they were made. Kept with the model and replayed in that order to tokenise any later text
Input tokens	Everything you send on a call — instructions, attached material and the request. Countable exactly before you send
Output tokens	Everything that comes back. Priced several times higher than input, one forward pass each, and not knowable in advance

Worth Memorising

Item	Value or formula
Rule of thumb, English prose	1 token ≈ 4 characters ≈ ¾ of a word; 1,000 tokens ≈ 750 words
What a token really measures	Frequency in the corpus the vocabulary was built from, not meaning or length
The merge step	Merge the commonest adjacent pair; recount; repeat
Why a word is one token	It was common enough for a merge to be spent on it
Which half is dear	Output, at several times input, and one forward pass per token
What a token count is a fact about	One particular model. A count does not carry across vocabularies
The worst case	One token per byte — the most byte-pair encoding can ever charge you

Review Questions

1. Why not just give the model a vocabulary of words? Give both reasons.
2. A corpus has `low` 8 times and `lowest` 2 times. On the first merge, what is the count for the pair `lo`, and why?
3. Your colleague says a document is "about 3,000 tokens" because they counted 3,000 words. What do you tell them?
4. Why does the same sentence, translated, cost several times as many tokens in one language as in another?
5. A model tells you "strawberry" has two letter R's. Explain what went wrong without using the word "bug".
6. You trim 200 words of instructions from a prompt that runs ten thousand times a day. What do you need to know before claiming a saving?

Review Answers

1. First, size: the embedding table holds one row per entry and the output step produces one score per entry, so a 500,000-word vocabulary at a width of 4,096 is over two billion parameters in the lookup table alone, and the same again wherever the output matrix is separate rather than the embedding table read backwards. Second, and worse, coverage: a word not in the list has no row at all, so every name, typo, new product and unplanned language would arrive as one shared "unknown" entry, with the word itself thrown away. Subwords fix the second completely — all 256 single bytes are in the vocabulary from the start, so any text at all can be represented — and the first well enough.

2. Ten. The pair is counted once per occurrence of each word, weighted by that word's count, so `lo` scores 8 from `low` plus 2 from `lowest`. The thing to get right is that you are counting across the corpus with weights, not counting distinct words.
3. That words and tokens are different units, and the direction of the error is predictable: English prose runs about three-quarters of a word to the token, so 3,000 words is nearer 4,000 tokens. Then that the rule of thumb is only for sizing a decision — if the document is going into a feature they are billed for, count it with the tokeniser of the model they are calling, because the number is a property of that model's vocabulary and not of the text.
4. Because the merges were spent on whichever language dominated the corpus the vocabulary was built from, which has been English. A language in another script may have no merged pieces at all and fall back to one or more tokens per character. Nothing about the language is harder; the compression simply was not bought for it.
5. A token's spelling is not in the model's input. The word arrives as one index, or two, and from the lookup onwards the arithmetic works on a vector and never on the characters — so it is answering a question about something it was never given, from what it absorbed about the word's spelling in training. That is recall, not counting. If the answer must be right, have it write code that counts, or count in your own code.
6. Three things. How many tokens those 200 words actually are, for the model you call, rather than 267 by the rule of thumb — and if the text is code or identifiers, the gap can be large. That input is the cheaper half, so the saving is smaller than the same cut to the output would be. And whether quality held: a shorter instruction is only a saving if the feature still works, which is a measurement and not a reading.

> **You understand this when** you can explain why "cats" and "catalyst" cost one token and six, without mentioning length or difficulty.

CHAPTER 11

Attention, Worked by Hand

This is the chapter the rest of the book has been pointing at. Chapter 9 called a layer a box of the right shape and promised to open it; this opens the half of it that everybody asks about in interviews. Everything here is arithmetic at the level of Chapter 8 — dot products, softmax, and multiplying vectors by numbers and adding them up — assembled in one particular order. By the end you will have worked the whole formula on a five-word sentence with four-dimensional vectors, and you will be able to say what every symbol in it stands for.

The problem. Every explanation says the model "pays attention to the relevant words". I want to know what is actually computed.

The question. What arithmetic turns a list of vectors into a list of vectors in which each position has drawn on the others, and where exactly does "relevant" enter?

What Each Position Needs

Start with the difficulty rather than the solution.

Chapter 9 left you with one vector per position, each one standing for a single token and nothing else. The vector at position 5 knows what token sits at position 5. It knows nothing about positions 1 to 4.

That is useless on its own. Consider the five tokens

```
the   cat   sat   on   it
 1     2     3     4    5
```

To do anything sensible with position 5, the model has to bring in something from the other positions. And the difficulty is that **which other positions matter depends on the content**, not on anything fixed. Sometimes the useful information is one token back, sometimes forty. A fixed rule — always look at the previous three — cannot be right in general, because the useful distance changes with the sentence.

So the thing we need is a procedure that, for each position, works out *for itself* how much of each other position to take, and then takes that much. **Attention** is that procedure, and it is built from three ideas you already have.

- Each position produces a request saying what it is looking for.
- Each position produces a label saying what it offers.
- Scoring every request against every label says who should take how much from whom.

Request, label and content are the three views, and the next section gives them their names.

Three Views of the Same Vector: Query, Key, Value

Every position's vector is turned into three different vectors, by multiplying it by three different matrices. The matrices are parameters — fitted in training, fixed while you use the model, exactly as Chapter 9 said.

View	Written	Read it as
Query	q	What this position is looking for
Key	k	What this position offers to others looking
Value	v	What this position actually hands over if chosen

The three matrices are written W_Q, W_K and W_V, and the arithmetic is Chapter 8's matrix-times-vector: each output slot is a dot product of one row of the matrix with the input vector.

Here is the whole example, and everything in this chapter is computed from it. Five tokens, width four:

```
pos  token   x
 1   the     (1, 2, 2, 0)
 2   cat     (0, 2, 1, 2)
 3   sat     (1, 2, 0, 0)
 4   on      (0, 1, 1, 1)
 5   it      (2, 0, 2, 0)
```

The three matrices, kept to zeroes and ones so the arithmetic stays visible:

```
W_Q = [ 1 0 0 0 ]    W_K = [ 0 0 0 1 ]    W_V = [ 1 0 0 0 ]
      [ 0 1 1 0 ]          [ 0 1 0 0 ]          [ 0 1 0 1 ]
      [ 0 1 0 0 ]          [ 1 0 0 0 ]          [ 0 1 0 0 ]
      [ 0 0 0 1 ]          [ 0 1 1 0 ]          [ 0 0 1 0 ]
```

Work one out by hand before trusting the table. Position 5 is $x = (2, 0, 2, 0)$, and its query is $W_Q x$ — row by row:

```
row 1   (1,0,0,0) . (2,0,2,0) = 2
row 2   (0,1,1,0) . (2,0,2,0) = 0 + 2 = 2
row 3   (0,1,0,0) . (2,0,2,0) = 0
row 4   (0,0,0,1) . (2,0,2,0) = 0

q(it) = (2, 2, 0, 0)
```

All fifteen vectors, the same way:

```
pos  token   q              k              v
 1   the     (1, 4, 2, 0)   (0, 2, 1, 4)   (1, 2, 2, 2)
 2   cat     (0, 3, 2, 2)   (2, 2, 0, 3)   (0, 4, 2, 1)
 3   sat     (1, 2, 2, 0)   (0, 2, 1, 2)   (1, 2, 2, 0)
 4   on      (0, 2, 1, 1)   (1, 1, 0, 2)   (0, 2, 1, 1)
 5   it      (2, 2, 0, 0)   (0, 0, 2, 2)   (2, 0, 0, 2)
```



The three matrices above were chosen by hand so that the arithmetic comes out legible and the result is worth looking at. In a real model they are fitted, nobody can say in advance where the weight will land, and a matrix that merely selects and adds slots the way these do would be unusually simple. What is real here is the procedure, not the numbers.

Why three views and not one? Because what a position is looking for and what it offers are different questions. "it" is looking for a thing it might refer to; what "it" offers to other positions is something else entirely. With one vector doing both jobs, the two uses would pull it in different directions. And the value is separate again, because *how much to take* and *what to take* are different decisions — which is the hinge the rest of the chapter turns on.

The Scores: Every Query Against Every Key

Now the comparison. The score of position i attending to position j is the dot product of i's query with j's key.

Chapter 8 built the dot product as agreement: large when both vectors are large in the same slots. That is exactly what is wanted here. A query and a key that agree slot by slot produce a high score, which is this procedure's entire definition of "relevant".

Work one by hand. Position 5's query against position 2's key:

```
q(it) . k(cat) = (2,2,0,0) . (2,2,0,3)
               = 2x2 + 2x2 + 0x0 + 0x3
               = 4 + 4 + 0 + 0
               = 8
```

Doing that for all 25 pairs gives a grid — queries down the side, keys along the top:

```
          k:the  k:cat  k:sat  k:on  k:it
q:the       10     10     10     5     4
q:cat       16     12     12     7     8
q:sat        6      6      6     3     4
q:on         9      7      7     4     4
q:it         4      8      4     4     0
```

In matrix terms this grid is $QK^\top$ — the matrix of queries times the matrix of keys with its rows and columns swapped, which is what the small $\top$ means, so that every row meets every column. Chapter 8's shape rule says why it comes out square:

```
(5 x 4) times (4 x 5)  gives  (5 x 5)
      ^         ^
      these must match
```

That grid being square in the number of tokens is what decides the cost of a long input, and the last section of this chapter is about what it costs.



Read the grid one row at a time. A row is one position asking "how much do I want from each of you?" — and it is rows, not columns, that get turned into weights. The columns have no meaning of their own.

Why Divide by the Square Root

Before the scores become weights, every one of them is divided by $\sqrt{d_k}$, the square root of the key's width — the number of slots in it, not its length in Chapter 8's sense. Here the keys have four slots, so the divisor is $\sqrt{4} = 2$:

```
          k:the  k:cat  k:sat  k:on  k:it
q:it        2      4      2      2     0
```

This looks like fussiness and it is not. Here is the same row turned into weights both ways — first without the division, then with it. Softmax is Chapter 8's: exponentiate each, divide by the total.

```
WITHOUT the division, scores (4, 8, 4, 4, 0)

  e^4 = 54.598   e^8 = 2980.958   e^0 = 1.000
  total = 54.598+2980.958+54.598+54.598+1.000 = 3145.752

  weights   0.017   0.948   0.017   0.017   0.000


WITH the division, scores (2, 4, 2, 2, 0)

  e^2 = 7.389    e^4 = 54.598     e^0 = 1.000
  total = 7.389 + 54.598 + 7.389 + 7.389 + 1.000 = 77.765

  weights   0.095   0.702   0.095   0.095   0.013
```

Undivided, position 2 takes 95 per cent and everything else is noise: attention has stopped blending and started picking. Divided, position 2 still wins clearly at 0.70 while the others keep a real share.

And the reason this matters more as models get bigger is the dot product itself. A dot product is a sum of d_k terms, so as the vectors get wider the sum has more terms in it. It does not grow in proportion to d_k, though, because the slot-by-slot products do not all pull the same way — some are positive, some negative, and they partly cancel. A sum of many terms that partly cancel typically ends up about $\sqrt{d_k}$ times the size of one of them, which is why the divisor is the square root and not the width itself. **That is the answer to "why the square root" rather than just "why divide".** Chapter 8's softmax property — *gaps get exaggerated* — then does the rest: wider vectors give larger scores, larger scores give larger gaps after exponentiating, and the weights collapse onto a single position. Dividing by $\sqrt{d_k}$ pulls the scores back to roughly the same size whatever the width, so the same design works at 64 slots and at 4,096.



The weights in the undivided block add to 0.999 rather than 1 as printed, because the fifth is 0.0003 and rounds to 0.000. The arithmetic is exact; the printing is to three places. This is Chapter 8's rounding point, and it is worth noticing here rather than being puzzled by it later.

Masking: No Looking Ahead

One more thing happens before softmax, and it is there because of how these models are used.

Chapter 9 showed the model producing one token at a time, each pass over an input one token longer. Position 3's output must not depend on anything after it, because the same arithmetic has to give the same answer when positions 4 and 5 genuinely do not exist yet — which is the situation every time the model produces a token. A position that used what follows it would behave one way on a finished sentence and another way while the sentence was being written.

The **causal mask** enforces that. Before softmax, every score where the key's position is later than the query's is set to minus infinity. Exponentiating minus infinity gives zero, so those positions get exactly zero weight and the rest share the whole of it.

```
          k:the  k:cat  k:sat  k:on  k:it
q:the       5      .      .      .     .
q:cat       8      6      .      .     .
q:sat       3      3      3      .     .
q:on      4.5    3.5    3.5      2     .
q:it        2      4      2      2     0
```

The dots are the masked entries. Note that the bottom row — the one this chapter works in full — is unaffected, because position 5 is last and has nothing ahead of it.



This triangle is what "decoder" means in practice. A model that is allowed to see the whole input at once, in both directions, is a different design used for different jobs. Everything in this book is the triangle.

Softmax, and the Weighted Blend

Take the last row, masked and scaled: $(2, 4, 2, 2, 0)$. Softmax turns it into **attention weights** — one per position, all positive, adding to 1.

```
step 1   exponentiate
         e^2 = 7.389   e^4 = 54.598   e^2 = 7.389
         e^2 = 7.389   e^0 = 1.000

step 2   add them
         7.389 + 54.598 + 7.389 + 7.389 + 1.000 = 77.765

step 3   divide each by the total
```

```
the   7.389 / 77.765 = 0.095
cat  54.598 / 77.765 = 0.702
sat   7.389 / 77.765 = 0.095
on    7.389 / 77.765 = 0.095
it    1.000 / 77.765 = 0.013
                       -----
                       1.000
```

Position 5 has decided to take about seven tenths of its information from position 2. Nothing told it to. It fell out of a dot product between two vectors that W_Q and W_K arranged — here by hand, in a real model by training.

Now spend the weights. The output for position 5 is the weighted sum of the *value* vectors — each value multiplied by its weight, all added up:

```
the   0.095 x (1, 2, 2, 2)  =  (0.095, 0.190, 0.190, 0.190)
cat   0.702 x (0, 4, 2, 1)  =  (0.000, 2.808, 1.404, 0.702)
sat   0.095 x (1, 2, 2, 0)  =  (0.095, 0.190, 0.190, 0.000)
on    0.095 x (0, 2, 1, 1)  =  (0.000, 0.190, 0.095, 0.095)
it    0.013 x (2, 0, 0, 2)  =  (0.026, 0.000, 0.000, 0.026)
                               --------------------------
                               (0.216, 3.378, 1.879, 1.013)
```

Add each column and check it yourself — that is the point of the layout. The result, $(0.216,\ 3.378,\ 1.879,\ 1.013)$, is one vector of width four, the same shape as the vector that went in, which is what makes a stack of these possible at all.

And look at what it is made of: mostly v for "cat", with a little of everything else. **Position 5 went in standing for "it" and comes out standing largely for "cat".** That is the whole trick, and there is no step in it that is not arithmetic.



It is tempting to say the model "resolved the pronoun". It did not. It computed a weighted average in which one term happened to dominate, because of matrices chosen — here by me, in a real model by training — to make that happen. Attention weights are often described as showing what a model "looked at", and they are evidence about one step of arithmetic rather than an account of why an answer came out as it did. Chapter 16 is where the general version of that caution lives.

The Formula, All at Once

Everything above, in one line:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Read it with Chapter 8's four steps, innermost bracket outward.

Piece	What it is	Which section
Q, K, V	The queries, keys and values, one row per position	Three views
$QK^\top$	Every query dotted with every key: the square grid	The scores
$\sqrt{d_k}$	The square root of the key width, here $\sqrt{4} = 2$	Why divide
softmax	Each row turned into positive weights adding to 1	Softmax
$\cdots V$	The weights spent on the value vectors	The blend

The mask is not in the formula. It is applied to the grid before softmax, and it is left out of the written form by convention rather than because it is optional.

That is the entire mechanism, and it is worth noticing how little of it there is: one matrix multiplication for each of the three views, a fourth to build the grid, a division, a softmax, and a fifth to spend the weights. Nothing in it is harder than Chapter 8.

This Formula Cannot Tell the Order

Chapter 9 said position has to be supplied separately, and promised the reason here. This is it — and it is checkable with the numbers already on the page.

Swap two tokens in the input — "cat" and "sat", positions 2 and 3 — and recompute position 5's row. Nothing else changes.

```
original    the  cat  sat  on  it
  scores     4    8    4   4   0
  weights   0.095 0.702 0.095 0.095 0.013
  output   (0.216, 3.378, 1.879, 1.013)

swapped     the  sat  cat  on  it
  scores     4    4    8   4   0
  weights   0.095 0.095 0.702 0.095 0.013
  output   (0.216, 3.378, 1.879, 1.013)
```

The weight of 0.702 followed "cat" to its new seat, exactly as it should. **And the output is identical.** Of course it is: the output is a sum, and addition does not care what order you add in.

So "the cat sat on it" and "the sat cat on it" produce the same vector at position 5. Nothing in this formula puts the order back — and *this formula* is the careful phrasing, because the mask is not in it. The mask is defined by position, so masked attention is not order-blind; swap positions 2 and 3 and the output at position 2 changes, since it blends a different pair. The order of your words carries no information in it at all.

That is why position has to be supplied separately. Something must put "I am the second token" into the arithmetic, because the shape of the computation never will. There are two families in use. One adds a position-dependent vector to each token's embedding before the stack begins, so the vector for "cat" at position 2 differs from the vector for "cat" at position 7. The other rotates each query and key by an amount that depends on its position, so that the dot product between them depends on how far apart they are. Both are called **positional encoding**, and between them they account for nearly everything in current models. (A third approach adds a position-dependent amount to the scores themselves, just before softmax.) All of them exist for this one reason.

And it takes two dot products to see it work. Take the first family and add to each token's vector a tiny marker of where it sits — $(0, 0, 0, 0)$ at position 1, $(0, 0, 0, 1)$ at position 2, $(0, 0, 0, 2)$ at position 3, and so on. Now "cat" is no longer one vector. At position 2 it is $(0, 2, 1, 2) + (0, 0, 0, 1) = (0, 2, 1, 3)$ and at position 3 it is $(0, 2, 1, 2) + (0, 0, 0, 2) = (0, 2, 1, 4)$. Push both through W_K, and push "it" at position 5 through W_Q:

```
cat at position 2   k = (3, 2, 0, 3)
cat at position 3   k = (4, 2, 0, 3)
it  at position 5   q = (2, 2, 0, 4)


cat at 2   2x3 + 2x2 + 0x0 + 4x3 = 6 + 4 + 0 + 12 = 22
cat at 3   2x4 + 2x2 + 0x0 + 4x3 = 8 + 4 + 0 + 12 = 24
```

Without the markers both of those scores were 8, which is why the swap changed nothing. With them the same word scores 22 in one seat and 24 in another, the two rows of weights no longer match, and the sum they feed comes out different. **Position is now in the arithmetic**, and it got there before the dot product rather than inside it.



Those markers are made up and deliberately crude — a real scheme has to keep working at position 4,000 as well as position 2, and must not swamp the token's own meaning with a number that grows without limit. What is real here is the shape of the fix: something position-dependent enters the vectors, and the scores stop being blind.



This is the answer to "why do models need positional encodings at all", and the demonstration above is the answer to give. It takes thirty seconds: swap two words, get the same answer, therefore the formula is blind to order. **Swap two words that are not the last one.** The final position supplies the query, so changing the token there changes what is being asked and the output moves for a reason that has nothing to do with order — which would lose you the argument rather than win it.

Several Heads

One more structural piece, and it is simpler than it sounds.

Everything above is one **attention head**: one set of W_Q, W_K, W_V, one grid, one set of weights. A real layer runs several heads at once, and they divide the width between them rather than each taking all of it. With a width of 4 and two heads, each head works in two dimensions, so $d_k = 2$ and the divisor becomes $\sqrt{2}$, which is 1.414 to three places. The scaled column below divides by the square root itself rather than by the rounded figure, so your own arithmetic may differ in the last digit.

Here are two heads over the same five tokens. They differ only in W_K, so you can see the effect of one change. Head 1's keys are $(x_4,\ x_2 + x_4)$; head 2's are $(x_4,\ x_1)$. Both queries are $(x_4,\ x_3)$, so q for position 5 is $(0, 2)$ in both.

HEAD 1	k	q.k	scaled	weight
the	(0, 2)	4	2.828	0.050
cat	(2, 4)	8	5.657	0.847
sat	(0, 2)	4	2.828	0.050
on	(1, 2)	4	2.828	0.050
it	(0, 0)	0	0.000	0.003

HEAD 2	k	q.k	scaled	weight
the	(0, 1)	2	1.414	0.152
cat	(2, 0)	0	0.000	0.037
sat	(0, 1)	2	1.414	0.152
on	(1, 0)	0	0.000	0.037
it	(0, 2)	4	2.828	0.623

The two heads attend to different places. Head 1 puts 0.847 of its weight on "cat"; head 2 puts most of its on position 5 itself. Neither is right or wrong — they are two different questions being asked of the same five tokens, and that is what the extra heads buy. One head has one set of weights and therefore one opinion about what is relevant; a layer with sixteen heads has sixteen, computed independently and in parallel.

Each head produces its own output vector, of its own narrower width, by spending its weights on its own values. Give head 1 the value projection (x_1, x_2) and head 2 (x_3, x_4), and the blends are:

```
HEAD 1    weight    v       contribution
  the      0.050  (1, 2)   (0.050, 0.100)
  cat      0.847  (0, 2)   (0.000, 1.694)
  sat      0.050  (1, 2)   (0.050, 0.100)
  on       0.050  (0, 1)   (0.000, 0.050)
  it       0.003  (2, 0)   (0.006, 0.000)
                           ----------------
                           (0.106, 1.944)

HEAD 2    weight    v       contribution
  the      0.152  (2, 0)   (0.304, 0.000)
  cat      0.037  (1, 2)   (0.037, 0.074)
  sat      0.152  (0, 0)   (0.000, 0.000)
  on       0.037  (1, 1)   (0.037, 0.037)
  it       0.623  (2, 0)   (1.246, 0.000)
                           ----------------
                           (1.624, 0.111)
```

Two vectors of two slots. Lay them end to end and you have one of four again:

```
concatenated   (0.106, 1.944, 1.624, 0.111)
```

and that is multiplied by one further matrix, W_O, which mixes the heads back together. Take

```
W_O = [ 1 0 0 1 ]      row 1   0.106 + 0.111 = 0.217
      [ 0 1 1 0 ]      row 2   1.944 + 1.624 = 3.568
      [ 1 1 0 0 ]      row 3   0.106 + 1.944 = 2.050
      [ 0 0 1 1 ]      row 4   1.624 + 0.111 = 1.735

output   (0.217, 3.568, 2.050, 1.735)
```

One vector of width four, the same shape that went in. **That last matrix is why the heads do not simply sit in separate lanes**: every row of W_O above draws on both of them, so what leaves the layer is a mixture rather than two answers stapled together.

Nobody tells a head what to specialise in. There is no instruction anywhere saying that head 7 handles pronouns and head 12 handles punctuation. Every head has the same shape and runs the same procedure; the only thing distinguishing them is that they hold different numbers, and those numbers are whatever training left behind. People do go looking

afterwards, and sometimes find a head whose weights land somewhere describable across many inputs — but that is a finding about a particular model, arrived at by measurement, not a design anybody specified. This is the same honesty Chapter 9 required about the dimensions of an embedding, and for the same reason.



Head 2's weights add to 1.001 as printed, for the rounding reason given earlier. And the two heads here share W_Q so that one variable changes at a time; real heads differ in all three matrices.

What It Costs: Big-O at Last

Chapter 8 gave you big-O and promised $O(n^2 d)$ would turn up here. This is where.

First, why every pair at all? It is a fair question: most of those 25 scores in the example are small, and in a 10,000-token input the overwhelming majority of the grid will be near zero. Could the model not compare only the pairs that matter?

It could not, because of the first section of this chapter. Which positions matter depends on the content, and the content is what the scores are being computed to find out. There is nothing to prune on until after you have the number you were trying to avoid computing. That is the circularity at the heart of the cost: the comparison is exhaustive precisely because the design refuses to decide in advance which comparisons are worth making.

There are designs that compare fewer pairs anyway — by restricting each position to a window, or to a pattern of positions, or by approximating the grid. The first two buy their speed by reinstating exactly the kind of fixed rule this chapter opened by rejecting. All of them are a specialist topic rather than what you will meet in the models this book is about.

Count the work for a single head first. Its grid has n^2 entries, one for every pair of positions, and each entry is a dot product of two vectors of width d_k — that many multiplications and that many additions. With one head taking the whole width, $d_k = d$ and the grid costs about $n^2 d$. With h heads it is the same total, because each of the h grids is only d/h wide and $h \times n^2 \times d/h = n^2 d$. Either way:

$$\text{scores} \;=\; n \times n \times d \;=\; O(n^2 d)$$

Put real numbers in it, at a width of 4,096:

tokens	pairs	multiply-adds
1,000	1,000,000	4,096,000,000
2,000	4,000,000	16,384,000,000
10,000	100,000,000	409,600,000,000

Double the input and the work goes up four times. That is the practical content of the n^2, and three things follow from it that you will meet all through this book.

Long inputs are superlinearly expensive. Chapter 1 told you cost grows with the text you move. This is the part that grows faster than the text: the dot products in the grid, and the grid itself, which at 10,000 tokens is a hundred million numbers *per head, per layer*.

It is a large part of why there is a limit on how much you may send at all. There is a length past which this stops being affordable, and where exactly that length sits is an engineering decision somebody made about memory and money rather than a preference. Chapter 16 names that limit and is about what else happens near it.

And it is why re-reading the whole input on every token would be ruinous. Chapter 9 flagged that a hundred-token reply means a hundred passes over a growing input. Now you can see the size of the problem: doing this grid again from scratch on every pass, when all but the last row and column are unchanged, is the obvious waste. Chapter 15 is where it gets fixed, and what the fix costs in memory.



$O(n^2 d)$ is the attention step. It is not the cost of a whole layer, which has other arithmetic in it that grows differently — Chapter 12 is where the rest of the layer lives. At short inputs the other part dominates; the squared term wins eventually, which is exactly what big-O is for.

Lab: Two More Rows

Pen, paper and a calculator. Thirty minutes, and there is a key for every step. Everything you need is the table from "Three views" — queries, keys and values for all five positions — and the score grid. No code, and nothing to install.

Step 1. Compute $q(\text{sat}) \cdot k(\text{the})$ by hand, slot by slot, and check it against the grid.

Expected result: $q(\text{sat}) = (1, 2, 2, 0)$ and $k(\text{the}) = (0, 2, 1, 4)$, so the terms are $0 + 4 + 2 + 0 = 6$. The grid's row 3, column 1 says 6.

Step 2. Write out row 3 of the grid, apply the causal mask, and divide by $\sqrt{d_k}$. Which entries survive, and what are they?

Expected result: row 3 is $(6, 6, 6, 3, 4)$. Position 3 may see positions 1, 2 and 3 only, so the last two are masked away. Dividing the survivors by 2 gives $(3, 3, 3)$. If you divided before masking you got the same three numbers, which is fine — the order of those two steps does not matter.

Step 3. Apply softmax to those three scores. What do you get, and what does it tell you about what attention is allowed to do?

Expected result: $e^3 = 20.086$ three times, total 60.258, and each weight is $20.086/60.258 = 0.333$.
A perfectly even blend. Three equal scores give three equal weights, which is worth meet-

ing: nothing in this procedure forces it to pick a winner. It takes an even third from each when the scores say so. (The three thirds print as 0.999; the exact value is one third each.)

Step 4. Compute position 3's output vector: the weighted sum of the value vectors for positions 1, 2 and 3.

Expected result:

```
the   0.333 x (1, 2, 2, 2)  =  (0.333, 0.666, 0.666, 0.666)
cat   0.333 x (0, 4, 2, 1)  =  (0.000, 1.332, 0.666, 0.333)
sat   0.333 x (1, 2, 2, 0)  =  (0.333, 0.666, 0.666, 0.000)
                               --------------------------
                               (0.666, 2.664, 1.998, 0.999)
```

Step 5. Now row 2, which is the interesting case for the mask. Write it out, mask it, scale it, and turn it into weights.

Expected result: row 2 is $(16, 12, 12, 7, 8)$, and position 2 may see only positions 1 and 2, so three of the five go. Scaled, the survivors are $(8, 6)$. Then $e^8 = 2980.958$ and $e^6 = 403.429$, total 3384.387, giving weights $(0.881,\ 0.119)$.

Step 6. Compare step 5's weights with step 3's. Both came from the same procedure. Why is one almost a decision and the other an even split?

Expected result: the gap between the scores, not their size. In step 3 all three scores were equal, so the exponentials were equal. In step 5 the two scores differ by 2, and softmax exaggerates gaps — e^8 is about 7.4 times e^6, so one position takes nearly nine tenths. This is the same property that made the division by $\sqrt{d_k}$ necessary, seen from the other side.

Step 7. Without recomputing anything: a sixth token is appended, so n goes from 5 to 6. How many new entries appear in the grid, and how many of the old ones change?

Expected result: the grid goes from 5 × 5 to 6 × 6, so from 25 entries to 36 — **eleven new ones**, which is one new row of 6 and one new column of 5. None of the old 25 changes, because each depends only on its own pair of positions. That observation is the whole idea behind Chapter 15's fix, and you have just derived the reason it is possible.

Step 8. A model has a width of 4,096 and allows 8,000 tokens of input. How many entries are in one attention grid, and how many multiply-adds does building it take?

Expected result: $8{,}000^2 = 64{,}000{,}000$ entries, and $64{,}000{,}000 \times 4{,}096 = 262{,}144{,}000{,}000$ multiply-adds — about 262 billion — and that is one layer's attention scores, all of its heads together, for one forward pass. The heads divide the width between them, so adding heads does not multiply this. Multiply by the layers to feel why long inputs are priced the way they are.

Step 9. Someone shows you a diagram of attention weights and says it proves the model "understood" that "it" refers to "cat". What two things do you say?

Expected result: first, that the weights are a real measurement of one step of arithmetic —

they are not made up, and position 5 genuinely did take 0.702 of its value from position 2. Second, that this is one head in one layer out of dozens of each, that the blend is then combined with every other head's by W_O and passed through the rest of the stack, and that a weight is not a reason. The honest sentence is "this head put most of its weight there", and anything stronger is a claim nobody has the evidence for.

Summary, Key Terms, and Review Questions

Summary

Attention gives every position a way to take information from the others, in amounts it works out for itself. Each position's vector is turned into three — a **query** (what it is looking for), a **key** (what it offers) and a **value** (what it hands over) — by three fitted matrices. Every query is dotted with every key, giving a square grid of scores, $QK^\top$; the dot product is the procedure's whole definition of relevance. The scores are divided by $\sqrt{d_k}$ because a dot product grows with width and softmax exaggerates gaps, so without it the weights collapse onto one position — and the divisor is the *square root* of the width rather than the width itself because the slot-by-slot products partly cancel, so the total grows like $\sqrt{d_k}$ and not like d_k. A **causal mask** sets every score for a later position to minus infinity, which softmax turns into exactly zero weight. Softmax then turns each row into **attention weights** that are positive and add to 1, and the output for that position is the weighted sum of the value vectors — the same width in as out, which is what lets layers stack. The whole of it is $\text{softmax}(QK^\top/\sqrt{d_k})V$. Because the output is a sum, **that formula is blind to word order**, which is why **positional encoding** exists at all. A layer runs several **attention heads** in parallel over slices of the width, each with its own matrices and its own opinion, and a final matrix mixes their outputs. Building the grid costs $O(n^2 d)$: double the input and the work goes up four times.

Key Terms

Term	Meaning
Attention	The step in which each position takes a weighted blend of information from the others, in amounts computed from the content rather than fixed in advance
Query	The vector saying what a position is looking for, made by multiplying its vector by W_Q
Key	The vector saying what a position offers to others, made with W_K. Scored against every query
Value	The vector a position hands over when it is attended to, made with W_V. What the weights are spent on
Attention weights	One row of the grid after scaling, masking and softmax: positive numbers adding to 1, saying how much of each position to take
Causal mask	Setting every score for a later position to minus infinity before softmax, so it gets exactly zero weight. It makes a position's output depend only on itself and what precedes it, so the arithmetic is the same whether or not the rest of the sentence exists
Attention head	One complete set of query, key and value matrices and the weights they produce. A layer runs several in parallel over slices of the width, and a final matrix mixes their outputs
Positional encoding	How a token's position is put into the arithmetic, since the attention formula carries no notion of order. Added to the embedding before the stack, or applied as a rotation to queries and keys, or added to the scores themselves

Worth Memorising

Item	Value or formula
The attention formula	$\text{softmax}(QK^\top/\sqrt{d_k})\,V$
What the three views are for	Query = what I want, key = what I offer, value = what I give
Why divide by $\sqrt{d_k}$	A dot product grows with width; without it softmax saturates and attention stops blending. The *square root*, because the slot products partly cancel, so the sum grows like $\sqrt{d_k}$
The score grid	$n \times n$, one entry per pair of positions
Attention cost	$O(n^2 d)$ — double the tokens, four times the work
What a row of weights is	Positive, adds to exactly 1, one number per visible position
Why positional encoding exists	The formula's output is a sum, so *the formula* is blind to the order of its inputs. Masked attention is not — the mask is positional

Review Questions

1. Walk through attention on a short sentence, naming every step in order.
2. Why is the score divided by the square root of the key width? What goes wrong without it?
3. Why does attention cost grow with the square of the input length, and name two things that follow in practice.
4. What do several heads give you that one does not?
5. Your colleague says the attention weights prove the model linked "it" to "cat". How do you answer?
6. Why can the attention formula not tell "the cat sat on it" from "the sat cat on it"? Why would swapping the *last* word not make the same point?

Review Answers

1. Each position's vector is multiplied by three matrices to give a query, a key and a value. Every query is dotted with every key, giving a square grid of scores — one per pair of positions. Every score is divided by the square root of the key width. Scores for positions later than the query are set to minus infinity, so they will vanish. Softmax turns each row into positive weights adding to 1. The output for a position is the weighted sum of the value vectors, which has the same width as the vector that went in. That is the whole of it, and it is one line: $\text{softmax}(QK^\top/\sqrt{d_k})V$.
2. Because a dot product is a sum over the width, so it grows as the vectors get wider, and softmax exaggerates gaps between large numbers. It is the *square root* of the width because the slot-by-slot products do not all pull the same way — they partly cancel, so a sum of d_k of them ends up about $\sqrt{d_k}$ times the size of one, not d_k times. Without

the division the biggest score runs away with nearly all the weight — in this chapter's example, 0.948 instead of 0.702 — and attention stops being a blend and becomes a hard pick. Dividing by $\sqrt{d_k}$ keeps the scores roughly the same size whatever the width, so one design works at 64 slots and at 4,096.

3. Because every position is compared with every other, so there are n^2 pairs per head, each costing a dot product of the head's width — and since the heads divide the width between them, the whole layer comes to about $n^2 d$ whatever the number of heads. Double the input and the work goes up four times. Two consequences: long inputs cost more than proportionally, which is a large part of why a context limit is a number somebody chose rather than a preference; and recomputing the whole grid for every token of a reply would be ruinous, since all but the newest row and column are unchanged — which is what the key–value cache exists to avoid.
4. Several independent opinions about what is relevant, computed in parallel. One head produces one set of weights, so it can express one view of which positions matter; with sixteen heads there are sixteen, each with its own query, key and value matrices and each free to attend somewhere different. Their outputs are laid end to end and mixed by a further matrix, so nothing downstream has to choose between them. The width is divided among them rather than multiplied, so the cost is roughly the same as one head of the full width.
5. That the weights are a genuine measurement — that head really did take about seven tenths of its blend from that position — and that it is one head in one layer out of dozens of each, after which everything is mixed again and passed up the stack. A weight says where a blend came from. It is not a reason, and "the model understood" is a claim about the whole computation made from a reading of one step of it.
6. Because the output of the attention formula is a sum of the value vectors, and addition does not depend on order. Swap two tokens and their weights swap with them, so the total is unchanged — this chapter shows exactly that, with the same output vector before and after. Swapping the last word would not make the point, because the last position is the one supplying the query: change the token there and you change what is being asked, so the output changes for a reason that has nothing to do with order. The demonstration only works on a swap that leaves the querying position alone. Word order therefore has to be put in separately — added to the embedding before the stack, applied to the queries and keys as they are formed, or added to the scores themselves.

You understand this when you can compute one row of attention weights by hand from a table of queries and keys, and say what goes wrong if you skip the division.

CHAPTER 12

The Stack: Layers and the Residual Stream

Chapter 11 opened half of a layer. This chapter opens the other half, puts the two together, and then stacks the result ninety-six times. It also answers a question that sounds reasonable and has no good answer — where in a model a particular fact is kept — and it is worth knowing why the question is wrong rather than merely being told it is.

The problem. One attention step is clear enough. A model has ninety-six of them and I don't know what that accumulates.

The question. What does a layer actually do to the vectors passing through it, what is left after ninety-six of those, and where do the parameters sit?

A Layer Does Not Replace, It Adds

Start with the single most useful fact about the design, because it is not what people expect.

A layer does not take a vector and hand back a different vector. It takes a vector, works out a **change**, and adds that change to what it was given:

```
out  =  in  +  (something computed from in)
```

That addition is called a **residual connection**, and the vector being added to as it passes up through the stack is the **residual stream**. It starts as the token's embedding, with whatever Chapter 11's positional encoding adds to it, and from there everything a layer computes is written into that one running vector, by addition, one contribution at a time.

A picture that helps, with the usual caution about pictures. Think of the residual stream as a long document that every layer may annotate. A layer reads the whole document, decides what to add, and appends its contribution. It cannot erase what is there — it can only add

something that cancels it out — and the next layer reads the sum of everything written so far.

Where that picture stops being accurate: the contributions are not kept separate. They are added into the same four thousand numbers, so once two layers have written, no inspection of the result can separate them. That matters later in this chapter.

Why build it this way rather than letting each layer transform the vector outright? One reason is plain enough to state without any theory: if each layer only ever *adds*, then a layer with nothing useful to contribute can add approximately nothing and the information already in the stream passes through untouched. A stack where each layer replaced its input would have to relearn, at every level, how to carry forward what it did not want to change.

What Is in a Layer

Each layer makes two contributions to the stream, not one, and each gets its own residual connection.

```
x                      the stream arriving

x  =  x + attention(x)      the attention block writes
x  =  x + feedforward(x)    the feed-forward block writes

                       the stream leaving, same width
```

The **attention block** is Chapter 11 in full: queries, keys, values, the grid, the mask, softmax, the blend, the heads and their mixing matrix. Its defining property is that it moves information *between positions*. It is the only part of the model that does.

The **feed-forward block** is the other half, and it is the opposite: it works on each position entirely on its own, with no reference to any other position. Position 5's vector goes in, position 5's contribution comes out, and if you deleted every other token it would make no difference to this block at all.

	Attention block	Feed-forward block
Looks at	Every position	One position, in isolation
What it is	Chapter 11's formula	Two matrices with a function between them
Cost grows	With n^2	With n
Share of a layer's parameters	Smaller	Larger — the arithmetic is below

So a layer is: *gather from elsewhere, then think about it here.* Twice per layer, ninety-six times over, into one running vector.

The Feed-Forward Block, Worked

It is two matrix multiplications with one non-linear step between them, and the middle is *wider* than the ends. That middle width is written d_{ff}, and it runs from about two and a half times the width to four times it, depending on the design — the smaller end being usual where the block has three matrices rather than two.

```
width 4  -->  [ W_up ]  -->  width 8  -->  activation
                                              |
width 4  <--  [ W_down ]  <--  width 8  <---
```

Carry on with Chapter 11's example. Its worked attention produced $(0.216,\ 3.378,\ 1.879,\ 1.013)$ for position 5, whose vector arriving at the layer was $(2, 0, 2, 0)$. The attention block's residual connection adds them:

```
arriving        (2,     0,     2,     0    )
attention   +   (0.216, 3.378, 1.879, 1.013)
                ---------------------------
stream now      (2.216, 3.378, 3.879, 1.013)
```

That sum is what the feed-forward block receives. Here is W_{up}, eight rows of four, chosen so you can follow it:

```
W_up = [  1  1  0  0 ]      [  1 -1  0  0 ]
       [  0  1  1  0 ]      [  0  1 -1  0 ]
       [  1  0  0  1 ]      [  0  0  1 -1 ]
       [  0  0  1  1 ]      [ -1  0  1  0 ]
```

Rows 1 to 4 add pairs of slots. Rows 5 to 8 *subtract* them, which is the interesting part. Multiplying out:

```
row 1   2.216 + 3.378  =   5.594
row 2   3.378 + 3.879  =   7.257
row 3   2.216 + 1.013  =   3.229
row 4   3.879 + 1.013  =   4.892
row 5   2.216 − 3.378  =  −1.162
row 6   3.378 − 3.879  =  −0.501
row 7   3.879 − 1.013  =   2.866
row 8   3.879 − 2.216  =   1.663
```

Now the **activation function**, which is the one step in this block that is not a matrix multiplication. The simplest and the one you can do in your head is to keep positives and replace negatives with zero:

```
before  5.594 7.257 3.229 4.892 −1.162 −0.501 2.866 1.663
after   5.594 7.257 3.229 4.892  0.000  0.000 2.866 1.663
```

Look at what that did. Rows 5 to 8 were comparisons — is slot 1 bigger than slot 2, is slot 3 bigger than slot 4. Two of them came out negative and have been switched off entirely. So what passes on from here depends on *which comparisons held*, and that is something no amount of matrix multiplication can do on its own: multiplying by a matrix and then by another matrix is the same as multiplying by one combined matrix, however many you put in a row. Without the activation, this block's W_{up} and W_{down} would do no more than a single 4-by-4 matrix could, and no condition could be expressed in it at all.

Then W_{down} takes the eight back to four:

```
W_down = [ 0 1 0 0 1 0 0 1 ]
         [ 1 0 0 1 0 1 0 0 ]
         [ 0 0 1 0 0 0 1 0 ]
         [ 0 1 1 0 0 0 0 1 ]


row 1   7.257 + 0.000 + 1.663          =   8.920
row 2   5.594 + 4.892 + 0.000          =  10.486
row 3   3.229 + 2.866                  =   6.095
row 4   7.257 + 3.229 + 1.663          =  12.149
```

And the second residual connection adds that to the stream:

```
stream       ( 2.216,  3.378,  3.879,  1.013)
feed-forward ( 8.920, 10.486,  6.095, 12.149)
             ------------------------------
stream now   (11.136, 13.864,  9.974, 13.162)
```

That is one complete layer. A vector of four numbers went in, two blocks wrote into it, and a vector of four numbers came out — ready for the next layer, which will do the same thing with its own numbers.



You will often hear the stronger version of this — that without the activations the whole stack would collapse into one matrix. That is not true, and Chapter 11's numbers show why: attention is not a linear step either. Double every input vector in Chapter 11's example and its output does not double. It goes from $(0.216, 3.378, 1.879, 1.013)$ to $(0.001, 7.996, 3.999, 2.000)$, because softmax turned the weights $(0.095, 0.702, 0.095, 0.095, 0.013)$ into $(0.000, 0.999, 0.000, 0.000, 0.000)$, which print as adding to 0.999 for the rounding reason you have met twice — the saturation of Chapter 11's scaling section, arriving uninvited. **The collapse argument is about the block, not the stack.** It is still the reason the activation is there.



Real models use a smoother activation than the keep-positives rule above, and most now use a gated form with three matrices rather than two. The shape of the argument does not change: two or three matrices, a non-linear step, applied at each position on its own, with the middle wider than the ends.

Normalisation

One step has been left out of the diagram, and it belongs in a sentence.

Before each block reads the stream, the stream is **normalised** — rescaled so its numbers sit in a predictable range, with a small set of learned parameters allowing the scale to be adjusted.

Why it is there: **without it the numbers grow as contributions are added layer after layer, and a very deep stack stops computing reliably.** That is the whole of the honest answer, and this book does not go further, because the rest of it is a subject — numerical stability — that Chapter 17 lists among the things you are not going to be able to explain.



This is a good example of a question where the right interview answer is short and stops. "It keeps the scale of the stream under control as layers add to it, which matters because the stack is deep" is correct and complete at this level. Reaching for more is how people talk themselves into trouble.

Ninety-Six of Them

Now stack it. Every layer has the same shape and its own numbers — Chapter 9's point, now with the inside visible — and the stream arrives at the end carrying the sum of every contribution made along the way. Depths of a few dozen to a hundred are usual; the worked model later in this chapter has thirty-two layers, and the arithmetic is the same at any depth.

What more layers let the model do is easy to say at the level of mechanism: more layers means more separate opportunities to gather from other positions and more separate opportunities to compute something at each position.

And the second layer is where that stops being abstract. The vector this chapter built — (11.136, 13.864, 9.974, 13.162) — is what passes into layer 2 at position 5. It is not the embedding of "it" any more. It is the embedding of "it" plus what position 5 gathered from "cat" plus what the feed-forward block made of the two together, and layer 2's queries and keys are computed from *that*. So position 5 now offers a different key to every other position than it did at layer 1, and asks a different question.

That is what depth buys, mechanically: each round gathers from, and computes on, what the last round assembled. A result put together at layer 20 is available to be gathered from, by a different position, at layer 21 — and the thing being gathered did not exist in the input at all.

And then there is the part nobody can tell you. *Why* a deeper stack is better at the things these models are better at — why thirty-two layers beats sixteen by more than it costs, where the returns stop — is an open question in the research, not a settled explanation anybody is keeping from you. It is on Chapter 17's list, and the honest version in an interview is "the mechanism is more rounds of gather-and-compute; why depth buys what it buys is not something I can derive, and nobody has a clean account of it."



You will meet confident stories about what particular layers "do" — early layers handle syntax, middle layers handle facts, late layers handle output format. There is real measurement behind some of this, and it is a finding about particular models arrived at by probing them, not a design anybody specified and not shown to be a rule that transfers. Treat it the way Chapter 6 says to treat any claim: ask what it was measured on.

Where Is the Fact Stored?

This is the question, and it is worth being precise about why it has no clean answer.

The question assumes a filing system: that somewhere in the model there is a place holding *Paris is the capital of France*, which could in principle be pointed at. Nothing in the mechanism supports that assumption, for three reasons that are each visible in the arithmetic above.

Everything is added into the same numbers. The stream at position 5 after one layer was (11.136, 13.864, 9.974, 13.162). Which part of that came from attention, which from the feed-forward block, which was the original token? The three contributions were $(2, 0, 2, 0)$, $(0.216, 3.378, 1.879, 1.013)$ and $(8.920, 10.486, 6.095, 12.149)$ — but having added them, there is no operation you can perform on the result that recovers them. A sum does not say what was added. Ninety-six layers in, the stream is a sum of a very large number of contributions and nothing marks where any of them came from.

No parameter belongs to anything. W_{down}'s row 3 above was used to compute part of this token's contribution, and the identical row will be used again on unrelated tokens in unrelated sentences. A parameter is not allocated to a fact. It is used again on input after input, with nothing to say which of them it was for.

And a fact is not a thing the model holds. What the model holds is numbers that make likely continuations likely. "Paris" being a probable continuation of "the capital of France is" is a property of how the whole computation lands, in the same way that a bridge's ability to hold a lorry is a property of the bridge rather than of a part of it you could point to.

So the honest answer to "where is it stored" is: **it is not stored anywhere, in the sense the question means.** What can be said is weaker and true. Measurement can show that changing particular parameters changes particular behaviour — that is a real research result and techniques exist that exploit it. What it does not show is an address, and a model with billions of parameters contributing to every output does not have one.



This is not a limitation of current understanding that better tools will clear up. The filing-system picture is wrong about the mechanism rather than merely incomplete, which is why "we don't know yet" is a worse answer than "the question assumes something that is not there".

Where the Parameters Actually Sit

Chapter 9 said a parameter is a fitted number and that the count is what a model's name reports. Now you can see where they are. Take a model of a common shape: width 4,096, a feed-forward middle of 11,008, and 32 layers.

One attention block. Four matrices — W_Q, W_K, W_V and the mixing matrix W_O — each 4,096 by 4,096:

```
4,096 x 4,096 = 16,777,216
16,777,216 x 4 = 67,108,864
```

One feed-forward block. Two matrices, 4,096 by 11,008 and back:

```
4,096 x 11,008 = 45,088,768
45,088,768 x 2 = 90,177,536
```

One layer, and then all of them.

```
attention         67,108,864
feed-forward      90,177,536
                 -----------
one layer        157,286,400
x 32 layers    5,033,164,800
```

Two things in that table are worth carrying.

The feed-forward block holds more parameters than attention does — 57 per cent of a layer here — even though attention is the part everybody talks about and the part that was hard to understand. The thing that gathers between positions is the cheaper half.

And the layers hold almost everything. Add Chapter 9's embedding table at a vocabulary of 128,000:

```
the layers    5,033,164,800
embedding       524,288,000    counted once, as in the
                               models that reuse it for
                               the output step (ch 9)
              --------------
              5,557,452,800
```

About nine parameters in ten are in the stack. That is why "where do the parameters sit" has a short answer — *in the layers, mostly in the feed-forward blocks* — even though "where is the fact" has none.



The three shape numbers above are illustrative of a common size and not any particular model. Widths, feed-forward multipliers and depths all vary, newer designs add matrices the arithmetic above does not count, and the count a model is sold under may include or exclude the embedding table. The *method* — count a matrix, multiply by how many of them there are — is what transfers.

Lab: One Layer, By Hand

Pen, paper and a calculator. Twenty-five minutes, and there is a key for every step. Nothing to install.

The stream arriving at a position is $(1, 2, 0, 1)$, and this layer's attention block has computed a contribution of $(0.5, -1, 3, 1)$.

Step 1. Apply the attention block's residual connection. What is the stream now?

Expected result: $(1.5,\ 1,\ 3,\ 2)$. Add, do not replace — if you wrote $(0.5, -1, 3, 1)$ you have done what most people expect a layer to do, and it is the thing this chapter exists to correct.

Step 2. The feed-forward block uses the same W_{up} as the chapter. Compute all eight numbers.

Expected result:

```
row 1   1.5 + 1    =   2.5       row 5   1.5 - 1    =   0.5
row 2   1   + 3    =   4.0       row 6   1   - 3    =  -2.0
row 3   1.5 + 2    =   3.5       row 7   3   - 2    =   1.0
row 4   3   + 2    =   5.0       row 8   3   - 1.5  =   1.5
```

Step 3. Apply the activation. Which slots are switched off, and what does that tell you about what this block can express?

Expected result: only row 6, which becomes 0. Everything else is already positive. The block can express a condition — *this signal passes on only when slot 2 exceeds slot 3* — and that is something no sequence of matrix multiplications could do by itself, because a product of matrices is just another matrix.

Step 4. Apply the chapter's W_{down} to those eight numbers.

Expected result:

```
row 1   4.0 + 0.5 + 1.5          =   6.0
row 2   2.5 + 5.0 + 0.0          =   7.5
row 3   3.5 + 1.0                =   4.5
row 4   4.0 + 3.5 + 1.5          =   9.0
```

Step 5. Apply the second residual connection. What leaves the layer?

Expected result: $(1.5, 1, 3, 2) + (6, 7.5, 4.5, 9) = (7.5,\ 8.5,\ 7.5,\ 11)$. Note the width: four in, four out, which is what lets the next layer run at all.

Step 6. Someone asks you which part of $(7.5, 8.5, 7.5, 11)$ came from the attention block. What do you answer, and what would you need to answer it?

Expected result: you cannot tell from the vector. You happen to know, because you did the addition and still have the pieces on your page — but the vector itself does not carry them, and the model keeps no record. You would need the inputs and all the matrices, and to redo the computation. After ninety-six layers nobody has that on a page.

Step 7. A model has a width of 2,048, a feed-forward middle of 5,632, and 24 layers. How many parameters are in its layers, and what share is feed-forward?

Expected result:

```
attention     2,048 x 2,048 x 4 = 16,777,216
feed-forward  2,048 x 5,632 x 2 = 23,068,672
one layer                        39,845,888
24 layers                       956,301,312
```

The feed-forward share is $23{,}068{,}672 / 39{,}845{,}888 = 0.579$, so about 58 per cent — the same story as the chapter's larger example, because the ratio depends on the feed-forward multiplier and not on the size.

Step 8. Without computing: your 24-layer model is replaced by a 48-layer one of the same width. Roughly what happens to the parameter count, and what happens to the attention cost on a long input?

Expected result: the layer parameters roughly double, since each layer holds the same number and there are twice as many. The attention cost also doubles, because $O(n^2 d)$ is per layer and there are twice as many layers — depth multiplies the cost, it does not change how it grows with n.

Summary, Key Terms, and Review Questions

Summary

A layer does not replace the vector it is given; it computes a change and **adds** it, which is a **residual connection**, and the running vector every layer adds into is the **residual stream**. Each layer writes twice: the **attention block** of Chapter 11, which is the only part that moves information between positions, and the **feed-forward block**, which works on each position in isolation — two matrices with an **activation function** between them and a middle between about two and a half and four times the width of the ends. The activation is what stops that block's two matrices from collapsing into one, and it is where a condition is expressed — not, as is often said, what stops the whole stack collapsing, since attention is not a linear step either. The stream is **normalised** before each block, which keeps its numbers in range as a deep stack adds to it. More layers means more rounds of gather-and-compute; why depth buys what it buys is on Chapter 17's list. **Where a fact is stored has no clean answer** because contributions are added into the same numbers and cannot be separated afterwards, because a parameter is used again on input after input rather than belonging to anything, and because what the model holds is numbers that make likely continuations likely rather than facts at addresses. Most parameters sit in the layers, and more of them

in the feed-forward blocks than in attention — about 57 per cent of a layer at a common shape, with roughly nine in ten of all parameters in the stack rather than the embedding table.

Key Terms

Term	Meaning
Residual connection	Adding a block's output to the vector that went into it, rather than replacing it. What lets a layer contribute nothing and pass its input through
Residual stream	The running vector passing up through the stack, which every block adds into. By the end it is a sum of every contribution made along the way, with nothing marking which is which
Attention block	The half of a layer that moves information between positions — Chapter 11's formula in full. The only part of a model that does
Feed-forward block	The half of a layer that works on each position alone: two matrices with an activation between them, the middle between about two and a half and four times the width of the ends. Holds more of a layer's parameters than attention does
Activation function	The step in the feed-forward block that is not a matrix multiplication, such as keeping positives and zeroing negatives. Without it the block's two matrices would collapse into one
Normalisation	Rescaling the stream to a predictable range before each block reads it, so the numbers do not grow out of hand as layer after layer adds to them

Worth Memorising

Item	Value or formula
What a layer does	$x \leftarrow x + \text{attention}(x)$, then $x \leftarrow x + \text{feedforward}(x)$
Attention block parameters	$4d^2$ — four matrices of $d \times d$
Feed-forward block parameters	$2 \times d \times d_{ff}$, with the middle width d_{ff} between about $2.5d$ and $4d$
Which half is bigger	Feed-forward, at roughly 57 per cent of a layer
Where the parameters are	In the layers — about nine in ten, at a common shape
Matrices compose	$W_2(W_1x) = (W_2W_1)x$, which is why a block without its activation is one matrix

Review Questions

1. What is the residual stream, and what does a layer do to it?

2. Why does "where is that fact stored in the model" have no clean answer?
3. What would go wrong if you removed the activation function from every feed-forward block?
4. Which of a layer's two blocks holds more parameters, and which costs more on a long input?
5. Why is normalisation there? Answer in one sentence and stop.
6. A colleague says layer 40 of a model "is where it does arithmetic". What do you ask?

Review Answers

1. It is the running vector that passes up through the stack, one per position. A layer does not replace it: it computes a change and adds it, twice — once from the attention block, which gathers from other positions, and once from the feed-forward block, which works on this position alone. Because each block only adds, a layer with nothing to contribute can add nearly nothing and the stream passes through. By the end the stream is a sum of every contribution every layer made.
2. Because the mechanism has no addresses. Every block's output is *added* into the same numbers, and a sum does not say what was added, so after a few layers nothing in the stream marks what came from where. A parameter is used again on input after input, so no parameter belongs to a fact. And what the model holds is numbers that make a likely continuation likely, not propositions in slots. Measurement can show that changing particular parameters changes particular behaviour, which is real, but that is not an address.
3. Every feed-forward block would collapse. A matrix multiplication followed by another is a single matrix multiplication, so without a non-linear step between them the block's two matrices would do no more than one matrix of the same shape, and no condition could be expressed in it — *pass this on only when that comparison holds*, which is why the chapter's example sends differences through it. Note what would **not** happen: the whole stack would not collapse to one matrix, because attention is not linear either. That is the common overstated version of this answer.
4. The feed-forward block holds more parameters, at roughly 57 per cent of a layer at a common shape, because its two matrices are 4,096 by 11,008 while attention's four are 4,096 by 4,096. But attention costs more on a long input, because its work grows with n^2 and the feed-forward block's grows with n — it is applied at each position independently. More parameters and more cost are different questions.
5. It keeps the scale of the stream under control as layer after layer adds to it, which matters because the stack is deep. That is the whole answer at this level, and the rest of it is a subject this book deliberately does not teach.
6. What it was measured on, and how. There is genuine work probing what particular layers respond to, and it produces findings about particular models rather than rules that transfer — so the questions are which model, which probe, and whether anybody has

shown the same thing elsewhere. The mechanism gives no reason to expect a layer to have a job, since every layer has the same shape and differs only in the numbers training left in it.

> **You understand this when** you can say what a layer adds to, in what order, and why that makes "which part of this vector came from attention" an unanswerable question.

CHAPTER 13

From Scores to a Word

Chapter 9 ended with a list of probabilities and a flat refusal to pick one. This chapter picks one. It is the last step of the pipeline and the only one you control from the outside: the settings on your call act here and nowhere else. It also answers the question in its opening problem, which is the most common surprise in this whole field and has nothing to do with machine learning.

The problem. I set temperature to zero and still got two different answers.

The question. How does a distribution become one word, what do the settings actually do to the numbers, and why is none of it a guarantee?

Where We Are

Chapter 9's worked example ended here. A vocabulary of five, and one forward pass just finished:

```
token   score     p
the       2     0.012
cat       5     0.234
sat       3     0.032
mat       4     0.086
on        6     0.636
```

Those raw scores have a name. They are the **logits** — one per vocabulary entry, straight out of the final matrix multiplication, before anything has been done to them. They are not probabilities: they can be negative, they do not add to anything in particular, and only their *differences* matter — for a reason the next section works out in one line.

Everything in this chapter happens between that table and one token. The model is finished. It has handed over five numbers and taken no view on which of them you should use.

Greedy: Take the Biggest

The simplest rule is to take the highest-scoring token every time. That is **greedy decoding**, and it is what "temperature zero" means in practice.

Here it picks "on", because 6 is the largest logit. Note that you do not need the softmax at all to do this: softmax preserves order — Chapter 8's third property — so the biggest logit is the biggest probability, and converting them changes nothing about which one wins.

And this is where the earlier claim about differences gets paid. Add 5 to every logit, so they become $(7, 10, 8, 9, 11)$. Every term on the top of the softmax gains a factor of e^5, and so does the total on the bottom — so it cancels, and every probability comes out exactly as before. Logits carry no zero point. Only the gaps between them mean anything, which is why a logit of 6 tells you nothing on its own.

Greedy sounds like it should be the good option, and it is the right choice for some work and the wrong one for a surprising amount of it. Two reasons.

It is locally greedy, and text is not local. Taking the most likely next token at every step is not guaranteed to produce the most likely sentence, and usually does not. A high-scoring first token can lead into a corner where every continuation is poor, while a second-choice first token opens onto something much better. Nothing in the procedure looks ahead.

It makes the model repeat itself. Once a loop begins — a phrase, a sentence shape, a list item — the tokens that continue the loop are exactly the ones made likely by the loop so far, and greedy takes them every time. Anyone who has watched a model produce the same paragraph four times has watched this.

So the alternative is to *sample*: treat the probabilities as probabilities and draw one. "on" about 636 times in a thousand, "cat" about 234 times, "the" about 12. The rest of this chapter is about reshaping that distribution before you draw from it.

Temperature

Temperature divides the logits before softmax. That is the whole definition:

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Chapter 8's softmax with one extra symbol. And its effect follows from the property Chapter 8 ended on — *gaps get exaggerated* — because dividing the logits divides the gaps between them.

Work the same five logits at three settings.

```
T = 1     logits unchanged       (2, 5, 3, 4, 6)
          e^2 = 7.389   e^5 = 148.413   e^3 = 20.086
          e^4 = 54.598  e^6 = 403.429
          total = 633.915

          the 0.012   cat 0.234   sat 0.032
          mat 0.086   on  0.636


T = 0.5   logits doubled         (4, 10, 6, 8, 12)
          total = 188,220.242

          the 0.000   cat 0.117   sat 0.002
          mat 0.016   on  0.865


T = 2     logits halved          (1, 2.5, 1.5, 2, 3)
          total = 46.857

          the 0.058   cat 0.260   sat 0.096
          mat 0.158   on  0.429
```

Read down the "on" column: 0.636, then 0.865, then 0.429. **Below 1, temperature sharpens the distribution; above 1, it flattens it.** The order never changes — dividing every logit by the same positive number cannot reorder them — so temperature does not change what the model thinks is most likely. It changes how much of the rest is in play.

	Low temperature	High temperature
The distribution	Sharper	Flatter
Top token's share	Goes up	Goes down
What you get	Predictable, repetitive	Varied, sometimes incoherent
Where it belongs	Extraction, classification, code	Drafting, brainstorming, variety



The numbers at $T = 2$ print as adding to 1.001, for the rounding reason you have now met three times. And at $T = 0.5$ "the" prints as 0.000 — it has not been removed, it is about 0.0003. Nothing here deletes an option; temperature only moves weight around.

And now the thing the formula makes obvious. Temperature zero would mean dividing by zero, which is not defined. What most providers do at $T = 0$ is not compute that limit — they switch to greedy decoding and take the biggest logit, and the rest clamp the temperature to something tiny, which comes to the same thing. So "temperature zero" is not a temperature at all. It is a different procedure wearing the same dial, and that matters for the last section of this chapter.

Top-k and Top-p

Temperature reshapes the whole distribution. The other two settings cut parts of it away entirely, and then renormalise what is left so it still adds to 1.

Top-k keeps the k highest-scoring tokens and discards the rest. At $k = 2$, on the $T = 1$ distribution above:

```
kept     on  0.636    cat  0.234      total kept 0.870
renormalised
         on  0.636 / 0.870 = 0.731
         cat 0.234 / 0.870 = 0.269
```

Three tokens now have probability exactly zero and cannot be produced however many times you sample.

Top-p, also called nucleus sampling, keeps the smallest set of tokens whose probabilities add to at least p. Sort by probability and accumulate:

```
on    0.636    running total 0.636
cat   0.234    running total 0.870
mat   0.086    running total 0.956   <- first to reach 0.9
sat   0.032
the   0.012
```

At $p = 0.9$ you keep *three* tokens, not two: after two the total is 0.870, which is short of 0.9, so "mat" comes in and the total overshoots to 0.956. Renormalising:

```
on  0.636 / 0.956 = 0.665
cat 0.234 / 0.956 = 0.245
mat 0.086 / 0.956 = 0.090
```

The difference between them is the whole reason top-p exists. Top-k keeps the same *number* of options whatever the distribution looks like. When the model is confident — one token at 0.98 — top-k with $k = 40$ keeps thirty-nine options that should never have been in play. When the model is genuinely uncertain across a hundred near-equal tokens, the same setting throws away sixty of them for no reason. Top-p keeps however many it takes to cover the mass, so it is narrow where the model is sure and wide where it is not. The cost is that the number of options is no longer something you can state in advance.



These interact, and the order is: temperature first, then the cut, then draw. Raising temperature and tightening top-p at the same time is a common way to get much less than you asked for, because the tail the flattening opened up is cut away again. Change one at a time — which is Chapter 18's discipline, arriving early.

How a Reply Ends

A reply has to stop. Three things stop it in the ordinary case, and only one of them is the model's doing.

The model emits a stop token. A vocabulary also holds entries that are not pieces of text — Chapter 10's list of merged pieces is not the whole of it — and one of them means *this is the end*. That entry is the **stop token**, often called an end-of-sequence token. When it is sampled, generation stops. This is the mechanism to understand, because it is not special: the stop token is a seat in the vocabulary like any other, it gets a logit like any other, and it is chosen by exactly the procedure above. **A model finishes talking by predicting that it has finished.**

Which means it has a probability, and you can read it. Suppose a forward pass somewhere in a reply gives these three logits:

```
token     logit     p
more        2      0.245
text        1      0.090
<end>       3      0.665


e^2 = 7.389   e^1 = 2.718   e^3 = 20.086
total = 7.389 + 2.718 + 20.086 = 30.193
7.389 / 30.193 = 0.245
2.718 / 30.193 = 0.090
20.086 / 30.193 = 0.665
```

Two times in three this reply stops here; one time in three it carries on. Nothing about that is a special case — it is the softmax of Chapter 8 over a vocabulary that happens to contain an entry meaning *stop*. And at temperature zero it would stop every time, which is one reason a greedy reply and a sampled reply from the same prompt can be very different lengths.

You set a maximum. Every API takes a limit on how many tokens may be generated. Hitting it truncates the reply mid-sentence, which is a different outcome from finishing, and your code should be able to tell them apart — the response says which happened.

You supply a stop condition. A **stop condition** — providers call it a stop sequence, and it is not Chapter 4's stopping condition — is a string you nominate; when the generated text contains it, generation halts and the string is normally not returned. Useful when you have asked for one item of a list and do not want four.



Two more things end a reply outside the ordinary case, and neither is in the list above. The input can simply fill up, which Chapter 16 is about. And your own code can cancel the request part-way, which Chapter 19 covers. If you are writing the code that handles a reply, those are two more outcomes to tell apart.



The maximum is a guillotine, not a budget. A model asked for a long answer with a short maximum does not write a shorter answer — it writes the same long answer and has it cut off. If you want brevity, ask for brevity in the instructions and set the maximum as a safety net above what you expect, not at it.

Why Temperature Zero Is Still Not a Guarantee

Now the opening problem, and it deserves a real answer rather than a shrug. Set temperature to zero, get greedy decoding, take the biggest logit every time — and still get different text on two identical calls. There is no randomness in that procedure. So where does the variation come from?

Four places that bite in practice, and none of them is the model being unpredictable.

Floating-point addition is not associative. Adding a list of numbers in a different order can give a slightly different answer, because each intermediate result is rounded to a fixed number of digits. This is a property of the arithmetic hardware, not of models. A GPU splits a long addition across thousands of units and then combines their partial sums, so the answer's last digits depend on how the work was split — and nothing in your call fixes that split.

Your request is batched with other people's. Providers run many requests together for efficiency. How many, and which ones, depends on who else is calling at that moment — and the batch's shape changes how the arithmetic is grouped, which by the point above changes the last digits. Chapter 15 is where batching is explained properly.

Those two combine into the case that is easiest to see. Suppose two logits are 6.0000001 and 6.0000000; greedy takes the first. Change the last digit through a different summation order and greedy takes the second — **a different token, and from there the reply may diverge entirely.** The difference is not in the model, the prompt or the settings. It is a tie broken the other way.

Some models route tokens to different parts of themselves. In a model built with experts, which part handles your token can depend on what else is in the batch, so your output can depend on your neighbours. Chapter 15 explains what those experts are.

And what runs underneath you can change. Providers update models, sometimes without a new name — and they also change the hardware and the libraries the same model runs on, which moves the last digits without moving a single parameter. Chapter 54 is about pinning versions and noticing when one moves.



Some providers offer a seed parameter and describe the result as *best effort* rather than guaranteed. That wording is accurate and worth taking literally: a seed fixes the random draw, and the sources above are not random draws.

What to do about it, in one line each. Do not write tests that assert on exact output strings;

assert on shape, on a property, or on a check. Log the output you actually got, so "it changed" is a comparison and not a memory. And where the answer must be stable, stabilise it in your code rather than hoping — a fixed format, a validator, a cache — rather than in the model.

Why One Good Output Proves Nothing

This chapter ends with the consequence, because it is the most expensive mistake in the material so far.

You have seen a distribution with five entries, three of which have a real chance of being drawn. One call samples one path through that distribution, at every token. The reply you got is one draw from a very large space of replies your prompt could have produced, and the next one will be a different draw.

So when you change a prompt, run it, and the output looks better: **you have one sample from each of two distributions.** That is not evidence that the new prompt is better. It is not even weak evidence, unless you know something about how much the output varies from call to call — and you do not, because you ran it once.

This is the single most common error made by people who are otherwise doing everything right, and it is why Chapter 20 exists: repeated runs, a measurement with an interval around it, and a verdict you could defend. This half of the book is mechanism; Chapter 20 is where the book turns to how you would know.



The habit to build now, before Chapter 20 gives you the statistics: **run it five times before you believe it.** Five is not a number with a theory behind it. It is small enough to do every time and usually enough to show you the spread is there at all — and Chapter 20, which says how small a spread can be, is why *usually* is the honest word.

Lab: Draw From a Distribution

Pen and paper for the first half, then about twenty lines of Python. Thirty minutes, and there is a key for every step.

A vocabulary of four and one forward pass, with these logits:

```
token    logit
yes        3
no         1
maybe      2
.          0
```

Step 1. Compute the probabilities at $T = 1$. The exponentials you need are $e^0 = 1$, $e^1 = 2.718$, $e^2 = 7.389$ and $e^3 = 20.086$.

Expected result: total $20.086 + 2.718 + 7.389 + 1 = 31.193$, and the probabilities are yes 0.644, no 0.087, maybe 0.237, "." 0.032.

Step 2. Now $T = 0.5$. What are the new logits, and without computing the softmax, what will happen to "yes"?

Expected result: the logits double to $(6, 2, 4, 0)$, and "yes" goes up, because halving the temperature doubles every gap and softmax exaggerates gaps. The order cannot change. (For the record it becomes 0.865.)

Step 3. Which token does greedy decoding pick, and does it pick a different one at $T = 0.5$?

Expected result: "yes", both times. Greedy takes the largest logit, and dividing every logit by the same positive number cannot change which is largest. **Temperature has no effect on greedy decoding at all** — which is worth knowing before you spend an afternoon tuning it on a call that is already greedy.

Step 4. Apply top-k with $k = 2$ to the $T = 1$ distribution.

Expected result: keep yes (0.644) and maybe (0.237), total 0.881. Renormalised, $0.644/0.881 = 0.731$ and $0.237/0.881 = 0.269$.

Step 5. Apply top-p with $p = 0.8$ to the same distribution. How many tokens survive, and why is it not one?

Expected result: two. "yes" alone is 0.644, which is short of 0.8, so "maybe" is added and the running total reaches 0.881. The rule is *the smallest set that reaches p*, so it overshoots — it is not allowed to stop below the threshold.

Step 6. In a new empty folder, save this as `sample.py` and run it:

```
import math, random

vocab = ["yes", "no", "maybe", "."]
logits = [3, 1, 2, 0]

def probs(logits, T):
    scaled = [z / T for z in logits]
    total = sum(math.exp(z) for z in scaled)
    return [math.exp(z) / total for z in scaled]

random.seed(13)
for T in (0.5, 1.0, 2.0):
    p = probs(logits, T)
    draws = random.choices(vocab, weights=p, k=1000)
    counts = {w: draws.count(w) for w in vocab}
    print(f"T={T}  p={[round(x, 3) for x in p]}")
```

```
    print(f"      counts {counts}")
```

Expected result: exactly this, because the seed is fixed.

```
T=0.5  p=[0.865, 0.016, 0.117, 0.002]
      counts {'yes': 873, 'no': 17, 'maybe': 108, '.': 2}
T=1.0  p=[0.644, 0.087, 0.237, 0.032]
      counts {'yes': 629, 'no': 96, 'maybe': 241, '.': 34}
T=2.0  p=[0.455, 0.167, 0.276, 0.102]
      counts {'yes': 467, 'no': 153, 'maybe': 271, '.': 109}
```

The counts track the probabilities, which is the only thing a probability means. At $T = 2$ the full stop came up 109 times in a thousand; at $T = 0.5$ it came up twice.

Step 7. Change the seed to any other number and run it again. What stays the same and what moves?

Expected result: the probabilities are identical, because they do not depend on the seed. The counts all move, most of them by tens. **That is what one sample looks like against a thousand**, and you have just watched a thousand of them to see the shape — which is exactly the evidence you do *not* have when you change a prompt, run it once, and like the answer.

Step 8. No arithmetic. You ask for a single JSON object, set the maximum to 50 tokens and supply the stop condition `\n\n`. Name the four different ways that call can come back, and say how your code tells them apart.

Expected result: (1) the model samples its stop token, and the reply is complete; (2) the maximum is reached at 50 tokens and the JSON is cut off mid-string; (3) the model emits a blank line, the stop condition matches, and generation halts wherever it happened to be; (4) your own code cancels or times out first. The response says which of the first three happened — every API reports a reason for finishing — and **that field is the thing to log**, because cases 2 and 3 both produce text that looks like an answer and is not one. Note what this means for the maximum: at 50 tokens you have not asked for a shorter object, you have arranged for a broken one.

Step 9. A colleague says their feature is deterministic because they set temperature to zero. Name two reasons that may not hold.

Expected result: any two of: floating-point addition is not associative, so a different summation order on the GPU changes the last digits and can flip which of two near-equal logits is largest; their request is batched with others, and the batch's composition changes that order; a model with experts may route their token differently depending on the batch; and the provider may have changed the model, or the hardware and libraries it runs on.

Summary, Key Terms, and Review Questions

Summary

The raw scores coming out of the final matrix multiplication are the **logits**, one per vocabulary entry, and only their differences matter. **Greedy decoding** takes the largest and is what temperature zero does in practice; it is locally greedy, so it is not guaranteed to produce the most likely sentence, and it makes models repeat themselves. **Temperature** divides the logits before softmax — below 1 it sharpens the distribution, above 1 it flattens it, and it never changes the order, so it has no effect at all on greedy decoding. **Top-k** keeps a fixed number of tokens and **top-p** keeps the smallest set whose probabilities reach p, both renormalising what survives; top-p adapts to how confident the model is and top-k does not. In the ordinary case a reply ends when the model samples a **stop token** — an ordinary vocabulary entry meaning *finished* — or when your maximum truncates it, or when a **stop condition** you supplied is matched; outside it, the input can fill up or your code can cancel. **Temperature zero is not a guarantee**: floating-point addition is not associative, batching changes the summation order, expert routing can depend on your neighbours, and what the provider runs underneath you can change. Some of those move the last digits of a score, which flips a near-tie; others change the computation outright. Where a near-tie does flip, the reply may diverge from that token on; where none is close enough, the greedy path is identical. And because one call is one draw, **one good output is not evidence**, which is the problem Chapter 20 solves.

Key Terms

Term	Meaning
Logits	The raw scores from the final matrix multiplication, one per vocabulary entry, before softmax. Can be negative; only the differences between them matter
Greedy decoding	Always taking the highest-scoring token. What "temperature zero" does in practice, since dividing by zero is not defined
Temperature	A number the logits are divided by before softmax. Below 1 sharpens the distribution, above 1 flattens it, and it never changes the order
Top-k	Keeping only the k highest-scoring tokens and renormalising. Keeps the same number of options however confident the model is
Top-p	Also called nucleus sampling. Keeping the smallest set of tokens whose probabilities add to at least p, then renormalising. Narrow where the model is confident, wide where it is not, at the cost of not knowing in advance how many survive
Stop token	An entry in the vocabulary meaning *finished*, scored and sampled like any other token. Often called an end-of-sequence token. A model stops talking by predicting that it has stopped
Stop condition	A string you nominate that halts generation when it appears. Providers call it a stop sequence; it is not Chapter 4's stopping condition. One of the ordinary ways a reply ends, alongside the model's own stop token and your maximum

Worth Memorising

Item	Value or formula
Temperature	$p_i = \exp(z_i/T) / \sum_j \exp(z_j/T)$
What temperature does	Below 1 sharpens, above 1 flattens, never reorders
Temperature and greedy	No effect whatsoever — greedy takes the largest logit either way
Top-p against top-k	Top-p keeps the smallest set reaching p; top-k keeps a fixed count
The order of operations	Temperature, then the cut, then draw
Why $T = 0$ is not a guarantee	Float order, batching, expert routing, provider change

Review Questions

1. What does temperature do to the numbers?
2. Why is temperature zero still not a guarantee of the same output twice?
3. Your call is set to greedy decoding and a colleague suggests lowering the temperature

to make it more reliable. What do you say?
4. When would you prefer top-p to top-k, and why?
5. Name the ordinary ways a reply can end, say which one is the model's doing, and name one way that is not on that list.
6. You change a prompt, run it once, and the answer is better. What have you learned?

Review Answers

1. It divides every logit by T before softmax. Because softmax exaggerates gaps and dividing the logits divides the gaps, a temperature below 1 sharpens the distribution towards the top token and a temperature above 1 flattens it towards the rest. In the chapter's example the top token goes from 0.636 at $T = 1$ to 0.865 at $T = 0.5$ and down to 0.429 at $T = 2$. It never changes the order, because dividing every logit by the same positive number cannot reorder them — so it changes how much of the rest is in play, not what the model thinks is most likely.
2. Because temperature zero is greedy decoding, and the thing greedy depends on — which logit is largest — can change for reasons that have nothing to do with the model. Floating-point addition is not associative, so a different summation order gives different last digits; a GPU splits a long addition across many units and combines their partial sums, and how it splits depends on what else was in your batch. Two logits a hair apart can swap, greedy takes the other token, and from there the reply may diverge. Models with expert routing can also depend on batch composition, and the provider may have changed the model, or the hardware and libraries it runs on. Nothing there is randomness in the usual sense, which is why a seed does not fix it.
3. That temperature has no effect on greedy decoding at all. Greedy takes the largest logit, and dividing every logit by the same positive number cannot change which is largest — so the dial they are reaching for is already disconnected. If the output is varying anyway, the cause is one of the four in this chapter, and the fix is in their code: assert on shape rather than on strings, log what came back, and stabilise with a validator or a cache rather than a setting.
4. When the model's confidence varies across the generation, which it usually does. Top-k keeps the same number of options whether the model is certain or not — so it leaves thirty-nine bad options in play when one token is at 0.98, and cuts away good ones when a hundred tokens are near-equal. Top-p keeps however many it takes to cover the mass, so it is narrow where the model is sure and wide where it is not. The cost is that the number of options is no longer something you can state in advance.
5. The model samples a stop token — an ordinary entry in the vocabulary meaning *finished*, chosen by the same procedure as any other token, and the only one of the ordinary three that is the model's doing. Your maximum truncates the reply, which is a cut rather than an ending and reports itself differently. And a stop condition you nominated is matched in the generated text. Off that list: the input can fill up, and your own

code can cancel the request.

6. Almost nothing. You have one sample from each of two distributions, and you have no idea how much either varies from call to call, because you drew once from each. The output you liked may be an ordinary result of the new prompt or an unusually good one, and nothing distinguishes those from where you are standing. Run it several times before believing it, and Chapter 20 is where that becomes a measurement with an interval around it.

> **You understand this when** you can explain to a colleague why their temperature-zero pipeline is not reproducible, without using the word "random".

CHAPTER 14

How It Learned to Do This

Every chapter so far has taken the model's numbers as given. This one says where they came from — far enough to answer the questions an interview asks, and then it stops, on purpose and visibly. By the end you will be able to write down what training optimises, say what each of the three stages changes, and say exactly which sentence is the last one this book will explain.

The problem. Somebody trained this on "the internet". That sentence explains nothing.

The question. What was being optimised, how were the numbers changed, and what does each stage of training actually buy?

The Objective, and Why It Needs No Labels

Chapter 9 said the model produces one probability for every vocabulary entry. Training is the process of making those probabilities better — and "better" has a precise meaning that is worth getting exactly right, because it explains nearly everything else.

The model is asked to predict the next token, and the text supplies the answer.

Take a sentence from the training data. Cut it anywhere. Everything before the cut is the input; the token immediately after the cut is the correct answer. Move the cut along one token and you have another example. A thousand-token document is a thousand training examples, and nobody had to label any of them.

```
the cat sat on the mat
^^^^^^^^^^ input       ^^^ answer
```

This is why the stage can consume an enormous amount of text: **the text is its own answer key.** There is no annotation step, no agreement between labellers, no cost per example beyond the compute. It is the single most consequential fact about how these models came to exist, and it is the reason the word "unsupervised" gets used, though "self-supervised" is the more accurate one — there *are* labels, and the text is where they come from.

The objective, written down:

$$\mathscr{L} = -\frac{1}{N}\sum_{i=1}^{N} \log p(t_i \mid t_1 \ldots t_{i-1})$$

Read it with Chapter 8's four steps. t_i is the token at position i; $p(t_i \mid t_1 \ldots t_{i-1})$ is the probability the model gave to that token, having seen everything before it; log is Chapter 8's logarithm; $\sum$ runs over every position; $\frac{1}{N}$ averages. The minus sign makes it a quantity to *minimise* rather than maximise.

In words: **the loss is the average surprise, over every position, at the token that actually came next.** Training adjusts the numbers to reduce it.

Cross-Entropy: How Wrong Was That Guess?

That quantity has a name. $-\log p$ of the correct answer is the **cross-entropy** loss, and it is worth computing once to see how it behaves.

Chapter 9's worked example ended with this distribution after "the cat sat":

```
the 0.012   cat 0.234   sat 0.032   mat 0.086   on 0.636
```

Suppose the text really did continue with "on". The loss for that one prediction is

```
-ln(0.636) = 0.453
```

Now suppose it continued with "cat" instead — a reasonable word the model thought less likely:

```
-ln(0.234) = 1.452
```

And suppose it continued with "the", which the model nearly ruled out:

```
-ln(0.012) = 4.423
```

Three properties fall straight out of those numbers, and all three matter.

The loss depends only on the probability of the right answer. What the model thought about the other four tokens never enters. A model that puts 0.636 on the correct token scores 0.453 whether the remaining 0.364 was spread evenly or piled on one rival.

Being confidently wrong is punished enormously. Going from 0.234 to 0.012 — still not zero — takes the loss from 1.452 to 4.423. As the probability of the true token approaches zero the loss grows without limit. That asymmetry is the point: a model that assigns almost nothing to something that then happens is making the worst kind of mistake.

And a perfect prediction scores zero. $-\log 1 = 0$. The loss is never negative, because a probability is never above 1.



Logarithms here are natural logs, as in Chapter 8. Some sources use base 2, which multiplies every number above by about 1.44 and changes the units from *nats* to *bits*. Nothing about the argument changes; if you compare a loss figure with somebody else's, check which they used.

Perplexity

Cross-entropy is in awkward units. **Perplexity** fixes that by exponentiating it:

$$\text{perplexity} = \exp(\mathscr{L})$$

And it has a reading that makes it genuinely useful. Suppose a model is completely undecided across a vocabulary of five, giving each token 0.2. The loss is $-\ln(0.2) = 1.609$, and the perplexity is $\exp(1.609) = 4.998$ — which is exactly 5, since exponentiating $-\ln(0.2)$ undoes the logarithm and leaves $1/0.2$. Exactly the number of options.

That is the general case: **a uniform choice among V options has perplexity V.** So perplexity reads as *the number of equally likely options the model is effectively choosing between*. A perplexity of 3 means the model is about as uncertain as someone picking at random from three things. A perplexity of 128,000 — the vocabulary size Chapter 9 uses for a real model — means it did no better overall than a uniform guess.

Work a short sequence. Four predictions, with these probabilities for the tokens that actually came next:

```
position   p       -ln p
   1     0.636     0.453
   2     0.234     1.452
   3     0.086     2.453
   4     0.636     0.453
                   -----
         average   1.203

perplexity = e^1.203 = 3.330
```

So across those four tokens the model was about as uncertain as a fair choice between three and four options. That is a number you can compare: between two models on the same text, or for one model as training proceeds.



Perplexity is a measure of how well a model predicts *some particular text*. It is not a measure of whether answers are correct, useful or safe, and two models' perplexities are only comparable on the same text with the same vocabulary — a model whose tokeniser cuts the text into fewer pieces is being asked a different question. Interview answers that treat perplexity as a quality score are the ones that get followed up on.

The Loop, From Outside

The training loop has four steps, and it is simpler than people expect.

```
1. take a batch of text
2. run it forward, getting a distribution at every position
3. compute the loss against the tokens that actually came next
4. change every parameter a little, in the direction
   that reduces the loss

then go back to 1, a very large number of times
```

How many times is set by how much text there is: the first stage works through its corpus once, or close to it, and the number of steps is the number of batches that takes. Running over the same text many times is not what these models do, and *why* is outside this book along with everything else about why the training works.

Steps 1 to 3 you can already do — they are Chapters 9 to 13 run forwards and then one logarithm. Step 4 is the one this book does not open, and here is the whole of it:

$$\theta \leftarrow \theta - \eta \, \nabla_\theta \mathscr{L}$$

One sentence of notation, and then we stop. θ stands for all the parameters at once. $\nabla_\theta \mathscr{L}$ is the **gradient** — for each parameter, how much the loss would rise if that parameter rose, computed for all of them together. η is the step size. The rule says: move every parameter a small distance in the direction that reduces the loss, and repeat.

That line is the edge of this book. How the gradient is computed for billions of parameters is called backpropagation. Why repeating this rule arrives anywhere useful is optimisation theory. What step size to use, and how to adjust it, is optimiser design. None of the three is in these pages, Chapter 17 lists all of them, and the honest interview answer is "I can state the update rule and I can tell you what it does; I have not studied why it works."



That answer is a good one, and it is better than an improvised version of the real explanation. Interviewers can tell the difference between a boundary somebody has drawn deliberately and one they have fallen off.

What is worth carrying instead is the *shape* of the loop, because three practical facts come straight out of it. Training happens once, before you ever call the model — Chapter 1's point, now with a mechanism. Nothing you send at inference runs step 4, so answering your request changes no parameter. (Whether what you send is kept and used in a *later* training run is a different question, and a matter of the provider's policy rather than of the loop.) And steps 1 and 4 are the same whatever the data is. What step 3 compares changes for one of the three stages, and so does what step 2 has to run for it — a point the third stage's own section makes concrete. That is why the next section is about data rather than about algorithms.

Three Stages, Three Kinds of Data

A model you can talk to was made in three stages. The loop above is involved in all three; what differs is the data — and, for the third stage only, the loss at step 3.

Pretraining runs the loop on an enormous quantity of general text. This is where essentially all of the model's knowledge of language, facts, code and document structure comes from, and it accounts for the overwhelming majority of the computation ever spent on the model. What comes out is a model that *continues text*. Ask it a question and it may well continue with more questions, because that is a thing documents do.

Supervised fine-tuning runs the same loop, with the same cross-entropy objective, on a much smaller set of curated examples: a request, and a good response to it. Nothing about the mechanism changes — it is still predicting the next token — but the text it is now predicting is the text of a helpful reply. This is the stage that turns a text continuer into something that answers.

That distinction has a name you will meet on every provider's model list. A model that has had only the first stage is a **base model**; one that has had the later stages is usually called an instruct or chat model. You can call a base model, and the result is instructive: give it "What is the capital of France?" and it may well continue with "What is the capital of Spain? What is the capital of Italy?", because a list of questions is a thing documents contain. It is not being unhelpful. It is doing the only job it was ever trained for.

Preference optimisation is the one that is genuinely different. Its data is not a single correct answer. It is *pairs*: for the same request, one reply people preferred and one they did not. Training adjusts the model to make the preferred kind more likely relative to the other.

	Data	What it gives	Rough share of the compute
Pretraining	Enormous, general, unlabelled text	Knowledge, language, code, structure	Nearly all of it
Supervised fine-tuning	Thousands to millions of request–response pairs, curated	Answering rather than continuing; format; tone	A small fraction
Preference optimisation	Pairs of replies, one preferred	Which of the things it could say it tends to say	A small fraction

And the sizes are worth putting numbers on, because the ratio is the point. Orders of magnitude, for a model of the moment:

```
pretraining     trillions of tokens
fine-tuning     tens of thousands to a few million examples
preference      tens of thousands of pairs
```

Put one pair of those side by side. Ten trillion tokens is about 7.5 trillion words, at Chapter 10's three-quarters of a word to the token — which at a hundred thousand words a book is **seventy-five million books**. A hundred thousand fine-tuning examples of five hundred tokens each is fifty million tokens — **one part in two hundred thousand** of what the first stage saw.

```
pretraining     10,000,000,000,000 tokens
fine-tuning             50,000,000 tokens

the second is 0.0005 per cent of the first
```

That ratio is the whole argument of the last section of this chapter, arrived at by arithmetic rather than by assertion. It is also where the compute shares in the table come from: the work of training is dominated by running the loop over tokens, so a stage that sees one part in two hundred thousand of the tokens accounts for about that share of the computation. A stage that sees one part in two hundred thousand of the tokens cannot be where the *bulk* of a model's knowledge came from. Note what that does and does not reach: it is a claim about share of exposure, and it says nothing on its own about whether one small corpus comes back.



Those figures are dated details of the worst kind — they have moved by an order of magnitude more than once and will again. What does not move is the *shape*: the first stage is larger than the other two by a factor with many zeroes in it. Check the ratio, not the numbers.



The names vary and the boundaries blur. You will meet "instruction tuning" for the second stage, and "alignment", "RLHF" or "post-training" covering the second and third together. The three-way split by *what the data looks like* — raw text, correct answers, ranked pairs — is the durable version and the one to reason with.

The Preference Objective

Chapter 8 said you should be able to read a formula by naming its symbols. Here is one worth being able to recognise, because interviewers ask about it by name.

$$\mathcal{L}_{\text{DPO}} = -\log \sigma\left(\beta \log \frac{\pi(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)$$

Name the symbols. $\mathcal{L}_{\text{DPO}}$ is this stage's own loss, and the subscript is there because it is not the cross-entropy above. x is the request. y_w is the reply people preferred — w for winner — and y_l the one they did not. π is the model being trained and π_{ref} is the same model as it was before this stage started, kept fixed as a reference. β is a setting controlling how hard to push. σ is the logistic function, written $\sigma(z) = 1/(1 + e^{-z})$, which squashes any number into the range 0 to 1 and gives exactly 0.5 at zero.

In words, working outward from the inner brackets: for each of the two replies, take how much *more* likely this model makes it than the reference model did. Subtract the loser's figure from the winner's. The loss is small when that difference is large and positive. So the procedure **raises the preferred reply's probability relative to the rejected one, measured against where the model started** — and subtracting the loser's figure is what rules out the easy win. Make *both* replies ten times more likely and the two figures each rise by the same amount, so the difference is unchanged and the loss does not move. Only the gap counts. The reference model in the denominators does something else: it sets the zero point, so both figures are measured from where the model was before this stage began.

And this is what the third stage does to the loop. Step 2 no longer runs one model over one stretch of text: it runs *two* models — the one being trained and the frozen reference — over *two* replies, to get the four probabilities the formula needs. Step 3 then compares two whole replies rather than scoring one next token. Steps 1 and 4 are unchanged.

This is **direct preference optimisation**, usually called DPO. It is stated here and not derived; why this particular expression is the right thing to minimise is outside this book.

But you can put numbers in it, and you should. Take $\beta = 0.5$, and write r_w and r_l for the two logarithms — how much more likely this model makes each reply than the reference did. They are logarithms, so $r_w = 2$ means the winner has become $e^2 = 7.389$ times as likely and $r_l = -1$ means the loser has fallen to $e^{-1} = 0.368$ of what it was. Three cases:

state of training	r_w	r_l	arg	sigma	loss
before this stage	0.0	0.0	0.0	0.500	0.693
moved the right way	2.0	-1.0	1.5	0.818	0.201
moved the wrong way	-1.0	2.0	-1.5	0.182	1.701

The middle column is $0.5r_w - 0.5r_l$, and the loss is $-\ln\sigma(\text{arg})$. Check the first row by hand: both logarithms are zero, so the argument is zero, $\sigma(0) = 0.5$, and $-\ln(0.5) = 0.693$. **That row is where every example starts**, because at the moment the stage begins the model *is* the reference.

Now the point from two paragraphs ago, in one line. Make both replies ten times more likely: both logarithms rise by $\ln 10$, the subtraction cancels it, and the loss stays at 0.693. The model has changed and the objective has not noticed.



There is an older approach to the same job, where human rankings are used to train a separate model that scores replies, and the main model is then trained against that scorer. That is reinforcement learning from human feedback, RLHF. DPO's selling point is that it skips the separate scorer and optimises the preferences directly. Both are in use; the thing to be able to say is what the *data* is — pairs of replies with a preference between them — because that is what does not change.

What Each Stage Cannot Do

This is the part with practical consequences, and it is where most expensive mistakes in this material begin.

No stage gives the model a way to look things up. Chapter 1 said the model itself cannot fetch anything, and no amount of training changes that. Training moves numbers; it does not attach a database.

Knowledge is overwhelmingly a pretraining phenomenon, and that much is arithmetic. Whether fine-tuning on a few thousand of your own documents installs their contents as recallable facts is a separate question the ratio does not settle — a small curated set is exactly the case where repeated exposure might help rather than hurt. What practitioners report is that it does not work reliably, and Chapter 20 is how you would find out for your own case. The stage is small, the examples are few, and what it is good at is shaping *how* the model responds rather than *what it knows*. The reliable way to make a model use your information is still Chapter 1's: put the information in the request.

And nothing in training is a test of whether anything is true. Every stage optimises agreement with text that people produced — tokens that continue a document, replies people wrote, replies people preferred. None of those is a test of correctness. Chapter 16 is where that becomes the characteristic failure of these systems.



"We'll do some fine-tuning on our docs" is the most common wrong first answer in this field. It is not never right — Chapter 39 is about when it genuinely is, and what it costs you — but it is the last lever to reach for and not the first, and the reason is in this section rather than in anybody's opinion.

What each stage *can* do is worth stating just as plainly. Pretraining gives the model essentially everything it knows. Supervised fine-tuning gives it a job and a format. Preference optimisation chooses between the things it was already capable of saying. That last one is the subtlest and the most useful to be able to say out loud: **a preference stage is not where knowledge comes from — nothing in its objective rewards adding any, and what it changes is which of its existing behaviours come out.**

Lab: Compute a Loss

Pen, paper and a calculator, then ten lines of Python. Twenty-five minutes, and there is a key for every step.

A model with a vocabulary of four has just made three predictions. In each case the probability it gave to the token that actually came next is shown.

position	token that came next	p
1	yes	0.500
2	no	0.250
3	yes	0.125

The exponentials and logs you need: $\ln 2 = 0.693$, $\ln 4 = 1.386$, $\ln 8 = 2.079$, $e^{1.386} = 3.999$.

Step 1. Compute the cross-entropy loss for each of the three predictions.

Expected result: $-\ln(0.5) = \ln 2 = 0.693$; $-\ln(0.25) = \ln 4 = 1.386$; $-\ln(0.125) = \ln 8 = 2.079$. Note what happened: each halving of the probability added the same 0.693 to the loss. That is the logarithm of Chapter 8 doing what logarithms do.

Step 2. Compute the average loss over the three positions, then the perplexity.

Expected result: $(0.693 + 1.386 + 2.079)/3 = 4.158/3 = 1.386$, and $\exp(1.386) = 3.999$ — which is 4, since 1.386 is $\ln 4$ rounded. The model was about as uncertain, on average, as a fair choice between four things — which over a vocabulary of four means it did no better overall than a uniform guess, even though it was better than uniform at position 1 and worse at position 3.

Step 3. Without computing anything: the model is retrained and now gives the correct token a probability of 1.0 at all three positions. What are the loss and the perplexity?

Expected result: loss 0 and perplexity 1. $-\log 1 = 0$ and $\exp(0) = 1$. A perplexity of 1 means no uncertainty at all — one option. It is the floor, and no model reaches it on real text, because

real text is not perfectly predictable.

Step 4. The model gives the correct token 0.5 at position 1 and the *wrong* tokens 0.3, 0.1 and 0.1. A second model gives the correct token 0.5 and the wrong ones 0.48, 0.01 and 0.01. Which has the lower loss at that position?

Expected result: **neither — they are identical, at 0.693.** The loss depends only on the probability assigned to the token that actually came next. Everything the model thought about the alternatives is invisible to it. This is worth meeting, because it is the first thing people guess wrong about cross-entropy.

Step 5. In a new empty folder, save this as `loss.py` and run it:

```
import math

# probability given to the token that actually came next
ps = [0.636, 0.234, 0.086, 0.636]

losses = [-math.log(p) for p in ps]
average = sum(losses) / len(losses)

for p, l in zip(ps, losses):
    print(f"  p={p:.3f}   -ln p = {l:.3f}")
print(f"average loss = {average:.3f}")
print(f"perplexity   = {math.exp(average):.3f}")
```

Expected result:

```
  p=0.636   -ln p = 0.453
  p=0.234   -ln p = 1.452
  p=0.086   -ln p = 2.453
  p=0.636   -ln p = 0.453
average loss = 1.203
perplexity   = 3.329
```

Those are the chapter's own four positions. Note that the printed perplexity is 3.329 while computing $e^{1.203}$ from the rounded average gives 3.330 — the program averaged before rounding and you did not.

Step 6. Change the first probability to 0.001 and run it again. What happens to the average, and what does that tell you about one bad prediction in a long document?

Expected result: the loss at that position jumps to 6.908 and the average to 2.817, giving a perplexity of 16.719. **One confidently wrong prediction dominated three reasonable ones.** That is the asymmetry from earlier in the chapter, and it is why one confidently wrong prediction costs more than several mildly wrong ones.

Step 7. Name the three stages of training, say what data each one consumes, and say which one your company would be changing by "fine-tuning on our support tickets".

Expected result: pretraining on enormous general text; supervised fine-tuning on curated request–response pairs; preference optimisation on pairs of replies with a preference between them. Fine-tuning on support tickets is the second stage, and what it would reliably change is the *shape* of replies — tone, format, the house way of saying things. What it would not reliably do is install the tickets' contents as facts the model can recall — reliably being the word, since the ratio argument bounds the share of knowledge and not this particular case.

Summary, Key Terms, and Review Questions

Summary

Training optimises next-token prediction, and the text supplies its own answers, which is why the first stage can consume so much of it. The objective is **cross-entropy**: the average of $-\log p$ over every position, where p is the probability the model gave to the token that actually came next. It depends only on the correct token's probability, it punishes confident wrongness without limit, and it is zero for a perfect prediction. **Perplexity** is its exponential, and reads as the number of equally likely options the model is effectively choosing between — a uniform choice among V options has perplexity V. The loop is: run forward, compute the loss, move every parameter a little in the direction that reduces it, repeat. That last step is $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$, and it is the edge of this book. Three stages use that loop on three kinds of data, the third with a loss of its own: **pretraining** on enormous general text, which is where essentially all knowledge comes from; **supervised fine-tuning** on curated request–response pairs, which turns a text continuer into something that answers; and **preference optimisation** on pairs of replies with a preference between them, stated once as the DPO objective. No stage gives the model a way to look anything up, knowledge is overwhelmingly a pretraining phenomenon, and nothing in training tests whether anything is true.

Key Terms

Term	Meaning
Cross-entropy	The training objective: the average of $-\log p$ over every position, where p is the probability given to the token that actually came next. Zero for a perfect prediction, unbounded for a confident mistake
Perplexity	The exponential of the cross-entropy. Reads as the number of equally likely options the model is effectively choosing between; a uniform choice among V options has perplexity V

Term	Meaning
Gradient	For each parameter, how much the loss would rise if that parameter rose. The update rule moves every parameter a small step against it. How it is computed is outside this book
Pretraining	Running the loop on an enormous quantity of general text. Where essentially all of a model's knowledge comes from, and nearly all of the computation
Supervised fine-tuning	Running the same loop with the same objective on curated request–response pairs. Turns a model that continues text into one that answers
Base model	A model that has had only pretraining. It continues text rather than answering, which is what the later stages change
Direct preference optimisation (DPO)	The preference objective written as one formula: raise the preferred reply's probability relative to the rejected one, both measured against the model as it was before this stage. Stated in this book, not derived
Preference optimisation	Training on pairs of replies with a preference between them, to make the preferred kind more likely relative to the other. Changes which of its existing behaviours come out; nothing in its objective rewards adding knowledge

Worth Memorising

Item	Value or formula
The objective	$\mathscr{L} = -\frac{1}{N}\sum_i \log p(t_i \mid t_1 \dots t_{i-1})$
Perplexity	$\exp(\mathscr{L})$; a uniform choice among V options gives V
The update rule	$\theta \leftarrow \theta - \eta \nabla_\theta \mathscr{L}$ — and that is the edge of this book
The three stages	Pretraining, supervised fine-tuning, preference optimisation
Where knowledge comes from	Pretraining, overwhelmingly
What the loss ignores	Everything except the probability of the token that actually came next

Review Questions

1. Name the training stages and say what each one changes.
2. Why can the first stage consume so much more data than the other two?
3. A model gives the correct next token a probability of 0.5. Another gives it 0.5 but is much more confident about its second choice. Which has the lower loss?
4. What does a perplexity of 20 mean, and what would you need to know before compar-

ing it with somebody else's figure?
5. Your colleague wants to do some fine-tuning on the company's documents so the model "knows" them. What do you say?
6. Which line of this chapter is the edge of the book, and what is on the other side of it?

Review Answers

1. **Pretraining** runs next-token prediction over an enormous quantity of general text; it is where essentially all the model's knowledge of language, facts, code and structure comes from, and it accounts for nearly all the computation. What it produces continues text rather than answering. **Supervised fine-tuning** runs the same loop with the same cross-entropy objective on a much smaller set of curated request–response pairs, which turns a continuer into something that answers, and sets format and tone. **Preference optimisation** uses pairs of replies with a preference between them and shifts which of the model's existing behaviours come out; it does not where knowledge comes from, because nothing in its objective rewards adding any.
2. Because its data needs no labels. Cut any text anywhere: what comes before is the input and the next token is the correct answer, so a thousand-token document is a thousand examples and the text is its own answer key. The other two stages need curated material — someone writing a good response, or people choosing between two replies — which is slow and expensive per example.
3. Neither; they are identical. Cross-entropy is $-\log p$ of the token that *actually came next*, so what the model believed about every other token is invisible to the loss. Two models that agree on the correct token's probability score the same however differently they spread the rest.
4. That the model was about as uncertain as a fair choice between twenty options, on the text it was measured on. Before comparing it you would need to know that the figure is over the same text, and above all that the tokeniser matches — a model whose vocabulary cuts the text into fewer pieces is being asked a different question, so the numbers are not comparable. And perplexity measures prediction of particular text, not whether answers are correct or useful. The one thing you do *not* need to check is the logarithm base: a base-2 loss is about 1.44 times a natural one, and exponentiating in the matching base cancels that exactly, so two perplexities over the same text agree whichever base was used to compute them.
5. That it is the last lever rather than the first, and that the stage they are reaching for shapes *how* the model responds rather than *what it knows*. Knowledge is overwhelmingly a pretraining phenomenon — that part is arithmetic — and practitioners report that a few thousand fine-tuning examples do not reliably install their contents as recallable facts, and nothing in training attaches a database. The reliable way to make the model use the documents is to put the relevant parts in the request. Chapter 39 is where fine-tuning is genuinely the right answer, and what it costs.

6. $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$ — move every parameter a small step in the direction that reduces the loss. On the other side of it are how the gradient is computed for billions of parameters, which is backpropagation; why repeating the rule arrives anywhere useful, which is optimisation theory; and how the step size is chosen and adjusted, which is optimiser design. Chapter 17 lists all three, and the right thing to say is that the boundary is deliberate.

> **You understand this when** you can write the training objective down, say what each of the three stages changes, and name the one line whose explanation you are deliberately skipping.

CHAPTER 15

What It Costs From the Inside

Chapter 1 told you the wait has two phases and that you feel them differently. This chapter says why, and the answer turns out to explain most of the cost of running these systems: why the first token is slow, why memory rather than arithmetic is usually the binding constraint, why your request is run alongside strangers', and why a model with a hundred billion parameters can be cheaper per token than one with fifty.

The problem. My first token takes two seconds and the rest arrive in a stream. Nobody has told me why.

The question. What is the machine actually doing during each of those two waits, what does it have to keep in memory while it does, and which of those things is the one that costs?

Two Different Computations

The two phases are not the same work at different speeds. They are different work.

Prefill is what happens to your input. The whole prompt is there at once, so every position can be computed at the same time: the embedding lookups, the attention grids, the feed-forward blocks, all of it, for all n tokens together. One pass through the stack handles the lot.

Decode is what produces the reply, and it is Chapter 9's loop: one forward pass per token, each pass producing one distribution, each token appended before the next pass runs.

	Prefill	Decode
Handles	The whole prompt at once	One token at a time
Passes through the stack	One	One per output token
Positions computed per pass	All of them	One
What limits it	Arithmetic	Memory traffic — see below
What you feel	The wait before the first word	The pace of the words after it

And that last row is the thing to understand, because it is where the surprise lives. The two phases are limited by different resources, which is why they behave so differently when you change things.

Why the First Token Is Slow

Prefill has to do all the work for every position in your prompt before a single token can be produced. Chapter 11 counted it: building the attention grid is about $n^2 d$ operations per layer, and the feed-forward blocks add their own work at every position.

So the wait before the first word grows with the prompt. Send twice as much and prefill does more than twice the work, for the reason Chapter 11 gave — the grid is square in n.

Then decode starts, and each token takes roughly the same time as the one before it — roughly, and two sections below says what makes it drift. The arithmetic for one new token is tiny by comparison: one position, not n of them.

But decode is not fast because it is doing less arithmetic. It is doing almost no arithmetic, and it is still not instant, and the reason is the one thing a beginner never guesses: **to compute one token, every parameter the pass uses has to be read out of memory.** Chapter 12 counted them — about five billion in the layers of a modest model. Reading five billion numbers takes time whatever you do with them, and for one token you do almost nothing with them.

Put the two side by side for one decode token of a seven-billion-parameter model at two bytes each. Each parameter the pass uses is used once, in a multiply and an add:

```
bytes read     7,000,000,000 x 2  =  14,000,000,000
operations     7,000,000,000 x 2  =  14,000,000,000

one operation per byte read
```

One operation per byte, on hardware designed to do hundreds of operations for every byte it fetches. The arithmetic unit is idle almost all the time, waiting. A step with that shape is called **memory-bound**, as against **compute-bound** when the arithmetic is the thing you

are waiting on — and decode is the most memory-bound step in this book. Prefill is the opposite: the same single read of the parameters serves every position in the prompt at once, so the operations multiply by n while the bytes do not.

That is why decode is described as limited by memory traffic rather than by arithmetic. The processor is waiting for numbers to arrive. It is also why the next two sections matter more than anything about raw speed.



This is the mechanism behind every "it is slow" conversation you will have about these systems, and the two halves have different cures. A long *prompt* makes prefill slow, and the cure is to send less. A long *reply* makes decode slow, and the cure is to ask for less. Chapter 52 is where the levers are; this chapter is why they exist.

The Key–Value Cache

Chapter 11 left a problem on the table. Each decode pass runs attention over the whole sequence so far — but all of it except the newest token is unchanged from last time, so recomputing the keys and values for every earlier position on every pass is pure waste.

The fix is to keep them. The **key–value cache** holds the key and value vector for every position, in every layer, for the whole sequence so far. Prefill writes it for the whole prompt at once; then on each decode pass the model computes k and v for the one new position, appends them, and attends against the lot.

That turns the work per decode token from "recompute everything" into "compute one position and read the rest", which is the difference between a system that works and one that does not.

And it is also the memory bill. Here is the calculation, and it is worth being able to do it from memory, because it is the one that decides how many users fit on a machine.

For each token you store a key and a value — that is the 2 — in every layer, each of width d, at some number of bytes per number:

$$\text{cache} = 2 \times \text{layers} \times d \times \text{tokens} \times \text{bytes}$$

Put the shape of Chapter 12's worked model in it: 32 layers, width 4,096, two bytes per number.

```
per token   2 x 32 x 4,096 x 2  =  524,288 bytes  =  512 KiB
```

Half a mebibyte per token. Now multiply by a conversation:

```
 1,000 tokens      524,288,000 bytes    0.49 GiB
 8,000 tokens    4,194,304,000 bytes    3.91 GiB
32,000 tokens   16,777,216,000 bytes   15.63 GiB
```



Two units, and the difference has embarrassed a lot of people at the moment a model did not fit. A **GiB** is $1{,}024^3$ bytes and a **GB** is $1{,}000^3$, about 7 per cent smaller. A card sold as "40GB" holds 40 GiB; a model quoted at 14 GB holds 14 thousand million bytes, which is 13.0 GiB. This chapter quotes memory in GiB and model sizes in GB, because that is how each is sold.

And then by the number of people talking to it at once:

```
ten users at 8,000 tokens each  =  39.1 GiB
```

That is on top of the model's own parameters, on a card with a fixed amount of memory. **The cache, not the model, is usually what decides how many conversations fit.** A model that takes 13 GiB of weights and leaves 27 GiB free on a 40 GiB card can serve six of those conversations at once, which is far fewer than the weights alone would suggest.



Notice what the cache does *not* depend on: how hard the question is, or how good the answer is. It depends on the number of tokens and nothing else. A user who pastes a long document into a chat and then asks three short questions is occupying that memory for the whole conversation. This is the mechanism behind every limit you have ever met on how much you may send.



Real systems shave this figure. Several heads can be made to share one set of keys and values, which divides the figure by the number of heads sharing; the cache can be stored at fewer bytes per number than the weights; and it can be held in pages rather than one block so the space is not wasted. The arithmetic above is the shape, and the shape is what you reason with.

Why Doubling the Prompt More Than Doubles the Work

Three things grow when your prompt gets longer, and they grow at different rates. Putting them side by side is the clearest way to see why "it is just a bit more text" is wrong.

```
what grows            how
-----------------------------------------
prefill, the rest     with n
prefill, attention    with n squared
cache memory          with n
decode, per token     with n, in cache reads
```

And the first two deserve numbers, because "with n squared" is the half everybody quotes and the smaller half for most prompts. Multiply-adds in one layer, at Chapter 12's shape:

```
tokens      the rest        attention       attention's
                                               share
 1,000   157,286,400,000     8,192,000,000       5%
 2,000   314,572,800,000    32,768,000,000       9%
19,200 3,019,898,880,000 3,019,898,880,000      50%
32,000 5,033,164,800,000 8,388,608,000,000      62%
```

So at a thousand tokens the squared term is one part in twenty, and doubling the prompt multiplies prefill by 2.1 rather than by 4. The two halves are equal at 19,200 tokens, and only past that does the squared term run the show. **"Prefill is quadratic" is true of attention and not of prefill**, until the prompt is long — and knowing where "long" starts is the difference between an answer and a slogan.

The first is Chapter 11's grid. The second is the section above. **The third is the one people miss**: every decode pass attends over the whole cache, so as the conversation gets longer each new token costs more to produce than the last. A reply that starts fast gets slower.

So doubling the prompt more than doubles the prefill arithmetic — the attention part of it quadruples, and that part is a minority until the prompt is long — doubles the memory you are holding, and makes every token of the reply more expensive. None of those is "a bit more text".



This is the honest answer to "why can't I just put everything in the prompt". Not "because the model gets confused" — that is Chapter 16's separate and also-true point — but because three different costs rise at once and one of them faster than proportionally, and somebody is paying for it.

Batching: Your Request Runs With Others

Decode is limited by reading the parameters out of memory. Read them once and you can compute one token. Read them once and you could just as easily compute a token for *fifty* different conversations, because the expensive part — getting the numbers — has already happened.

That is **batching**, and it is why providers do it. One read of the parameters serves every request in the batch, so the cost per request falls — but by how much depends entirely on how much cache each request is also reading, and that is worth doing rather than guessing. With short conversations the cache is almost nothing beside the 13 GiB of weights, and a batch of fifty really does approach a fiftyfold saving: at 100 tokens each it is about forty-two times. At the 8,000 tokens the next paragraph picks, it is about *four*.

But only the read of the parameters is shared, and nothing else is. Every request in the batch has its own cache to read and its own arithmetic to do, so the saving flattens as the batch grows. At 8,000 tokens each — on a machine large enough to hold them, which the lab's is not — fifty conversations read fifty lots of 3.91 GiB of cache against one shared 13 GiB of weights — the unshared reads have already overtaken the thing being shared. And past some batch size the machine runs out of arithmetic rather than memory traffic — it becomes compute-bound — and every token gets slower. That is the answer to *then why not batch ten thousand*, and it is the sentence to have ready when somebody asks.

Three consequences, and all three show up in your work.

It is why the price per token is what it is. You are not paying for a machine to yourself. You are paying a share of a machine that is serving many people at once, and the published price reflects that. Chapter 53 is what to do about the bill that results.

It is one of the reasons Chapter 13's temperature-zero problem exists. The batch's shape changes how the arithmetic is grouped, and a different grouping changes the last digits, and a near-tie between two logits can flip. Your output can depend on who else was calling at that moment. Chapter 13 gave the consequence; this is the mechanism behind two of its four causes.

And it is why your own latency is not entirely yours. If the system is busy, your request waits to join a batch. If it is quiet, it may run in a small batch and finish sooner.



Serving systems do not fill a batch and then run it to completion. They add and remove requests as they arrive and finish, so the batch is a moving population — which is a serving technique in its own right and belongs to Chapter 52. What matters here is the mechanism underneath it: the parameters are read once and shared.

Quantisation: Bits Per Parameter

If decode is limited by reading parameters, then making each parameter *smaller* makes decode faster. That is **quantisation**: storing each number in fewer bits.

The arithmetic is as simple as it looks. For a model with seven billion parameters:

```
16 bits   2 bytes each    14.0 GB
 8 bits   1 byte each      7.0 GB
 4 bits   half a byte      3.5 GB
```

Halve the bits, halve the memory, and roughly halve the time spent reading. It is the single biggest lever on the size of machine a model needs, which is why a model that will not fit on a card at 16 bits will often fit comfortably at 4.

What it costs you is precision. Each number is stored less exactly, and the outputs change — usually a little, sometimes in ways that matter for a particular task. What practitioners report is that the effect is small at 8 bits and usually acceptable at 4: a practitioner report on Chapter 6's ladder rather than anything this chapter measures, and that *usually* is doing real work in the sentence. It is a thing to measure on your own task rather than to assume, and Chapter 20 is how you would measure it.



Quantisation applies to the weights, and separately to the cache from two sections ago — the two are chosen independently, and halving the cache's bits halves that 512 KiB per token as well. Where a published figure says "a 7B model in 4 GB", it is counting weights.

Mixture of Experts

The last piece, and it is the one that makes model sizes confusing to compare.

Chapter 12 showed that most of a layer's parameters are in the feed-forward block. A **mixture-of-experts** model replaces that one block with several — say eight — and adds a small piece of arithmetic that chooses which two of the eight each token goes through.

So the model *holds* eight feed-forward blocks per layer and *uses* two. Take Chapter 12's numbers, with attention at 67,108,864 parameters and one feed-forward block at 90,177,536:

```
total     67,108,864 + 8 x 90,177,536  =  788,529,152
active    67,108,864 + 2 x 90,177,536  =  247,463,936

active share  247,463,936 / 788,529,152  =  0.314
```

Under a third of the parameters run for any given token. This is what the terms *total parameters* and **active parameters** mean, and why a model advertised at a very large number can be cheap per token: the number in the name is the first figure and the cost follows the second.

What it does not make cheap is memory. All eight blocks have to be *there*, because the next token may route differently — and in a batch they are all needed anyway, because a batch's

tokens between them route almost everywhere. The saving on reading the parameters is real at one request and shrinks as the batch fills, which is why serving these models well is harder than the active-parameter figure makes it sound. So this design trades memory — which you can buy — for arithmetic and memory traffic, which is what you are waiting on.



Eight experts with two active is one arrangement among many, and a dated detail: designs routing two of eight sit at the modest end, and some route eight of two hundred and fifty-six, which puts the active share near a thirtieth rather than near a third. The arithmetic is the same; the fraction is not.



Which experts a token goes through can depend on what else is in the batch, because the routing balances load across experts. That is the third of Chapter 13's four causes of non-determinism, and now you can see where it comes from.

Lab: Size a Machine

Pen, paper and a calculator. Twenty minutes, and there is a key for every step. Nothing to install.

You are sizing a machine to serve one model. Its shape: **32 layers, width 4,096, 7 billion parameters, weights at 16 bits.** That is more than Chapter 12 counted for the same shape, because this model's feed-forward blocks are the gated three-matrix form Chapter 12's Note on real feed-forward blocks mentions — three matrices of 4,096 by 11,008 instead of two comes to 7.0 billion. The card has **40 GiB** of memory. Your product allows conversations up to **8,000 tokens**, and the cache is stored at 2 bytes per number.

Step 1. How much memory do the weights take?

Expected result: $7{,}000{,}000{,}000 \times 2 = 14{,}000{,}000{,}000$ bytes — 14.0 GB, which is 13.0 GiB — the two units from the Note earlier in the chapter, and the reason this step is worth doing rather than skipping.

Step 2. How much cache does one 8,000-token conversation need?

Expected result: $2 \times 32 \times 4{,}096 \times 2 = 524{,}288$ bytes per token, and $524{,}288 \times 8{,}000 = 4{,}194{,}304{,}000$ bytes — **3.91 GiB.**

Step 3. How many such conversations fit at once?

Expected result: 40 − 13.0 = 27.0 GiB free, and $27.0/3.91 = 6.9$, so **six**. Not 6.9 — you cannot serve nine tenths of a conversation, and in practice you would leave headroom and say five.

Step 4. Your product manager wants to allow 32,000-token conversations instead. How many fit now, and what would you say?

Expected result: the cache per conversation becomes $524{,}288 \times 32{,}000 = 16{,}777{,}216{,}000$ bytes, 15.63 GiB, so $27.0/15.63 = 1.7$ — **one.** Quadrupling the allowed length cut capacity from six

to one, which is a sixfold increase in the cost per conversation for a feature most users will not use. That is the conversation to have, and the arithmetic is the whole of your side of it.

Step 5. Someone suggests 4-bit weights. How much does that free, and how many 8,000-token conversations fit then?

Expected result: the weights drop to $7{,}000{,}000{,}000 \times 0.5 = 3{,}500{,}000{,}000$ bytes — 3.5 GB, 3.3 GiB. Free memory rises from 27.0 to 36.7 GiB, and $36.7 / 3.91 = 9.4$, so **nine** conversations rather than six. Then the part that is not arithmetic: you would have to measure whether the 4-bit model is still good enough at your task, because nothing above tells you that.

Step 6. The cache is also stored at 4 bits instead of 16. What happens?

Expected result: the per-token figure falls from 524,288 bytes to a quarter of that, 131,072 bytes, so an 8,000-token conversation needs 0.98 GiB. With 4-bit weights too, 36.7 GiB free gives **thirty-seven** conversations. **The cache was the binding constraint all along** — and that is the point of the whole lab. Most people reach for the weights first, because that is the number in the model's name.

Step 7. Without computing: your traffic doubles overnight and every conversation is the same length. What happens to the arithmetic the machine does per token, and what happens to the memory it needs?

Expected result: the arithmetic per token barely changes, because the parameters are read once for the whole batch and twice as many requests share that one read. The memory roughly doubles, because every conversation needs its own cache. **Doubling the load is a memory problem, not an arithmetic one**, which is the opposite of most people's intuition and follows directly from the first two sections.

Summary, Key Terms, and Review Questions

Summary

The two waits are two different computations. **Prefill** handles the whole prompt in one pass, with every position computed together, and is limited by arithmetic — whose attention part grows with n^2, so a longer prompt makes the first token slower than proportionally. **Decode** is one pass per output token, does almost no arithmetic, and is limited by having to read every parameter it uses out of memory for each token. The **key–value cache** stores each position's key and value in every layer so decode does not recompute them, at a cost of $2 \times \text{layers} \times d \times \text{tokens} \times \text{bytes}$ — 512 KiB per token at a common shape, so 3.91 GiB for an 8,000-token conversation and 39 GiB for ten of them. That cache, not the weights, usually decides how many conversations fit. **Batching** works because the parameters are read once and shared across many requests, which is why the price per token is what it is and why your output can depend on who else was calling. **Quantisation** stores each parameter in fewer bits, halving memory and read time each time the bits halve, at a cost in precision you have

to measure. And a **mixture-of-experts** model holds many feed-forward blocks and runs a few, so its **active parameters** are a fraction of its total — under a third in the worked example — which is why a very large model can be cheap per token while still needing all of that memory.

Key Terms

Term	Meaning
Prefill	The pass that processes the whole prompt, every position at once. Limited by arithmetic, whose attention part grows with the square of the prompt length and overtakes the rest on a long one. What you wait for before the first word
Decode	One forward pass per output token. Does almost no arithmetic and is limited by reading every parameter the pass uses out of memory once per token
Key–value cache	The stored keys and values for every position in every layer, so decode attends to earlier positions without recomputing them. Costs $2 \times \text{layers} \times d \times \text{tokens} \times \text{bytes}$
Compute-bound	A step whose speed is set by the arithmetic it has to do rather than by how fast its numbers arrive. Prefill is; a large enough batch becomes so
Memory-bound	A step whose speed is set by how fast numbers arrive from memory rather than by the arithmetic done with them. At two bytes per parameter, decode does about one operation for every byte it reads, on hardware built for hundreds
Batching	Running many requests through the same forward pass, so one read of the parameters serves all of them. Close to free up to a point — each request still reads its own cache and does its own arithmetic — which is why the price per token is low and why your neighbours affect your output
Quantisation	Storing each parameter in fewer bits. Halving the bits halves the memory and roughly halves the time spent reading, at a cost in precision that has to be measured
Mixture of experts	Replacing one feed-forward block per layer with several and routing each token through a few of them. The model holds all of them and runs some
Active parameters	The parameters that actually run for a given token, as against the total the model holds. Cost follows the active figure; the name usually quotes the total

Worth Memorising

Item	Value or formula
Key–value cache	2 × layers × d × tokens × bytes
At a common shape	32 layers, d 4,096, 2 bytes: 512 KiB per token
An 8,000-token conversation	3.91 GiB of cache, on top of the weights
Weights at 7 billion parameters	14 GB at 16 bits, 7 GB at 8, 3.5 GB at 4
What limits each phase	Prefill: arithmetic. Decode: reading the parameters the pass uses
What batching shares, and what it does not	The parameters are read once and shared; each request's own cache is not. So the saving is large at short contexts and small at long ones — about 4x at 8,000 tokens each

Review Questions

1. Why is the first token slow and every token after it fast?
2. Your prompt doubles in length. Why does the work more than double?
3. What is the key–value cache, and how big is it?
4. Why do providers run your request alongside other people's, and what does that do to you?
5. A model is advertised with a very large parameter count but a low price per token. How is that possible?
6. You need to serve more conversations on the same card. Name two levers, say which one people reach for first, and name a third that is not about bits at all.

Review Answers

1. Because they are different computations. The first token requires prefill: the whole prompt processed in one pass, every position at once, with attention's grid growing as the square of the prompt length — so the wait before the first word grows faster than the prompt does, and faster still once that part dominates. Every token after it is one decode pass over a single new position, which is almost no arithmetic. Decode is not instant either, but what limits it is reading out of memory every parameter the pass uses, once per token, not the arithmetic it then does.
2. Three things grow and none of them grows linearly in the way people expect. Prefill's attention arithmetic grows with the square of the length, so it roughly quadruples. The key–value cache grows with the length, so the memory you are holding doubles. And every decode pass attends over the whole cache, so each token of the reply costs more to produce than it did — a reply that starts fast gets slower. "A bit more text" is three costs rising at once, one of them faster than proportionally.

3. The stored key and value vectors for every position in every layer, kept so that decode does not recompute them on every pass. Its size is 2 × layers × d × tokens × bytes — at 32 layers, a width of 4,096 and two bytes, that is 512 KiB per token, so 3.91 GiB for an 8,000-token conversation and about 39 GiB for ten of them. It depends on nothing but the number of tokens, and it is usually what decides how many conversations fit on a card rather than the weights.
4. Because decode is limited by reading the parameters out of memory, and that read serves the whole batch at once — so fifty requests cost far less than fifty, and the price per token reflects a shared machine. Only that read is shared, though: each request still reads its own cache and does its own arithmetic, so the saving flattens as the batch grows and eventually the machine becomes compute-bound instead. What it does to you: your request may wait to join a batch, so your latency depends on how busy the system is; and the batch's composition changes how the arithmetic is grouped, which changes the last digits of the scores and can flip a near-tie — which is why temperature zero is not a guarantee.
5. Because it is a mixture-of-experts model, and the number in the name is the total rather than the active parameters. Each layer holds many feed-forward blocks and routes each token through a few, so the arithmetic and the memory traffic per token follow the smaller figure — under a third of the total in this chapter's example. What it does not make cheap is memory: every block has to be there, because the next token may route differently.
6. Quantisation and the cache. Quantisation is what people reach for first, because the weights are the number in the model's name — and in this chapter's lab it took capacity from six conversations to nine. The cache is the bigger lever and the less obvious one: storing it at fewer bits as well took the same machine from nine to thirty-seven, because at a long conversation length the cache, not the weights, was the binding constraint. And the third lever is not about bits: shortening the conversations you allow. Step 4 priced it — going from 8,000 tokens to 32,000 took the same machine from six conversations to one.

> **You understand this when** you can work out how many conversations fit on a card, and say which of the weights and the cache is the one actually stopping you.

CHAPTER 16

Why It Fails the Way It Does

This half of the book has been notation and mechanism. This chapter spends them. Each of the characteristic failures of these systems turns out to be a consequence of something you have already worked by hand, and the useful thing about that is the conclusion it forces: most of them are not bugs, nobody is going to fix them, and knowing which is which is what tells you where to put your engineering.

The problem. It invents citations with a straight face, and I'd like to know whether that is a bug or the design.

The question. Which of these failures come from the mechanism itself, which come from how it is used, and what follows for the things I build?

There Is No Step Where Truth Is Checked

Walk the pipeline once more, which you can now do from memory. Text becomes tokens. Tokens become vectors. The vectors pass up a stack, each layer adding to a running sum. The last position's vector becomes one score per vocabulary entry. Softmax makes those scores into probabilities. Something picks one.

Point at the step that consults anything. There isn't one. There is no source consulted, no check, no moment where a claim is compared against anything. The whole computation is arithmetic on numbers that were fitted to make likely text likely, and "likely" is the only quantity in it.

So when a model produces a citation that does not exist, nothing has gone wrong in the sense that a program has gone wrong. A plausible-looking reference is exactly what a plausible continuation of "according to the paper by" looks like. The author name is a likely author name. The year is a likely year. The title is a likely title. **Each piece is individually the most ordinary thing in the world**, and the assembly is a citation that was never written.

This is what **hallucination** means, and the word is unfortunate because it suggests a malfunction. The mechanism is working exactly as described in Chapters 9 to 14. What is missing is a step that was never there.



That is why "the model lied" is the wrong frame, and not only because it is impolite. Lying requires knowing the truth and choosing otherwise. There is no represented truth to diverge from — there is a probability distribution over tokens, which is a different kind of object entirely. The practical consequence is that you cannot fix this by asking the model to be more careful, because carefulness is not the faculty that is missing.

And now the part that matters for your work. Since nothing in the model checks, the checking has to live outside it. That is not a workaround; it is the architecture, and it is one of the four gaps Chapter 2's parts answer. Retrieval puts the source in the request so the continuation is constrained by something real. Citations let a person verify. Checks test the output against a rule. None of these makes the model truthful. They make the system's output checkable, which is a different and achievable thing.



Fluency is not a signal here. The model's prose is as fluent for the invented citation as for the real one, because fluency is what the probabilities measure. A hedge — "I believe", "if I recall" — is itself a likely continuation in contexts where people hedge, not a report of an internal uncertainty. Chapter 31 is where you build something that does measure.

The Failures That Come From Tokens

Chapter 10 gave you the mechanism and one example. Here is the family, because an interviewer will ask about a different member of it than the one you prepared.

Counting letters. A word arrives as one index, or two, or three. A token's spelling is not in the input, so a question about the letters inside a word is a question about something the model was not given. What it has instead is whatever it absorbed about spelling in training, which is recall rather than counting.

Spelling tasks generally. Reversing a word, finding words that rhyme, counting syllables, working with anagrams. All of them ask about structure below the token.

Arithmetic with digits. Numbers are cut into pieces like everything else, and the pieces do not respect place value. Take an addition, and a tokeniser that happens to cut it this way:

```
the sum      3721  +  1949

as digits    3 7 2 1  +  1 9 4 9
as tokens    37  21   +  19  49
```

Addition is "add the units, carry, add the tens". **There is no units column here.** The 1 and the 9 that should be added first are each buried inside a two-digit piece, and the model is given the pieces. Whatever it does to produce an answer, it is not the procedure you were taught, because the procedure's first step refers to something it cannot see.

Models have got better at this without gaining the ability to see the digits as digits, and the mechanism underneath has not changed.



The split above is illustrative. Different tokenisers cut numbers differently and some cut every digit separately, which is reported to help — and if it does, the help comes from the cutting rather than from the model. What does not vary is that the model works on whatever pieces it is given, and that nothing guarantees those pieces line up with place value.



All of these have the same engineering answer, and it is the one from Chapter 10: if the answer must be right, do not ask the model to compute it — have it write code that computes it, or compute it in your own code. A model that cannot count the letters in "strawberry" can write a correct program to count the letters in anything.

The Context Window, and What Falls Out of It

Every model has a limit on how many tokens it will accept at once. That limit is its **context window**, and the two earlier chapters said where it comes from: Chapter 11's attention grid is square in the number of tokens, Chapter 15's key–value cache grows with it, and somebody has to decide where the cost stops being worth paying.

Two things follow that surprise people.

It counts the reply too. The limit is on the request and the reply together, so a long prompt leaves less room to answer in — and a reply can stop simply because the two reached the limit, which is one of the ways off Chapter 13's ordinary three, and the one it pointed here for. Spend one and see:

```
the window                      128,000 tokens
the document you pasted         120,000
your instructions and question    1,500
                                -------
left for the reply                6,500
```

Nobody told you that. You will find out when a long answer stops mid-sentence, and the reply will report that it hit the limit rather than that it finished.

It is not memory. Nothing persists between calls, as Chapter 9 said. The whole conversation is re-sent on every request, which is also why a long conversation costs more each turn.

And something has to go when it fills. This is where products differ, and the differences are invisible from outside. Some drop the oldest turns. Some summarise what they drop. Some refuse. Chapter 18 is where a conversation that will not fit gets truncated on purpose and Chapter 35 is where an agent's context is managed deliberately; the point here is that

when a model "forgets" what you said twenty minutes ago, the usual explanation is not that the model forgot, but that your words were not in the request.

The sharper version, and the one to know. When a conversation is truncated, what is dropped is chosen by a rule in somebody's product code, and that rule does not know what mattered. The instruction you gave at the start — the one that said "always answer in French" — is the oldest thing in the conversation.

Long Inputs Are Not Used Evenly

Here is a finding rather than a mechanism, and Chapter 6's discipline applies: say what kind of claim it is.

The claim. When a long input contains one relevant passage, models are better at using it when it sits near the beginning or the end than when it sits in the middle. The effect has been measured across several models and several tasks, and it is a repeated experimental result rather than a single study — more than one study, and short of settled practice, because Chapter 6's tell for settled practice is written-down failure modes and nobody has written down where this stops holding — and still a finding about particular models rather than a law of the design.

What it means for you. Putting more in the context does not reliably mean more of it gets used. Doubling the retrieved passages in the hope that the right one is in there somewhere is a strategy that costs twice as much and may work less well. Chapter 28 is where you decide what to put in and in what order, and this is the reason it is a decision at all.



Do not state this as "models ignore the middle". They do not ignore it; they use it less reliably. And the effect's size varies by model and by task, so the useful version in an interview is "it has been measured repeatedly, the position of a passage matters, and I would measure it on my own data before designing around a number from a paper."

Why Thinking Out Loud Works

This is the one with a genuinely satisfying explanation, and the mechanism follows from Chapter 9 alone.

Recall two facts. **One forward pass does a fixed amount of arithmetic for the input it is given** — a fixed number of layers, run once each, and no more of them for a harder question — and produces exactly one token. **And the model has no memory between passes**; the only thing carried forward is the text so far.

Put those together. A problem that needs several steps of work cannot get several steps of work out of one forward pass. The pass is as long as it is. And nothing carries from one pass

to the next: no scratchpad, no variable holding a partial result, nowhere to put an intermediate answer.

Except one place. The output.

If the model emits its intermediate steps as tokens, those tokens become part of the input for every later pass. The partial result is now in the context, where later tokens can condition on it. **The model's only scratchpad is its own output** — which is a strange and slightly undignified fact about the design, and it is why a problem needing several steps has nowhere to put them unless they are emitted. That the working *improves the answer* is a separate and weaker claim: the mechanism cannot reach it, because a model may compress the same computation into one pass and be right anyway. On Chapter 6's ladder it has been demonstrated and studied in particular settings and is short of settled practice, so how much it buys on your task is something to measure. Chapter 22 makes the same split for strict output formats.

```
without intermediate tokens
  [ question ] -> passes -> answer

with them
  [ question ] -> passes -> step 1
  [ question, step 1 ] -> passes -> step 2
  [ question, step 1, step 2 ] -> passes -> answer
```

Three rounds instead of one — and each round is one pass per token it emits, as Chapter 9 said — with the third round reading the first two. That is more total computation applied to the problem, and the extra computation is applied to a question that has already been partly answered.



That is the mechanism, and it is the whole of what this book says about it. Why models have got better at producing useful intermediate steps, and what training does to make them do it, is on Chapter 17's list.

What It Costs, and What the Trace Is Not

Two things to say about intermediate reasoning that people get wrong in opposite directions.

It is expensive, and in the dear half. Intermediate tokens are output tokens. A 200-token chain of working before a 50-token answer is 250 output tokens where you wanted 50 — five times the output bill — and Chapter 15 says each of those 250 passes reads the whole cache, which is growing as it goes. Then, where the product keeps it, the working sits in the context for the rest of the conversation, being re-read.

```
answer alone        50 output tokens
working + answer   250 output tokens     5x

and every one of the 250 is a pass over a growing cache
```

So it is a lever with a price, not a free improvement, which is why products increasingly make it conditional — on by default for hard tasks, off for easy ones. That last is a current convention rather than a property of the design, and it is the kind of detail that moves. Chapter 53 is the bill.

And the trace is not a log. This is the one that catches people out, and it follows from the first section of this chapter.

The intermediate steps are *generated text*. They are produced by exactly the same process as everything else: plausible continuations, one token at a time, with no step that consults the computation that is actually happening in the layers. So a chain of working is a plausible account of how one might reach that answer. It is not a record of how this answer was reached, because no such record exists — there is nothing in the mechanism that writes one.

The consequence is sharper than it sounds. A model can produce correct working and a wrong answer, or wrong working and a right answer, and both are ordinary. You cannot read the trace to find out why it got something wrong; you can only read it as more output, to be checked like any other output.



This is worth being precise about in an interview, because the sloppy version — "the reasoning is fake" — is wrong in the other direction. The intermediate tokens are doing real work: later passes genuinely condition on them, which is why the working can change the answer at all. They are causally involved and they are not a description of the cause. Both halves are true and the useful answer says both.

Which of These You Can Fix

Worth separating, because the engineering is different for each.

Failure	Where it comes from	What you can do
Invented facts	No step checks truth	Put sources in the request; check the output; cite
Letters, spelling, digits	Tokens, from Chapter 10	Move the task into code
Forgetting things said earlier	Something was dropped from the request	Decide what to drop, deliberately
Uneven use of long inputs	Measured behaviour, not a design rule	Put less in, and order it; measure
Needing to think out loud	One pass gets no deeper for a harder question	Ask for working where it earns its cost

None of the rows says "use a better model". Better models are reported to make the first and the second less frequent — Chapter 10 says so for the token-level case — and nothing about either mechanism predicts a rate, so that half is a report rather than a consequence. What does not change is why either happens. The third and fifth are properties of how the thing is used and of the design. The fourth is measured behaviour that varies by model, so a better model can move it — and what you put in the context is still yours.

Lab: Diagnose Six Failures

No code. Twenty minutes, and there is a key for every step. For each, say which mechanism produced it and what you would do.

Step 1. A model summarises a document and attributes a quotation to a person who is not mentioned in it.

Expected result: no step checks truth, so a plausible attribution is a plausible continuation. The engineering is outside the model: the source in the request, a citation the reader can follow, and a check that every quoted string appears in the source. Note that the last of those is a rule your code can apply, not a judgement.

Step 2. Asked to add two four-digit numbers, a model is confidently wrong in the third digit.

Expected result: tokenisation. The digits are cut into pieces that do not respect place value, so there is no units column to carry from. Have it write the code, or do the arithmetic in yours.

Step 3. Your chat application is told at the start to answer only in French. Forty turns later it answers in English.

Expected result: the first thing to check is whether the instruction is still in the request, and

the usual answer is that it is not. It was the oldest thing in the conversation, and whatever rule drops text when the window fills does not know it mattered. Check what you actually send before blaming the model: forty turns of chat may still fit inside a large window, in which case the instruction is present and simply not being followed. The fix is not a sterner instruction — it is to put the instruction back in on every request, which costs tokens and is worth it.

Step 4. A model asked to count the letters in a word gets it wrong, then writes a correct program to count them.

Expected result: both behaviours are the same mechanism. A token's spelling is not in its input, so counting the letters is recall; writing the program is ordinary text production, which is what it is good at. This is the single most useful pattern in the chapter: move the task to where the answer is checkable.

Step 5. You add twenty more retrieved passages to a prompt and quality falls.

Expected result: two things at once, and you should name both. Long inputs are not used evenly, so the relevant passage may now sit where it is used less reliably. And the extra passages cost tokens on every request. Neither is a reason never to retrieve more; both are reasons to measure rather than assume.

Step 6. A model produces three paragraphs of correct-looking working and then a wrong answer. Your colleague says the working proves it was "reasoning properly" and something went wrong at the end.

Expected result: the working is generated text, produced the same way as everything else, and is not a record of the computation. It can be correct while the answer is wrong, and that is ordinary rather than a sign of a late failure. Say both halves: the intermediate tokens did real work, because later passes conditioned on them — and they are not an account of how the answer was produced, because nothing in the mechanism writes one.

Step 7. Which of the six could be fixed by a better model, and which could not?

Expected result: steps 1, 2 and 4 are *reported* to get less frequent with better models, and do not change in kind — the mechanism that produces them is the mechanism that produces everything else. Step 3 is not about the model at all — it is a decision in somebody's product code. Step 5 is half yours: how much you put in is your decision, and how evenly a long input is used varies by model, so measure it. Step 6 is a property of the design and will not change while the scratchpad is the output.

Summary, Key Terms, and Review Questions

Summary

No step in the computation consults anything or checks anything, so a fabricated citation is an ordinary plausible continuation rather than a malfunction — which is what **hallucination** means and why asking the model to be careful cannot fix it. The checking lives outside the model, which is what Chapter 2's checks were for. A family of failures comes from tokens: counting letters, spelling, reversal, rhyme and arithmetic with digits all ask about structure below the token, and the engineering answer to all of them is to move the task into code. The **context window** is a limit on the request and the reply together, it is not memory, the whole conversation is re-sent every call, and something has to give when it fills — commonly the oldest thing, which is often the instruction that mattered. Long inputs are not used evenly: a repeatedly measured finding that a passage's position affects how reliably it is used, which makes what you put in the context a decision rather than a default. And **intermediate reasoning** works because one forward pass runs its layers once however hard the question is, and the model has no memory between passes, so emitting the steps as tokens is the only way to carry partial work forward — the model's only scratchpad is its own output. That costs output tokens, the dear half, and the visible trace is generated text rather than a record: the intermediate tokens do real work *and* are not an account of how the answer was produced.

Key Terms

Term	Meaning
Hallucination	Output that is fluent, plausible and not true. Not a malfunction: no step in the computation checks anything, so a plausible continuation is all the mechanism can produce
Context window	The maximum number of tokens a model will accept at once. A cost limit rather than a memory, counting the reply as well as the request. The whole conversation is re-sent every call, and something is dropped — or the call refused — when it fills
Intermediate reasoning	Emitting the steps of a problem as tokens so that later passes can condition on them. The only way to carry partial work forward, because one pass runs its layers once however hard the question is, and nothing persists between passes

Review Questions

1. Why is hallucination the default behaviour rather than a defect?
2. Why does asking a model to work step by step improve accuracy, and why is the visible

reasoning not a reliable account of how it answered?
3. Your chat application forgets an instruction given forty turns ago. What happened, and what do you do?
4. Name three failures that come from tokenisation and say what they have in common.
5. A colleague wants to double the retrieved passages in the prompt "so the answer is in there somewhere". What do you say?
6. Which of this chapter's failures would a better model fix?

Review Answers

1. Because there is no step in the computation where anything is checked. Text becomes tokens, tokens become vectors, the vectors pass up the stack, the last one becomes a score per vocabulary entry, and softmax makes those into probabilities — and "likely" is the only quantity anywhere in it. A citation that does not exist is assembled from a likely author, a likely year and a likely title, each the most ordinary continuation of what came before. The mechanism is working as described; what is missing was never there. Which is why the checking has to live outside the model — sources in the request, citations a reader can follow, rules your code can apply.
2. Because one forward pass does a fixed amount of arithmetic for the input it is given — no more layers for a harder question — and produces one token, and nothing persists between passes. A problem needing several steps cannot get several steps out of one pass, and there is no scratchpad to hold a partial result — except the output. Emitting the steps as tokens puts them in the context, where later passes can condition on them, so more computation is applied and it is applied to a partly-answered question. The trace is not a reliable account because it is generated text, produced the same way as everything else, with no step consulting what the layers actually did. Both halves matter: the tokens do real work, and they are not a description of the cause.
3. Most likely the instruction is no longer in the request — it was the oldest thing in the conversation, and the rule that drops text when the window fills does not know what mattered. Nothing was forgotten by the model, because the model has no memory: your words simply were not in the request. The fix is to re-send the instruction on every call rather than to word it more firmly, and to decide deliberately what gets dropped rather than leaving it to a default.
4. Counting the letters in a word, reversing or rhyming or counting syllables, and arithmetic with digits. What they have in common is that they all ask about structure *below* the token: the model is given seat numbers, not spellings, so the thing being asked about is not in its input. They also share an answer — if it has to be right, move the task into code, which the model is good at writing.
5. That it costs twice as much on every request and may work less well. Long inputs are not used evenly: a passage's position affects how reliably it is used, measured repeatedly across models and tasks, so burying the right passage among twenty others is not a

neutral act. The honest version is that this is a thing to measure on their own data rather than to assume in either direction — and that ordering what goes in is a decision, not a default.

6. The rate of invented facts and token-level mistakes is reported to fall with better models, without the reason for either changing. The only other one that moves with the model is the uneven use of long inputs, which varies by model and by task and is a thing to measure rather than to assume. The rest are not about the model: an instruction falling out of the window is a decision in somebody's product code, how much you put in the context is a decision in yours, and needing to think out loud is a property of the design that will hold for as long as one pass does fixed work and the output is the only scratchpad.

> **You understand this when** you can say why a fabricated citation and a correct answer are produced by exactly the same process, without calling either one a bug.

CHAPTER 17

Where the Theory You're Skipping Begins

Nine chapters ago this book started opening the model, and six times it stopped and said *not this* — about eight things in all, and there are five more it never needed, which makes thirteen. This chapter collects those points in one place. It is short, it is a list, and it is the one chapter here you should be able to recite — because the difference between a gap you can name and a gap you cannot is the difference between sounding bounded and sounding ignorant.

The problem. I can explain the computation now. I want to know exactly what I still cannot explain.

The question. What did this book deliberately leave out, what is each of those things actually called, and what do I say when somebody asks me one of them?

The List

Every row is a place where an earlier chapter stopped on purpose. None of them is a gap in the explanation; each is a decision, and here is the whole of it.

What you cannot explain	Where you met it	What it actually is
Why the embedding geometry carries meaning	9	**Representation learning**: why fitting a model to predict text arranges its vectors so that the arrangement is useful for anything else
Anything beyond one sentence on why normalisation is there	12	**Numerical stability**: how quantities behave through long chains of arithmetic, and what keeps them in a usable range
Why depth helps	12	An open research question. Not a settled account being withheld
How the gradient is computed	14	**Backpropagation**: the procedure that gets a derivative for every parameter in one backward sweep
Why repeating the update rule — gradient descent — arrives anywhere	14	**Optimisation theory**: when a procedure like this converges, to what, and how fast
What step size to use and how to change it	14	**Optimiser design**
Why the preference objective is the right thing to minimise	14	**Objective design**: why that expression, rather than another, is what you want a model to minimise
Why models got better at producing useful intermediate steps	16	What training does to make a model emit intermediate work that helps. There is no tidy name to give this one at the level this book works at

And five more that the book never needed, listed so the set is complete rather than convenient:

What you cannot explain	What it actually is
Initialisation	What the parameters are set to before training begins, and why the choice matters
Regularisation	The techniques that stop a model fitting its training data too closely
Inductive bias	What a design assumes about the world before it sees any data
Generalisation theory	Why a model fitted to one body of text works on text it never saw — the bias–variance trade-off, and what learning theory proves about it
Scaling laws	The relationships between model size, data, compute and loss — and their derivation

Three further subjects are outside this book without being gaps in it: convolutional and recurrent architectures, and classical machine learning. They are other kinds of model rather than parts of this one, which is why they are not on the list above.

Thirteen things. You can read that list in a minute, which is the point: a boundary you can recite is a boundary you can defend.

What They Have in Common

Look down the "what it actually is" column and one pattern runs through all of it.

Not one of them is needed to follow the arithmetic of a single reply. That is the claim that holds of all thirteen, and it is worth being exact about it, because the tidier version — that every item is about why the numbers came to be what they are — is not quite true. Most are. Two are not: *why depth helps* is about what a deeper computation can do, and *numerical stability* is about the arithmetic itself, beyond the one sentence Chapter 12 gives it. What all thirteen share is that a single reply can be followed without any of them.

Chapters 9 to 13, 15 and 16 took that computation apart — tokens, the embedding table, attention, the stack, the output distribution, the sampling, the cost, the failures — and at no point did any of it require knowing why the numbers came to be what they are.

That is not a coincidence; it is the line the book was built on. The numbers are given, and everything downstream of them is arithmetic you can do by hand. **The things you skipped are the things that computation never needed.**

Which has a consequence worth stating plainly, because it is the thing to say when somebody suggests you have missed the real subject. For the work of building applications on top of these models — choosing what goes in the request, deciding what the model may do, finding out whether the output was any good, controlling what it costs — none of the thir-

teen is load bearing — with one edge: if you ever do your own fine-tuning, which is Chapter 39, you will be choosing a step size. They are load bearing for building a model, which is a different job.



"Different job" is not "lesser job", and the boundary is not a judgement about what is worth knowing. It is a judgement about what fits in one book read in the time you have. Chapter 56 is where to go next, and it names the books.

What to Say When Somebody Pushes

The practical half of this chapter. An interviewer asks why depth helps, or how backpropagation works, and you have not studied it. There is a good answer and it has three parts, in this order.

Name the thing correctly. "That is optimisation theory" or "that is representation learning" or "that is backpropagation — the backward sweep that gets a derivative for every parameter". Naming it accurately is most of the work, because it demonstrates that you know where you are rather than that you have wandered off a map.

Say what you do know that borders it. You know a great deal that touches each of these. You can state the training objective as a formula. You can state the update rule and say what each symbol means. You can say what each of Chapter 14's three stages changes. What you have not done is derive why the rule converges — and the border is exactly that sharp.

Say what you would do if you needed it. Not "I would look it up", which says nothing. Something specific: "if I were debugging a training run I would need it, and I would start with the standard text rather than a blog post, because this is settled material and not a moving target."

Put together, for the depth question:

> *"Honestly, nobody has a clean account of that. The mechanism is that more layers means more rounds of gathering from other positions and computing at each one, and a result assembled at layer twenty can be gathered from at layer twenty-one.* Why *that buys what it buys — why thirty-two layers beats sixteen by more than it costs, where the returns stop — is an open research question, and I have not studied the attempts at it. What I can tell you is what a layer does, which half of it holds the parameters, and what depth costs at inference."*

That is a good answer to a question you cannot answer, and it is better than a confident reconstruction, because an interviewer who asks it usually knows the answer is not clean.



The failure mode is not saying "I don't know". It is saying "I don't know" and stopping — or worse, producing a plausible-sounding derivation. You have spent nine chapters learning that a plausible continuation is not the same as a true one. Do not become one.

What a Boundary Is Not

Three things it is not, because each is a way people get this wrong.

It is not a claim that the material is easy. Nothing above was omitted because it is simple. Backpropagation is a few lines of calculus and a great deal of care; optimisation theory is a field.

It is not permanent. You are weeks from an interview, not at the end of your career. Chapter 56 exists because the sensible order is to learn the system first and the theory second — not instead.

And it is not the same as not knowing it is there. This is the whole reason the chapter exists. A person who has never heard of representation learning and a person who can say "that is representation learning, and I have not studied it" are in completely different positions, and only one of them can have a conversation about it.

Lab: Write Your Own Boundary

No code and no calculator. Fifteen minutes, and there is a key for the parts that have one.

Step 1. Close the book. Write down as many of the thirteen as you can, from memory — and for the eight that came from a chapter, the chapter where you met each.

Expected result: the key is the two tables above — the first carries the chapters, the second is five items that were never needed and so never met. Score yourself out of thirteen, and look up every one you missed: those are the ones most likely to come up unprepared.

Step 2. For each of the thirteen, write one sentence saying what it actually is. Then check them against the tables above.

Expected result: the table's sentence is the key for each. These are one-line definitions, not subjects you are expected to summarise, and the test is whether you can say one without reaching for the book.

Step 3. Pick the three you find hardest to say out loud. For each, write the three-part answer from this chapter: name it, say what you know that borders it, say what you would do if you needed it.

Expected result: three short paragraphs, each about forty words. The middle part carries the answer: if you cannot say what you know that borders a gap, you do not yet know where the gap is.

Step 4. Now the other direction. Write down three things you *can* explain about what a model computes, which you could not have explained before Chapter 8.

Expected result: no key, and that is deliberate — this one is about you. If the list is hard to write, the fix is not more theory; it is to go back and do one chapter's arithmetic with a pen.

Step 5. A colleague says the book you have read is "just the API layer" and that you do not really understand the models. Write two sentences in reply.

Expected result: something close to: *"I can compute an attention row by hand and tell you what a key–value cache costs at eight thousand tokens. What I have not studied is why training converges — which is a different subject, and I can tell you exactly which one."* The reply works because it is specific in both directions. A vague defence of the boundary sounds like a vague understanding of the material.

Summary, Key Terms, and Review Questions

Summary

Thirteen things were left out of this book on purpose, and they are listed here so that each is a gap you can name rather than one you do not know about. Eight of them are places an earlier chapter stopped and said so: why the embedding geometry carries meaning (Chapter 9); the subject behind Chapter 12's one sentence on normalisation, and why depth helps (Chapter 12); how the gradient is computed, why repeating the update rule — gradient descent — converges, how a step size is chosen, and why the preference objective is the right thing to minimise (Chapter 14); and why models got better at producing useful intermediate steps (Chapter 16). Five more — initialisation, regularisation, inductive bias, scaling laws and generalisation theory — were never needed. **Not one of them is needed to follow the arithmetic of a single reply**, which is the line the book was built on. When somebody pushes past it, the answer has three parts: name the thing correctly, say what you know that borders it, and say what you would do if you needed it. A boundary is not a claim that the material is easy, it is not permanent, and it is not the same as not knowing the material is there.

Key Terms

Term	Meaning
Backpropagation	The procedure that computes a derivative for every parameter in one backward sweep through the model. How the gradient of Chapter 14's update rule is obtained. Not taught in this book
Optimisation theory	The study of when a procedure like the update rule converges, to what, and how fast. Not taught in this book
Optimiser design	Choosing and adapting the step size and the form of the update. Not taught in this book

Term	Meaning
Objective design	Why one expression rather than another is the right thing for a model to minimise. The question Chapter 14 states its preference objective without answering. Not taught in this book
Representation learning	Why fitting a model to predict text arranges its vectors so that the arrangement is useful for other things. The question Chapter 9 refused. Not taught in this book
Numerical stability	How quantities behave through long chains of arithmetic, and what keeps them in a usable range. The subject behind Chapter 12's one sentence on normalisation. Not taught in this book
Initialisation	What the parameters are set to before training begins, and why the choice matters
Regularisation	The techniques that stop a model fitting its training data too closely
Inductive bias	What a design assumes about the world before it sees any data
Generalisation theory	Why a model fitted to one body of text works on text it never saw — the bias–variance trade-off, and what learning theory proves about it
Scaling laws	The relationships between model size, data, compute and loss, and their derivation

Review Questions

1. An interviewer asks why depth helps. You have not studied that. What do you say?
2. What do all thirteen items on the list have in common, and why does that matter?
3. Somebody says you have learned "just the API layer". What is the two-sentence reply?
4. What is the difference between a boundary and ignorance?
5. Name the four items that came from Chapter 14 and say what each one is.

Review Answers

1. Name it, border it, and say what you would do. "Nobody has a clean account of that — it is an open research question rather than a settled explanation I have skipped. The mechanism is that more layers means more rounds of gathering from other positions and computing at each one, so a result assembled at layer twenty can be gathered from at layer twenty-one. Why that buys what it buys, and where the returns stop, I have not studied. What I can tell you is what a layer does, that the feed-forward block holds more of the parameters than attention does, and what depth costs at inference." The answer works because every sentence in it is true and the boundary is drawn exactly.
2. **None is needed to follow the arithmetic of a single reply.** That matters for two reasons. It explains why the book could take the whole computation apart without any of them — the numbers are given, and everything after that is arithmetic. And it gives you

the sentence to use when somebody suggests you have missed the real subject: these are load bearing for building a model, which is a different job from building applications on one.

3. "I can compute an attention row by hand and tell you what a key–value cache costs at eight thousand tokens. What I have not studied is why training converges — which is a different subject, and I can tell you exactly which one." It works because it is specific in both directions: a concrete thing known and a named thing not known. A vague defence of a boundary sounds like a vague grasp of the material.
4. A boundary is a decision you can state, with a name on each side of it. Ignorance is not knowing the thing exists. The person who says "that is representation learning and I have not studied it" and the person who has never heard the phrase are in entirely different positions, and only the first can have the conversation. That is the whole reason this chapter is in the book.
5. Backpropagation — the procedure that computes a derivative for every parameter in one backward sweep, which is how the gradient in the update rule is obtained. Optimisation theory — when repeating that update rule, which is gradient descent, converges, to what, and how fast. Optimiser design — what step size to use and how to adjust it as training proceeds. And why the preference objective is the right thing to minimise, which Chapter 14 states and says outright it is not deriving. All four sit on the far side of the two lines Chapter 14 wrote down and did not derive: the update rule and the preference objective.

> **You understand this when** you can name all thirteen, say which side of the line each sits on, and answer the depth question out loud without either guessing or stopping at "I don't know".

PART II

Use



What to ignore for now: serving your own models, and the bill — Part IV takes both.

CHAPTER 18

The Prompt as a Specification

Part I was about what the thing is. Part II is about driving it, and this is the first control. The chapter's claim is that a prompt is not a magic phrase you discover by rubbing it, but a document with parts, each of which does a different job and can be changed on its own — and that most of what makes a prompt work is not the wording at all.

The problem. I keep rewriting my instructions until something works, and I couldn't tell you what I changed or why it helped.

The question. What is a prompt actually made of, what is the thing I am really changing when I edit one, and how do I find out whether a change helped?

Four Separable Parts

Chapter 2 named everything sent on a call the **context**, and split it into instructions, added material and the request. That was the right split for understanding what a system is *for*. For editing one the cut falls differently: the instructions turn out to be three separate things, and the added material and the request turn out to be one — because what matters now is not who wrote a piece but how often it changes. (Chapter 2 keeps the word *context* for everything sent, because *prompt* is used three ways. This chapter says *prompt* where the thing being edited is the specification, which is the third of those three and the one a person means when they say they are writing one.)

Part	What it does	How often it changes
Instructions	What to do, what not to do, who to be	Rarely — it is your specification
Examples	What a good answer looks like, shown rather than described	Rarely
Output format	The shape the answer must take	Rarely
Supplied information	The request, and the material it is about	Every request

The first three are the *specification*. They are written once, changed deliberately, and belong in version control next to your code. The fourth is *data*, and it is different on every call.

Separating them matters for three concrete reasons, and none of them is tidiness.

You can change one and measure it. If the instructions, the examples and the format are tangled into one paragraph, "I changed the prompt" is all you can say about a change. If they are separate blocks, you can change the examples and leave everything else alone — which is the only way to find out what the examples were doing.

You pay for the first three on every single call. Chapter 1 did this arithmetic and Chapter 10 gave it its unit: a page of standing instructions is a page of input tokens on every request for the lifetime of the feature. Knowing which block is which is knowing where your bill is.

And three of the four are the same for every user. That matters more than it sounds, because a prefix that never changes is exactly the thing a provider can cache — which is Chapter 52's lever, and it only works if the unchanging part is actually at the front.



Put them in that order — instructions, examples, format, then the supplied information — and the stable part is at the front by construction, which is where it is cheapest to leave alone. Chapter 52 is what a provider will do with that, and what it takes to qualify.

Here is one pulled apart. The four blocks are marked; nothing else about it is special.

```
INSTRUCTIONS
  You classify support messages. Answer with one category and
  nothing else. If none fits, answer "other".

EXAMPLES
  "my card was declined"        -> billing
  "the app crashes on startup"  -> bug
  "how do I export my data?"    -> question

OUTPUT FORMAT
  One word, lower case, from: billing, bug, question, other.

SUPPLIED INFORMATION
  "I was charged twice this month."
```

The first three are 332 characters, so eighty-odd tokens by Chapter 10's rule, and they never change. The fourth is a line of about thirty characters, so eight or so tokens, and it changes every time.

The Message List

What you actually send is not one block of text. It is a list of messages, each with a role, and the roles mean something to the model because its training put them there.

Role	Who it is	What goes in it
System	You, the operator	The specification — instructions, usually the format, and often the examples. Set once per conversation
User	The person, or your code on their behalf	The supplied information — the request and the material it is about
Assistant	The model	What it said last time. You send its own replies back to it
Tool	Your code	The result of something the model asked for. Chapter 23

The classification above, as a message list:

```
system     You classify support messages... (instructions,
           examples and format)
user       I was charged twice this month.
```

That is the shape, and this is roughly what goes over the wire — the same two messages, as a client library takes them. Providers differ in the details; several carry the system text as a field of its own rather than as an entry in the list, which changes nothing about the idea:

```
[
  {"role": "system", "content": "You classify support messages..."},
  {"role": "user",   "content": "I was charged twice this month."}
]
```

And a second turn, after the model has answered:

```
system     You classify support messages...
user       I was charged twice this month.
assistant  billing
user       What about "the export button does nothing"?
```

Look at the third line of that four-message list. The model's own previous answer is in the input, sent by you. It did not remember saying it. Chapter 1 said the model carries nothing from one call to the next, and this is what that looks like in practice: **the conversation is a list your code keeps and re-sends.**



Where the examples go is a real choice and not a settled one. They can sit in the system message, or as alternating user and assistant messages that look like a conversation that already happened. Both are in use. The second often works better for chat-trained models, which is a current convention rather than a law, and the thing to do is try both and measure — which is the next section but one.

Every Turn Re-Sends Everything

Now the arithmetic, because it is the thing people are most surprised by and it falls straight out of the list above.

A **turn** is one exchange: the user's message and the model's reply to it. A conversation is a list of them, and the list is what gets re-sent.

Take a conversation with a 200-token system message, 100-token user turns and 150-token replies. Every turn re-sends the whole list.

```
turn   input   output
   1     300      150
   2     550      150
   3     800      150
   4   1,050      150
   5   1,300      150
   6   1,550      150
   7   1,800      150
   8   2,050      150
   9   2,300      150
  10   2,550      150
       ------   ------
       14,250    1,500
```

Check turn 3 by hand: 200 for the system message, three user turns at 100, two replies at 150 — $200 + 300 + 300 = 800$. Each turn adds one user message and one reply, so each row is 250 more than the one above it.

Two numbers to take away.

Turn ten's input is 8.5 times turn one's — 2,550 tokens against 300 — for a question that is no longer and no harder.

And the ten turns together take 14,250 input tokens, where ten independent questions with the same system message would take 3,000. **A conversation's input is about five times the input of the same questions asked separately** — the replies are the same 1,500 tokens either way, and Chapter 1 is the reminder that they are the dearer half, so the bill does not rise by five. Nobody is being overcharged; you are re-sending a growing document every time, because that is the only way the model can see it.



This is why "it's just a chat" is a dangerous sentence in a cost estimate. The input of each call grows by a fixed amount per turn, so the total you pay for over n turns grows like n^2 — and Chapter 15 adds that the attention cost inside each of those calls is growing too.

When the History Will Not Fit

Eventually the list exceeds the window, and something has to go. Chapter 16 said this happens; here is what to do about it, because it is your decision and not the provider's.

Four strategies, and each loses something different.

Strategy	What it does	What it loses
Drop oldest	Remove turns from the front	Your specification, if it is in the list rather than pinned
Keep a window	Last *k* turns, always	Anything established early — the specification too, unless pinned
Summarise	Replace old turns with a summary	Whatever the summary left out, silently
Start again	New conversation, carry nothing	Everything, but visibly

The one rule that comes closest to not being a trade-off: **never drop your specification.** That is the system message — and the example turns too, if that is where your examples live. It is a small share of a long list, and dropping any of it changes the behaviour of everything that follows. Pin it and truncate around it.

The one that catches people: **summarising is lossy and the loss is silent.** A summary is itself model output, with every property Chapter 16 gave it, and nothing tells you what it dropped. If you summarise, you need a way to find out what was lost, which is Chapter 35's subject and not a detail.

Which to choose, in one line each. The real difference between the first two is predictability: drop-oldest keeps as much history as fits, so the input and the cost per turn keep rising until the window is full; a fixed window of the last *k* turns makes the cost of a turn the same every time. So keep a window when you need a predictable bill or a predictable latency, and drop oldest when you would rather use whatever room is going and only lose the oldest when you must. Summarise when early turns *do* matter and you can afford a mechanism that proves what survived, which is Chapter 35 and is more work than it sounds. Start again when the task has changed — more often than people admit, and the only one of the four whose loss the user can see.



Decide this before you need it. A truncation strategy chosen in production at the moment a conversation first overflows is chosen by whoever is on call, and it is usually "drop oldest", which is the one that can silently delete your instructions.

Change One Thing at a Time

The opening problem of this chapter is a process problem, and it has a process answer.

Rewriting a prompt until something works is a search with no record. You cannot say what helped, you cannot undo the part that hurt, and you cannot tell whether the last edit did anything at all — because one run of a prompt is one sample, which Chapter 13 made the case for at length.

So: **change one block, keep the others fixed, and run it more than once.** That is the whole discipline, and the four-part split is what makes it possible. "I added two examples" is a change you can describe, repeat and reverse. "I rewrote the prompt" is not.

What this gets you is not rigour for its own sake. It is the ability to answer "why is this prompt like this?" six months later — which is a question somebody will ask, probably you.



Running it more than once, and knowing how many times is enough, is Chapter 20. This chapter asks only that you change one thing; Chapter 20 is how you find out whether the change was real.

What Current Practice Actually Weights

A last thing, and it is the one that most changes how people spend their time.

An enormous amount of writing about prompts is about *wording*: the phrasing that unlocks better answers, the magic preamble, the role to assign. Some of it has an effect. Very little of it has as much effect as **what information is in the request at all.**

The ordering that practitioners converge on, roughly:

1. **Is the information the model needs actually in the request?** If it is not, no wording fixes it — Chapter 1's point, and the reason Part II spends chapters 24 to 29 on retrieval.
2. **Is the task clearly specified, with the output shape stated?** Ambiguity costs far more than infelicity.
3. **Are there examples?** Showing beats describing for anything with a format or a style.
4. **Is the wording good?** Last, and a real but smaller lever.

In Chapter 6's terms that ordering is a **current convention** resting on a **practitioner report**, and short of settled practice because Chapter 6's tell for settled practice is written-down failure modes, and nobody has written down where this ordering stops holding — though its first item is a derivation from Chapter 1 and needs no survey. It is also the opposite of how most beginners allocate their afternoon.



The corollary is uncomfortable. If a prompt is failing and the information it needs is not in the request, you can rewrite it for a week without improving it — and you will feel busy the whole time. The first question is always what the model was given, not how it was asked.

Lab: Take a Prompt Apart

Pen and paper, then a calculator. Twenty-five minutes, and there is a key for every step.

Here is a prompt somebody wrote in one go:

```
You are an expert technical writer with 20 years of
experience. Read the release note below and write a
changelog entry. Make it concise and professional. Use the
past tense. For example, if the note says "we fixed the
thing where saving failed", you'd write "Fixed an error
that prevented documents from saving." Output just the
entry, no preamble. Release note: {note}
```

Step 1. Split it into the four parts. Which sentences belong to which?

Expected result: **Instructions** — "You are an expert technical writer…", "Read the release note below and write a changelog entry", "Make it concise and professional", "Use the past tense". **Examples** — the "for example" sentence. **Output format** — "Output just the entry, no preamble". **Supplied information** — `{note}`. The format instruction is at the end and the supplied information follows it, which is the wrong way round for caching and makes the format easy to miss.

Step 2. Rewrite it as four labelled blocks in the order this chapter recommends, changing no wording.

Expected result: instructions, examples, format, then the note. The point of the step is that you changed nothing and the prompt is already better to work with: you can now see that there is exactly one example, and that "concise and professional" may be doing no work the example is not doing better — which is now a hypothesis you can test on its own.

Step 3. Write it as a message list.

Expected result: a system message holding the first three blocks, and a user message holding the note. Optionally the example as a user/assistant pair instead — and if you do that, say which version you would test first.

Step 4. The system message is 180 tokens. A user turn is 120 and a reply is 200. Compute the input tokens for turns 1, 2 and 5 of a conversation.

Expected result: turn 1 is $180+120 = 300$. Turn 2 is $180+240+200 = 620$. Turn 5 is $180+600+800 = 1{,}580$. Each turn adds 320 to the one before, which is one user message plus one reply.

Step 5. The window is 4,000 tokens. At which turn does this conversation stop fitting, and what do you do?

Expected result: **turn 12.** The input at turn k is $180 + 320k - 200$, so turn 12 is 3,820 — and the window counts the reply too (Chapter 16), so a 200-token answer needs 4,020 and there is room for 180. Input alone passes 4,000 one turn later, at $180 + 4{,}160 - 200 = 4{,}140$. The turn that stops fitting is the one where the answer stops fitting, not the one where the question does. What to do is the four strategies, and whichever you pick, the system message stays.

Step 6. The prompt opens "You are an expert technical writer with 20 years of experience".

Which of the four parts is that, where does it sit in the last section's ordering, and what would you do with it?

Expected result: it is instructions — the *who to be* clause — and in the ordering it is item 4, wording. It is among the most repeated pieces of prompt advice in circulation, and it is a cheap thing to test: delete it, run both versions several times, and see whether anything moves. The point is not that personas never help. It is that this one costs tokens on every call, and the question of whether it earns them is a measurement nobody on the team has taken.

Step 7. Someone proposes changing "concise and professional" to "clear, concise and professional", and separately adding two more examples. What do you say?

Expected result: do them one at a time, and run each more than once. Also note which one you would expect to matter: by this chapter's ordering, two more examples is a bigger lever than an adjective. But the point of measuring is that expectations like that are sometimes wrong, and one run of each tells you nothing either way.

Step 8. The prompt is failing on release notes that reference an internal project code name the model has never heard of. What is the first thing to change?

Expected result: not the wording. The information is not in the request — the model cannot know what the code name refers to, which is Chapter 1's point and no amount of rewriting touches it. Put the code name's meaning in the supplied information, or retrieve it. This is the chapter's last section as a concrete case, and it is the most common way a week is wasted.

Summary, Key Terms, and Review Questions

Summary

A prompt has four separable parts — **instructions**, **examples**, **output format** and **supplied information** — and the first three are a specification that changes rarely while the fourth is data that changes every call. Separating them lets you change one and measure it, shows you where your bill is, and puts the unchanging part at the front where it can be cached. What you send is a list of messages with roles: **system** for your specification, **user** for the request, **assistant** for what the model said last time, and **tool** for results your code returns. You send the model's own replies back to it, because it remembers nothing. Every turn resends the whole list, so a ten-turn conversation's input is about five times the input of the same ten questions asked separately, and turn ten's input is eight and a half times turn one's. When the history will not fit, four strategies each lose something different — and your specification is never the thing to drop — the system message, and the example turns if that is where they live — while summarising loses things silently. Change one block at a time and run it more than once, because one run is one sample. And current practice

weights what the model receives far above how the instruction is worded: if the information is not in the request, no wording fixes it.

Key Terms

Term	Meaning
Instructions	What to do, what not to do and who to be. Part of the specification, so it changes rarely and is paid for on every call
Examples	What a good answer looks like, shown rather than described. Part of the specification
Output format	The shape the answer must take, stated rather than hoped for. Part of the specification
System message	The message holding your instructions and usually the output format. Your specification, set once per conversation and never the thing to drop when the history will not fit — nor are the example turns, if that is where your examples live
User message	The request and the material it is about, from the person or from your code on their behalf
Assistant message	What the model said earlier in the conversation, sent back to it by your code. The model does not remember saying it
Tool message	A result your code returns after the model asked for something. Chapter 23
Turn	One exchange — the user's message and the model's reply to it. A conversation is a list of turns, and the whole list is re-sent on each one
Supplied information	The part of a prompt that is different on every call. The only one of the four parts that is data rather than specification

Worth Memorising

Item	Value or formula
The four parts	Instructions, examples, output format, supplied information
The order to put them in	Specification first, data last, so the unchanging part is at the front
Cost of a conversation	Every turn re-sends the whole list; the total over n turns grows like n^2
Worked: ten turns	14,250 input tokens against 3,000 for ten separate questions
What never gets dropped	Your specification: the system message, and the example turns if that is where they live
What to change per experiment	One block

Review Questions

1. Which of a prompt's parts changes on every call, and what three things follow from that?
2. Why does your code send the model's own previous answers back to it?
3. A ten-turn conversation sends far more input than ten separate questions. Why, roughly how much more, and why is the bill not up by the same factor?
4. The history will not fit. Name the strategies and say what each loses.
5. Your prompt is not working. In what order do you try things?
6. A colleague has rewritten a prompt six times this afternoon and says the last version is best. What is wrong with that?

Review Answers

1. **Supplied information** — the request and the material it is about. The other three parts — instructions, examples and output format — are a specification that changes rarely; only the fourth is data. Separating them matters for three reasons that are not tidiness: you can change one block and measure it, where "I changed the prompt" is not a change you can describe or reverse; you pay for the first three on every request, so knowing which block is which is knowing where the bill is; and the three that never change can be placed at the front, where a provider can cache them.
2. Because the model has no memory between calls — nothing of one request survives into the next. The conversation is a list your code keeps, and the model's previous replies are in that list as *assistant* messages. What looks like memory from the outside is entirely your code re-sending a growing document.
3. Because every turn re-sends the whole list, so the input grows by one user message and one reply each time. With a 200-token system message, 100-token turns and 150-token replies, the ten turns together take 14,250 input tokens where ten separate questions

would take 3,000 — about five times — and turn ten's input alone is 8.5 times turn one's. The bill is not up by five, because the replies are the same 1,500 tokens either way and Chapter 1 says they are the dearer half. The input total grows like the square of the number of turns, and Chapter 15 adds that the attention cost inside each call is growing as well.

4. Drop the oldest turns, which loses whatever was established early and can lose your instructions if they are in the list. Keep a fixed window of the last few turns, which loses anything established before it. Summarise the old turns, which loses whatever the summary left out — silently, because a summary is model output and nothing reports what it dropped. Or start a new conversation, which loses everything but does so visibly. Your specification is never dropped — the system message, and the example turns if that is where your examples live. Pin them and truncate around them.
5. First, is the information the model needs actually in the request — because if it is not, no wording will fix it. Then, is the task clearly specified with the output shape stated. Then, are there examples, since showing beats describing for anything with a format. And last, the wording, which is a real lever and a smaller one. In Chapter 6's terms that ordering is a current convention resting on a practitioner report — except its first item, which is a derivation from Chapter 1 — and it is the reverse of how most people spend the afternoon.
6. They have run a search with no record. They cannot say which change helped, cannot reverse the one that hurt, and — because each version was run once — cannot tell whether any of the differences they saw were real rather than the ordinary variation between two samples. The fix is to change one block at a time and run each version more than once. Chapter 20 is how many times is enough.

> **You understand this when** you can take somebody else's prompt, split it into four blocks without changing a word, and say which block you would change first and why.

CHAPTER 19

Calling a Model Reliably

Chapter 18 was about what you send. This is about the call itself, and it is the chapter that separates a script from a system. Everything in it is ordinary engineering — retries, limits, concurrency, resumption — applied to a component with two properties that make it unusual: it is slow, and it answers a piece at a time.

The problem. My script works for ten documents. For ten thousand it dies on a 429, and I can't tell how far it got.

The question. What goes wrong when you call a model ten thousand times instead of ten, and what does the code have to do about each of those things?

Streaming

Chapter 15 explained why a reply arrives in pieces. This is how you receive them.

A streaming response is a long-lived HTTP connection down which the server sends small events as it produces them, in a format called **server-sent events** — a line per event, each one a `data:` line holding a small piece of JSON, with a blank line between events. The last line is not JSON but a plain marker, so whatever reads the stream has to allow for it:

```
data: {"delta": "The"}

data: {"delta": " cat"}

data: {"delta": " sat"}

data: [DONE]
```

Each piece is a **delta** — what to add to what you have so far, not the whole answer again. Your code appends them. That is nearly the whole protocol, and it is deliberately dull. What the last event looks like — the `[DONE]` marker above, or a typed event, or simply the connection closing — is one of the things each provider decides for itself.

What streaming buys you is a number you can feel. The time to the first word falls to whatever prefill takes, instead of the whole generation. Chapter 1 said two systems with the same total time can feel completely different; this is the mechanism for the one that feels fast.

And now the thing that catches people, and it follows from Chapter 13. A reply stops for one of several reasons — the model finished, it hit your maximum, a stop condition fired — and only the response itself says which. A truncated reply looks exactly like text. So the only way to know you are holding a whole answer is to have reached the end of it.

You cannot stream a response that you must validate as a whole. If your code has to check the answer before acting on it — that it is valid JSON, that it names a real product, that it does not contain something forbidden — then you need all of it to run the check. And once you have all of it, you have waited for the whole generation, so streaming has saved you no time at all and has given you a more complicated code path for it.

Worse, if you stream it *to a user* and then your check fails, you have already shown them the thing you were going to reject.

The edge of that rule is worth knowing, because it shows why the rule holds. If the answer is a *sequence of independent units* — one JSON object per line, one row at a time — then each unit is whole the moment it lands, so you can validate it and act on it while the rest is still arriving. Streaming is fine there. What you cannot do is act on part of something that is only valid in one piece.

And a stream gives you one thing a validator can use even then: if the first few tokens are already wrong, you can close the connection instead of paying for the remaining nine hundred.

```
to a person                yes - reading, not checking
one whole answer to check  no  - you must wait for the end
independent units          yes - each is whole on arrival
```

So the rule is: **stream when the consumer can use a piece, and do not when it needs the whole.** A person reading can use a piece. A check that only makes sense over the entire answer cannot. Chapter 21 is where validation gets its own chapter, and the two decisions are connected.

Cancellation. A streaming connection is open while the model generates, so you can close it. A user navigates away, a timeout fires, a better answer arrives from somewhere else — shut the connection and the generation normally stops. You are usually billed for what was produced before you stopped, which is less than the whole. You can abandon a non-streaming call too, but you have nothing to show for the time already spent, and no natural point at which to stop.



The cancellation point is the one people forget to build. If your interface lets a user ask a second question before the first has finished, and you do not cancel the first, you are paying for an answer nobody will read — on every impatient user, forever.

Rate Limits and 429s

A provider limits how fast you may call. The limits are usually two: requests per minute, and tokens per minute. When you exceed one, the response is **HTTP 429** — *too many requests* — and it is not an error in your code.

This is the commonest way a working script dies at scale, and the reason is arithmetic. Ten documents at one a second is inside any limit. Ten thousand, loosed as fast as your machine can open connections, is not.

A 429 usually comes with a header saying how long to wait. Use it when it is there. The rest of this section is what to do when it is not, and what to do about the other failures that look like it.

Read the body of the error, not only the status. The same 429 is returned when you are going too fast, which waiting fixes, and when your account has run out of credit, which waiting never fixes. The status cannot tell them apart; the error body usually can, and that is where to look.

What you get back	What it means	What to do
429	Too fast	Wait and retry
429 with an exhausted quota	You are out of credit	**Do not retry.** Pay or ask for more
500, 502, 503	Their side broke	Wait and retry
Timeout, connection reset	Something in between broke	Wait and retry
400	Your request is malformed	**Do not retry.** Fix it
401, 403	Your key is wrong or lacks permission	**Do not retry**
413, context length	You sent too much	**Do not retry.** Send less

The line down the middle of that table is the whole decision. Retrying something retryable costs a wait. Retrying something that will never succeed costs the same wait, every time, forever — and a 400 retried five times with backoff spends the whole retry budget, on every malformed request, learning nothing.

Retries: Backoff and Jitter

Retry immediately and you make the problem worse: you were called too fast, so you call again at once. The fix has two halves and both matter.

Exponential backoff doubles the wait after each failure. **Jitter** picks a random wait *up to* that amount rather than exactly it.

```
backoff = min(cap, base * 2 ** attempt)
wait = random.uniform(0, backoff)
```

Backoff keeps you from hammering a system that is struggling. Jitter keeps your ten thousand parallel workers from retrying *in step* — without it, everything that failed at the same moment retries at the same moment, which is the original problem again with extra steps.

Here is the whole thing, in a new empty folder as `retry.py`:

```
import random

class RateLimited(Exception):
    pass

def call(attempt_log, fail_times):
    """Stands in for the API call. Fails the first `fail_times` times."""
    attempt_log.append(len(attempt_log) + 1)
    if len(attempt_log) <= fail_times:
        raise RateLimited("429")
    return "ok"

def with_retry(fail_times, tries=5, base=0.5, cap=8.0):
    log, waits = [], []
    for attempt in range(tries):
        try:
            return call(log, fail_times), waits
        except RateLimited:
            if attempt == tries - 1:
                raise
            backoff = min(cap, base * 2 ** attempt)
            waits.append(round(random.uniform(0, backoff), 2))
    raise RuntimeError("unreachable")

random.seed(19)
for fails in (0, 3):
    result, waits = with_retry(fails)
    print(f"failed {fails} time(s) -> {result} after waits {waits}")
```

```
failed 0 time(s) -> ok after waits []
failed 3 time(s) -> ok after waits [0.34, 0.78, 1.04]
```

The real code would `time.sleep(wait)`; the example collects the waits instead so the output is instant and the same for every reader. The ceiling those waits are drawn from doubles each attempt and then stops:

```
attempt 1 fails: wait up to 0.5s
attempt 2 fails: wait up to 1.0s
attempt 3 fails: wait up to 2.0s
attempt 4 fails: wait up to 4.0s
attempt 5 fails: give up
```

Five attempts, four waits. The last attempt has nothing to wait for, which is easy to get wrong when you are counting. The worst case is 0.5 + 1 + 2 + 4 = 7.5 seconds of sleeping, and about half of that on average, because each wait is drawn from zero up to its ceiling. That is a reasonable place to stop and tell somebody.

The cap never binds at five attempts; `min(cap, …)` is there so that a larger `tries` cannot put you to sleep for an hour. At eight attempts the ceilings would be 0.5, 1, 2, 4, 8, 8, 8.



Retries hide problems as well as solving them. A pipeline that retries everything three times and logs nothing looks healthy at three times the cost and a third of the speed. Count your retries, and alarm on the rate rather than on individual failures.

Timeouts

Every call needs one, and the default is usually wrong.

The reason is Chapter 15's: a model's response time depends on the length of the reply and on how busy the provider is, neither of which you control. A call that normally takes three seconds can take sixty, and without a timeout one stuck connection holds a worker forever.

Three timeouts, not one, and this is where people get caught. A **connect** timeout — how long to wait for the server to answer at all — can be short, a few seconds. A **read** timeout, in most HTTP libraries, is *not* a budget for the whole response: it is the longest gap allowed between one piece of the body and the next. That is the right tool for noticing a dead connection, and it is useless for capping a long generation, because a model that sends a token every half second never trips a thirty-second read timeout however long it talks. The cap on the whole call is a third, **total timeout** — and some libraries do not have one, so you build it yourself by wrapping the call, with `asyncio.timeout` or the equivalent.

It is the total timeout you derive from the token cap. If you cap the reply at 1,000 tokens and tokens arrive at about twenty a second, fifty seconds is the floor.



`max_tokens / 20 + 10` is a crude rule and it is far better than the 30 you were going to write, because it makes the relationship explicit to whoever reads the code next. Check which of the three knobs your library's `timeout=` argument actually sets. The names are not consistent between libraries, and the default is usually wrong for this work.



Several numbers in this chapter are the shape of the thing rather than the thing: how many limits a provider publishes, whether a 429 carries a retry-after header, how a cancelled stream is billed, whether idempotency keys are supported, and twenty tokens a second. All of them vary by provider and all of them move. Measure yours; the reasoning is what carries over.

Concurrency, From Zero

Ten thousand documents at three seconds each is eight hours if you do them one at a time. You do not have to.

The calls spend nearly all of that time waiting for a reply, so they can wait at the same time. Python's `asyncio` is how you do that in one process, and the whole of what you need is three ideas.

`async def` makes a function that can be paused. Calling it does not run it; it hands you a coroutine, which is a job that has not started.

`await` runs one and waits for it — and while it waits, the program is free to run other jobs. This is the whole point: `await` is where control is handed back.

`asyncio.gather` runs many at once and gives you the results in order.

That is enough to write it:

```
import asyncio, time

async def fetch(doc, limiter):
    async with limiter:
        await asyncio.sleep(0.1)          # stands in for the call
        return f"done {doc}"

async def main(n, at_once):
    limiter = asyncio.Semaphore(at_once)
    started = time.perf_counter()
    results = await asyncio.gather(*(fetch(d, limiter) for d in range(n)))
    assert results == [f"done {d}" for d in range(n)]      # in order, always
    elapsed = time.perf_counter() - started
    print(f"{n} documents, {at_once} at once: {elapsed:.1f}s")

for at_once in (1, 4, 20):
    asyncio.run(main(20, at_once))
```

```
20 documents, 1 at once: 2.0s
20 documents, 4 at once: 0.5s
20 documents, 20 at once: 0.1s
```

Twenty documents at a tenth of a second each: two seconds in a row, half a second four at a time, a tenth of a second all at once. The arithmetic is exactly what you would expect, which is the point of running it. The `assert` is doing real work — it is the claim that `gather` hands back results in the order you asked for them, whatever order they finished in, and the program would stop if that were not so.

And now the line that matters. A **semaphore** is a counter that bounds how many jobs may be inside a block at the same time. `asyncio.Semaphore(at_once)` makes one. The statement `async with limiter` takes a unit and gives it back at the end of the block, and when none is left the next job waits. It is three words of code and it is what stands between you and the previous section.

Without it, `gather` over ten thousand documents opens ten thousand connections at once, and every one of them comes back 429. The semaphore is not an optimisation. It is the difference between concurrency and a denial-of-service attack on your own account.

Work the number out from the provider's limit, not from your machine. The rate you produce is the bound divided by how long a call takes: 3 at once at 3 seconds each is one a second, or 60 a minute. So the bound you want is *rate × duration* — 60 a minute is one a second, times 3 seconds, is 3.

But a semaphore bounds concurrency, not rate, and the difference is the thing to say out loud in an interview. If your calls come back faster than you assumed, the same bound produces a higher rate and you are over the limit again — 3 at once at 1 second each is 180 a minute. That is why you leave a margin, and it is why a tool that limits *rate* directly, a token bucket, is the better answer when the limit is the one that actually hurts.

One more thing about `gather`, and it is this chapter's own subject. If any one job raises, `gather` raises too — and the caller gets *nothing*, including the results of the jobs that had already finished. Ten thousand documents, one failure at document four thousand, and you are back at the problem this chapter opened with, caused by the tool that was meant to solve it.

The fix is one argument:

```
results = await asyncio.gather(*jobs, return_exceptions=True)
```

Now a failure arrives as an exception *object* in the results list, in the position of the job that raised it, and every other result is intact. You walk the list, keep what worked, and decide what to do about the rest — which is exactly what the record in *When It Dies Half-Way*, two sections below, is for.

Batch Endpoints

There is a second way to do ten thousand documents, and it is often the right one.

Providers offer a **batch endpoint**: you upload a file of requests, come back later, and download a file of results. It is slower to finish — hours rather than minutes — and substantially cheaper. Chapter 15 is the reason it can be: a batch reads each parameter once and shares it across every request in the batch, so a full batch costs much less than the same requests sent one at a time — up to a point, as that chapter is careful to say. Work you let the provider schedule is work they can use to fill a batch that would otherwise run half empty, and the discount is what they pay you for the flexibility.

	Many concurrent calls	Batch endpoint
Finishes in	Minutes	Hours
Costs	Full price	Substantially less
Rate limits	Yours to manage	Theirs
Good for	Anything a person is waiting for	Anything overnight

The question to ask is whether anybody is waiting. Re-processing an archive, generating embeddings for a corpus, scoring last month's conversations — nobody is waiting, and paying full price for speed you will not use is a decision somebody should make deliberately rather than by default.



The discount, the turnaround time and even the existence of a batch endpoint are provider-specific and they move. The question — *is anybody waiting?* — does not.

Idempotency

An operation is **idempotent** if doing it twice has the same effect as doing it once.

This matters here because of the retry section. When a call times out, you do not know whether it arrived. Retrying a request that actually succeeded means paying twice; if the request did something in the world, it means doing that thing twice.

Two defences, and you want both.

An idempotency key. You generate an identifier, send it with the request, and the provider undertakes to return the first response rather than doing the work again if it sees the same key. Generate it from the *content* — a hash of the document and the prompt — so that a retry of the same work produces the same key and a different piece of work does not.

A record of what you have done. Before you act on a result, check whether you already have one for this input. That is a dictionary, a file or a table, and it is the same thing as the next

section's.

One caution, because it bites immediately. A key is for a retry of a call you believe failed. It is not for a *deliberate* re-run: if you send the same prompt five times on purpose — to see how much the answers vary, which is the next chapter — a content-derived key may hand you the first answer five times and tell you the model is perfectly consistent.



The expensive case is not the doubled API bill. It is the pipeline step that sends an email, charges a card or writes a row — where a retry after a timeout does the thing twice and nothing in the model's world knows it happened. Chapter 4 said approval points belong in code; so does this.

When It Dies Half-Way

The opening problem, and the last piece. Ten thousand documents, a failure at four thousand, and no idea which four thousand.

The fix is to write down what you have finished, as you finish it. Not at the end — at the end there is nothing to write, because you crashed. One line per completed item, flushed, in a file you can read back.

```
import json, pathlib

DOCS = [f"doc-{i:03d}" for i in range(10)]
DONE = pathlib.Path("done.jsonl")
FIXED = False                       # set to True for the second run

def already_done():
    if not DONE.exists():
        return set()
    with DONE.open() as f:
        return {json.loads(line)["doc"] for line in f}

def process(doc):
    if doc == "doc-004" and not FIXED:
        raise RuntimeError("the API fell over")
    return doc.upper()

def run():
    done = already_done()
    print(f"  starting with {len(done)} already done")
    with DONE.open("a") as f:
        for doc in DOCS:
            if doc in done:
                continue
```

```
            try:
                result = process(doc)
            except RuntimeError as e:
                print(f"  stopped at {doc}: {e}")
                return False
            f.write(json.dumps({"doc": doc, "result": result}) + "\n")
            f.flush()
    print("  all done")
    return True

run()
```

Run it once with `FIXED = False`:

```
starting with 0 already done
stopped at doc-004: the API fell over
```

Then set `FIXED = True` — standing in for whatever you had to repair — and run it again, without deleting `done.jsonl`:

```
starting with 4 already done
all done
```

Four things make that work, and all four are easy to leave out.

One line per item, appended. Not a list held in memory and written at the end. A crash loses memory and keeps the file.

Flushed. Be precise about what `f.flush()` buys, because the common version of this claim is wrong. An ordinary exception loses nothing: the `with` block closes the file on the way out, and closing flushes. What `flush()` protects you against is the process being *killed* — out of memory, a container stopped, a `kill -9` — where nothing runs on the way out. And what you lose then is not the last line or two: it is everything still sitting in the buffer, which is thousands of bytes, so it can be a great many records. One word of code.



Flushing is still not durability. `flush()` hands the bytes to the operating system, not to the disk, so if the machine itself loses power they can go with it. `os.fsync(f.fileno())` is what waits for the disk, and it is slow enough that you would not do it per record without a reason — which is a real decision to make, not a detail.

Keyed by input, not by position. "I did the first four thousand" breaks the moment the input list changes order. "I did these four thousand identifiers" does not.

And the results are in it. The file is both a record of what is done *and* the output. A separate "progress" file that does not hold the answers means a crash costs you the answers anyway.



This file is also your idempotency record from the last section, and it is where you will look when somebody asks why document 7,214 is wrong. One file, three jobs. Build it on day one; it is twenty lines and it never stops being useful.

When the Pipeline Has Stages

The last section does one job to many items. Real work is usually a pipeline: extract the text, embed it, score it. That changes the question, because now a failure lands *between* stages, and "I got to document four thousand" does not say which stage document four thousand got to.

So the key is the pair, not the item. One line per `(document, stage)`, and the resume skips a step only if that document has been through that stage.

```
import json, pathlib

DOCS = [f"doc-{i:03d}" for i in range(5)]
STAGES = ["extract", "embed", "score"]
LEDGER = pathlib.Path("stages.jsonl")
BROKEN = True                       # set to False for the second run

def done_steps():
    if not LEDGER.exists():
        return set()
    with LEDGER.open() as f:
        return {(r["doc"], r["stage"]) for r in map(json.loads, f)}

def work(doc, stage):
    if stage == "embed" and doc == "doc-002" and BROKEN:
        raise RuntimeError("the embedding API fell over")
    return f"{stage}:{doc}"

def run():
    done = done_steps()
    total = len(DOCS) * len(STAGES)
    print(f"  {len(done)} of {total} steps already done")
    with LEDGER.open("a") as f:
        for stage in STAGES:
            for doc in DOCS:
                if (doc, stage) in done:
                    continue
                try:
                    result = work(doc, stage)
                except RuntimeError as e:
                    print(f"  stopped at {stage} of {doc}: {e}")
```

```
                return
            f.write(json.dumps({"doc": doc, "stage": stage,
                                "result": result}) + "\n")
            f.flush()
    print(f"  all {total} steps done")

run()
```

Five documents, three stages, fifteen steps. The first run:

```
0 of 15 steps already done
stopped at embed of doc-002: the embedding API fell over
```

Then `BROKEN = False`, and run it again:

```
7 of 15 steps already done
all 15 steps done
```

Seven is the number to look at. Five extracts and two embeds had got through, and the second run did not repeat one of them — which is the entire point, because extraction was the slow part. A ledger keyed by document alone could not have expressed "extracted but not embedded".

Stage at a time, or item at a time? The program above finishes every document's extraction before it starts embedding. The other order — each document all the way through before the next one begins — is equally easy to write and behaves differently:

	Stage at a time	Item at a time
First complete result	After the last stage	Early
A stage you can batch	Yes, all of it together	One at a time
A broken stage	Fails everything at once, visibly	Fails one item, then the next
Re-running one stage	Delete its lines, run again	Same, but interleaved with the rest

Stage at a time for bulk work you are going to batch; item at a time when somebody is waiting for the first answers, or when the items are independent enough that one bad document should not stop the others.

And now the part that is actually hard. A stage that only computes is safe to retry. A stage with an *external* effect — it sends the email, it writes the row, it charges the card — is not, and a resume will do that effect twice unless you have done the work in the last section but one: make the stage idempotent, key it by content, and write its ledger line *after* the effect has landed rather than before.



The failure mode here is quiet. A pipeline whose every stage only computes re-runs cleanly for a week and teaches you to trust the resume. Add a notification stage and the ledger looks no different, but a crash now costs a duplicate email for whatever was in flight — one, with the ledger line written after the effect, and every message in the batch if somebody writes it before. The day it is thousands is the day somebody re-runs the stage on purpose to fix something else.

Keep each stage's output, not just a tick. The ledger above stores `result` beside the pair, which means a stage can be re-run on its own: change the scorer, delete the `score` lines, and run again — the extracts and embeddings are still there. If the ledger held only "done", you would have to start from the beginning to change the last step.

Lab: Make a Fragile Script Robust

Four programs, all pure Python with nothing to install and no account anywhere. Forty minutes, and there is a key for every step. Work in a new empty folder.

Step 1. Save the retry program from this chapter as `retry.py` and run it.

Expected result: exactly this, because the seed is fixed.

```
failed 0 time(s) -> ok after waits []
failed 3 time(s) -> ok after waits [0.34, 0.78, 1.04]
```

Step 2. Make the call fail five times: change the last loop to `for fails in (0, 5)`. What happens, and why is that right?

Expected result: the first line still prints, and then `RateLimited` comes out as an uncaught traceback and the program exits 1. Five attempts means four retries, and the call is still failing on the fifth, so the code gives up — which is what you want, because something that has failed five times with growing waits is not going to succeed on the sixth. The alternative, retrying forever, is a pipeline that never finishes and never tells you. In real code you would catch it at the top and record which document it was.

Step 3. Remove the jitter — use `wait = backoff` instead of `random.uniform(0, backoff)`. What breaks, and when would you notice?

Expected result: nothing breaks in this program, because there is only one caller. You would notice with a thousand workers: they all failed at the same moment, so without jitter they all wait the same 0.5 seconds and all retry at the same moment, and the provider sees the same spike again. Jitter spreads them. **The defect is invisible at the scale you test at**, which is the general shape of everything in this chapter.

Step 4. Save the concurrency program as `concur.py` and run it. Then change `asyncio.Semaphore(at_once)` to `asyncio.Semaphore(20)`, leaving the loop alone, and say what

you have done.

Expected result: the first run gives about 2.0s, 0.5s and 0.1s. With the bound hard-coded to 20 it no longer varies, so all three lines print about 0.1s — you have removed the bound while leaving code that still looks as though it is there, which is how most unbounded pipelines come to exist. Note that the `assert` keeps passing: ordering has nothing to do with the bound.

Step 5. Your provider allows 120 requests a minute. A call takes 2 seconds. How many should you run at once?

Expected result: `120/60 = 2` requests a second, and each occupies a worker for 2 seconds, so four at once keeps you at the limit. Go to 8 and you are asking for 240 a minute and you will spend half your time in backoff. The formula is *rate × duration*, and it is worth writing in a comment next to the number.

Step 6. Save the resume program as `resume.py` and run it. Then set `FIXED = True` and run it again, without deleting `done.jsonl`.

Expected result: the two runs from the chapter — stopped at `doc-004` with none done, then starting with four already done and finishing. Check `done.jsonl`: ten lines, each with its document and its result.

Step 7. Delete the `f.flush()`, delete `done.jsonl`, and run both runs again. Does anything change? Then make the process die without running anything on the way out: set `FIXED = True`, delete `done.jsonl`, `import os`, and put `os.kill(os.getpid(), 9)` as the last line inside the `with` block, after the loop. Count the lines in `done.jsonl` — once with the flush, once without.

Expected result: deleting the flush changes nothing in the ordinary runs, because the `with` block closes the file on the way out and closing flushes — an exception is not what `flush` protects you from. Under `kill -9` the difference is total: with the flush you keep every line written so far, and without it you can lose all of them, because the buffer is thousands of bytes and ten short records never fill it. That is why the bug survives into production: the only thing that exposes it is the one event you do not test. Put the flush back.

Step 8. You are processing 10,000 documents. Your code streams each response, validates it as JSON, and writes a progress file with the index of the last document completed. Name three things wrong with that.

Expected result: streaming saves no time here, because the answer has to be valid JSON as a whole and you cannot check that until it has all arrived. All it buys is the chance to give up early on an answer that is already malformed. The progress file holds an index rather than identifiers, so it breaks if the input list is ever reordered or filtered. And it holds no results, so a crash at document 9,000 means you know where you were and have lost the answers. There are two more if you spotted them: nothing said anything about a bound on concurrency, and nothing said what happens when one document fails.

Step 9. Save the stages program as `stages.py`. Run it, set `BROKEN = False`, run it again.

Then answer this: why is seven the interesting number, and what would break if the ledger were keyed by document alone?

Expected result: `0 of 15` and then `stopped at embed of doc-002`; on the second run, `7 of 15 steps already done` and `all 15 steps done`. Seven is five extractions plus two embeddings — work the resume did not repeat. Keyed by document alone the ledger could only say "doc-000: done", which is false (it has been extracted and embedded but not scored) or useless (it is not finished, so redo all three stages). The pair is what makes a partial document expressible.

Step 10. In `concur.py`, make `fetch` raise on one document: put the line `if doc == 7: raise RuntimeError("boom")` before the sleep, and run it. Then add `return_exceptions=True` to the `gather` call and run it again.

Expected result: first the whole program dies with that `RuntimeError` and prints no timing line at all — not even for the documents that had already finished before doc 7 ran, whose results are simply gone. The `assert` never even runs. With `return_exceptions=True` the call returns, and the `assert` fails instead, because `results[7]` is a `RuntimeError` object rather than `"done 7"`. Both failures teach something: the first throws away good work, and the second shows you that dealing with the exception objects is now *your* job.

Summary, Key Terms, and Review Questions

Summary

A streaming response is **server-sent events** carrying **deltas** your code appends, and it buys a shorter wait before the first word — but **you cannot stream a response you must validate as a whole**, because the check needs all of it, so stream when the consumer can use a piece and not when it needs the whole; a sequence of independent units is the case where it works. Streaming also gives you a cancellation point, which is the saving people forget to build. **429** means too fast and is not an error in your code; it is retryable, as are 500s and timeouts, while a malformed request, a bad key, an over-length request and a 429 raised by an exhausted quota are not, so read the error's body and not only its status. Retry with **exponential backoff** so you stop hammering a struggling system, and **jitter** so your workers do not retry in step; five attempts make four waits. Every call wants three timeouts — connect, read between chunks, and a **total** derived from the token cap, which is the only one that caps a long generation. `asyncio` lets calls wait at the same time, and a **semaphore** bounds how many: without it, `gather` over ten thousand documents opens ten thousand connections and every one comes back 429. But a semaphore bounds concurrency, not rate, so leave a margin or use a **token bucket**; and `gather` throws away every good result when one job raises, unless you pass `return_exceptions=True`. A **batch endpoint** is slower and substantially cheaper, and the question it turns on is whether anybody is waiting. **Idempotency** matters because a timed-out call may have succeeded: use a content-derived key,

keep a record, and do not let that key defeat a re-run you meant. And write down what you have finished as you finish it — one flushed line per item for one job over many items, one per `(item, stage)` for a pipeline, keyed by input and holding the results.

Key Terms

Term	Meaning
Server-sent events	A long-lived HTTP response carrying one small event per line as the server produces them. How a streamed reply arrives
Delta	One streamed piece: what to add to what you have, not the whole answer again
429	*Too many requests.* The provider's rate limit, on requests per minute and on tokens per minute. Retryable when you are going too fast; the same status comes back when the account is out of credit, which no wait fixes
Exponential backoff	Doubling the wait after each failed attempt, up to a cap, so a struggling system is not hammered
Jitter	Waiting a random amount up to the backoff rather than exactly it, so many workers do not retry in step
Total timeout	The cap on a whole call, as distinct from the gap between chunks that most libraries call the read timeout. Derived from the token cap
Semaphore	A counter that bounds how many jobs run at once. Three words of code, and the difference between concurrency and attacking your own account
Token bucket	A limiter on the *rate* rather than on how many run at once, for when the requests-per-minute limit is the one that hurts
Batch endpoint	Upload requests, collect results hours later, pay substantially less. For work nobody is waiting on
Idempotency	Doing an operation twice has the same effect as doing it once. What makes a retry after a timeout safe

Worth Memorising

Item	Value or formula
Backoff with jitter	`wait = random.uniform(0, min(cap, base * 2 ** attempt))`
Attempts and waits	*n* attempts make *n* – 1 waits. Five attempts at base 0.5 is 7.5 s at worst
How many at once	rate × duration — 120 a minute at 2 seconds each is 4. It bounds concurrency, not rate, so leave a margin
Timeouts	Three: connect, read between chunks, total. Derive the *total* from the token cap
Retryable	429, 5xx, timeouts. *Not* a malformed request, a bad key, an over-length one, or a 429 from an exhausted quota
What to stream	To a person, yes. One answer you must validate whole, no. Independent units, yes
`asyncio.gather`	One failure loses every result unless you pass `return_exceptions=True`
The resume file	One flushed line per item, or per `(item, stage)` for a pipeline. Keyed by input, holding the result

Review Questions

1. Your script works on ten documents and dies on a 429 at document four thousand of ten thousand. What do you do?
2. Why can you not stream a response that you have to validate?
3. What is idempotency, and where does it matter in a pipeline like this?
4. Why is jitter necessary when backoff alone seems to solve the problem?
5. Your provider allows 300 requests a minute and a call takes 4 seconds. How many do you run at once, and how did you get that?
6. When would you use a batch endpoint instead of running many calls yourself?
7. A multi-step pipeline fails half way. How do you resume it?

Review Answers

1. Four things, and the order matters. First, make the failure survivable: write one flushed line per completed document, keyed by the document's identifier and holding its result, so a crash costs you one document rather than four thousand. Second, stop causing the 429: bound the concurrency with a semaphore set from the provider's limit — rate times duration — rather than letting `gather` open as many connections as it likes, and pass `return_exceptions=True` so one failure does not discard the rest. Third, retry what is retryable with exponential backoff and jitter, and do not retry a malformed request at all. Fourth, give every call a total timeout derived from the token cap, so one stuck connection cannot hold a worker for ever. Then re-run it; it starts from where it

stopped.

2. Because the check only makes sense over the whole answer — that it parses, that it has the fields you need, that it says something permissible — and you cannot run it until the whole answer has arrived. Once it has, you have waited for the entire generation, which is what streaming was supposed to avoid, so you have taken on a more complicated code path and saved no time by it. Worse, if you streamed it to a user as well, you have already shown them the answer you are about to reject. The edge is worth giving too, because it shows why the rule holds: if the output is a sequence of independent units, each one is whole as it lands, so you can validate and act on them one at a time and streaming is fine.
3. An operation is idempotent if doing it twice has the same effect as doing it once. It matters because a timed-out call may well have succeeded — you have no way to tell — so every retry risks doing the work twice. For a model call that means paying twice. For a pipeline step that sends an email, charges a card or writes a row, it means doing that thing twice. The defences are an idempotency key derived from the content, so the same work produces the same key, and a record of what you have already done, checked before you act. The thing to add unprompted is that the key is for a retry of a call you think failed, not for a re-run you meant: send the same prompt five times on purpose under a content key and you may get the first answer five times.
4. Because backoff alone is a *per-worker* fix and the problem is usually *across* workers. A thousand callers that all failed at the same moment will all wait the same half second and all retry at the same moment, so the provider sees the same spike it rejected, and the cycle repeats with longer gaps. Jitter spreads the retries over the interval instead. The reason it is easy to leave out is that it makes no difference at all with one caller, which is the scale you test at.
5. Twenty at once, or sixteen with a margin. The arithmetic is rate times duration: $300/60 = 5$ requests a second, each occupying a worker for 4 seconds, so $5 \times 4 = 20$ in flight holds you at the limit. The margin is the part worth saying out loud: a semaphore bounds concurrency, not rate, so twenty in flight only produces 300 a minute while calls really take 4 seconds. Sixteen buys you a fifth of headroom for being a little wrong about that. It does not buy you much more than that, and it is worth saying so: if a burst of calls comes back in 2 seconds, sixteen in flight is 480 a minute and you are over anyway. No concurrency bound fixes that, because the quantity it bounds is not the quantity the provider limits. A token bucket does.
6. When nobody is waiting. Re-processing an archive, embedding a corpus, scoring last month's conversations — work that has to be done by tomorrow rather than in the next minute. The batch endpoint takes hours instead of minutes and costs substantially less, because work the provider can schedule is work they can use to fill a batch that would otherwise run half empty. Paying full price for speed nobody will use should be a decision somebody made, not the default that happened because the synchronous call was the one in the example.

7. Key the record by the pair, not the item: one line per `(item, stage)`, so the resume skips a step only when that item has been through that stage. A ledger keyed by the item alone cannot express “extracted and embedded but not scored”, which is exactly the state a half-way failure leaves. Store each stage’s output beside the pair, not just a tick, so a later stage can be re-run on its own without redoing the earlier ones. Then say the hard part before you are asked: a stage that only computes is safe to repeat, but a stage with an external effect — it sends the email, it charges the card — is not, so it needs the idempotency work, and its ledger line must be written after the effect has landed rather than before. Decide also whether to run a stage at a time, which batches well and fails visibly, or an item at a time, which gets the first complete results out early.

> **You understand this when** you can take a script that calls a model in a loop and name the four things it needs before it can be run on ten thousand inputs.

CHAPTER 20

Is the Difference Real?

Chapter 18 said to change one thing and run it more than once. This chapter is the second half of that sentence: what *more than once* has to mean, and what you are allowed to conclude when you have done it.

The reason it matters is unflattering. Most prompt work is done by changing something, looking at a handful of outputs, deciding it is better, and keeping it. That procedure cannot distinguish an improvement from a coincidence, and the arithmetic below says it usually is not distinguishing them.

The problem. I changed the prompt and it looks better. I have no idea whether it actually is.

The question. What does it take to know that a change helped, and what does a single comparison really tell you?

The Set You Measure On

You cannot answer the question with examples you look at. You need a fixed set of cases and a judgement you can apply to each one the same way every time.

For this chapter a **case** is an input plus whatever lets you mark the answer right or wrong. That is all the statistics need. Three properties matter:

The judgement is mechanical. Given the same answer twice, it gives the same verdict twice. If a person is reading the answers and deciding, you have a second source of variation on top of everything below, and you cannot separate it from the first. A judgement that wanders is a problem to fix rather than to average away — though where you cannot fix it, repeated judging at least bounds it, which is what trials are for below. Chapter 31 is where a judgement gets built and checked properly.

The cases are independent. Forty rephrasings of one question are worth far less than forty cases — closer to one — and the arithmetic in this chapter cannot tell. It counts rows, so it will happily hand you a narrow interval on a set that is four questions wearing forty hats.

The set does not change while you are comparing. Add a case because B got it wrong

and you have stopped measuring. Chapter 33 is about the set growing over time, which is a different activity from running a comparison.



Where the cases come from, how to turn real failures into them, and how to build a judgement you can trust are chapters 30, 31 and 33. This chapter assumes you have forty cases and a verdict for each, because that is the smallest thing you can do statistics to. Forty is not a recommendation; by the end of the chapter it will be clear what it is.

One Run Is Not a Score

Start with a single prompt, forty cases, and a model that gets any given case right about 70 per cent of the time. Run the whole set five times.

```
import random, statistics

CASES = 40
P = 0.70          # this model gets each case right 70% of the time

def one_run():
    return sum(1 for _ in range(CASES) if random.random() < P)

random.seed(20)
for run in range(1, 6):
    right = one_run()
    print(f"  run {run}: {right}/{CASES} = {right / CASES:.3f}")

scores = sorted(one_run() / CASES for _ in range(10_000))
print(f"  over 10,000 runs: lowest {scores[0]:.3f}, highest {scores[-1]:.3f}")
print(f"  middle 95%: {scores[250]:.3f} to {scores[9750]:.3f}")
print(f"  standard deviation: {statistics.stdev(scores):.3f}")
```

```
  run 1: 29/40 = 0.725
  run 2: 30/40 = 0.750
  run 3: 35/40 = 0.875
  run 4: 25/40 = 0.625
  run 5: 31/40 = 0.775
  over 10,000 runs: lowest 0.450, highest 0.925
  middle 95%: 0.550 to 0.825
  standard deviation: 0.073
```

Read runs 3 and 4 again. 0.875 and 0.625. Same prompt, same forty cases, same model, nothing changed between them. A quarter of the whole scale, from nothing at all.

Now imagine those two runs were your before and after. You would have measured a

twenty-five point improvement from a change you did not make, and you would have believed it, because twenty-five points is a lot and your eyes were on the outputs rather than on the arithmetic.

And the spread is not luck, it is arithmetic. Each case is a coin weighted to 0.70, so forty of them have a standard deviation of

$$\sqrt{np(1-p)} = \sqrt{40 \times 0.70 \times 0.30} = \sqrt{8.4} = 2.90 \text{ cases}$$

which as a score is $2.90/40 = 0.072$ — one standard deviation — and the program measured 0.073 over ten thousand runs. Two standard deviations either side is about ± 0.14.

Hold on to the number 0.14. The difference this chapter goes on to chase is 0.075.

Now be careful what that spread is measuring, because this program makes two different things coincide. `one_run()` treats every case as its own coin at 0.70, so a fresh run of the same forty cases is arithmetically the same thing as drawing forty new cases. Real evaluation sets are not like that: most cases are reliably right or reliably wrong, and only a few are genuinely uncertain. Three sets that all score about 0.700:

```
import random, statistics

# Three evaluation sets that all score about 0.700, differing only in
# how uncertain the model is on each case.
SETS = {
    "every case a 0.70 coin":       [0.70]*40,
    "28 right, 12 wrong, reliably": [0.98]*28 + [0.02]*12,
    "24 right, 8 wrong, 8 coins":   [0.98]*24 + [0.02]*8 + [0.50]*8,
}

def run(per_case, repeats=1):
    """Score the set once, judging each case `repeats` times and averaging."""
    total = 0.0
    for q in per_case:
        hits = sum(1 for _ in range(repeats) if random.random() < q)
        total += hits / repeats
    return total / len(per_case)

random.seed(20)
for name, per_case in SETS.items():
    scores = [run(per_case) for _ in range(5_000)]
    print(f"  {name:30s} {statistics.fmean(scores):.3f}"
          f"   sd {statistics.stdev(scores):.3f}")
```

```
every case a 0.70 coin          0.699   sd 0.074
28 right, 12 wrong, reliably    0.691   sd 0.022
24 right, 8 wrong, 8 coins      0.693   sd 0.040
```

The middle line is the one to notice. A system that is reliably right on twenty-eight cases and reliably wrong on twelve scores exactly 0.700 in about half its runs, and lands within one case of it in nine out of ten; its run-to-run spread is under a third of the coin model's. So 0.14 means two different things and you have to say which:

- **It is always the spread over which forty cases you happened to draw.** That is what the next section computes, and it does not depend on how certain the model is per case.
- **It is the spread over repeated runs only when the model is as uncertain per case as the coin.** A near-deterministic system moves much less between runs.

The coin is the worst case, which is why a plus-or-minus is built from it. Runs 3 and 4 above are real and so is the arithmetic; just do not quote fourteen points as run-to-run noise for a system you have not measured that way.



The seed is in that program so that your numbers match the book's. It does not make the variation go away, and nothing you can set makes a real provider repeat itself exactly. Temperature zero does not either: Chapter 13 gives four reasons — floating-point addition order, batching, expert routing and the provider changing something under you — and none of them is a setting you control. What those four cause is the *run-to-run* kind of variation, the second reading above, and at temperature zero it is small — near-ties flipping, not a coin on every case. Small is not zero, and you do not know how small on your own set until you run it twice. **Repeated runs are not a workaround for a temperature you forgot to set. They are the measurement.**

An Interval You Can Justify

So a score needs an interval around it. The textbook move is to reach for a named distribution and a formula. This book does not, and the reason is worth saying plainly, because it is the part people get wrong when asked.

There is a formula for this one, and it is not the reason to learn the formula. A proportion has a well-known interval, and at forty cases and a score of 0.70 it is perfectly serviceable. The trouble is that it is a formula for *one* statistic. The number this chapter actually needs an interval on is the **paired difference** between two prompts on the same cases, and its formula needs more than the two scores — you have to bring in how often the two disagree. For right-or-wrong marks that formula is still a one-liner, and it agrees with the bootstrap everywhere in this chapter. But change the score to an average of per-case marks, or to a ranking measure, and the shape of the formula changes with it. Every one is a thing to derive, look up, or get subtly wrong.

The bootstrap is one procedure for all of them. Draw forty cases from your forty, with replacement, compute *whatever your statistic is* on that sample, and do it ten thousand times. The spread of those ten thousand numbers is an estimate of how much your statistic would move if you had drawn a different forty cases of the same kind. Take the 2.5th and 97.5th percentiles and you have a 95 per cent interval.

Its assumption is weaker than a named distribution's and it is one you can state out loud: **that your cases are a sample of the kind of input you care about.** If that is false, no method on this page helps you, and you have an evaluation-set problem rather than a statistics problem.

That is the whole method, and it is three lines of the program below.

```
import random

# 40 cases. 1 means the answer was judged right.
# Both right 26, A only 2, B only 5, both wrong 7.
A = [1]*26 + [1]*2 + [0]*5 + [0]*7
B = [1]*26 + [0]*2 + [1]*5 + [0]*7

def score(results):
    return sum(results) / len(results)

def interval(results, trials=10_000):
    n = len(results)
    scores = []
    for _ in range(trials):
        sample = [results[random.randrange(n)] for _ in range(n)]
        scores.append(score(sample))
    scores.sort()
    return scores[int(0.025 * trials)], scores[int(0.975 * trials)]

random.seed(20)
for name, results in (("A", A), ("B", B)):
    lo, hi = interval(results)
    print(f"  {name}: {score(results):.3f}   95% interval {lo:.3f} to {hi:.3f}")
```

```
  A: 0.700   95% interval 0.550 to 0.825
  B: 0.775   95% interval 0.650 to 0.900
```

Prompt A scores 0.700, with an interval from 0.550 to 0.825 — the same two numbers the previous section's ten thousand actual runs produced.

That match is exact for a reason, and the reason is worth more than the match. A holds exactly 28 ones out of 40, so resampling it with replacement *is* drawing from a list that is exactly 70 per cent ones — which is the distribution the previous section sampled di-

rectly. The two procedures are not independent and then agreeing; they are one distribution reached two ways. The exact binomial percentiles are 0.550 and 0.825 as well, so all three agree, and they had to.

What that does show is that the bootstrap recovers the right *width*. What it cannot show is that your interval is in the right *place*: it is anchored on the score you measured — look at A's, which runs further below 0.700 than above it — so if you drew an unlucky forty cases, the whole interval moves with them. A confidence interval is a statement about the procedure, not a promise about this one run.



Ten thousand resamples is not a tuning parameter to agonise over. A thousand is visibly lumpy at the percentiles; ten thousand is smooth and takes a fraction of a second on forty cases. The number that matters is the number of *cases*, not the number of resamples — resampling cannot create information that forty cases do not contain.

Overlapping Intervals Are Not the Test

A's interval runs to 0.825 and B's starts at 0.650, so they overlap heavily. The obvious conclusion is that there is no evidence of a difference.

That conclusion happens to be right here and the reasoning is wrong, and it is wrong in a way that will cost you later. Two intervals overlapping does not mean the difference is uncertain, and two intervals not overlapping is a stricter condition than the one you actually care about. The question is not *where is A* and *where is B*. The question is *where is B minus A*, and that has an interval of its own.

So compute the thing you want an interval on.

Compare on the Same Cases

Here is the move that buys more than any other in this chapter, and it costs nothing.

Run both prompts on the *same* cases, keep each case's two results together, and resample *cases* rather than results. Each resample draws forty cases and asks what B minus A was on those forty.

```
import random

A = [1]*26 + [1]*2 + [0]*5 + [0]*7
B = [1]*26 + [0]*2 + [1]*5 + [0]*7

def difference(pairs):
    return sum(b - a for a, b in pairs) / len(pairs)
```

```
def interval(pairs, trials=10_000):
    n = len(pairs)
    diffs = []
    for _ in range(trials):
        sample = [pairs[random.randrange(n)] for _ in range(n)]
        diffs.append(difference(sample))
    diffs.sort()
    return diffs[int(0.025 * trials)], diffs[int(0.975 * trials)]

pairs = list(zip(A, B))
random.seed(20)
lo, hi = interval(pairs)
print(f"  B - A = {difference(pairs):+.3f}   95% interval {lo:+.3f} to {hi:+.3f}")

agree = sum(1 for a, b in pairs if a == b)
a_only = sum(1 for a, b in pairs if a == 1 and b == 0)
b_only = sum(1 for a, b in pairs if a == 0 and b == 1)
print(f"  {agree} cases agree, {a_only} only A gets right, {b_only} only B")
```

```
  B - A = +0.075   95% interval -0.050 to +0.200
  33 cases agree, 2 only A gets right, 5 only B
```

The interval on the difference runs from −0.050 to +0.200, and **it contains zero.** B might be seven and a half points better, and it might be five points worse, and forty cases cannot tell you which.

Now see what the pairing bought. Run the same comparison without it — resample A's results and B's results independently, as if they had been measured on different sets:

```
random.seed(20)
out = []
for _ in range(10_000):
    a = sum(A[random.randrange(40)] for _ in range(40)) / 40
    b = sum(B[random.randrange(40)] for _ in range(40)) / 40
    out.append(b - a)
out.sort()
print(f"  unpaired: 95% interval {out[250]:+.3f} to {out[9750]:+.3f}")
```

```
  unpaired: 95% interval -0.125 to +0.275
```

0.400 wide against 0.250, on exactly the same data — the unpaired interval is about 1.6 times the paired one. Note what pairing did and did not do. It did *not* remove the variation that comes from which cases you drew: the 0.250 that is left is entirely that. What it removed is the part of that variation the two prompts *share* — a hard case hurts both of them,

and when you subtract, the shared part cancels. What survives is how the two prompts differ on the same inputs. Nothing was assumed and no case was added.

And the last line of the program is the one to take to an interview. Thirty-three of the forty cases came out the same way under both prompts. Those cases carry no information about which prompt is better — they are the same evidence for both. The entire comparison rests on the seven cases where the two prompts disagree: five for B, two for A. Lab step 6 measures this: move cases between both-right and both-wrong and the interval does not budge.

Five against two. Put that way, nobody would claim a result. It is the same number as +0.075 out of 40, which sounds much more impressive, and that is exactly why it is worth computing both ways.

How Many Cases, and How Many Trials

Two different questions with two different answers, and they are easy to confuse because both sound like "run it more".

Cases tell you about the range of inputs. More cases narrow the interval on the difference. Keep the shape of the forty-case result and ask what a larger set of the same kind would show:

```
import random

# Keep the pattern of the 40 cases and ask what a bigger set of the same
# kind would show: 65% both right, 5% only A, 12.5% only B, 17.5% both wrong.
PATTERN = [(1, 1)]*26 + [(1, 0)]*2 + [(0, 1)]*5 + [(0, 0)]*7

def interval(pairs, trials=2_000):
    n = len(pairs)
    diffs = []
    for _ in range(trials):
        s = [pairs[random.randrange(n)] for _ in range(n)]
        diffs.append(sum(b - a for a, b in s) / n)
    diffs.sort()
    return diffs[int(0.025*trials)], diffs[int(0.975*trials)]

random.seed(20)
for k in (1, 2, 5, 10, 25):
    pairs = PATTERN * k
    lo, hi = interval(pairs)
    verdict = "B is better" if lo > 0 else "cannot tell"
    print(f"  {len(pairs):5d} cases   {lo:+.3f} to {hi:+.3f}   {verdict}")
```

```
  40 cases   -0.050 to +0.200   cannot tell
  80 cases   -0.013 to +0.163   cannot tell
 200 cases   +0.020 to +0.135   B is better
 400 cases   +0.035 to +0.113   B is better
1000 cases   +0.049 to +0.101   B is better
```

Two hundred cases is where this difference starts to become visible. Not forty, and not eighty. If somebody asks how many cases are enough, that table is the answer in its proper form: enough to see the difference you care about, and you find the number by computing it rather than by choosing a round one.

Read the table for what it is, though. `PATTERN * k` replicates the forty-case mix exactly, so every row shows what the interval looks like when the larger set has precisely this composition. A real draw of two hundred cases varies, and you can work out exactly how much: across every possible draw of two hundred from those rates, weighted by how likely each is, this interval clears zero 72 times in a hundred. At four hundred cases it is 95. So two hundred is where you would *start* to see this difference, and four hundred is where you could count on it — which is the distinction between a typical interval and a detection rate, and it is the one people skip.

The shape of the table is the familiar one. Width shrinks with the square root of the count, so each halving of the interval costs four times the cases: eighty gives you 0.176 of width, and a thousand gets you to 0.052, which is 3.4 times narrower for 12.5 times the cases — and $\sqrt{12.5} = 3.54$.

Trials tell you about the model's own variation on one input. That is the first section's problem, and more cases do not touch it. If you run each case once, the right-or-wrong you record is one sample of something that was never certain.

And there is arithmetic for how many, which is the same law applied to a different quantity. Judge a case `\(k\)` times and average, and that case's own variance is divided by k, so the run-to-run spread shrinks as $1/\sqrt{k}$. The `run()` function above already takes the argument; this continues that program:

```
flips = SETS["24 right, 8 wrong, 8 coins"]
random.seed(20)
for k in (1, 2, 3, 5, 10):
    scores = [run(flips, repeats=k) for _ in range(5_000)]
    print(f"  {k:2d} repeat(s) per case   run-to-run sd "
          f"{statistics.stdev(scores):.3f}")
```

```
 1 repeat(s) per case   run-to-run sd 0.041
 2 repeat(s) per case   run-to-run sd 0.028
 3 repeat(s) per case   run-to-run sd 0.023
 5 repeat(s) per case   run-to-run sd 0.018
10 repeat(s) per case   run-to-run sd 0.013
```

Look at what each repeat costs and buys. The second one takes the spread from 0.041 to 0.028, a saving of 0.013 for one extra pass over the set. Going from three repeats to ten — seven more passes — buys 0.010, less than the second repeat managed on its own, for seven times the work. That is the square-root law being unkind in the usual way, and it gives you the rule: **two or three repeats, then stop**, unless the within-case variation is itself the thing you are measuring.

Repeats never narrow the interval that matters for a comparison, because they add no new inputs. What they do is stop run-to-run noise from masquerading as a difference.

The practical rule follows from what each one buys:

If	Then
The judgement is noisy, or the model is sampled at a real temperature	Repeat the cases and average before you compare
You want to detect a smaller difference	More cases
The interval is wide and nothing else is wrong	More cases
The interval is wide and repeated runs of *the same* prompt disagree	Fix that first, with repeats or a lower temperature



There is a wrong way to spend repeats, and it is the obvious one. Running the same case three times and recording three separate rows triples your apparent case count and narrows your interval, while adding no new inputs at all. Average the repeats *within* the case, so one case is still one row. The interval is computed over cases, because cases are what you drew from the world.

Every Change Is an Experiment

The statistics above are easy. The discipline around them is what people do not do.

Write the prediction down before you run it. Not the hope — the number. *I expect this to move the score from 0.70 to about 0.80.* A prediction you wrote down turns a run into a test of something; a prediction you form afterwards turns every result into a confirmation, because the result is always consistent with some story.

One change at a time, which is Chapter 18's rule and the reason it exists. Two changes and a better score tell you that the pair is better than the pair, which is not something you can build on, and if one of them was a regression you have hidden it.

Keep the runs. A file with the date, what changed, the score, the interval, and the prediction. It costs a line per experiment and it is the only thing that will tell you, in two months, whether the last twenty changes added up to anything — which is a question people ask and almost nobody can answer.

And accept "cannot tell" as a result. It is the honest reading of the paired interval above, and it is more useful than a decision, because it tells you what to do next: get more cases, or go and find a difference big enough to see. What it does not license is keeping B because B *looks* better. You measured, the measurement came back inconclusive, and "it looks better" is the thing the measurement was there to replace.



The cheapest way out of "cannot tell" is often not more cases. What you are really short of is effect against noise, and a verdict that is only right-or-wrong throws away most of what happened — so a measurement aimed at the specific failure you care about can move further between the two versions for the same forty cases. It is the ratio that improves, not the width on its own: a richer measurement can be noisier too. Designing one is chapters 30 and 31, and that is why they come before the rest of the measurement chapters.

Lab: Measure a Difference You Cannot See

Five programs, pure Python, nothing to install. Every number is reproducible from the seed except in step 5, which asks for a shuffle and promises no number. Forty-five minutes, and there is a key for every step.

Step 1. Save the five-runs program as `runs.py` and run it.

Expected result: exactly the output in the chapter, because the seed is fixed: 0.725, 0.750, 0.875, 0.625, 0.775, then a lowest of 0.450, a highest of 0.925, a middle 95 per cent of 0.550 to 0.825, and a standard deviation of 0.073.

Step 2. Change `CASES` to 400, leaving `P` alone. What happens to the standard deviation, and does it match $\sqrt{np(1-p)}/n$?

Expected result: the standard deviation falls to about 0.023, and the formula gives $\sqrt{400 \times 0.7 \times 0.3}/400 = 9.17/400 = 0.023$. Ten times the cases for about a third of the spread — the square root again.

Step 3. Save the bootstrap program as `boot.py` and run it. Then change `trials` to 100, and change `random.seed(20)` to `random.seed(21)` and then to `22`.

Expected result: first `A: 0.700  0.550 to 0.825` and `B: 0.775  0.650 to 0.900`. With 100 resamples A's interval comes out 0.575 to 0.825 at seed 20, 0.525 to 0.825 at 21 and 0.550 to 0.800 at 22 — the endpoints move by a few hundredths. Nothing is averaged at either count: `interval()` sorts and returns one order statistic, the third smallest of 100 or the 251st of 10,000. At 100 a single number moving moves the endpoint; at 10,000 that order statistic sits in a part of the distribution that has been sampled far more densely, so it barely moves. The score in the middle never moves, because it is not resampled. Note what you had to do to see this: the seed is why re-running the program unchanged gives the same answer every time, which is the point of having one.

Step 4. Halve the set. Save the paired program as `paired.py` and give it a twenty-case set of the same shape — `[1]*13 + [1]*1 + [0]*3 + [0]*3` for `A` and `[1]*13 + [0]*1 + [1]*3 + [0]*3` for `B`. What happens to the interval, and what happens to the measured difference?

Expected result: A scores 0.700 and B scores 0.800, so the difference *grew* to +0.100 — and the interval went from 0.250 wide to 0.400 wide, −0.100 to +0.300. A smaller set gave you a bigger apparent improvement and less ability to believe it, which is the trap this whole chapter is about. Four informative cases, three to one.

Step 5. Put `paired.py` back to forty cases and run it. Then break the pairing: shuffle `B` before zipping it with `A`.

Expected result: the paired interval is `-0.050 to +0.200`, and the difference `+0.075` does not change when you shuffle, because a sum does not care about order. The interval does change, and it gets wider — you have thrown away the information about which case each result belongs to. That is the whole value of pairing, visible as a number.

Step 6. Forty cases, and the honest answer is "cannot tell". Without running anything, predict what the interval would do if B had won 8 of the disagreements and A had won 2. Then edit the lists and check.

Expected result: ten informative cases instead of seven, with B taking eight, so the difference doubles to $(8-2)/40 = +0.150$ — and the interval is +0.000 to +0.300. It lands on zero rather than clearing it, which is still not a result — and the landing is that tidy partly because a score on forty cases moves in steps of 1/40, so the 2.5th percentile has few places it can fall. The closed form puts the lower end at +0.002, which clears zero by nothing worth having. Doubling the effect on forty cases was not enough; that is how little forty cases can see. The point to take is that the interval is driven by the disagreements and the total count; the cases the two prompts agree on are ballast, whether they agree on a right answer or a wrong one. The next part of this step measures that.

Keeping the two and the five, move cases between both-right and both-wrong: build `A` as `[1]*20 + [1]*2 + [0]*5 + [0]*13` and `B` as `[1]*20 + [0]*2 + [1]*5 + [0]*13`.

What happens to the interval?

Expected result: **nothing at all.** It is −0.050 to +0.200, the same two numbers, although A has fallen from 0.700 to 0.550 and B from 0.775 to 0.625. Try 15 both-right and 18 both-wrong, or 30 and 3 — still −0.050 to +0.200 every time. That is the chapter's claim turned into a measurement: for this statistic the interval depends on how many cases disagree, which way, and how many cases there are in total. The scores themselves do not enter it.

Step 7. Save the sample-size program as `howmany.py` and run it. Then work out, from the table alone, roughly how many cases you would need to detect a difference of 0.02 of this shape.

Expected result: the printed table, and then an estimate. At 1,000 cases the half-width is $(0.101 - 0.049)/2 = 0.026$, which is already bigger than 0.02, so 1,000 is not enough. The half-width shrinks with the square root, so the count you need is about $1000 \times (0.026/0.02)^2 = 1690$ — call it seventeen hundred. Changing the loop to `for k in (50,):` runs 2,000 cases and gives +0.057 to +0.093, a half-width of 0.018, which is inside 0.02 with a little room. The useful conclusion is not the number. It is that small differences are expensive to prove, and that is a reason to stop chasing them rather than a reason to build a bigger set.

Step 8. You read that a new prompt improved a score from 71 per cent to 78 per cent on an internal evaluation. What do you ask?

Expected result: how many cases, and were both prompts run on the same ones. Then: how many times was each case run, what was the interval on the difference, and who or what decided right or wrong. Seven points on forty cases is inside the noise of a single run; seven points on two thousand paired cases with a mechanical judgement is a result. The number you were given distinguishes the two not at all, which is why it is the wrong number to report on its own.

Step 9. Save the three-sets program as `models.py` and run it. All three sets score about 0.700. Before you look at the numbers, say which one you expect to vary most between runs, and why.

Expected result: 0.074, 0.022 and 0.040. The coin set varies most, because every one of its forty cases is genuinely uncertain; the reliable set varies least, because thirty-seven or thirty-eight of its forty cases come out the same way every time. All three have the same case-sampling spread of about 0.072, which is the distinction this chapter asks you to keep: *which cases you drew* against *what the model did on them*.

Step 10. Append the repeats loop to `models.py` and run it. Then decide, from the output alone, how many repeats you would actually pay for.

Expected result: 0.041, 0.028, 0.023, 0.018, 0.013 at one, two, three, five and ten repeats. The second repeat buys 0.013 for one extra pass; the seven passes from three to ten buy 0.010 between them. Two or three, and spend the rest of the budget on cases — which buy something repeats never do, namely a wider range of inputs. Check the law while you are there: 0.041 divided by $\sqrt{10}$ is 0.013.

Summary, Key Terms, and Review Questions

Summary

A single run is not a score. Forty cases at a true 0.70 have a standard deviation of $\sqrt{np(1-p)}/n = 0.072$, so two of those span about ±0.14, and five seeded runs of one unchanged prompt gave 0.625 and 0.875 among them. **Say which spread you mean**: ±0.14 is always the spread over which forty cases you drew, and it is the run-to-run spread only when the model is as uncertain per case as a 0.70 coin — a system reliably right on 28 and wrong on 12 hits exactly 0.700 in half its runs and stays within one case of it in nine out of ten, at a run-to-run spread of 0.022. A seed makes the book's numbers match yours; nothing makes a provider repeat itself, and temperature zero does not, for Chapter 13's four reasons, which cause the run-to-run kind and cause little of it. So a score needs an interval, and the **bootstrap** gives you one with no formula to derive: resample your cases with replacement, compute your statistic on each resample, and take the 2.5th and 97.5th percentiles. A proportion has a perfectly good closed form; the reason to use the bootstrap is that it is the same three lines for the paired difference, for an average of per-case marks and for a ranking measure. On this data it returns 0.550 to 0.825 for a score of 0.700, which is exactly what ten thousand real runs give — and it had to be, because resampling a list of exactly 28 ones *is* that distribution. It shows the bootstrap recovers the right **width**; the interval is anchored on the score you measured, so it says nothing about being in the right place. **Overlapping intervals are not the test**: compute an interval on the difference, and compute it **paired**, both prompts on the same cases, resampling cases rather than results. That took the width from 0.400 to 0.250 here, about a factor of 1.6 — not by removing the variation from which cases you drew, which is all that 0.250 is, but by cancelling the part of it both prompts share. It still contains zero around +0.075, so the answer is *cannot tell*, and the reason is the last line of the program: 33 cases agree, 7 disagree, five to two. Move cases between both-right and both-wrong and the interval does not move at all. **Cases** buy resolution on the range of inputs and **trials** buy it on the model's own variation; repeats divide a case's own variance by k, so the second is worth more than the seven passes from three to ten — two or three, then stop — and they are averaged *within* a case, never recorded as extra rows. Two hundred cases of this shape is where +0.075 starts to be visible, but a real draw of two hundred clears zero only 72 times in a hundred and four hundred manages 95, which is the difference between a typical interval and a detection rate. Width shrinks with the square root, so each halving costs four times the cases. Write the prediction down first, change one thing, keep a file of the runs, and let **cannot tell** be an answer.

Key Terms

Term	Meaning
Case	One input plus whatever marks the answer right or wrong. The unit the statistics count — and they count rows, so forty rephrasings of one question are worth nearer one case than forty
Bootstrap	Estimating how much a statistic would move by resampling your own cases with replacement. One procedure for every statistic you score, where a closed form is a different formula each time
Confidence interval	The range a measurement would fall in most of the time if you repeated it. Here, the 2.5th to 97.5th percentiles of the resampled scores
Paired comparison	Running both versions on the same cases and keeping the results together, so the part of case-to-case variation the two versions share cancels when you subtract
Trials	Repeated runs of the same case, which measure the model's own variation. Averaged within the case, never counted as extra cases

Worth Memorising

Item	Value or formula
Spread of a score	$\sqrt{np(1-p)}/n$ — one standard deviation over *which cases you drew*. 40 cases at 0.70 is 0.072, so two of them is the ±0.14 that spans 95 per cent. Run-to-run spread is a different number and is smaller unless the model is uncertain on every case
The bootstrap	Resample your cases with replacement, score each, take the 2.5th and 97.5th percentiles
What to put an interval on	The *difference*, paired. Never two separate intervals and a look
Cost of resolution	Width shrinks as $1/\sqrt{n}$, so halving it costs four times the cases
What carries the comparison	Only the cases where the two versions disagree. Count them and say them out loud
What pairing is worth	Here, 0.400 wide down to 0.250. It cancels the variation the two versions share, so the more they agree, the more it buys
Cases against trials	Cases for the range of inputs, trials for the model's variation. Neither fixes the other
How many trials	Averaging k judgements divides a case's own variance by k. Two or three, then stop
Typical interval vs detection	A table built from a fixed mix shows the usual interval. Whether a real draw clears zero is a separate number, and lower

Review Questions

1. How do you decide whether prompt B is better than prompt A?
2. What is a bootstrap interval, and why use one here rather than a named distribution?
3. How many cases and how many trials are enough?
4. Two scores have overlapping confidence intervals. What, if anything, does that tell you?
5. Why does pairing narrow the interval when it adds no data?
6. Your evaluation runs each of 40 cases three times and records 120 rows. What is wrong?

Review Answers

1. Run both on the same fixed set of cases with a mechanical judgement, keep the results paired per case, and compute a bootstrap interval on **B minus A** rather than on each score separately. If that interval contains zero, the honest answer is that you cannot tell, and the next question is whether the set is big enough to see a difference the size you care about. Two things to say unprompted. First, forty cases carry a plus-or-minus of about fourteen points on which cases you drew, so a seven-point gap between two single runs of a forty-case set is nothing — and be precise about which spread that is,

because the run-to-run spread is that large only when the model is genuinely uncertain on most cases, and far smaller when it is reliably right or wrong on each. Second, the comparison rests only on the cases where the two prompts disagree, so count those and say the number: five against two persuades nobody, and it is the same data as "+7.5 per cent".

2. A bootstrap interval is built by drawing a sample of the same size from your own cases, with replacement, computing your statistic on it, repeating that ten thousand times, and taking the 2.5th and 97.5th percentiles. It estimates how much your statistic would move if you had happened to draw a different set of cases of the same kind. Be careful not to overclaim the reason: a proportion does have a perfectly good closed-form interval, and at forty cases it works. The reason to use the bootstrap is that it is **one** procedure for every statistic you will score. The number you actually want an interval on is the paired difference between two versions on the same cases, whose formula needs more than the two scores — it needs how often they disagree, and for right-or-wrong marks it is still a one-liner that agrees with the bootstrap everywhere in this chapter. Move to an average of per-case marks or a ranking measure and it is a different formula again, each one a thing to derive, look up or get subtly wrong. The bootstrap needs no formula, its assumption is one you can state out loud: that your cases are a sample of the inputs you care about. On the honest side, do not oversell the chapter's consistency check. The interval the bootstrap gave from one run of forty cases matched the spread of ten thousand actual runs exactly — but it had to, because the list being resampled held exactly 28 ones out of 40, so resampling it *is* drawing from that same distribution. What it demonstrates is that the method recovers the right width. It cannot tell you the interval is in the right place: a bootstrap interval is anchored on the score you measured, so an unlucky set of cases moves the whole thing.
3. They are different questions. **Cases** buy resolution on the range of inputs: the interval narrows as one over the square root of the count, so each halving costs four times as many, and the honest answer is "enough to see the difference you care about" — for a difference of 0.075 of the shape in this chapter, two hundred is where it starts to show and four hundred is where you could count on it, because a real draw of two hundred clears zero only about three times in four. You find those numbers by computing, not by picking a round one. **Trials** buy resolution on the model's own variation on a single input, which more cases never touch, and they have arithmetic of their own: averaging k judgements of a case divides that case's variance by k, so on this chapter's mixed set the run-to-run spread goes 0.041, 0.028, 0.023 at one, two and three repeats and only reaches 0.013 at ten. The second repeat buys more than the seven passes from three to ten, so two or three and stop, unless the within-case variation is itself what you are measuring. The trap worth naming is recording repeats as extra rows: that narrows the interval while adding no new inputs, which is the appearance of precision rather than precision.
4. On its own, almost nothing. Overlap is not the test, in either direction. The quantity

you care about is the difference, and it has an interval of its own, which is narrower than you would guess from two overlapping intervals when the comparison is paired — and *clearer* than you would guess from two that barely miss each other, because non-overlap is a stricter condition than the one you want, so two intervals that just fail to touch mean a difference comfortably clear of zero. Compute the interval on the difference, and compute it paired. In this chapter's data the intervals overlapped heavily *and* the paired interval on the difference also contained zero — but the first fact was not the evidence for the second.

5. Because it removes a source of variation rather than adding information. Unpaired, each resample draws one set of cases for A and a different set for B, so the difference it computes is charged for the luck of both draws. Paired, each resample draws one set of cases and both prompts face it, so the case difficulty cancels and what is left is how the two prompts differ on the same inputs. On this chapter's forty cases that took the width from 0.400 to 0.250, a factor of 1.6, with no extra runs and no assumption. It works because the two prompts agree on most cases: the more they agree, the more pairing buys.
6. The 120 rows are being counted as 120 cases, and they are not — they are 40 cases run three times. The interval is computed over the things you drew from the world, and you drew forty inputs. Recording them as 120 narrows the interval by about the square root of three without a single new input, which is a precision you have not earned. Average the three runs within each case so one case stays one row; the repeats then do the job they are for, which is reducing the model's own run-to-run variation inside each case rather than pretending to be more cases.

> **You understand this when** somebody tells you a change improved their score by seven points and your first two questions are how many cases and whether both versions saw the same ones.

CHAPTER 21

Structured Output and Schemas

Chapter 2 said that a shape check is what notices when a model's answer is not the answer your code expected. This chapter is how you ask for a shape and how you check it, and it is the shortest distance between a demo and a program.

The problem. The model gives me a paragraph when I need a field my code can read.

The question. How do you get output a program can act on, and what do you do on the occasions when you do not get it?

Why a Program Must Never Trust Prose

Start with the reason, because it is not "prose is untidy".

A paragraph has no contract. When your code reads *The sentiment is positive, with high confidence*, it has to find the word "positive" somewhere in a sentence whose wording nobody promised. That works until the model writes *mostly positive*, or *positive, though the second half is negative*, or *I would call this positive*. Each of those contains the word. Two of them mean something else.

And the deeper reason is Chapter 9's. The model produces one token at a time, each chosen from a distribution over the vocabulary. Nothing in that machinery has a notion of *the format I promised*. A reply that came out as clean JSON ninety-nine times came out that way because clean JSON was the likeliest continuation ninety-nine times, not because anything enforced it. The hundredth reply is not a bug in the model; it is the same unconstrained process arriving somewhere else. Nor does turning the sampling off change it — at temperature zero the token chosen is simply the highest-scoring one, and nothing in *that* enforces a format either. What *can* constrain the choice is a provider's strict mode, which is the next section but one and then Chapter 22.

So the rule, and it is the answer to give when asked:

A program must never trust prose, because prose has no contract — your code would have to guess what the words mean, and a guess that is right ninety-five times in a hun-

dred is a defect five times in a hundred. What you want instead is a format narrow enough that a parser either accepts it or tells you it did not, which converts an unbounded guessing problem into a check with two outcomes.



This is not an argument that the model is unreliable and prose is bad. For a person reading the answer, prose is the right output and a schema would be worse. The rule is about what *code* consumes. Chapter 19 draws the same kind of line for streaming: who the consumer is decides what you can do with a partial reply.

Asking for a Fixed Format

Asking is the easy half, and it is one of Chapter 18's four parts — the output format block. Three things make it work.

Give the shape, not a description of the shape. Put the exact skeleton you want — the field names, the types, a `null` where a value may be absent — in Chapter 18's **output format** block, rather than a sentence about what the fields are. Keep the examples block for a worked input-and-output pair, which is a different job: one shows the contract, the other shows what a good answer looks like inside it.

Say what to do when the answer is not available. A model asked for a `confidence` field will produce one whether or not it has any basis for it. If you want `null`, say `null`; otherwise you have invited an invention and Chapter 16 says why you will get one.

Ask for nothing but the format. *Return only the JSON object, with no explanation and no code fence.* You will still get code fences sometimes, which the last section of this chapter deals with, but you get fewer.



The validator here is hand-written so that you can see what a schema check *is*, and so the lab needs nothing installed. The published standard for writing the same thing declaratively is **JSON Schema** — an object with `"type"`, `"properties"` and `"required"`, which is also the form a provider's strict mode expects, though each engine accepts its own subset of it. In production you declare the schema that way and let a library check it; what you do not do is skip the check because the library is not installed.

And then there is the other way of asking, which is to use a provider's own facility for constraining output to a schema — a strict mode, where the format is enforced during generation rather than requested in the prompt. That changes the odds considerably and it changes them in ways worth understanding, including a case where it makes answers worse. **Chapter 22 is about that**, and everything in this chapter applies whichever way you asked, because none of it trusts the asking.

A Schema Is a Contract Your Code Enforces

Here is a validator. It is about thirty lines, it has no dependencies, and writing one by hand is worth doing once, because it makes plain what a **schema** actually is: a list of claims about a value, tested in order, and enforced by you.

```
import json

SCHEMA = {
    "sentiment":  {"type": str,   "allowed": {"positive", "negative", "neutral"}},
    "confidence": {"type": float, "min": 0.0, "max": 1.0},
    "reason":     {"type": str,   "max_len": 200},
}
```

Three fields, each with the claims you are prepared to enforce on it. And the function that enforces them:

```
def validate(text):
    """Return (ok, why). The first three layers, in order; it stops at the
    first failure. `e.msg` and `e.pos` are worth logging, but their wording
    changes between Python versions, so this returns a stable reason."""
    try:
        data = json.loads(text)
    except json.JSONDecodeError:
        return False, "not JSON"
    if not isinstance(data, dict):
        return False, f"not an object: {type(data).__name__}"
    for field, rule in SCHEMA.items():
        if field not in data:
            return False, f"missing field: {field}"
        v = data[field]
        if rule["type"] is float and type(v) is int:
            v = float(v)
        if type(v) is not rule["type"]:
            return False, f"{field}: wanted {rule['type'].__name__}, got
                ↪ {type(v).__name__}"
        if "allowed" in rule and v not in rule["allowed"]:
            return False, f"{field}: {v!r} not allowed"
        if "min" in rule and not rule["min"] <= v <= rule["max"]:
            return False, f"{field}: {v} outside {rule['min']} to {rule['max']}"
        if "max_len" in rule and len(v) > rule["max_len"]:
            return False, f"{field}: {len(v)} characters, limit {rule['max_len']}"
    extra = sorted(set(data) - set(SCHEMA))
    if extra:
        return False, f"unexpected: {extra}"
    return True, "ok"
```

The use of `type()` rather than `isinstance()` is deliberate, and the place it matters is the coercion line. `type(v) is int` is what decides whether to promote an integer to a float, and in Python `isinstance(True, int)` is true — so an `isinstance` version of that line turns a JSON `true` into 1.0, which then passes the range check and is accepted as a confidence. By default `json.loads` returns only exact built-in types, so `type()` costs nothing here and the one case where the two differ is the one that would hurt. Do not tidy it.

And here are eleven replies of the kind you actually get, in the same file:

```
REPLIES = [
    '{"sentiment": "positive", "confidence": 0.9, "reason": "battery"}',
    '```json\n{"sentiment": "positive", "confidence": 0.9, "reason": "ok"}\n```',
    'Sure! Here is the JSON:\n{"sentiment": "negative", "confidence": 0.8,
        ↪ "reason": "no"}',
    "{'sentiment': 'neutral', 'confidence': 0.5, 'reason': 'mixed'}",
    '{"sentiment": "positive", "confidence": 0.9, "reason": "ok",}',
    '{"sentiment": "Positive", "confidence": 0.9, "reason": "ok"}',
    '{"sentiment": "positive", "confidence": 90, "reason": "ok"}',
    '{"sentiment": "positive", "reason": "ok"}',
    '{"sentiment": "positive", "confidence": 0.9, "reason": "ok", "notes": "hm"}',
    '{"sentiment": "positive", "confidence": 0.9, "reason": "the review',
    '{"sentiment": "Positive", "confidence": 90, "reason": "ok"}',
]

if __name__ == "__main__":
    for i, r in enumerate(REPLIES, 1):
        ok, why = validate(r)
        print(f"  {i:2d}  {'OK ' if ok else 'BAD'}  {why}")
```

```
 1  OK   ok
 2  BAD  not JSON
 3  BAD  not JSON
 4  BAD  not JSON
 5  BAD  not JSON
 6  BAD  sentiment: 'Positive' not allowed
 7  BAD  confidence: 90.0 outside 0.0 to 1.0
 8  BAD  missing field: confidence
 9  BAD  unexpected: ['notes']
10  BAD  not JSON
11  BAD  sentiment: 'Positive' not allowed
```

One of eleven. Ten failures, every one of them something a real model does, in nine distinguishable ways — and the important thing about the list is not how long it is but how *different* its entries are — the eleventh is the exception that makes the point, being replies 6 and 7 at once.

Replies 2 to 5 and 10 never reach the schema at all: they are not JSON. **Notice that all five print the same reason**, because the status *not JSON* is all a parser gives you about the category, while the five causes are completely different: a code fence, prose around the object, single quotes, a trailing comma, and a reply that stopped mid-string. This is Chapter 19's point about reading the error's body and not only its status, arriving in a new place. `json.loads` raises a `JSONDecodeError` carrying `e.msg` and `e.pos`, and those are what you log.



The validator above deliberately does not *return* `e.msg`. The wording of a `JSONDecodeError` is the interpreter's, and it changes between Python versions — the trailing comma in reply 5 is reported differently on 3.12 and on 3.13 — so a message built from it would make this chapter's printed output depend on which Python you have. Log it; do not branch on it, and do not show it to a user.

Reply 10 is the one to look at twice, because an unterminated string is usually not a formatting mistake — it is a reply that hit the token cap, which Chapter 13 calls a guillotine rather than a budget. Your validator reports *not JSON* and is telling you the truth while burying the cause, so read the response's own **finish reason** as well.

Replies 6 to 9 and 11 are JSON, and they are the ones worth the chapter. `"Positive"` is valid JSON, the right type, in the right field, and not a value your code can use. `90` is a plausible confidence on a scale nobody specified. Reply 8 simply left a field out. Reply 9 added one. And reply 11 has *both* of reply 6's and reply 7's faults, which is why it reports only the first: a validator that stops at the first failure tells you the first thing wrong, not every thing wrong.



Reply 9 is a policy decision disguised as a failure, and you have to make it deliberately. Rejecting unexpected fields catches a model that is drifting away from your format, and it breaks the day you add a field and an old consumer sees it. Accepting them lets you add fields freely, and it lets an invented `"confidence_explanation"` through unnoticed for a year. Pick one, write down why, and apply it everywhere.

The Four Layers, and Which Two You Get for Free

The validator above checks a stack of claims, and it is worth naming the stack — the **validation layers** — because the top of it is not a validation problem at all.

Layer	Question	Who answers it
1	Does it parse?	`json.loads`
2	Is it the right shape?	Your schema
3	Are the values possible?	Your schema, if you wrote the ranges
4	Are the values *true*?	Nothing on this page

Layers 1 and 2 are mechanical and you should never skip them. Layer 3 is mechanical

too, and it is the one people leave out: a `confidence` field with no range, a `quantity` with no upper bound, a date with no sanity check. Reply 7 passes layers 1 and 2 and fails 3, and it took one line of schema to catch.

Layer 4 is a different activity. Take a reply that calls a review positive with a confidence of 0.95 and a reason of `"praises the battery"`, about a review whose whole complaint is the battery. It is perfectly valid and completely wrong. No schema can see that, and a stricter schema will not help. Chapters 30 and 31 are where measuring *correctness* lives, and Chapter 31 in particular is where a judgement of content gets built and checked.

So say this part out loud when asked, because it is the distinction that separates somebody who has run this in production from somebody who has read about it: **a schema check tells you the answer is shaped like an answer. It does not tell you the answer is right, and the two failures need completely different machinery.**

Measuring How Often It Fails

You cannot decide what to do about validation failures until you know the rate, and the rate is a proportion — which Chapter 20 has already given you the tools for. Its bootstrap works here unchanged.

```
import random

def interval(results, trials=10_000):
    """Chapter 20's bootstrap, on a failure rate."""
    n = len(results)
    rates = []
    for _ in range(trials):
        s = [results[random.randrange(n)] for _ in range(n)]
        rates.append(sum(s) / n)
    rates.sort()
    return rates[int(0.025 * trials)], rates[int(0.975 * trials)]

TRUE_RATE = 0.02        # 1 reply in 50 fails validation

random.seed(21)
for n in (40, 200, 2_000):
    failures = [1 if random.random() < TRUE_RATE else 0 for _ in range(n)]
    seen = sum(failures)
    lo, hi = interval(failures)
    print(f"  {n:5d} replies: {seen:3d} failed = {seen/n:.3f}"
          f"   interval {lo:.3f} to {hi:.3f}")

print("\n  and when you have seen no failure at all:")
```

```
lo, hi = interval([0] * 40)
print(f"    40 replies, 0 failed: bootstrap says {lo:.3f} to {hi:.3f}")
for n in (40, 200, 2_000):
    print(f"    0 failures in {n:5d}  ->  rule of three: up to {3/n:.3f}")
```

```
     40 replies:   1 failed = 0.025   interval 0.000 to 0.075
    200 replies:   5 failed = 0.025   interval 0.005 to 0.050
   2000 replies:  31 failed = 0.015   interval 0.011 to 0.021

  and when you have seen no failure at all:
    40 replies, 0 failed: bootstrap says 0.000 to 0.000
    0 failures in    40  ->  rule of three: up to 0.075
    0 failures in   200  ->  rule of three: up to 0.015
    0 failures in  2000  ->  rule of three: up to 0.002
```

Forty replies tell you almost nothing. One failure in forty reads as two and a half per cent, and the interval runs from zero to seven and a half. The lower end is an artefact rather than a finding: you *saw* a failure, so the rate is certainly not zero, and the bootstrap only reaches zero because thirty-six per cent of resamples of a list with one `1` in it happen to contain no `1` at all. What forty replies cannot tell you is whether the rate is one in four hundred or one in thirteen, and those two need different plans. Two thousand replies put it between 1.1 and 2.1 per cent, which is a number you can plan around.

And now the part Chapter 20 did not need, because this chapter has rare events and that one did not. Look at the zero-failure line. The bootstrap returns 0.000 to 0.000, and it is not being modest — it is resampling a list of forty zeros, and no amount of resampling a list with no failures in it will ever produce one. **The bootstrap cannot see a failure it has never observed.** Quoting that interval would be the most confident possible way of being wrong.

What you use instead is one line of arithmetic. If the true rate were p, the chance of seeing no failure in n tries is $(1 - p)^n$, and that is above five per cent — so still quite possible — for any p up to

$$1 - 0.05^{1/n} \;=\; 1 - 0.05^{1/40} \;=\; 0.072$$

which is where the **rule of three** comes from: $\ln 20 = 2.996$, so the bound is about $3/n$, and $3/40 = 0.075$ against the exact 0.072. Clean runs on forty replies are consistent with one reply in thirteen failing — fourteen on the exact bound, thirteen on the rule of three, and the rule of three is the one you will quote.

Do not read the two 0.075s as the same number. The bootstrap's upper end for *one* failure in forty is 0.075, and the rule of three for *zero* failures in forty is also 0.075, and they are different quantities that happen to coincide at this sample size. A rate on forty replies moves

in steps of 1/40, and for one failure in forty the 97.5th percentile lands on the third of those steps — $P(X \leq 2) = 0.922$ and $P(X \leq 3) = 0.983$, so 3/40 is where it has to fall, whatever the seed. Three fortieths is also what the rule of three gives. Change the forty and they part company.



This is the arithmetic to reach for whenever somebody says a thing has never happened. It converts "we have never seen it" into a number, and the number is usually larger than the person expects. If you need to be sure a failure rate is under one in a thousand, you need about three thousand clean runs, and there is no way to talk your way out of it.

What to Do With a Failure

Four options, and they are in order of how much you should like them.

Reject. The reply failed, so you do not use it. For anything where a wrong answer is worse than no answer, this is the right default and the rest of this section is a distraction.

Retry — and know what your temperature setting does to that. Above temperature zero the model samples, so a second attempt is a genuinely different draw and a plain retry often works. **At temperature zero it is not.** Greedy decoding takes the highest-scoring token every time, so a retry is no longer a fresh draw from a distribution and retrying unchanged is close to paying twice for one answer. Close to, not exactly: Chapter 13 is clear that temperature zero is not a guarantee either, for reasons that have nothing to do with sampling. What you cannot count on is the retry *helping*. Structured extraction is usually run at temperature zero, which makes this the common case rather than the exception, so **change something in the retry**: send the validator's own message back, because *missing field: confidence* is a far more useful instruction than *that was wrong*. Cap the attempts and count them either way, or you have built an expensive loop that hides a broken prompt.

Repair, but only the kinds that cannot change what the reply means. Three edits qualify:

```
import re
from check import REPLIES, validate

def repair(text):
    """Three edits that cannot change what the JSON means."""
    t = text.strip()
    t = re.sub(r"^```[a-zA-Z]*\n|\n```$", "", t)      # a fenced code block
    i, j = t.find("{"), t.rfind("}")                   # prose either side
    if i != -1 and j > i:
        t = t[i:j+1]
    t = re.sub(r",(\s*[}\]])", r"\1", t)               # a trailing comma
    return t

before = after = 0
for n, r in enumerate(REPLIES, 1):
    ok0, _ = validate(r)
    ok1, why = validate(repair(r))
    before += ok0
    after += ok1
    mark = "ok" if ok1 else "still bad"
    moved = " <- repaired" if ok1 and not ok0 else ""
    print(f"  {n:2d}  {mark:9s} {why if not ok1 else ''}{moved}")
print(f"\n  {before}/{len(REPLIES)} valid as sent,"
      f" {after}/{len(REPLIES)} after repair")
```

```
   1  ok
   2  ok         <- repaired
   3  ok         <- repaired
   4  still bad not JSON
   5  ok         <- repaired
   6  still bad sentiment: 'Positive' not allowed
   7  still bad confidence: 90.0 outside 0.0 to 1.0
   8  still bad missing field: confidence
   9  still bad unexpected: ['notes']
  10  still bad not JSON
  11  still bad sentiment: 'Positive' not allowed

  1/11 valid as sent, 4/11 after repair
```

One in eleven becomes four in eleven, and every one of those three edits removes something that was never part of the data: a code fence, text outside the braces, a comma JSON does not allow. None of them touches a value.

Two of the three are doing the work. Disable the fence rule and the count is still four,

because the brace slice already throws away the fence along with everything else outside the object. The fence rule earns its place only when a reply fences something that is not an object — a bare array, or an object with a comment after the closing brace — and it costs one line, so keep it. But do not count edits and assume each one bought you something; disable them one at a time and look.

One more thing to notice before the next part. Reply 7's message says `90.0` where the model actually sent `90`. That is the int-to-float coercion showing in the output, and if you log the validator's message beside the raw reply, somebody will one day compare the two and wonder which is wrong. Log the raw text as well as the reason.

Now look at what is left, because the line between repair and invention runs straight through it. Reply 4 uses single quotes, and `ast.literal_eval` would parse it happily — which is why you should not, because you would have quietly accepted Python literals from a model asked for JSON, and a Python literal can contain things JSON cannot. Reply 6 could be lower-cased. Reply 7 could be divided by a hundred. Both of those are **guesses about what the model meant**, and both will be right most of the time, which is exactly what makes them dangerous: you will not find out about the time `90` meant ninety.

So the line is: **repair the envelope, never the contents.** If the fix requires you to know what the model intended, it is not a repair, and the honest version of it is a retry or a rejection.

Record. Whatever you do with the reply, keep the failure: the raw text, the validator's message, and the input that produced it. That file is how you find out that eighty per cent of your failures are one prompt and one field, which is a prompt you can fix — and Chapter 30 is the discipline of actually reading them.



The failure you must never choose is the fifth one, which is to catch the exception, log a line nobody reads, and carry on with a default value. That turns a loud failure into a wrong answer with no fingerprints, and the field it silently sets to `"neutral"` will be in a report somebody makes a decision from.

Lab: Nine Ways to Fail, and What to Do About Them

Three programs, pure Python, nothing to install, every number reproducible from the seed. Forty minutes, and there is a key for every step. **Work in a new empty folder**, because `repair.py` imports from `check.py` and a stale copy of either will quietly change your answers.

Step 1. Save the schema, the validator and the eleven replies as `check.py` and run it.

Expected result: exactly the eleven lines in the chapter — one `OK` and ten `BAD`. The reasons are stable across Python versions because the validator does not pass the interpreter's own parse message through.

Step 2. Reply 7 is `"confidence": 90`. Delete the `"min"` and `"max"` from the schema's

`confidence` rule and run it again. What happens, and which layer did you just remove?

Expected result: reply 7 now passes, so the count goes to two of eleven. You removed layer 3, the check that the values are *possible*, and the result is a program that will cheerfully act on a confidence of ninety. This is the layer people leave out, and it costs one line to put back. Reply 11 still fails, because it meets the enum check first.

Step 3. Now delete the `type` check as well, and run it again. Then look at reply 11 and say what the validator has not told you.

Expected result: reply 7 still passes and nothing else changes, because every other reply fails earlier — at the parse, at a missing field, at the enum or at the extra field. Reply 11 is the point of the step: it has `"Positive"` *and* a confidence of 90, and the validator reports only the enum, because it returns at the first failure. So a clean second run after a fix is not evidence the reply was nearly right; it is evidence you have reached the next problem. If you want all of them at once, collect the failures into a list instead of returning — which is worth doing for a report and not worth doing in the request path, where the first failure is already enough to reject on.

Step 4. Put `check.py` back the way it was first — both the `type` check and the `min` and `max` you deleted in steps 2 and 3 — because `repair.py` imports from it and will report a different answer against a weakened schema. Then save the repair program as `repair.py` in the same folder and run it.

Expected result: the chapter's eleven lines and `1/11 valid as sent, 4/11 after repair`. If you get `2/11` and `5/11`, you did not restore the schema and reply 7 is passing a range check that is no longer there. If you get an import error, `check.py` must be in the same directory; if you see check.py's own eleven lines first, its demo loop is not under `if __name__ == "__main__":`.

Step 5. Add a fourth repair that lower-cases the `sentiment` value, and run it again. How many pass now, and what did you just agree to?

Expected result: five of eleven — reply 6 joins them, and reply 11 does not, because lower-casing its sentiment only moves it on to its second fault. What you agreed to is that `"Positive"` and `"positive"` mean the same thing, which they almost certainly do. But you have moved from repairing the envelope to editing the contents, and the next request of this kind will be to map `"very positive"` onto `"positive"`, which is a different claim entirely. If you want this, put it in the schema as a case-insensitive enum rather than in the repair step, so it reads as a decision about the contract instead of a patch.

Step 6. Save the rate program as `rate.py` and run it. Then explain why the forty-reply interval contains zero when you know the true rate is 0.02.

Expected result: the chapter's figures. The interval contains zero because one failure in forty is also what you would often see if the rate were very small, and the bootstrap is resampling a list with a single 1 in it — plenty of resamples contain no 1 at all. Knowing the true rate is a luxury of the simulation; the interval is what you would have without it.

Step 7. Set `TRUE_RATE` to 0.0 and run it. What does the bootstrap report for all three sample sizes, and what should you report instead?

Expected result: 0.000 to 0.000 at every size, because there is no failure in any of the lists to resample. You should report the rule-of-three bound for the size you ran: up to 0.075 at forty, 0.015 at two hundred, 0.002 at two thousand. "We saw no failures" and "the failure rate is zero" are different sentences and only the first is true.

Step 8. You need confidence that a format failure happens less than once in five hundred replies. How many clean runs does that take, and what do you do if you cannot afford them?

Expected result: the rule of three inverted — $3/n = 1/500$ gives $n = 1500$ clean runs. If you cannot afford fifteen hundred, the answer is not to run fewer and claim the same thing; it is to stop needing the guarantee. Make the failure cheap instead: reject and retry, cap the attempts, record every failure, and alarm on the rate. A system that handles a one-in-fifty failure gracefully does not need to prove it is one in five hundred.

Step 9. A colleague's extraction pipeline parses the model's reply with a regular expression that pulls out the first number it finds, and it has worked for three months. Name three ways it fails, and what you would replace it with.

Expected result: it finds a number in a date, a citation, or a quantity in the input that the model repeated; it finds the right number in the wrong units because nothing fixed the units; and it silently picks up the first of two numbers when the model starts answering in a sentence — and because every one of those produces *a number*, nothing ever raises. The replacement is a field with a name, a type, and a range, in a schema, validated on every reply, with failures recorded. Three months of working is not evidence of a contract; it is evidence that the likeliest continuation has been stable.

Summary, Key Terms, and Review Questions

Summary

A program must never trust prose, because prose has no contract: your code would have to guess what the words mean, and a guess that is right ninety-five times in a hundred is a defect five times in a hundred. The reason is Chapter 9's — a reply that was clean JSON ninety-nine times was clean because that was the likeliest continuation, not because anything enforced it. So ask for a fixed format: show the exact shape rather than describing it, say what to do when a value is unavailable, and ask for nothing but the format. A provider's **strict mode** changes the odds and Chapter 22 is about it; nothing in this chapter depends on how you asked. **A schema is a contract your code enforces**, and a hand-written validator makes plain what it is: a list of claims tested in order. Of eleven realistic replies, one passed. Five were not JSON at all, and **all five report the same reason**, because a parse status names the category and not the cause — one of them was an unterminated string, which usually

means the token cap rather than a formatting slip, so read the finish reason too. The other five were valid JSON and unusable: `"Positive"` for an enum, `90` for a confidence, a missing field, an extra one, and one reply with two faults that reports only the first. Validation has **four layers** — it parses, it is the right shape, the values are possible, the values are true — and the first three are mechanical while the fourth is a different activity belonging to chapters 30 and 31. **A schema check tells you the answer is shaped like an answer; it does not tell you the answer is right.** Measure the failure rate with Chapter 20's bootstrap: one failure in forty gives an interval of 0.000 to 0.075, whose lower end is an artefact of resampling a single `1` rather than evidence that the rate might be zero, while two thousand replies place it between 0.011 and 0.021. But **the bootstrap cannot see a failure it has never observed** — on forty clean replies it reports 0.000 to 0.000 — so for a rare event use **the rule of three**: zero failures in n tries leaves the rate as high as $3/n$, which at forty is one reply in thirteen. Then handle the failures: **reject** by default, **retry** with the validator's own message fed back, **repair the envelope and never the contents** (three meaning-preserving edits took one of eleven to four of eleven, and only two of the three did any of the work), and **record** every failure with its input. The option never to choose is the silent default.

Key Terms

Term	Meaning
Schema	The list of claims your code makes about a reply — fields, types, allowed values, ranges — and enforces on every one. A contract you check, not a request you hope was honoured
Validation layers	Parses / right shape / values possible / values true. The first three are mechanical; the fourth needs a judgement and belongs elsewhere
Rule of three	With no failures in n tries, the rate can still be about $3/n$. What to use when a bootstrap has no failure to resample
Repair	Editing a reply so it parses, restricted to things that were never part of the data: a code fence, text outside the braces, a comma JSON disallows
Finish reason	The response's own statement of why generation stopped. What distinguishes a truncated reply from a malformed one

Worth Memorising

Item	Value or formula
Why not prose	No contract, so your code guesses. Right 95 times in 100 is a defect 5 times in 100
The four layers	Parses · right shape · values possible · values true. Only the first three are free
Rule of three	0 failures in n → the rate may be $3/n$. Exactly, $1 - 0.05^{1/n}$
0 in 40	Consistent with a 7.5 per cent failure rate
The repair line	Repair the envelope, never the contents. Needing to know what the model meant makes it a retry
Failure options	Reject · retry · repair · record. Never a silent default

Review Questions

1. Why must a program never trust prose?
2. You are asking for JSON. How do you measure how often you do not get it, and what do you do with the failures?
3. Your validator has passed every reply for a month. What can you conclude?
4. A reply is valid JSON, matches your schema, and is wrong. What caught it, and what should have?
5. When is repairing a reply acceptable, and when is it not?

Review Answers

1. Because prose has no contract. To act on a paragraph your code has to locate meaning inside wording that nobody promised — it finds "positive" in *mostly positive* and in *positive, though the second half is negative*, and two of those mean something else. Underneath that is the mechanism from Chapter 9: the model emits one token at a time from a distribution, and nothing in it has a notion of the format you asked for. A reply that arrived as clean JSON ninety-nine times did so because that was the likeliest continuation, not because anything enforced it, so the hundredth is the same unconstrained process arriving somewhere else rather than a malfunction, and that holds at temperature zero too, where the token taken is just the highest-scoring one. A fixed format replaces the guess with a parser that either accepts the reply or tells you it did not. And the limit is worth volunteering: this is a rule about what code consumes. For a person reading the answer, prose is the right output.
2. Measure it as a proportion, with an interval, because the raw rate is misleading at small numbers: one failure in forty looks like two and a half per cent and the bootstrap interval runs from 0.000 to 0.075, containing zero, while two thousand replies place the rate between 0.011 and 0.021. The thing to add, because it is where people go wrong, is that

a bootstrap cannot see a failure it has never observed — forty clean replies give an interval of 0.000 to 0.000, which is the most confident possible way of being wrong — so for a run with no failures you use the rule of three: the rate may still be about $3/n$, one reply in thirteen at forty tries. For the failures themselves there are four options in order of preference: **reject**, which is right whenever a wrong answer is worse than no answer; **retry**, which works unusually well above temperature zero, where sampling makes the second attempt a genuinely different draw, and is worth nothing at temperature zero unless you change something — so feed the validator's own message back; **repair**, strictly limited to edits that cannot change meaning — a code fence, prose outside the braces, a trailing comma, which together took one of eleven replies to four of eleven; and **record**, always, with the raw text and the input, because the failures cluster and the clusters are fixable. The fifth option, catching the error and substituting a default, is the one to name as unacceptable.

3. That the rate is low, not that it is zero, and you can put a number on it. If the month was forty replies, the rule of three says the rate could still be seven and a half per cent — one in thirteen. If it was two thousand, you are entitled to about one in six hundred and seventy. The bootstrap is no help here and will tell you the rate is exactly zero, because it has no failure to resample. The second thing to conclude is narrower than it sounds: a validator that has never fired has also never been tested, so you do not know it works. Feed it a deliberately bad reply and watch it fail before you trust its silence.
4. Your schema caught nothing, because there was nothing for it to catch: layers one, two and three were all satisfied. What should have caught it is a measurement of *correctness*, which is a different activity from validation and needs different machinery — a check or a judge over a set of cases with known answers, built and validated as chapters 30 and 31 describe. The useful way to say this is that a schema check tells you the answer is shaped like an answer. Tightening the schema will not help, and the instinct to tighten it after a wrong-but-valid answer is the mistake worth naming.
5. Acceptable when the edit removes something that was never part of the data and could not change what the reply means: a surrounding code fence, text outside the braces, a comma JSON does not permit. Not acceptable when the edit requires you to know what the model intended — lower-casing an enum value, dividing a number by a hundred because you think it was a percentage, or parsing single-quoted output with `ast.literal_eval`, which quietly accepts Python literals from a model you asked for JSON. Those are guesses, they are right most of the time, and being right most of the time is what hides the case where `90` meant ninety. The rule is to repair the envelope and never the contents; if you want case-insensitive values, that belongs in the schema as part of the contract, not in a patch applied after the fact.

> **You understand this when** you can say what a schema check does *not* tell you, and put a number on a failure you have never seen.

CHAPTER 22

Constrained Decoding and Strict Modes

Chapter 21 built a validator and never once trusted the model to honour a format. This chapter is about the other end of the problem: what happens when the provider enforces the format in a **strict mode**, how that is done, and why it is not the end of the matter.

The problem. One provider guarantees my format and another ignores it, and on one task the strict mode made the answers worse.

The question. How is a format constraint actually enforced while the model generates, and what does enforcing it cost?

What a Constraint Actually Does

Chapter 13 left the model at the moment of choosing: a score for every token in the vocabulary — its *logit*, Chapter 13's word — a softmax to turn those scores into probabilities, and a rule for picking one. **A constraint goes in between.** Before the pick, something crosses out every token the format does not allow, and the pick happens among what is left.

That is **constrained decoding**, and the implementation is one line.

Here is a working decoder. The model is a stand-in — the score of the next token depends only on the last one, and the numbers are written out rather than computed, so every figure in this chapter can be checked by hand.

```
import math

VOCAB = ["Sure", "!", "{", "}", ":", '"sentiment"', '"positive"', '"negative"']

# A stand-in model. The score of the next token depends only on the last one,
# and the numbers are written out rather than computed, so every figure below
# can be checked by hand. Anything not listed scores 0.0.
```

```
NEXT = {
    None:          {"Sure": 6.0, "{": 4.0, '"sentiment"': 1.0},
    "Sure":        {"!": 5.0, "{": 3.0},
    "!":           {"{": 4.0, "Sure": 0.5},
    "{":           {'"sentiment"': 5.0, "}": 1.5},
    '"sentiment"': {":": 6.0, "}": 1.0},
    ":":           {'"positive"': 4.0, '"negative"': 3.0, "Sure": 2.0},
    '"positive"':  {"}": 5.0, ":": 1.0},
    '"negative"':  {"}": 5.0},
    "}":           {},
}

def scores(last):
    row = NEXT[last]
    return [row.get(t, 0.0) for t in VOCAB]

def softmax(xs):
    m = max(xs)
    e = [math.exp(x - m) for x in xs]
    total = sum(e)
    return [v / total for v in e]
```

The format, written as a *grammar* — a set of rules saying which characters may follow which — compiles into a **state machine**: in each state, the set of tokens that may come next. Engines call the same object an automaton; this chapter says state machine throughout.

```
# The grammar, as a state machine: which tokens may come next in each state.
ALLOWED = [
    {"{"},
    {'"sentiment"'},
    {":"},
    {'"positive"', '"negative"'},
    {"}"},
]

def generate(constrained, steps=5):
    out, last, state = [], None, 0
    for _ in range(steps):
        s = scores(last)
        if constrained:
            s = [v if VOCAB[i] in ALLOWED[state] else -math.inf
                 for i, v in enumerate(s)]
            state += 1
        token = VOCAB[max(range(len(VOCAB)), key=lambda i: s[i])]
        out.append(token)
        last = token
```

```
    return out

if __name__ == "__main__":
    print("  free:        " + " ".join(generate(False, 7)))
    print("  constrained: " + "".join(generate(True)))
```

```
  free:        Sure ! { "sentiment" : "positive" }
  constrained: {"sentiment":"positive"}
```

The free run is the reply from Chapter 21's list — a polite preamble and then perfectly good JSON, which does not parse, because the preamble is part of the string. It is printed token by token above; the string itself is `Sure!{"sentiment":"positive"}`, and `json.loads` rejects it at character 0. The constrained run cannot produce the preamble, because at the first step the only token with a score left is `{`.

The line that does the work is the one that sets a score to negative infinity. That is **masking**: a token with a score of $-\infty$ gets $e^{-\infty} = 0$ from the softmax, so — as long as at least one token survives — its probability is exactly zero and no decoding rule can choose it. That condition is not pedantry: mask every token and this softmax returns `nan` rather than zero, because it shifts by the maximum and $e^{-\infty-(-\infty)}$ is undefined. The lab builds that state on purpose. Nothing is bolted onto the model. The model computes exactly what it would have computed; the constraint edits the numbers on their way out.



A real engine does not carry a list of allowed token strings per state. It compiles your schema into a state machine over characters and then, for each state, works out which of the vocabulary's tokens are legal — which is where the interesting difficulties live, and the section after next is about them.

The Mask, Step by Step

The mechanism is simple. What it costs is the thing worth measuring, and you can measure it: the **masked mass** at each step, which is how much probability the mask threw away.

```
import math
from decode import VOCAB, ALLOWED, scores, softmax

last, state = None, 0
print("  step  free pick   p     allowed pick  p      mass masked")
for step in range(1, 6):
    s = scores(last)
    p = softmax(s)
    free = max(range(len(VOCAB)), key=lambda i: s[i])
    keep = [i for i in range(len(VOCAB)) if VOCAB[i] in ALLOWED[state]]
    masked = 1.0 - sum(p[i] for i in keep)
    best = max(keep, key=lambda i: s[i])
    q = softmax([s[i] if i in keep else -math.inf for i in range(len(VOCAB))])
    print(f"  {step:^4}  {VOCAB[free]:<11} {p[free]:.3f} "
          f"{VOCAB[best]:<13} {q[best]:.3f}  {masked:.3f}")
    last, state = VOCAB[best], state + 1
```

```
  step  free pick   p     allowed pick  p      mass masked
   1    Sure        0.866 {             1.000  0.883
   2    "sentiment" 0.934 "sentiment"   1.000  0.066
   3    :           0.979 :             1.000  0.021
   4    "positive"  0.627 "positive"    0.731  0.142
   5    }           0.945 }             1.000  0.055
```

Look at step 1 and then at steps 2, 3 and 5.

At step 1 the model wanted to write *Sure* with a probability of 0.866, and the grammar allowed only `{`. The mask deleted 88.3 per cent of the probability mass — you can check it by hand, as $1 - 0.117 = 0.883$ — and the single survivor was renormalised to 1.000. The constraint did not nudge the model. It overruled it.

At steps 2, 3 and 5 the mask removed almost nothing: 6.6, 2.1 and 5.5 per cent. The model was already going where the grammar wanted, so the constraint was free.

Most of the time a constraint costs nothing, because a model asked for JSON mostly produces JSON — which is a practitioner's observation rather than a measurement, so hold it loosely. **And the mask deletes whatever the model was going to emit — and whether that costs you anything depends entirely on what that was.** Chapter 21 drew the same line for repairs: the envelope or the contents. Step 1 deleted the envelope. *Sure* is a preamble, deleting it is the whole purpose of the feature, and the 88.3 per cent is therefore the chapter's largest number and its cheapest deletion. Nothing was lost.

Step 4 is where a cost could have arisen, and did not. The grammar allowed two values, so the mask removed 14.2 per cent — 8.5 of it the probability on *Sure*, and the remaining 5.7 spread over the five tokens the stand-in model scores at zero, which is worth noticing on its own: a mask deletes the floor as well as the rival. What it left was a genuine choice between

`"positive"` and `"negative"`, decided by the model, raising the winner from 0.627 to 0.731.

A constraint that leaves a real choice is doing what you asked. A constraint that leaves one option is making the decision itself — and that is one of the two ways a strict format can cost you something, which lab step 3 makes you do deliberately. Set state 3 to allow only `"negative"` and the output becomes `{"sentiment":"negative"}`, against a model that scored `"positive"` at 4.0 and `"negative"` at 3.0. You did not constrain the format. You picked the answer. Notice what this run does *not* show: nowhere in it does the mask delete anything the model needed. For that you need a schema that forbids the working, and the section after next is about it.



If you can get at the logprobs, the quantity in that last column is worth logging on a sample of calls — not because a large number is bad in itself, as step 1 shows, but because *where* it is large tells you something. Large on the envelope means the model keeps trying to write a preamble, which is a prompt problem showing up as a decoding statistic. Large on a value slot means the model wanted to say something your schema does not allow, which is the case worth reading the examples for.

Why the Support Varies Between Engines

Now the part that answers the first half of the opening problem, and it is not vendor capriciousness. It is a genuinely awkward computation.

Your schema is written over characters. Generation happens over tokens. JSON's syntax is defined in characters — a `{`, then a `"`, then field characters. The model does not emit characters; it emits tokens from Chapter 10's vocabulary, and a token can carry several characters at once. So "which tokens are legal here" is not a question the schema answers directly, and the cheap way of answering it is wrong.

```
# The grammar is written over characters. Generation happens over tokens,
# and a real vocabulary has tokens that span more than one character.
VOCAB = ["{", "}", '"', "{}", "}}", '":', '"}', '{"s']

NEXT_CHARS = '{"'      # the characters the grammar will accept next, in order

def first_char_ok(tok):
    """The cheap check: does the token start with a character we allow?"""
    return NEXT_CHARS.startswith(tok[0])

def every_char_ok(tok):
    """Correct, as far as this window reaches: every character the token
    contributes must fit. A token that runs past the window is reported
    illegal, which is the limitation step 5 asks you to find."""
    return NEXT_CHARS.startswith(tok)
```

```
print(f"  the grammar will accept {NEXT_CHARS!r} next")
print(f"  {'token':8} {'first char':>11} {'every char':>11}")
for tok in VOCAB:
    print(f"  {tok!r:8} {str(first_char_ok(tok)):>11}
        ↪ {str(every_char_ok(tok)):>11}")
```

```
  the grammar will accept '{"' next
  token     first char  every char
  '{'             True        True
  '}'            False       False
  '"'            False       False
  '{}'            True       False
  '}}'           False       False
  '":'           False       False
  '"}'           False       False
  '{"s'           True       False
```

Two rows matter, and they are wrong in opposite directions.

`{}` begins with a character the grammar wants and then emits one it does not — it closes the object before the field has been written. The cheap first-character check admits it. **So checking the first character is too loose**, and it is *almost* right, which is exactly why it ships.

`{"s` is the other failure. In real JSON it is perfectly legal: `{`, then `"`, then the first letter of a field name. The window check calls it illegal, because the window is only two characters long and the token runs past the end of it. **So checking against a short window is too strict**, and a decoder built that way masks out legal tokens and can leave itself with none — the dead state the lab builds in step 4.

So neither shortcut works, and the real check runs the *whole* grammar over every character a candidate token would contribute, for every token in the vocabulary, in every state — asking not "does this fit what comes next" but "does this leave the grammar satisfiable". With a vocabulary of Chapter 9's size that is not something you do per step, so engines precompute it: for each state of the state machine, the set of legal tokens, built once and cached — in practice lazily, as each state is first reached, and with the cache bounded.

And that precomputation is where the feature differences come from. A schema feature that keeps the state machine small and deterministic is cheap to support. One that makes it large, or that cannot be decided without looking far ahead, is expensive or impossible:

Schema feature	Why it is easy or hard
A fixed set of field names, in order	A short, deterministic state machine
An enum of string values	A small branching table of the allowed strings; the easy case
`anyOf` between similar shapes	Branches that stay open for many tokens
A regular-expression `pattern`	Fine if it compiles to a small state machine, awful if it does not
`minLength` or `maxLength`	Needs a counter, and tokens advance it by varying amounts
Recursion, or unbounded nesting	The state is a stack rather than a number

So when one provider guarantees your schema and another ignores half of it, the difference is usually which features survived that compilation — not how much either of them cares. Read what the engine says it accepts, and assume the parts it is silent about are not enforced.



Those feature names and which engine supports which are the most perishable facts in this chapter, and the table above is about the *reason*, not a product comparison. The reason does not date. Check the current support yourself, and keep Chapter 21's validator either way, because a feature silently not enforced looks exactly like a feature enforced and satisfied.

When a Strict Format Makes the Answer Worse

This is the second half of the opening problem and the better interview question, because the honest answer has two parts that are at very different levels of confidence.

The mechanism is a derivation, and you have already seen it. The mask deletes probability mass, and the mass it deletes is what the model was going to say. Now put that next to Chapter 16's finding: the model's only scratchpad is its own output, so emitting intermediate tokens is how later tokens get to condition on partial work. A schema that permits only `{"answer": …}` forbids the intermediate tokens *entirely*. The model cannot work through the problem first, because there is nowhere to work: every token it emits must be part of the final object.

Be exact about what that establishes. It is a derivation in Chapter 6's sense, and what it derives is that **the scratchpad is gone** — which follows from what the grammar admits plus Chapter 16's architectural fact, and the reader can check both. It does *not* derive that the answers get worse. That needs a third premise the chapter cannot supply: that writing the working out actually improves the answer on this task. A model may compress the same computation into one forward pass and be right anyway. So the capability claim is off the ladder and the accuracy claim is not, and it is the capability claim that tells you where to look — tasks that need working out rather than looking up.

The size of the effect is a different matter, and it is much weaker evidence. Put it on

Chapter 6's ladder honestly: that strict formats cost accuracy on reasoning-heavy tasks has been *demonstrated* by people showing before-and-after examples, and *studied* in particular settings with particular schemas and models. It is not settled practice, and the tell is Chapter 6's own: nobody has written down where it stops holding. Anyone who tells you strict mode costs you *n* per cent is quoting one setting.

And the other way is simpler and entirely under your control. A schema can be so narrow that the right answer is not expressible in it. An enum of two values when the honest answer is a third, a `maxLength` shorter than the shortest correct reply, a required field the case has no value for — in each the mask deletes the right answer because you forbade it. That one is not a subtle effect at the edge of the evidence; it is arithmetic, it is visible in lab step 3, and it is far commoner than the reasoning effect.

The fix for the first follows from the mechanism rather than from the evidence, which is why it is worth knowing even though the size is unsettled. Give the model somewhere to work:

```
bad   {"answer": "..."}
good  {"reasoning": "...", "answer": "..."}
```

A field for the working, *before* the answer field, restores the scratchpad inside the schema. The order matters and it matters for a mechanical reason: tokens condition on what came before them, so a `reasoning` field after `answer` is written after the answer has been committed and cannot inform it.

That fix depends on something worth checking. It only works if the engine holds you to your schema's field order — the first row of the table above. JSON itself does not care about key order, so a grammar that admits any order lets the model emit `answer` first and the field you added does nothing at all. It is then text that cannot have influenced the answer, which is a different and much less useful thing.



Whether to show that field to anyone is a separate decision. Chapter 16 is clear that a visible trace is not an account of how the answer was reached, so a `reasoning` field is there to improve the answer, not to explain it to a user.

What You Still Have to Test

A guaranteed format is worth having. Here is what it does not buy you.

It does not buy you correctness. This is Chapter 21's four layers again: a strict mode moves layers 1, 2 and often 3 to the provider, and layer 4 — *are the values true* — is untouched. A constrained decoder will produce `{"sentiment": "positive"}` about a furious review with the same confidence it produces the right answer. Measuring that is chapters 30 and 31.

It does not buy you your whole schema. The engine enforces the subset it compiled. Keep the validator, and if it never fires, Chapter 21's rule of three tells you what you are entitled to conclude from that silence.

And turning it on is a change, so measure it. This is the part people skip. A strict mode alters what the model emits, which means it can alter the answers — that is what the previous section was about. So it is a change like any other, and Chapter 20 applies unchanged: run both versions on the same cases, keep the results paired, and put an interval on the difference. Nothing about *a provider guarantees the format* tells you whether your answers got better or worse, and the people who are surprised by a strict mode making things worse are the people who turned it on without measuring.

Test the format as well as the content, because they fail independently and for different reasons. Two measurements, not one:

Measure	Catches
Validation failure rate	A schema feature the engine did not enforce; a truncated reply
Task accuracy, paired against the unconstrained version	The answers getting worse because the model lost its scratchpad

The first can be zero while the second is terrible. That combination — perfect format, worse answers — is the exact shape of the opening problem, and it is invisible to anybody measuring only the first.

Lab: Build a Constrained Decoder and Break It

Three programs, pure Python, nothing to install, every number checkable by hand. Forty minutes, and there is a key for every step. Work in a new empty folder.

Step 1. Save the model and the grammar — *both* code blocks above, in that order — as `decode.py`, and run it.

Expected result: `Sure ! { "sentiment" : "positive" }` free, and `{"sentiment": "positive"}` constrained.

Step 2. Save the mask program as `mask.py` in the same folder and run it. Check step 1's masked mass by hand from the `NEXT[None]` row. An `ImportError` naming `ALLOWED` means you saved only the first block in step 1; seeing `decode.py`'s own two lines first means its demo is not under `if __name__ == "__main__":`.

Expected result: the chapter's table. By hand: the scores are 6.0 for `Sure`, 4.0 for `{`, 1.0 for `"sentiment"` and 0.0 for the other five, so $p(\{) = e^{4-6}/(e^{0} + e^{-2} + e^{-5} + 5e^{-6}) = 0.117$ and the mass masked is $1 - 0.117 = 0.883$.

Step 3. In `ALLOWED`, make state 3 allow only `"negative"`. Run `decode.py` again. What changed, and what did it cost?

Expected result: the output becomes `{"sentiment":"negative"}`. The model preferred `"positive"` at 4.0 against 3.0, so you have not constrained the format — you have chosen the answer. A schema whose enum has one member is not a format constraint, and this is the thing to look for in somebody else's schema.

Step 4. Give state 0 an empty allowed set: `set()`. Run `decode.py`. What happens, and why is that the worst possible behaviour?

Expected result: the free line is unchanged and the constrained line becomes `Sure"sentiment":"positive"}` — the decoder emits `Sure`, the one token the grammar most clearly forbids, and then carries on through the remaining states as if nothing had happened. Every score is `-inf`, so `max` returns the first index and the mask silently ceases to mean anything. Nothing raises. `generate` never calls `softmax`, but try `softmax([-math.inf]*8)` yourself: you get eight `nan` values rather than eight zeros, because `exp(-inf - -inf)` is not defined — which is why the claim earlier in this chapter that a masked token gets probability zero needed its condition. A **dead state** — a grammar state with no legal token — is a bug that produces output instead of an error, which is why you would want an engine to check its state machine for dead states when it compiles the schema rather than when it decodes.

Step 5. Save the token program as `tokens.py` and run it. Then lengthen the window: set `NEXT_CHARS` to `'{"sentiment"'` and run it again.

Expected result: first the chapter's table. With the longer window, `'{"s'` flips to `True` and `True`, because the token no longer runs past the end of what the check can see — and `'{}'` still disagrees, because it is genuinely illegal however far you look. That is the distinction to take away: `{}` is a *wrong* token and no window length will make it right, while `{"s` was only ever reported wrong because the check could not see far enough. One is a correctness problem and the other is a horizon problem, and only the second goes away by looking further ahead.

Step 6. Without running anything: your schema is `{"answer": "yes" | "no"}` and your task is a question that needs three steps of arithmetic. Predict what strict mode does to your accuracy and say why.

Expected result: the mechanism says the scratchpad is gone, and it says nothing about how much accuracy you lose — keep those apart, because that is the whole discipline of the section above. The schema permits no tokens except the object, so there is nowhere for the model to do the three steps, and Chapter 16 says its only scratchpad is its own output. What follows is that the model cannot show the working, not that it will get the answer wrong — those are different claims and only the first is derivable. Measure it; do not predict it. The fix is a `reasoning` field before `answer`. The honest part of the answer is the second half: you do not know *how much* it will fall, that is not settled, and the way to find out for your task is

Chapter 20 — both versions, same cases, paired, with an interval on the difference.

Step 7. Your provider's strict mode is on. Your validation failure rate over two thousand replies is zero. Name two things you still do not know.

Expected result: whether the parts of your schema the engine did not compile are being enforced — a `maxLength` or a `pattern` silently ignored looks exactly like one satisfied — and whether the answers are right, which no constraint on shape can tell you. A third, if you spotted it: whether the answers got *worse* when you turned it on, which you cannot know unless you measured before.

Summary, Key Terms, and Review Questions

Summary

A format constraint is enforced by **masking**: before each pick, every token the format forbids has its score set to $-\infty$, so the softmax gives it probability exactly zero and — as long as one token survives — no decoding rule can choose it. The model computes what it would have computed; the constraint edits the numbers on the way out, and the format itself is a **state machine** over the tokens allowed next. Measured on this chapter's decoder, the mask deleted 88.3 per cent of the probability mass at step 1 — where the model wanted to write *Sure* — and 6.6, 2.1 and 5.5 per cent at three of the other four. **The mask deletes whatever the model was going to emit, and whether that costs you anything depends on what that was**: step 1's 88.3 per cent was a polite preamble, so the chapter's largest deletion is also its cheapest, and nowhere in that run does the mask remove something the model needed. Support varies between engines because **your schema is written over characters and generation happens over tokens**: the token `{}` starts with a character the grammar wants and then emits one it does not, so the cheap first-character check is too loose — while a check against a short window is too strict, calling the perfectly legal `{"s` illegal because it runs past the end of what the window can see. The real check runs the whole grammar over every character a candidate token would contribute, for every token, in every state, asking whether it leaves the grammar satisfiable — which engines precompute, and which is why features that keep the state machine small and deterministic are supported while recursion, length bounds and open branches often are not. A strict format can make answers worse, and the two halves of that claim sit at very different strengths: the **mechanism is a derivation** — the schema forbids intermediate tokens and Chapter 16 says the model's only scratchpad is its own output — while the *size* of the effect is a handful of studies, each in one setting, with nobody having written down where it stops holding. There is also a second and much commoner way to make answers worse, which is arithmetic rather than subtlety: **a schema too narrow to express the right answer**, which deletes it because you forbade it. The fix for the first follows from the mechanism: a **`reasoning` field before the answer field**, in that order, because tokens condition only on what came before them —

and it works only if the engine holds you to your schema's field order, since JSON itself does not. And a guaranteed format buys shape, not sense: keep Chapter 21's validator, measure the failure rate and the task accuracy separately because they fail independently, and treat turning strict mode on as a change to be measured Chapter 20's way — paired, same cases, an interval on the difference.

Key Terms

Term	Meaning
Constrained decoding	Restricting each token choice to what the format allows, by editing the scores before the pick. The model is unchanged
Masking	Setting a forbidden token's score to $-\infty$, which makes its probability exactly zero after the softmax — as long as one token survives. Mask them all and you get `nan`, not zero
Strict mode	A provider's facility for enforcing a schema during generation, as against asking for it in the prompt
Masked mass	How much probability the constraint deleted at a step. Small means the constraint was free; large means it overruled the model
Dead state	A grammar state with no legal token. It does not raise — the mask stops meaning anything and the decoder emits whatever was first
State machine	What a schema compiles into: a set of states, each with the tokens allowed next. Engines call it an automaton

Worth Memorising

Item	Value or formula
The mechanism	Score → $-\infty$ for forbidden tokens → softmax gives 0 → pick. One line, and only while one token survives
What a constraint costs	The mass it deletes, which costs you only when that was the answer rather than the envelope
Why engines differ	Schemas are over characters, generation is over tokens; the legal-token set is precomputed per state
Both wrong checks	First character only is too loose (admits `{}`); a short window is too strict (rejects `{"s`)
Why strict modes can hurt	No room for intermediate tokens, and the model's only scratchpad is its own output. Also: a schema too narrow to hold the right answer
The fix, and its order	A `reasoning` field *before* the answer field — and only if the engine fixes field order
What it never buys	Correctness, or the parts of your schema the engine did not compile

Review Questions

1. How is a format constraint actually enforced while the model generates?
2. When can a strict output format make the answer worse?
3. Why does schema support vary between providers?
4. Your strict mode has produced valid JSON on every one of two thousand calls. What have you learned?
5. A colleague turns on strict mode and reports that the format problem is solved. What do you ask them to measure?

Review Answers

1. By masking the scores before the token is picked. The model runs exactly as it would otherwise and produces a score for every token in the vocabulary; a state machine compiled from the schema says which tokens are legal at this point; every illegal token's score is set to negative infinity, so the softmax gives it $e^{-\infty} = 0$ and no decoding rule — greedy, top-k, top-p — can select it; and the pick happens among what is left. State the condition, because it is the thing somebody will push on: that holds while at least one token survives, and a state where none does is a dead state, where the mask stops meaning anything and the decoder emits whatever came first rather than raising. Nothing is added to the model and nothing is retried. Two details worth volunteering: the renormalisation means the surviving tokens' probabilities go up, so a state with one legal token has that token at probability 1 whatever the model thought; and the legal-token

set is precomputed per state rather than worked out per step, because the vocabulary is the size Chapter 9 gives.

2. When the schema leaves the model nowhere to think. The answer has two parts at different strengths, and saying so is most of a good answer. The mechanism is a derivation: masking deletes the probability the model had put on the tokens it was going to emit, and if the schema permits only the final object then the tokens it forbids are the intermediate ones — and the model's only scratchpad is its own output, so there is nowhere else for the working to happen. That predicts the damage lands on tasks that need working out rather than looking up. The size of the effect is much weaker evidence: demonstrations and a few studies in particular settings, not settled practice, and nobody has written down where it stops holding, so anybody quoting a percentage is quoting one setting. The fix comes from the mechanism rather than the evidence: a `reasoning` field *before* the answer field, in that order, because tokens condition only on what precedes them — after the answer it is text that cannot have influenced it. One caveat worth volunteering: that only helps if the engine holds you to your schema's field order, since JSON does not care about it.

And name the second way, because it is commoner and it is entirely your own doing: a schema too narrow to express the right answer. An enum of two values when the honest answer is a third, a `maxLength` shorter than the shortest correct reply, a required field the case has no value for. The mask deletes the right answer because you forbade it, and no amount of reasoning space helps. 3. Because the schema is written over characters and the model generates tokens, so an engine has to work out, for every state of the state machine it compiles from your schema, which of the tokens in the vocabulary are legal. Checking only the first character is too loose — `{}` begins legally where `{"` is wanted and then closes the object — and checking against a short lookahead window is too strict, because a token like `{"s` is entirely legal and simply runs past the end of the window. The real check asks whether a token leaves the grammar satisfiable. That set is precomputed and cached, and the cost of the precomputation depends on the schema's features: a fixed set of fields in order, or an enum of strings, gives a small deterministic state machine, while `anyOf` keeps branches open, `minLength` needs a counter that tokens advance by varying amounts, and recursion needs a stack instead of a state number. The features that survive that compilation are the features the engine supports. So the practical rule is to read what the engine says it enforces and assume the rest is not enforced — and keep validating, because a constraint silently not applied is indistinguishable from one applied and satisfied. 4. That the shape is right, and almost nothing else. You have not learned that your whole schema is being enforced: the engine compiled a subset, and a `pattern` or a length bound it quietly dropped produces exactly the result you are looking at. You have not learned that the values are right, which is Chapter 21's fourth layer and needs a measurement of correctness rather than of shape. And you have not learned that the answers are as good as they were before you turned it on, which is the failure this chapter is about — perfect format, worse answers — and which is invisible to anybody measuring only validity. On the rate itself, two thousand clean calls

does entitle you to something: by the rule of three the format failure rate is under about one in six hundred and seventy. 5. Two things, separately, because they fail independently. The validation failure rate, which should now be very low and which still needs measuring, because it is what catches a schema feature the engine did not compile and a reply truncated at the token cap. And task accuracy, paired against the unconstrained version on the same cases with an interval on the difference, because turning strict mode on is a change to what the model emits and therefore a change that can move the answers. The question to ask them is simply whether they ran the evaluation set before and after; if they ran it only after, they know the format is fixed and have no idea what it cost. If the task needs working out rather than looking up, also ask whether the schema has a field for the working, and whether it comes before the answer.

> **You understand this when** you can say in one sentence what a constraint does to the numbers, and name the thing a guaranteed format cannot tell you.

CHAPTER 23

How a Tool Call Works, and Who Decides

The problem. People say the model "called an API". Models generate text, so something here is being skipped.

The question. What actually happens when a model uses a tool, and at which step does something get decided?

Nothing Called Anything

A model emits tokens. That is the whole of what it does, and Chapter 9 is why. It has no network access, no file handle and no way to run a function. So when somebody says the model called an API, the sentence has compressed five steps into one, and four of them are yours.

Step	Who does it	What it is
1	Your code	Send the tool descriptions, as text, with the prompt
2	The model	Emit tokens that your code can read as a proposal
3	Your code	Parse it, validate it, and decide whether to run it
4	Your code	Run the function and get a result
5	Your code	Send the result back as another message, and ask again

The model's entire contribution is step 2, and step 2 is text. Everything that touches the world is yours.

So there are two decisions, not one, and conflating them is the mistake in the opening sentence. The model decides *what to propose*. Your code decides *what runs*. People say the model called the API because the two decisions sit next to each other in time and the second one is invisible — it happens inside code you wrote, so it does not feel like a decision at all.

Only one of the two can refuse, and this chapter is about that one. Chapter 2 drew the same line first: both of them decide something, and the two decisions are different.

Chapter 4 owns the loop this sits inside — what makes it run again, and what stops it. This chapter owns the five steps themselves, in detail, and it goes round twice where it has to in order to show step 5.

Describing a Function to a Model

Step 1. Your code has functions; the model cannot see them. What it gets is text.

```
import json

def get_balance(account_id):
    return {"account_id": account_id, "balance_pence": 184_200}

def refund(order_id, amount_pence):
    return {"refunded": order_id, "amount_pence": amount_pence}

# What your code knows. The model never sees the functions, only the text
# rendered from the "describe" fields below.
REGISTRY = {
    "get_balance": {
        "describe": "Return the balance of one account.",
        "params": {"account_id": {"type": str, "max_len": 12}},
        "run": get_balance,
    },
    "refund": {
        "describe": "Refund part or all of an order.",
        "params": {"order_id":     {"type": str, "max_len": 12},
                   "amount_pence": {"type": int, "min": 1, "max": 50_000}},
        "run": refund,
    },
}
```

Three things are in each entry and they go to three different places. The `describe` text and the parameter *names and types* are sent to the model, and together they are the **tool description**. The constraints — the maximum length, the range — are not sent; they are for your code. And `run` never leaves your process.

```
import json
from tools import REGISTRY

def render(name):
    """The text your code sends. The model sees this, never the function."""
    spec = REGISTRY[name]
    params = {k: r["type"].__name__ for k, r in spec["params"].items()}
    return json.dumps({"name": name,
                       "description": spec["describe"],
                       "parameters": params})

for name in REGISTRY:
    spec = REGISTRY[name]
    print(f"  {name}")
    print(f"    {spec['describe']}")
    for k, r in spec["params"].items():
        print(f"    {k}: {r['type'].__name__}")
    print(f"    -> {len(render(name))} characters as sent")
```

```
  get_balance
    Return the balance of one account.
    account_id: str
    -> 113 characters as sent
  refund
    Refund part or all of an order.
    order_id: str
    amount_pence: int
    -> 126 characters as sent
```

That is the whole of "giving a model tools". It is a paragraph of text per function, placed in the input. There is no registration, no handshake and no capability granted. Chapter 18's message list gained a block, and that is all that happened.



How providers carry that text differs — a dedicated `tools` field, or the system message, with their own field names and their own way of marking the proposal when it comes back. The arrangement varies and will keep varying. What does not vary is that it is input going in and text coming out, which is why this chapter is about the shape rather than the field names.

The Proposal

Step 2, the model's only step. What comes back is tokens, and your code reads them as a proposal:

```
{"tool": "refund",
 "arguments": {"order_id": "ORD-9", "amount_pence": 2500}}
```

Three things about that are worth being exact about.

It is a **proposal**, not a call. Nothing has happened. Chapter 2 put it exactly: a proposed call is a suggestion arriving in text.

It can be anything the model can emit. A tool you never described. An argument of the wrong type. A refund of nine thousand pounds. Or a sentence instead of a proposal — *I will now issue the refund for order ORD-9* — which is the most instructive failure, because it shows there was never a mechanism to begin with, only a convention the model was following.

And **a provider's strict mode can make the proposal reliably parseable**, which is Chapter 22, and worth having. It constrains the *shape*. It has nothing to say about whether `amount_pence` should be 2,500 or 900,000.

Your Code Decides

Step 3, and this is the chapter. Here is what your code checks, and then the **dispatcher** — the code that takes a proposal, validates it, and either runs the function or refuses.

```
def validate(name, args):
    """What your code checks before anything runs. Not a complete list:
    it accepts an unexpected key at the top level, and an empty string
    where a schema would want a minimum length."""
    if name not in REGISTRY:
        return f"no such tool: {name}"
    spec = REGISTRY[name]["params"]
    missing = sorted(set(spec) - set(args))
    if missing:
        return f"missing argument: {missing[0]}"
    extra = sorted(set(args) - set(spec))
    if extra:
        return f"unexpected argument: {extra[0]}"
    for key, rule in spec.items():
        v = args[key]
        if type(v) is not rule["type"]:     # type, not isinstance: ch 21
```

```
            return f"{key}: wanted {rule['type'].__name__}"
        if "max_len" in rule and len(v) > rule["max_len"]:
            return f"{key}: longer than {rule['max_len']}"
        if "min" in rule and not rule["min"] <= v <= rule["max"]:
            return f"{key}: outside {rule['min']} to {rule['max']}"
    return None

def handle(proposal):
    """Parse, validate, decide, and run or refuse."""
    try:
        call = json.loads(proposal)
    except json.JSONDecodeError:
        return "not JSON", None
    if not isinstance(call, dict):
        return "not a proposal", None      # valid JSON, wrong shape
    name = call.get("tool")
    args = call.get("arguments") or {}
    if not isinstance(name, str) or not isinstance(args, dict):
        return "not a proposal", None
    why = validate(name, args)
    if why:
        return why, None
    return None, REGISTRY[name]["run"](**args)
```

Six proposals, of the kinds you actually get:

```
PROPOSALS = [
    '{"tool": "get_balance", "arguments": {"account_id": "ACC-00041"}}',
    '{"tool": "refund", "arguments": {"order_id": "ORD-9", "amount_pence":
        ↪ 2500}}',
    '{"tool": "refund", "arguments": {"order_id": "ORD-9", "amount_pence":
        ↪ 900000}}',
    '{"tool": "drop_table", "arguments": {"name": "orders"}}',
    '{"tool": "refund", "arguments": {"order_id": "ORD-9"}}',
    'I will now issue the refund for order ORD-9.',
]

if __name__ == "__main__":
    for i, p in enumerate(PROPOSALS, 1):
        why, result = handle(p)
        print(f"  {i}  {'REFUSED  ' + why if why else 'ran -> ' +
            ↪ json.dumps(result)}")
```

```
1  ran -> {"account_id": "ACC-00041", "balance_pence": 184200}
2  ran -> {"refunded": "ORD-9", "amount_pence": 2500}
3  REFUSED  amount_pence: outside 1 to 50000
4  REFUSED  no such tool: drop_table
5  REFUSED  missing argument: amount_pence
6  REFUSED  not JSON
```

Two ran and four did not, and the four are the chapter.

Proposal 3 is well-formed, correctly typed, names a real tool — and asks for nine thousand pounds. Chapter 21's layers again: it passes the shape checks and fails on range, and the range check is one line that nobody puts in until the first time it is needed.

Proposal 4 is the one to remember. `drop_table` was never described to the model and does not exist in the registry. The model produced it anyway, because producing plausible text is what it does, and `drop_table` is extremely plausible text. A **registry** lookup is what stands between that proposal and a **dispatcher** that does something clever with names.

Proposal 5 is missing an argument. Proposal 6 is not a proposal at all.

Then step 5 closes the loop. The result — or the refusal — goes back as a message, and the model's next turn sees it as more input. Chapter 18 named that role and owns it: a tool message is a result your code returns after the model asked for something. Send the refusal text, not a generic failure: *amount_pence: outside 1 to 50000* lets the model propose 50,000 instead, and *that was invalid* does not. The edge on that advice is Chapter 4's: a specific refusal also tells the model exactly where the fence is, and a loop that is refused and then tries a slightly different route to the same place has been inconvenienced rather than controlled.

Say the limit, and keep the limit.



A refusal is still an outcome, and the loop can spin. If the model proposes an out-of-range refund three times, something is wrong with the prompt or with the limit, and Chapter 4's step limit is what stops you paying to find out forever.

Why Validation Cannot Live in the Instructions

This is the second question, and the answer people give is usually a preference. It is not a preference, and the reason is mechanical.

Chapters 2 and 4 both state the rule — Chapter 4's approval point has to be in your code and not in the instructions. Here is the mechanism underneath it, which the five steps make plain: **nothing in the loop compares a proposal's *values* against your instructions except your own code.** Be precise about that, because Chapter 22 has just shown you the exception that proves the shape of the rule: a strict mode genuinely does check the model's emission during step 2 — against a *schema*. It can hold a proposal to a shape, and to a closed set of values where you can enumerate them — Chapter 22's enum, its easy case. What it does not do is hold a *number* inside a range you decided: nothing in principle forbids it, and Chapter 22's rule is that the features which survive the compilation are the features an engine supports, so assume a bound you did not see enforced is not enforced. And nothing at all can check a value against the world — whether this order exists, whether it has already been refunded, whether this caller may refund it. Those are the limits that matter, and they are all yours. Look at the table again. Step 1 is you sending text. Step 2 is the model emitting text. Steps 3, 4 and 5 are yours. An instruction saying *never refund more than £500* is input at step 1; it influences what is likely at step 2; and nothing anywhere compares step 2's output against it. If your code does not check, the check does not exist.

And here is the second reason. Your instruction is text in the input, and so is everything else in there — the retrieved document, the user's message, the previous tool result. Nothing marks one of those as an instruction and the rest as data; they arrive as one sequence of tokens. So your instruction is not only unenforced, it is competing. Chapter 51 is where that becomes a security problem rather than a reliability one, and it owns the question properly.

And a third, which is the one that catches careful people. Suppose the model complies perfectly, every time, for a year. You still have no limit. You have a correlation, and the first time the input is unusual — a document that mentions a large refund, a user who asks twice, a model version change — the correlation is what you find out you were relying on. Chapter 21's language for this: a validator that has never fired has also never been tested.

So the rule, in a form you can say:

Validation belongs in your code because your code is the only participant that can refuse. Instructions shape what gets proposed, which is worth doing and is not a control. The limit is the line of code that returns before the function runs.

What the Model Sees Next

Step 5 deserves its own look, because it is where the loop actually closes. The result does not go back as a return value. It goes back as a message in the list, and the model's next turn reads it as input like everything else.

Chapter 18 defined that role and pointed here for the mechanism. Here it is, as `exchange.py`, over one exchange that needs two calls — the second because the customer asked for two things:

```
import json
from tools import handle

# What the model would emit at each turn, written out so the exchange is
# deterministic. Turn 3 quotes turns 1 and 2's results, which is the point.
TURNS = [
    '{"tool": "get_balance", "arguments": {"account_id": "ACC-00041"}}',
    '{"tool": "refund", "arguments": {"order_id": "ORD-9", "amount_pence":
        ↪ 2500}}',
    "Your balance is GBP 1842.00 and I have refunded GBP 25.00.",
]

messages = [
    {"role": "system", "content": "You help customers with orders."},
    {"role": "user",   "content": "What is my balance, and refund ORD-9 GBP 25."},
]

for turn in TURNS:
    messages.append({"role": "assistant", "content": turn})
    why, result = handle(turn)
    if why == "not JSON":
        break                                # the model answered in prose: done
    messages.append({"role": "tool",
                     "content": json.dumps(result) if why is None else why})

for m in messages:
    body = m["content"]
    print(f"  {m['role']:9} {body[:46]}")
print(f"\n  {len(messages)} messages, {sum(len(m['content']) for m in messages)}"
      f" characters of content")
```

```
system    You help customers with orders.
user      What is my balance, and refund ORD-9 GBP 25.
assistant {"tool": "get_balance", "arguments": {"account
tool      {"account_id": "ACC-00041", "balance_pence": 1
assistant {"tool": "refund", "arguments": {"order_id": "
tool      {"refunded": "ORD-9", "amount_pence": 2500}
assistant Your balance is GBP 1842.00 and I have refunde

7 messages, 369 characters of content
```

Read the roles down the left. A proposal is an `assistant` message — it is the model talking. In the text convention this chapter models, the only thing that makes it a tool call is that your code recognised it as one; where a provider marks the proposal for you, as the earlier Note says they do, the recognising is done before your code sees it and the deciding still is not. A result is a `tool` message, written by you. They alternate, and the alternation is the loop.

The last turn is prose, and nothing stopped it. The model was free to answer at every step and chose to call a tool twice and then answer — which is Chapter 4's first stopping reason, the model finishing, seen from inside step 5. Your dispatcher's `not JSON` branch is how the program notices that no tool was proposed — which is the same branch that handled proposal 6, being used for a success this time rather than a failure. **That is worth knowing, and it is a smell rather than a virtue.** *The model answered* and *the model emitted something I could not parse* arrive at your code through the same door, because both of them fail to parse as a proposal. A truncated reply — `{"tool": "refund", "arguments":` — therefore ends the loop and becomes the last thing the user sees.

And you cannot separate them by looking at the text, which is the part to get right. There are three cases, not two: a reply that parses as a proposal, a reply that is a plain answer, and a reply that is neither because it was cut off. The first is obvious. For the other two, Chapter 21 already named the signal — read the response's own **finish reason**, which says whether the model stopped because it had finished or because it hit the token cap. Only the second is a failure, and nothing in the reply's characters tells you which of the two you are holding.

And every one of those seven messages is sent on the next call, along with the tool descriptions: the system message, the user's question, both proposals, both results, and the answer it finally gave. Chapter 18's arithmetic for a conversation applies, and a tool exchange fills one faster than talking does. Count it here: the user wrote one message of 44 characters, and what came back was five further messages and 294 characters, none of it from the user. A plain question and its answer would have added two.



How a result is tied back to the call it answers differs between providers — an id on the proposal that the result quotes, or simply the order of the messages. You need some such mechanism as soon as a model proposes two calls at once, which providers also handle differently — and when it does, two results go back before the model emits again, so it is not one message each. The role *names* vary too: a result may come back as its own role or inside a user-role message. What does not vary is whose contribution comes next — the model emits, then your code puts results in, then the model again.

What the Descriptions Cost

One last mechanical fact, and it follows from Chapter 18. The descriptions are input, and input is sent on *every call*.

```
total = sum(len(render(n)) for n in REGISTRY)
each = total / len(REGISTRY)
print(f"\n  {len(REGISTRY)} tools = {total} characters, on every call")
for n in (2, 8, 30):
    print(f"  {n:2d} tools ~ {each*n:5.0f} characters ~ {each*n/4:4.0f} tokens")
```

```
  2 tools = 239 characters, on every call
   2 tools ~   239 characters ~   60 tokens
   8 tools ~   956 characters ~  239 tokens
  30 tools ~  3585 characters ~  896 tokens
```



Treat the token column as an estimate and the character column as a measurement. Four characters to the token is a rule of thumb, not a result: Chapter 10 is why the real number depends on the tokeniser and on the text, and it can err either way — densely punctuated JSON and ordinary prose do not tokenise at the same rate. The characters are solid, and they are what to reason with. The more important correction is that `render` is this book's shape, kept short to fit the page. A real tool definition carries a nested schema with `type`, `properties` and `required`, and the same two functions written that way measure 190 and 231 characters rather than 113 and 126 — about 210 a tool, so thirty tools is nearer 6,300 characters than 3,600, and the token figure moves with it.

Thirty tools is of the order of a thousand tokens on every call — nearer fifteen hundred with a real schema, as the Note works out — including the calls that need no tool at all, and before the conversation has said anything. Two consequences, and only the first one is obvious.

You are paying for capability you mostly do not use. The user asking *what is my balance* pays for the refund tool's description, the escalation tool's, and the twenty-seven others'.

And the model is choosing from thirty things. That is the cost people do not count: a longer menu is a harder choice, and Chapter 16's finding about uneven use of a long input applies to the tool block like any other input. What to do about either is split: fewer,

more purposeful tools and a description the model chooses well from is Chapter 36's, and a routing step or tools selected per request is Chapter 34's and Chapter 52's. The arithmetic is here so the decision is informed.

Lab: Run the Five Steps by Hand

Three programs, pure Python, nothing to install. Forty minutes, and there is a key for every step. Work in a new empty folder.

Step 1. Build `tools.py` from three of this chapter's code blocks, in this order: the one beginning `import json` and `def get_balance`, the one beginning `def validate`, and the one beginning `PROPOSALS = [`. Run it.

Expected result: the six lines in the chapter: two ran, four refused, each refusal naming its own reason.

Step 2. Save the `render` block and the cost block together as `describe.py`, in the same folder, and run it. Then answer: which parts of the registry did the model never see?

Expected result: the chapter's output. The model never saw `run`, nor `max_len`, nor the `min` and `max` on `amount_pence`. It saw two names, two sentences and three parameter names with their types — 239 characters in total. Everything that stops a bad call is on your side of that line.

Step 3. Delete the registry lookup — the two lines beginning `if name not in REGISTRY` — and run `tools.py` again. What happens to proposal 4, and what would happen in a dispatcher that looked tools up by name?

Expected result: proposals 1, 2 and 3 print as before and then the program dies with `KeyError: 'drop_table'` raised inside `validate`, because it now goes straight to `REGISTRY[name]["params"]`. Here that is a crash, which is survivable. The point is what the same missing line does in a dispatcher built to be helpful — one that falls back to a naming convention, or searches a module for a matching function. Then the model's invented name is a lookup key into your code, and `drop_table` is a plausible name for a real function somewhere.

Step 4. Put the line back. Now raise `amount_pence`'s maximum to 1,000,000 and run it again.

Expected result: proposal 3 runs and refunds £9,000. Nothing else changed — the proposal was always well-formed, correctly typed and named a real tool. The only thing that ever stood between it and your payment processor was a number in a dictionary in your code. Put the 50,000 back before you go on.

Step 5. Change the instructions instead. Add to `tools.py` a module-level string `INSTRUCTIONS = "Never refund more than 500 pence."` and run it again. What changed?

Expected result: nothing at all, which is the point of the exercise. The string is not read by anything. In a real system it *would* be read — it would be sent to the model and would change what step 2 proposes — and it still would not be a limit, because nothing in the loop compares the proposal's values against it. Keep the dictionary value.

Step 6. Proposal 6 is prose. Change the dispatcher to look for the word "refund" in a non-JSON proposal and act on it. Then say why you should not.

Expected result: you get part of the way and then stop, and where you stop is the lesson. *I will now issue the refund for order ORD-9* gives you the word "refund" and an order id a regular expression can find. It does not give you an amount — the only digit in the sentence is the `9` of `ORD-9` — so `refund(order_id, amount_pence)` cannot be called without you inventing a sum of money. That is the moment to stop. You have replaced a contract with a guess, which is Chapter 21's rule about never trusting prose, now attached to something that moves money. And the sentence *I will not issue the refund* contains the same word.

Step 7. Your service has 30 tools. A user says "hello". Roughly what does that cost, and what are the two separate costs?

Expected result: about 3,600 characters by the chapter's renderer, so of the order of 900 tokens — and nearer 5,900 characters and 1,500 tokens with a real tool schema, which is the number to quote. Either way it is sent before the user's word is even considered. The two costs are the input you pay for on every call, and the harder choice the model is making from a longer menu. The first is money, the second is accuracy, and only the first shows up on the bill.

Step 8. Save the exchange program as `exchange.py` and run it. Then truncate the second turn, cutting it down to `{"tool": "refund", "arguments":` and no more, and run it again.

Expected result: first the chapter's seven messages. After the truncation the loop stops at that turn, because `handle` returns `not JSON` and the program treats that as *the model answered*. The broken fragment is now the last assistant message, and in a real system it is what the user sees. You cannot fix this by inspecting the text, because the prose answer and the truncated proposal both fail to parse in exactly the same way — check it yourself by passing each to `handle`. The fix is the response's finish reason, which Chapter 21 named: it says whether the model stopped because it was done or because it ran out of tokens.

Step 9. Feed the dispatcher some valid JSON that is not a proposal — `[1,2,3]`, `"hello"`, `null`, and a proposal whose `tool` is a list. Then feed it a well-formed refund proposal with an extra key beside `tool` and `arguments`.

Expected result: the first four all return `not a proposal`, which is what the `isinstance` guards are for — every one of them is valid JSON, so the `JSONDecodeError` branch never sees them. The one with an extra top-level key *runs*, because nothing checks for unexpected keys there; the check only covers the inside of `arguments`. That is a real gap in the validator this chapter prints, which is why its docstring says the list is not complete. Decide whether

you want that check — and notice that you have just done the thing the chapter asks of you, which is to read the validator rather than trust it.

Step 10. A colleague says their agent "has access to the database". Rewrite that sentence accurately.

Expected result: their code has access to the database, and the model has a sentence describing a function their code will run if it decides to. The rewrite matters because the first version suggests the model holds a capability that could be taken away or misused directly, and the second locates the capability where it is — and therefore where the limit has to go.

Summary, Key Terms, and Review Questions

Summary

A model emits tokens and nothing else, so "the model called an API" has compressed five steps into one and four of them are yours: you send the **tool descriptions** as text, the model emits a **proposal**, and your code parses it, validates it, decides, runs the function, and sends the result back as a tool message — Chapter 18's word for it. **There are two decisions and only one of them can refuse** — the model decides what to propose, your code decides what runs, and the second is invisible because it happens in code you wrote. Describing a function is a paragraph of text per function placed in the input: 113 and 126 characters for the two here, and the model never sees the function, the maximum length or the range. A proposal can be anything the model can emit — a tool you never described, an argument of the wrong type, a refund of nine thousand pounds, or a sentence instead of a proposal — and of the chapter's six, two ran and four were refused, by a registry lookup, a range, a missing argument and a parse. **Validation cannot live in the instructions, and the reason is mechanical**: nothing in the loop compares a proposal's values against your instructions except your own code — a strict mode can hold a proposal to a shape, and to an enumerated set of values, and neither of those is a limit on a number or a check against the world — your instruction is competing with every other piece of text in the input because nothing in the input marks one part as the instruction, and a year of compliance is a correlation rather than a limit. **Your code is the only participant that can refuse.** And the descriptions are input, so they are sent on every call: thirty tools is of the order of fifteen hundred tokens before the conversation starts, which costs money on calls that need no tool and also makes the model choose from a longer menu.

Key Terms

Term	Meaning
Tool description	The text your code sends about one function: a name, a sentence and the parameter names and types. Not the function, and not your constraints
Proposal	What the model emits — tokens your code reads as a request to run something. Nothing has happened yet
Registry	The map from a name to a function your code is willing to run. The lookup that refuses a name the model invented
Dispatcher	The code that takes a proposal, validates it, decides, and runs the function or refuses. The only participant that can refuse

Worth Memorising

Item	Value or formula
The five steps	You send · model proposes · **you validate and decide** · you run · you return
Who decides	The model decides what to *propose*. Your code decides what *runs*
Why not in the instructions	Nothing in the loop checks a proposal's *values* against them except your code. A strict mode holds a shape, and an enum; not a range, and never a fact about the world
What the model never sees	The function, and every constraint you kept on your side
Description cost	Input on every call. A real tool schema is around 200 characters, so 30 tools is of the order of 1,500 tokens
The refusal to send back	The specific reason, so the next proposal can be better

Review Questions

1. Walk me through a tool call. Who decides what runs?
2. Where does validation of a tool call belong, and why not in the instructions?
3. The model proposes a tool your code has never heard of. What happens, and what should?
4. Your agent has thirty tools and most requests use none of them. What is that costing you?
5. A colleague says the model "has access to the database". What is wrong with the sentence?

Review Answers

1. Five steps, and the model is responsible for one. Your code sends the tool descriptions as text along with the prompt. The model emits tokens that your code reads as a proposal — a name and some arguments. Your code parses that, validates it, and decides whether to run it. Your code runs the function. Your code sends the result back as a tool message, and the model's next turn sees it as more input. **Who decides what runs is your code, always**, and the thing to say next is why the question gets asked at all: there are two decisions sitting next to each other, the model's about what to propose and yours about what to execute, and the second is invisible because it happens inside code you wrote. The model has no network, no file handle and no way to run anything; a tool call is a convention over text, which is why a proposal can also arrive as a plain sentence.
2. In your code, and the reason is mechanical rather than stylistic. Walk the loop: step 1 is your code sending text, step 2 is the model emitting text, and steps 3 to 5 are your code. Nothing in that compares a proposal's *values* against what you told the model — a strict mode can hold it to a shape, which is a different thing from a limit. An instruction like *never refund more than £500* is input — it changes what is likely at step 2, which is worth having — but nothing enforces it, so if your code does not check, the check does not exist. Two more reasons worth adding. Your instruction is competing: it is text in the input alongside the retrieved document, the user's message and the last tool result, and the whole lot arrives as one sequence of tokens with nothing marking which part is the instruction — which is also the mechanism behind a proposal that is there because something the model read told it to ask, and that is Chapter 51's subject. And compliance is not a limit: a model that has obeyed for a year has given you a correlation, and you discover what you were relying on the first time the input is unusual. The short form is that your code is the only participant that can refuse.
3. It arrives like any other proposal, as text, and whether anything bad happens depends entirely on your dispatcher. What *should* happen is a registry lookup that fails and a refusal sent back naming the unknown tool. What can happen instead is the problem: a dispatcher that is clever about names — one that falls back to a naming convention, or searches a module for a matching function — turns the model's invented name into a lookup key into your code. The chapter's example is `drop_table`, which was never described to the model and is extremely plausible text, which is exactly why the model produced it. So the answer to give is the registry: an explicit map from a name to a function you are willing to run, with the lookup before anything else, and no path that constructs a target from the name the model supplied.
4. Two costs, and only one of them shows up on the bill. The descriptions are input and input is sent on every call, so thirty tools at around 200 characters each — which is what a real schema with its `type`, `properties` and `required` comes to — is roughly 6,000 characters, of the order of 1,500 tokens, paid before the user's message is considered and including on every request that uses no tool at all. The second cost is accuracy: the

model is choosing from a thirty-item menu, and a longer input is used less evenly than a short one. What to do about it is a design question rather than a trick — fewer tools, a routing step, or selecting the tools for a request — and the arithmetic is what tells you whether it is worth doing.

5. Their *code* has access to the database. The model has a sentence describing a function their code will run if it decides to. The distinction is not pedantry, because the first version locates a capability in the model, and if that is where you think it lives then that is where you will try to control it — with an instruction, which cannot refuse anything. Put the capability where it actually is, in the dispatcher, and the limit goes in the same place.

> **You understand this when** somebody says the model called an API and you can name which of the five steps they skipped.

CHAPTER 24

Embeddings, and Choosing One for Your Data

The problem. Two sentences that mean the same thing share no words. I don't see how any program could notice that.

The question. What does an embedding model produce, and how do you choose one for your own documents?

What an Embedding Model Produces

Start by separating two things the word *embedding* is used for.

Chapter 9 has one already: the *embedding table* inside a generative model, a matrix with one row per vocabulary entry, where looking a token up is how it enters the computation. That table is internal to the computation, it holds one row per token rather than one vector per text, and it is not what you search with.

This chapter is about something else: an **embedding model**, a separate model with a different job. You give it a piece of text, it gives you back one vector of a fixed width, and that is all it does. It does not continue your text, it does not answer, and it has no output tokens. One text in, one list of numbers out, the same width every time.

	A generative model	An embedding model
You send	A prompt	A piece of text
You get	Tokens, one at a time	One vector, of fixed width
Width	A vocabulary's worth of scores per step	A few hundred to a few thousand numbers, once
Useful for	Producing text	Comparing texts with arithmetic

The vector it returns is that text's **embedding**, and its width is fixed by the model. A model

with 768 dimensions gives you 768 numbers for a word, for a sentence and for a page.



This book does not explain *why* the arrangement of those numbers carries anything useful. Chapter 9 says the same of its table and means it: the coordinates are whatever made the model's training objective work, nobody chose them, and no dimension means anything on its own. That question is representation learning, it is on Chapter 17's list of what this book deliberately leaves out, and you can do everything in this chapter and the five after it without an answer to it. What you can check is what the numbers *do*, which is the rest of this chapter.

Closeness, Applied

Chapter 8 built the tool for this and you already have it: the dot product grows with magnitude, which is why *cosine similarity* exists, and the cosine of two vectors is their dot product divided by the product of their lengths. Chapter 8 also showed that the cosine does not change when you scale a vector, which is worth having and is not the reason a page and a sentence are comparable. That reason is the one above: the model returns the same fixed width whatever goes in.

So apply it. Three sentences, with their vectors written out:

```
import math

# Four numbers per sentence, written out rather than computed. Producing them
# needs an embedding model; what this chapter is about is what you do with
# them once you have them.
VECTORS = {
    "the flight was delayed":        [0.80, 0.50, 0.20, 0.10],
    "my plane left late":           [0.70, 0.60, 0.10, 0.20],
    "the flight attendant was kind": [0.20, 0.10, 0.90, 0.40],
}

def dot(a, b):
    return sum(x * y for x, y in zip(a, b))

def length(a):
    return math.sqrt(dot(a, a))

def cosine(a, b):
    return dot(a, b) / (length(a) * length(b))

if __name__ == "__main__":
    names = list(VECTORS)
    for i, a in enumerate(names):
        for b in names[i + 1:]:
```

```
        print(f"  {cosine(VECTORS[a], VECTORS[b]):.3f}  {a}")
        print(f"         vs  {b}")

0.978  the flight was delayed
       vs  my plane left late
0.439  the flight was delayed
       vs  the flight attendant was kind
0.386  my plane left late
       vs  the flight attendant was kind
```

Check the first one by hand, because the whole chapter rests on being able to. Carry five decimals through the lengths — at four, the rounding is enough to push the quotient to 0.979:

$$\frac{0.90}{\sqrt{0.94}\,\sqrt{0.90}} = \frac{0.90}{0.96954 \times 0.94868} = 0.978$$

Now read the three numbers carefully, because the obvious reading is the wrong one.

The flight was delayed and *my plane left late* share **no content words at all** — not *flight*, not *plane*, not *delayed*, not *late* — and their vectors point almost the same way, at 0.978. *The flight was delayed* and *the flight attendant was kind* share three words including the subject, and sit at 0.439.

So the answer to the opening problem is this: **the program does not notice that two sentences mean the same thing.** What a model does is place each text somewhere, and *if* it places two texts close together, cosine will report that — which is a measurable fact about its output, and enough to build a search on, because you can rank by it.

Now the honest part, and it is the hinge of the chapter. Those four-number vectors were written down by hand, not measured. Nothing above shows that a real model puts *the flight was delayed* and *my plane left late* at 0.978; it shows that cosine reports 0.978 for the vectors I chose. Whether a real model does that for *your* paraphrases, in your language and your domain, is an empirical property of that model — and the second half of this chapter is how you find out whether a model works on your data, which is a different measurement and the one that decides anything. That is not a caveat bolted on to the demonstration. It is the reason the second half exists, and the reason you cannot take a leaderboard's word for it, or this book's. A high cosine is not evidence that the model understood anything, nor a demonstration that two texts mean the same — **Chapter 25 is about the gap between closeness and relevance**, and the answer there is not a simple yes.

That distinction is worth holding onto, because every one of the next five chapters depends on it. What you have is a number. What you want is relevance. They are not the same thing and the rest of Part II is largely about the distance between them.

Ranking

One more step and you have search. Embed the query with the same model — not a different one, or the numbers are not comparable — compare it with every document, and sort.

```
from close import VECTORS, cosine

QUERY = [0.75, 0.55, 0.15, 0.15]     # "why was my plane late"

ranked = sorted(VECTORS, key=lambda d: cosine(QUERY, VECTORS[d]), reverse=True)
for place, doc in enumerate(ranked, 1):
    print(f"  {place}  {cosine(QUERY, VECTORS[doc]):.3f}  {doc}")
```

```
  1  0.995  the flight was delayed
  2  0.994  my plane left late
  3  0.415  the flight attendant was kind
```

That is **dense retrieval**, complete. Embed everything once and keep the vectors; embed the query at search time; sort by cosine; take the top few. Five chapters from now you will be making this fast over millions of documents rather than three, which is Chapter 29's, and the arithmetic will not have changed.

And a raw cosine has no meaning on its own, which is the other thing to take from that output. 0.995 does not mean 99.5 per cent relevant. It is not comparable with another model's 0.995, because the two models' vectors live in different spaces; it is not comparable across queries, because the spread of scores depends on what the query was; and it is not comparable across corpora. So a cut-off learned in one place — *treat anything above 0.7 as a hit* — does not travel, and a number that worked for months can stop working the day you change model. What travels is the *order*, which is why everything in these chapters ranks rather than thresholds. If you must have a cut-off, derive it on your own set, with your own answer key, and re-derive it whenever the model changes.

Two more details are worth noticing in that output. The top two are separated by 0.0002, which is nothing — a ranking can be stable in its top *set* and unstable in its *order*, and if you care about which of two near-ties comes first you need a tie-break your code controls rather than a third decimal place. And the gap down to third place is huge, 0.994 to 0.415, which is the shape you hope for and will not always get.

Why the Best Model Is a Property of Your Data

Now the question that matters, and the one with a real answer.

There is no best embedding model, in the way there is no best lens. A model is fitted on some distribution of text, and it does well on text like that. Three things about *your* text decide which model suits it:

Language. A model fitted mostly on English does worse on your German support tickets, and a multilingual model usually trades some English quality for the rest. If your documents are in one language, say so when you choose.

Domain. Legal clauses, clinical notes, error logs and product reviews do not look like each other, and none of them looks much like the general web text most models see most of.

Document length. This is the one people forget, and it has two halves. The first is a hard limit: **every embedding model has a maximum input length**, counted in tokens, and text past it is cut off — commonly with no error at all, so you get a vector back and it describes the first part of your document. Some services reject the request instead, which is the kinder failure and not the one to plan for. Find that number before you do anything else, because it decides whether your documents can be embedded whole at all. The second half is softer: within the limit, a model fitted on sentences still has to compress two thousand words into the same fixed width as a sentence, and models differ a great deal in how well they do that. Those two facts are not in tension: the fixed width is what makes a page and a sentence *comparable* at all, and it is also what forces the compression that makes them unequally well *represented*. Whether your documents arrive as sentences, paragraphs or pages is a property of your corpus, and Chapter 27 is where you decide how to split them — a decision that interacts with this one.

So the way to choose is to measure, on your own documents, with your own queries. Here is the smallest version of it. Two candidate models, four of your documents, four of your queries, and the cosine scores each one gave:

```
# Two candidate embedding models, run over the same four documents and the
# same four queries. These scores are invented, exactly as the first
# program's vectors were, and they are here to show the shape of the
# comparison rather than to report a measurement.
DOCS = ["short-1", "short-2", "long-1", "long-2"]

SCORES = {
    "model A": {
        "q1 (short)": [0.91, 0.44, 0.30, 0.22],
        "q2 (short)": [0.38, 0.88, 0.25, 0.31],
        "q3 (long)":  [0.71, 0.40, 0.66, 0.35],
        "q4 (long)":  [0.33, 0.29, 0.41, 0.74],
    },
    "model B": {
        "q1 (short)": [0.62, 0.68, 0.40, 0.35],
        "q2 (short)": [0.30, 0.77, 0.41, 0.38],
        "q3 (long)":  [0.45, 0.39, 0.81, 0.44],
```

```
        "q4 (long)":  [0.36, 0.40, 0.48, 0.79],
    },
}

# Which document each query is actually about. This is the part only you can
# supply, and the chapter's whole argument is that it is the input that matters.
ANSWER = {"q1 (short)": "short-1", "q2 (short)": "short-2",
          "q3 (long)": "long-1",  "q4 (long)": "long-2"}

# Two ways of grouping the same four queries. Which grouping you use is
# itself a decision about your data, and that is what the second one shows.
BY_LENGTH   = {"q1 (short)": "short", "q2 (short)": "short",
               "q3 (long)": "long",   "q4 (long)": "long"}
BY_LANGUAGE = {"q1 (short)": "en", "q3 (long)": "en",
               "q2 (short)": "de", "q4 (long)": "de"}

def score(rows):
    """Which queries each model got top-1 right."""
    return {q: DOCS[max(range(len(DOCS)), key=lambda i: s[i])] == ANSWER[q]
            for q, s in rows.items()}

def tally(hits, grouping):
    out = {}
    for query, group in grouping.items():
        got, total = out.get(group, (0, 0))
        out[group] = (got + hits[query], total + 1)
    return "   ".join(f"{g} {a}/{b}" for g, (a, b) in sorted(out.items()))

for place, (model, rows) in enumerate(SCORES.items()):
    hits = score(rows)
    if place:
        print()
    for query, scores in rows.items():
        top = DOCS[max(range(len(DOCS)), key=lambda i: scores[i])]
        print(f"  {model}  {query}  top: {top:8}"
              f" {'ok' if hits[query] else 'MISSED'}")
    print(f"  {model}  overall {sum(hits.values())}/4")
    print(f"    by length    {tally(hits, BY_LENGTH)}")
    print(f"    by language  {tally(hits, BY_LANGUAGE)}")
```

```
  model A  q1 (short)  top: short-1  ok
  model A  q2 (short)  top: short-2  ok
  model A  q3 (long)  top: short-1  MISSED
  model A  q4 (long)  top: long-2   ok
  model A  overall 3/4
    by length    long 1/2   short 2/2
```

```
  by language  de 2/2   en 1/2

model B  q1 (short)  top: short-2  MISSED
model B  q2 (short)  top: short-2  ok
model B  q3 (long)  top: long-1   ok
model B  q4 (long)  top: long-2   ok
model B  overall 3/4
  by length    long 2/2   short 1/2
  by language  de 2/2   en 1/2
```

Both models score three out of four, and they are not interchangeable. A is perfect on the short documents and misses one of the long; B is the other way round. The overall figure — the one a benchmark gives you — is identical, so it cannot choose between them.

And now look at the second breakdown, because it is the one that catches people. Grouped by language the two models are *also* identical: de 2/2 and en 1/2, both of them. Grouped by length they are opposites. Same four queries, same eight results, two groupings, and only one of them distinguishes the models at all.

So the breakdown is not a trick that rescues you from the average. **It only separates the models if you group by a property they differ on**, and choosing that grouping is a judgement you make about your corpus before you run anything. Group by the wrong property and you are no closer to choosing than the headline left you — and you learn one real thing, that the property does not discriminate, with the added hazard of feeling thorough.

And look at what the program needed that no benchmark can give you. Not the vectors: those came from the models. The `ANSWER` line — your **answer key**, which document each query is actually about. That is the only input here that has to come from you, it is the thing that makes the measurement about *your* data, and it is also the thing people skip, because writing down forty queries and their answers is dull and reading a leaderboard is quick.



Counting how often the right document came top is the crudest possible score, and it is deliberately crude here. It ignores whether the right answer came second instead of tenth, which usually matters. **Chapter 26 builds the proper ranking measures from zero** — recall at k, mean reciprocal rank, normalised discounted cumulative gain — and Chapter 20 is how you tell whether a difference between two models is real rather than luck on four queries. Four queries decides nothing; the shape of the argument is what this program is for.

What a Public Benchmark Score Is

Benchmark tables for embedding models are genuinely useful and they are routinely read for more than they say. Put one on Chapter 6's ladder and the answer falls out.

A public benchmark is *one study*, run once, on a fixed set of documents and queries that somebody else chose. It supports exactly what Chapter 6 says one study supports: that it worked there. What it does not tell you is how the ordering transfers to a different language mix, a different domain, a different document length, or your queries — and the chapter you just read shows two models tying on a headline while differing completely underneath.

It is still the right place to start, and saying only the sceptical half is its own mistake. A leaderboard narrows a field of hundreds to a shortlist of three or four in an afternoon, and nothing else does that. Use it for the shortlist; decide with your own data.

Four things to look at while you are there, and none of them is the top line:

Look at	Because
Which tasks the score averages	Retrieval, classification and clustering are different jobs, and an average hides which one it won
The languages in the mix	A high average can be an English score with a multilingual tax
The document lengths	A model that tops a sentence-similarity table may not hold a page
Vector width and cost	3,072 numbers per document is four times the storage of 768, for a gain you have not measured



The specific benchmarks, which models lead them, and the vector widths on offer are the fastest-moving facts in this chapter and will be stale sooner than anything else in this book. The method survives: shortlist from the leaderboard, measure on your own set, break the result down by the property of your data you most care about.

Lab: Measure Two Models on Your Own Terms

Three programs, pure Python, nothing to install. Thirty minutes, and there is a key for every step. Work in a new empty folder.

Step 1. Save the first program as `close.py` and run it.

Expected result: 0.978, 0.439 and 0.386, in that order. Check the first by hand against the displayed arithmetic.

Step 2. Multiply every number in *my plane left late* by 10 and run it again.

Expected result: **nothing changes.** All three cosines are identical to three decimal places, because the cosine divides by the lengths, which is Chapter 8's result and the reason cosine is the standard choice here. Note what this does *not* show: nothing says a longer document gets a longer vector, and many models return vectors of length one anyway, so do not reach for this when you want to know why a page and a sentence are comparable.

Step 3. Now add a fourth sentence whose vector is `[0.80, 0.50, 0.20, 0.10]` — identical to the first — but call it something unrelated, say *the coffee was cold*. Run it again.

Expected result: the output grows from three pairs to six, because four sentences make six pairs — and two of those six are worth reading twice. *The flight was delayed* and *the coffee was cold* sit at exactly 1.000, and *my plane left late* and *the coffee was cold* sit at 0.978, which is the number the chapter's paraphrase pair got. You have not broken anything; you stated that two texts have the same embedding and the arithmetic believed you. The lesson is about where trust lives: the cosine is a fact about two vectors, and whether those vectors deserved to be that close is a fact about the model, which no arithmetic on the output can tell you.

Step 4. Save the ranking program as `rank.py` in the same folder and run it.

Expected result: the chapter's three lines. If you see `close.py`'s own output first, its demo loop is not under `if __name__ == "__main__":`.

Step 5. Change `QUERY` to `[0.20, 0.15, 0.85, 0.45]` and run it again.

Expected result: the flight attendant was kind comes first, at 0.996, and the other two fall to 0.467 and 0.430. Nothing about the ranking code changed — you moved the query, and the order followed. Worth doing once because it makes concrete that the index is fixed and the query is what moves.

Step 6. Save the comparison program as `choose.py` and run it. Then answer: if your corpus is two thousand pages of contracts, which model do you pick, and what does the overall score tell you?

Expected result: both models go 3/4 overall; A is 2/2 on short and 1/2 on long, B the reverse. On this evidence you would try *B* first, because your documents are long. The overall score tells you nothing at all — it is the same for both — and that is the point of the step. Four queries is not enough to decide, as the Note above says; it is enough to show you which number to look at.

Step 7. In `choose.py`, swap `BY_LANGUAGE` so that q1 and q2 are `en` and q3 and q4 are `de`. Run it. What happens, and what does it tell you about breakdowns?

Expected result: model A reads en 2/2, de 1/2 and model B reads en 1/2, de 2/2 — so grouped this way the two models now look different, where under the chapter's own grouping they looked identical. Nothing about either model changed. You relabelled the queries, and the grouping that distinguishes two models is a property of how you cut the data, not of the models. Which is why the grouping has to come from what you know about your corpus, and not from whichever cut makes the numbers look decisive.

Step 8. In `choose.py`, change `ANSWER` so that `q3 (long)` points at `short-1` instead of `long-1`. Run it again. What happens, and what have you actually done?

Expected result: model A now scores 4/4 and model B 2/4, so A looks clearly better. What you did was change the labels, not the models — and this is the one lever in the whole procedure that is yours, which is why the answers have to be written down by someone who knows the documents, before anybody runs a model. A wrong answer key does not produce an error; it produces a confident ranking of models, pointing the wrong way.

Step 9. A colleague picks the top model from a benchmark table and asks why you want to spend two days measuring. Give the one-sentence answer, and then the concession.

Expected result: because the table's ordering was produced on somebody else's documents and queries, and the chapter's own example has two models with identical headline scores that are not interchangeable — so the table cannot tell you which to pick, only which three to try. The concession matters: the leaderboard did the expensive part, which is narrowing hundreds to a shortlist, and two days is the right amount of work only for the shortlist.

Summary, Key Terms, and Review Questions

Summary

An **embedding model** is a separate model with a different job from a generative one: one piece of text in, one vector of fixed width out, every time — not Chapter 9's internal *embedding table*, which is not what you search with. The vector is that text's **embedding**, and this book does not explain why its arrangement carries anything useful; that is representation learning, it is on Chapter 17's list, and nothing in these chapters needs it. What you can check is what the numbers do. Chapter 8's *cosine similarity* is the tool, and it is scale-free. What makes a page and a sentence comparable is not that, but the fixed width. Applied to three sentences: *the flight was delayed* and *my plane left late* share no content words and sit at **0.978** on the chapter's stipulated vectors, while *the flight was delayed* and *the flight attendant was kind* share three words and sit at **0.439**. Those vectors were written down rather than measured, which is why the second half of the chapter exists. Read that correctly — the program did not notice that two sentences mean the same thing; it placed them close, which is a measurable fact about its output and enough to rank on, and **not** evidence of meaning. Chapter 25 owns the gap between closeness and relevance. Embed the query with the same model, sort by cosine, take the top few, and that is dense retrieval complete. A raw cosine has **no meaning on its own** — not across models, not across queries, not across corpora — so rank by it and do not threshold on it. Every embedding model also has a **maximum input length**, past which your text is cut off with no error, and that is the first thing to find out. **The best model is a property of your data**, through language, domain and document length, so measure: two candidate models over four documents and four queries both scored 3/4 overall while A went 2/2 on short documents and 1/2 on long, and B the reverse. Grouped by language, though, the two were *identical* — so a breakdown only helps when you group by the property that matters for your corpus, and choosing that property is a judgement you make before you run anything. The one input no benchmark can supply is **which document each query is actually about**, and it is the one people skip. A public benchmark is *one study* on Chapter 6's ladder — it worked there, on somebody else's corpus — which makes it the right way to shortlist three models in an afternoon and the wrong way to pick one.

Key Terms

Term	Meaning
Embedding model	A model whose whole job is to turn one piece of text into one vector of fixed width. No output tokens, nothing to continue
Embedding	The vector an embedding model returns for a given text. Its width is the model's, and it is the same for a word, a sentence and a page
Dense retrieval	Embedding every document once, embedding the query at search time with the *same* model, and ranking by cosine
Answer key	Which document each of your queries is actually about. The only input in the procedure that has to come from you

Worth Memorising

Item	Value or formula
What you get back	One vector, fixed width, per text. A few hundred to a few thousand numbers
The comparison	Chapter 8's cosine. Scale-free: multiplying a vector by 10 does not move it
What a high cosine is	A fact about two vectors. **Not** evidence that two texts mean the same
Dense retrieval	Embed the documents once, embed the query per search with the same model, sort by cosine
What decides the model	Your language, your domain, your document length. Length first: it is the one that gets forgotten
The hard limit	Every embedding model has a maximum input length. Past it, text is cut off, usually with no error
A raw cosine	Has no meaning on its own. Not comparable across models, queries or corpora — rank, do not threshold
What a benchmark is	One study, somebody else's corpus. A shortlist of three, not a choice of one
The input only you have	The answer key: which document each query is about

Review Questions

1. How would you choose an embedding model for your own data?
2. Why is a public benchmark score a starting point rather than a verdict?
3. Two sentences with no words in common get a cosine of 0.978. What has that established?

4. What is the difference between the embedding table in Chapter 9 and an embedding model?
5. Your documents are 40-page contracts. What does that fact alone tell you about model choice?

Review Answers

1. Shortlist from a public table, then measure the shortlist on your own documents with your own queries. The measurement needs one thing nobody can give you: an answer key, which is which document each query is actually about — forty or so queries, written down by somebody who knows the corpus, before any model is run. Then score each candidate and, crucially, **break the score down by whatever property of your data you most care about**, because an overall figure can be identical for two models that are not interchangeable: the chapter's two both score 3/4, while one is 2/2 on short documents and 1/2 on long and the other is the reverse. The properties worth breaking down by are language, domain and document length, and length is the one that gets forgotten. Two things to add unprompted: the crude score of *did the right document come top* is enough to show the shape but not enough to decide, so use Chapter 26's ranking measures and Chapter 20's interval to say whether a difference is real; and the choice interacts with how you split your documents, which is Chapter 27.
2. Because it is one study, in Chapter 6's sense, run on a fixed set of documents and queries somebody else chose, so what it supports is that those models ranked that way *there*. Your language mix, your domain and your document lengths are all different, and the ordering is not guaranteed to survive any of them — the chapter's own pair of models have identical overall scores and opposite strengths, which is what an average can do to a difference that matters. Say the concession too, because it is the part that shows you have used one: a leaderboard narrows hundreds of models to three or four in an afternoon and nothing else does that, so it is exactly right for the shortlist and wrong for the decision. And when you are reading one, look at what the average is averaging, at the language mix, at the document lengths and at the vector width, rather than at the top line.
3. That the model placed those two texts close together. That is a fact about the model's output, it is measurable, and it is enough to rank on — which is the whole basis of dense retrieval. It is **not** evidence that the model understood the sentences or that they mean the same thing, and the distinction is not pedantry: you can hand two unrelated texts the same vector and the cosine will report 1.000 quite happily, because the arithmetic has no access to anything but the numbers. Whether closeness tracks the relevance you actually want is a separate question with a genuinely unsatisfying answer, and Chapter 25 owns it.
4. Different models, different jobs, different outputs. Chapter 9's embedding table is a matrix *inside* a generative model with one row per vocabulary entry; looking a token up in it is how that token enters the computation. It is internal, it is per-token, and it is not

what you search with. An embedding model is a model of its own: you give it a piece of text of any length and it returns one vector of fixed width, with no output tokens and nothing to continue. The shared word is unfortunate and the two are not interchangeable in any direction.

5. First, that you need the maximum input length of every candidate before anything else, because forty pages will exceed most of them and the usual failure is silent: you get a vector describing the first part of the contract and no error. Then: document length is your first filter on quality too, and most of the shortlist will be wrong for you. A model fitted on sentences has to compress forty pages into the same fixed width as a sentence, and models differ a great deal in how well they do that — so a sentence-similarity leaderboard is close to useless for you, and you want candidates evaluated on long-document retrieval. It also tells you the choice is entangled with a decision you have not made yet: how to split the contracts before embedding them, because you will almost certainly not embed forty pages as one vector. That is Chapter 27, and the two decisions should be made together rather than in sequence.

> **You understand this when** you can say what a cosine of 0.978 does and does not establish, and name the one input a benchmark cannot give you.

CHAPTER 25

Where Dense Search Fails

Chapter 24 built dense retrieval and left a debt: it said twice that a high cosine is a fact about two vectors and not evidence that two texts mean the same, and handed the gap here. This chapter pays it, and what you get for it is the ability to say where ranking by closeness fails and why, to tell which of those failures is a derivation and which is an observation, and to know what a page of plausible results does and does not entitle you to conclude.

The problem. My semantic search is clever about meaning and completely useless when I search for a part number.

The question. Where does ranking by closeness fail, why, and what can you tell about it by looking?

A note on the first word of the problem. **Semantic search** is what people usually call this, and it is the phrase you will hear; Chapter 24's term for the mechanism is dense retrieval, and they mean the same thing. This chapter uses the book's term except where it is quoting the complaint.

A Part Number Is Almost No Signal

Take two maintenance notes that differ in one token out of ten, and a query naming one of the two parts.

```
import math

def cosine(a, b):
    dot = sum(x * y for x, y in zip(a, b))
    return dot / (math.sqrt(sum(x*x for x in a)) * math.sqrt(sum(y*y for y in b)))

# Two maintenance notes that differ in one token out of ten, and a query
# naming one of the two parts. The vectors are written out; they sit close
# together for the reason the chapter gives, not because that is convenient.
B = [0.71, 0.52, 0.30, 0.18]      # ...pump X-4471-B before servicing
D = [0.70, 0.53, 0.30, 0.19]      # ...pump X-4471-D before servicing
SCHEDULE = [0.64, 0.49, 0.33, 0.22]
QUERY = [0.69, 0.54, 0.29, 0.20]  # "X-4471-D seal replacement"

DOCS = {"X-4471-B note": B, "X-4471-D note": D, "maintenance schedule": SCHEDULE}

if __name__ == "__main__":
    print(f"  the two notes, compared with each other: {cosine(B, D):.5f}")
    print(f"  the whole distance between them: {1 - cosine(B, D):.5f}\n")

    ranked = sorted(DOCS, key=lambda d: cosine(QUERY, DOCS[d]), reverse=True)
    for place, doc in enumerate(ranked, 1):
        print(f"  {place}  {cosine(QUERY, DOCS[doc]):.5f}  {doc}")
    a, b = (cosine(QUERY, DOCS[d]) for d in ranked[:2])
    print(f"\n  the right note wins by {a - b:.5f}")
```

```
  the two notes, compared with each other: 0.99983
  the whole distance between them: 0.00017

  1  0.99978  X-4471-D note
  2  0.99928  X-4471-B note
  3  0.99712  maintenance schedule

  the right note wins by 0.00050
```

A reader doing the arithmetic will ask how the winning margin, 0.00050, can be three times the distance between the two notes, 0.00017. The margin is not a distance between a pair at all: it is the *difference of two distances from the query*, 0.00072 for the wrong note against 0.00022 for the right one. The query sits a little to one side of the pair, so it is slightly nearer one of them, and that difference is what the margin reports. What matters is that all three numbers are tiny.

The right note came first, and look at what by. Five parts in ten thousand, out of a distance between the two notes of 0.00017.

Be careful what that demonstrates, because it is less than it looks. These vectors were

written down, so the chapter has not shown that a model *does* place two such texts this close. What it shows is the consequence *if* it does, and the consequence is the point: when two documents sit 0.00017 apart, a search that ranks by cosine has nothing else to rank them on. The ordering is decided by the residual, and five parts in ten thousand is not a quantity any model is stable to.

That dense retrieval loses to lexical retrieval on identifier queries is an observation, not a derivation, and it belongs on Chapter 6's ladder rather than off it. Where on the ladder is not something this chapter can settle for you: that would take the practitioner reports and the written-down failure modes, and the honest statement is that the claim is widely made and that you should check it on your own identifier queries, which Step 8 of the lab asks you to do.

What a mechanism can offer in its place is a *sketch* of why, and the sketch is Chapter 24's fixed width — one vector of a few hundred numbers has to stand for the whole text, and a string appearing once has to compete for that room with everything else in the document. That is a reason to expect the failure. It is a sketch and not a calculation, and the reason it cannot be a calculation is simple: this book never says how the tokens become one vector. It declines to, in Chapter 9's Warning and again in Chapter 24's. Without that step there is no arithmetic to do, so nothing here computes what a token is "worth" — and the 0.00017 above is a distance between two stipulated vectors, not a token's share of anything.

The cases are the familiar ones: a part number, a surname, an invoice number, an error code, a SKU, a date, a version string. The thing you are searching *for* is a small part of the thing you are searching *with*.

And Chapter 10 has something to add, though not what you might expect. A rare code is not one token: `X-4471-B` is cut into several pieces, because no corpus saw it often enough to merge it. That raises the number of tokens it occupies and lowers the specificity of each — the pieces are mostly common fragments, a letter, a hyphen, a two-digit group, each appearing in countless other strings — so little of what survives is specific to *this* code.



The vectors here are written out, as Chapter 24's were, and for the same reason: a real embedding model is not available to a lab that installs nothing. Be exact about what that costs. The premise — that a model places two texts differing in one character close together — is an observation about trained models, and this book cannot derive it; it is the kind of claim Chapter 9's Warning and Chapter 24's both decline to explain. What *is* derivable, and what this section turns on, is the consequence: given two documents that close together, a cosine has nothing left to separate them with. Check the premise on your own data in one line as soon as you have a model.

Similarity Is Symmetric. Relevance Is Not.

Now the second failure. Part of this one is a derivation, and the derivation is the half that needs no numbers at all — so read the printed numbers as the illustration and the paragraph after them as the argument.

```
from codes import cosine

QUERY    = [0.62, 0.55, 0.31, 0.22]      # "was my refund approved?"
APPROVED = [0.64, 0.54, 0.30, 0.21]      # "Your refund was approved."
REFUSED  = [0.63, 0.55, 0.31, 0.20]      # "Your refund was not approved."

print(f"  approved  {cosine(QUERY, APPROVED):.5f}")
print(f"  refused   {cosine(QUERY, REFUSED):.5f}")
print(f"  and the two answers, compared with each other:"
      f" {cosine(APPROVED, REFUSED):.5f}\n")

# Cosine is symmetric. Relevance is not.
print(f"  cos(query, approved) = {cosine(QUERY, APPROVED):.5f}")
print(f"  cos(approved, query) = {cosine(APPROVED, QUERY):.5f}")
print(f"  identical: {cosine(QUERY, APPROVED) == cosine(APPROVED, QUERY)}")
```

```
  approved  0.99958
  refused   0.99970
  and the two answers, compared with each other: 0.99976

  cos(query, approved) = 0.99958
  cos(approved, query) = 0.99958
  identical: True
```

Read the third number first. *Your refund was approved* and *Your refund was not approved* are 0.99976 similar to each other — **closer to each other than either is to the query.** They are opposite in meaning and nearly identical as vectors, because *not* is one token among five.

So the ordering between the true answer and its negation is decided by whatever is left after that. Here the negation won; the point is not that it always will, and these are typed numbers.

The claim that survives is about the score rather than the outcome, and it is worth being careful about how it is put. It is tempting to say a cosine has no term representing polarity, but that proves far too much: a cosine has no term representing *anything*, since no single number in these vectors means a nameable thing — Chapter 9 says so outright. The same sentence would therefore deny the method any handle on topic, and Chapter 24's paraphrase with no shared words refutes that.

The narrow claim is about what the number gives you to work with. Whatever separates

these two sentences is whatever the model left after *not*, and that residual reaches you only as one number that also carries everything else about both texts. So you have *no handle on that ordering*: you cannot tune it, you cannot put a threshold on it, and you cannot tell from the score which of the two documents you are holding. A search that cannot tell *approved* from *not approved* is not a search for whether your refund was approved, and no amount of looking at the scores will tell you which case you are in.

Now the structural half, which holds for every model and needs no numbers. Cosine is symmetric: $\cos(a, b) = \cos(b, a)$, as the last two lines show, and it follows immediately from Chapter 8's formula: the dot product is symmetric, and the denominator is the two lengths multiplied together, which multiplication leaves unchanged.

Relevance is not symmetric. That word is worth pinning down, because it is what dense retrieval is estimating: **relevance** is whether this document answers that question, for this asker. A one-line answer can be exactly what a long question needs, while that long question is useless as an answer to the one-line query. A glossary entry is relevant to *what does idempotent mean*; the question is not relevant to the entry. A summary answers a request for a summary, and the document it summarises does not.

So similarity and relevance have different shapes. **One is a symmetric function of two texts. The other is a directed judgement about whether this document answers that question, and it depends on who is asking and why.**

Here is the impossibility that follows, stated no wider than it holds. Put two texts through *the same* encoder and compare them with a cosine, and you get one number — so that arrangement cannot distinguish *x answers y* from *y answers x*. The question has a direction and the number does not.

And here is what removes it, because an argument like that is only worth having if you know its escapes. Score the query and the document *together*, with a model that sees both at once, and the score is free to be asymmetric — that is Chapter 26's reranking. Or encode the two sides differently, with separate query and document encoders, or with a marker that tells one model which side it is embedding: then the score is $\cos(f(q), g(d))$ with $f \neq g$, which is *not* symmetric in the two texts.



That separately encoded sides are common in current retrieval models is a statement about today's practice, not a lasting principle, and it is the kind of detail to check rather than memorise. The lasting part is the shape of the argument: one encoder and a cosine give you one symmetric number, and two encoders or a joint score do not.

So the honest conclusion is narrow: a single shared encoder compared by cosine is the wrong shape for a directed question, and getting the right shape means changing the arrangement rather than improving the model.



This is the sentence to have ready, because it is the question's real answer rather than a list of failure cases: similarity is a symmetric number computed from two texts, and relevance is a directed judgement that depends on the asker. Dense retrieval uses the first as an estimate of the second, which is a useful approximation and never an identity. Then carry the edge, because it is what shows you have used this: the symmetry goes away as soon as the two sides are encoded differently or scored together, so the limit is a property of *one shared encoder with a cosine*, not of retrieval.

Why You Cannot Judge This by Eye

Here is the failure that costs the most, because it is the one that hides.

```
from codes import cosine

QUERY = [0.70, 0.50, 0.30, 0.20]   # "can I get a refund on an order
                                   #  I cancelled six weeks ago?"

# Six documents. Only one of them answers that question, and you know which
# because somebody wrote it down - not because anything here can tell.
DOCS = {
    "overview of refunds":        [0.71, 0.51, 0.29, 0.19],
    "refund policy summary":      [0.70, 0.52, 0.30, 0.18],
    "how refunds are calculated": [0.69, 0.49, 0.31, 0.21],
    "shipping and returns":       [0.68, 0.50, 0.32, 0.20],
    "payment methods":            [0.66, 0.48, 0.33, 0.23],
    "refund for a cancelled order after 30 days": [0.55, 0.61, 0.38, 0.30],
}
ANSWERS = {"refund for a cancelled order after 30 days"}

ranked = sorted(DOCS, key=lambda d: cosine(QUERY, DOCS[d]), reverse=True)
print("  what you would look at - the top three:")
for place, doc in enumerate(ranked[:3], 1):
    print(f"    {place}  {cosine(QUERY, DOCS[doc]):.4f}  {doc}")
print("\n  where the one that answers the question actually came:")
for place, doc in enumerate(ranked, 1):
    if doc in ANSWERS:
        print(f"    {place}  {cosine(QUERY, DOCS[doc]):.4f}  {doc}")
```

```
what you would look at - the top three:
  1  0.9998  overview of refunds
  2  0.9998  how refunds are calculated
  3  0.9996  shipping and returns

where the one that answers the question actually came:
  6  0.9716  refund for a cancelled order after 30 days
```

The top two print as the same number, 0.9998 against 0.9998. Printed to ten places they are 0.9998056809 and 0.9997993270, so the difference between them is about six parts in a million — Chapter 24's near-tie, arriving where it matters. Which of those two comes first is not a fact about the documents.

Now look at the top three and ask yourself whether anything is wrong. *Overview of refunds*, *how refunds are calculated*, *shipping and returns* — for a question about a refund, those are three entirely reasonable results. Nothing in them is absurd. Nothing in them is off-topic. There is nothing to notice.

And the only document that answers the question is sixth of six. Last.

This is the asymmetry that makes retrieval hard to improve by inspection:

What you can see by looking	What you cannot
Whether the results you got are plausible	Whether the result you needed was there
That nothing in the top few is obviously absurd	What the top few displaced
That the scores are high	That a high score is not evidence of meaning — which is this chapter's own conclusion, and the negation case is why

Everything in the first column is about the documents in front of you. Everything that matters is about the one that is not. A missing document leaves no trace in a result list. There is no gap where it should have been, no warning, no low score — the list is simply three plausible things, which is exactly what a good list also looks like.

And this one does not rest on the invented vectors, which is worth saying because the other two sections had to be careful about exactly that. Delete the answering document and the top three cannot change, because removing an item ranked below third cannot reorder the three above it — for any numbers whatsoever. A corpus that never held the answer and a ranking that buried it therefore produce the *same* result list, so no inspection of that list can tell them apart. The vectors decide which case this example is; they do not decide that the two cases look alike.

That is why the most common report about a search system is that it *seems* fine, and why seeming fine is consistent with being badly broken.

What You Can Conclude Without an Answer Key

So what does looking buy you? Something, and it is worth naming, because the answer is not *nothing*.

You can conclude that a result is wrong. If a query about refunds returns a page about payroll, you have found a real defect by inspection, and no judgements were needed. Inspection is a sound way to find *present* errors.

From the result list, you can conclude nothing about what is missing. Not whether anything is, not how much, not which queries are affected. The information is not in the list, and no amount of staring adds it. Someone who knows the corpus *can* tell you a document is missing — but they are using knowledge from outside the list, which is the whole point, and writing that knowledge down is what the next section is about.

The thing that closes the gap is the one Chapter 24 named: an *answer key* — for each query, which documents actually answer it, written down by somebody who knows the corpus, before any search is run. Each of those recorded decisions is a **relevance judgement**, and they are what the key is made of. With it you can say how often the right document was found and where it ranked. Without it, every statement of the form *our search works well* is a statement about how the results looked to whoever last looked.

Chapter 26 turns that key into measures — it builds the standard ranking measures from zero and names what each one does and does not capture. Chapter 20 is how you decide whether a difference between two systems is real rather than luck on a handful of queries. This chapter's job is narrower: to establish that the key is not optional, and why.



The expensive version of this mistake is a demonstration. Somebody runs five queries they thought of, in front of a room, and the results look reasonable — because results usually do. That is a demonstration on Chapter 6's ladder, which supports "it can work once", and the five queries were chosen by the person showing you. It is not evidence about the other ten thousand queries, and the thing it is least able to detect is the thing most likely to be wrong.

And What to Do About It

This chapter is a list of limits, so it owes you the shape of the answer even though it owns none of it.

Failure	Where it is handled
Part numbers, names, codes, rare words	Chapter 26: lexical retrieval, which matches the words and strings as written, combined with dense
Similarity is not relevance	Chapter 26's reranking, which scores a query and a document *together* rather than comparing two vectors
You cannot see what is missing	Chapter 26's measures, on Chapter 24's answer key
Documents too long or badly split to match	Chapter 27 — not a limit of dense retrieval, a limit of the pipeline around it
The right passage retrieved and still not used	Chapter 28 — the same

None of this makes dense retrieval a bad idea. Matching two texts with no words in common is something lexical retrieval cannot do at all, and dense retrieval can — that capability is real, and Chapter 24's vectors were stipulated, so take the capability on its own terms rather than from that demonstration. The conclusion is narrower and more useful: **dense retrieval is one signal, it is strong on meaning and weak on identifiers, it is symmetric where the task is directed, and its failures are invisible to inspection.** Each of those last three limits is what Chapter 26 takes up; Chapters 27 and 28 handle the two pipeline failures in the table above, which are not limits of dense retrieval at all.

Lab: Find the Failures Yourself

Three programs, pure Python, nothing to install. Thirty minutes, and there is a key for every step. Work in a new empty folder.

Step 1. Save the first program as `codes.py` and run it.

Expected result: the two notes at 0.99983 of each other, so a distance of 0.00017 between them, and the right note winning by 0.00050.

Step 2. Change `D`'s second number from `0.53` to `0.50` and run it again.

Expected result: the wrong note now comes first — `X-4471-B` at 0.99928 against `X-4471-D` at 0.99903 — and the two notes are still 0.99981 of each other, so they are still describing almost the same text. Three hundredths on one of four numbers was enough.

And read the last line of the output, which now says something false: *the right note wins by 0.00025*. It says that because the program takes the top result and calls it right, which is

the assumption this whole chapter is attacking, written into a print statement. Fix the label if you like; the more useful thing is to notice how easily that assumption gets into code, and that nothing failed when it did.

Step 3. Save the second program as `negate.py` in the same folder and run it.

Expected result: approved 0.99958, refused 0.99970, the two answers 0.99976 of each other, and `identical: True` for the two directions of the cosine. If you see `codes.py`'s own output first, its demo is not under `if __name__ == "__main__":`.

Step 4. Which number in that output is the one to quote, and why?

Expected result: 0.99976 — the two answers compared with each other. It is higher than either answer's similarity to the query, which says the true answer and its negation are closer to each other than either is to the question. That is the fact that makes the ordering between them arbitrary. The 0.99970 beating 0.99958 is just which way the arbitrary fell this time.

Step 5. Save the third program as `eye.py` and run it. Then cover the second half of the output and ask yourself, from the top three alone, whether this search is working.

Expected result: the top three are *overview of refunds*, *how refunds are calculated* and *shipping and returns*, and from them alone the honest answer is that you cannot tell. They are plausible. The document that answers the question is sixth. Nothing in the first three indicates that, and that is the whole point of the step.

Step 6. In `eye.py`, delete the line for *refund for a cancelled order after 30 days* entirely, and run it again.

Expected result: the top three are *identical* — and notice this is not an experiment, it is a consequence, the one the body states. Removing an item ranked below third cannot change the top three, for any numbers whatsoever. That is why this section does not depend on the invented vectors: a corpus missing the answer and a ranking burying it produce the same top three, so neither can be diagnosed from one. The one thing that does change is the last line, which now has a heading and nothing under it.

Step 7. Your colleague demonstrates the search with five queries and all twenty-five results look sensible. What have you learned, and what would you ask for?

Expected result: that the system returns plausible results on five queries somebody chose, which on Chapter 6's ladder is a demonstration and supports *it can work once*. You have learned nothing about what those five queries missed, and nothing about the queries nobody tried. Ask for an answer key — a set of queries with the documents that actually answer them, written down independently — and for where the right document ranked on each.

Step 8. Give an example of a query that meaning-based search gets wrong, and say why in one sentence of mechanism rather than one of complaint.

Expected result: any identifier will do — *invoice 4471-B*, *error E0432*, *the Nakamura contract* —

and the sketch is that the vector describes the whole text at a fixed width, so a string appearing once has to compete for that room with everything else in the document. Say *sketch* rather than *calculation*, because nothing here computes a token's share. The negation case is the other good answer: *was my refund approved* gives a score that cannot tell you whether you have the answer or its negation, because *not* is one token among five. Then do the check the body asks for: twenty identifier queries from your own data, with where each answer ranked.

Summary, Key Terms, and Review Questions

Summary

Dense retrieval fails in three ways, and they do not stand on the same kind of support. **A part number is almost no signal**: two maintenance notes differing in one token of ten sat 0.99983 from each other — a distance of 0.00017 — and the right note won by 0.00050, which is the difference of two distances from the query rather than a distance between any pair. That dense retrieval loses to lexical retrieval on such queries is an *observation*; what a mechanism offers is a sketch, Chapter 24's fixed width, since one vector must hold the whole text and a string appearing once competes for that room with everything else. It is a sketch and not a calculation, because this book never says how tokens become one vector. Names, invoice numbers, error codes, SKUs and version strings are all that case, and Chapter 10 compounds it, since a rare code is cut into pieces that are mostly common fragments, so little of what survives is specific to this code. **Similarity is symmetric and relevance is not.** *Your refund was approved* and *Your refund was not approved* sat 0.99976 from each other — closer to each other than either was to the question — so what separates them is a residual, and the claim that survives is about the score rather than the outcome: that residual reaches you as one number carrying everything else too, so you cannot tune it, threshold it, or tell from the score which document you hold. Structurally, $\cos(a, b) = \cos(b, a)$ follows from Chapter 8's formula, while relevance is a directed judgement that depends on who is asking: a glossary entry answers *what does idempotent mean* and the question does not answer the entry. A better model narrows that gap and cannot close it, because the two are not the same kind of object. **And the failures are invisible.** A top three of *overview of refunds*, *how refunds are calculated* and *shipping and returns* is entirely plausible while the only document that answers the question sits sixth of six; delete that document and the top three *cannot* change, which is why a buried answer and an absent one produce the same list. Everything inspection can see is about the documents in front of you, and everything that matters is about the one that is not. So you can conclude by eye that a *present* result is wrong, and from the list alone nothing about what is missing — someone who knows the corpus can tell you, and that is knowledge from outside the list. What closes the gap is Chapter 24's *answer key*, written before any search is run; Chapter 26 turns it into measures and Chapter 20 says whether a difference is real. None of this makes dense retrieval a bad idea: match-

ing two texts with no words in common is something lexical retrieval cannot do at all, and that capability is the reason to use it. It makes it *one signal*: strong on meaning, weak on identifiers, symmetric where the task is directed, and failing where you cannot see. Each of those last three is what Chapter 26 takes up.

Key Terms

Term	Meaning
Semantic search	What dense retrieval is usually called. The same thing, in the words people use when they complain about it
Relevance	Whether this document answers that question, for this asker. A directed judgement, not a distance
Relevance judgement	One recorded decision that a given document does or does not answer a given query. The unit an answer key is made of

Worth Memorising

Item	Value or formula
Why identifiers fail	A sketch, not a calculation: the vector holds the whole text at a fixed width, so a string appearing once competes for that room with everything else
What kind of claim that is	An observation on Chapter 6's ladder. The mechanism gives a reason to expect it, not a derivation
The illustration	Two notes differing in one character, on stipulated vectors: 0.99983 apart, so a distance of 0.00017
Similarity against relevance	$\cos(a, b) = \cos(b, a)$ always. Relevance depends on which is the question
The negation case	On stipulated vectors, a true answer and its negation sat closer to each other than either did to the query
What the negation case proves	Nothing about which one ranks first. It is a claim about the score: you get no handle on that ordering, so you cannot tune or threshold it
What inspection finds	A wrong result that is present. Not a right result that is missing from the list
What a five-query demo is	A demonstration on Chapter 6's ladder: it can work once

Review Questions

1. Give me a query that meaning-based search gets wrong, and say why.

2. Why is similarity not the same thing as relevance?
3. Your search returns three plausible results for every query you try. What do you know?
4. A colleague says a better embedding model will fix your part-number problem. Is that right?
5. What is the cheapest thing you can do to find out whether your retrieval is any good?

Review Answers

1. Any identifier. *Invoice 4471-B*, *error code E0432*, *the Nakamura contract* — and the mechanism is the thing to lead with, not the example. The embedding describes the whole text at a fixed width, so a string appearing once has to compete for that room with everything else in the document. Call that a *sketch* of why rather than a calculation, because it takes a step this book does not supply: how the tokens become one vector. On two stipulated vectors a token of ten left the two notes 0.99983 from each other — a distance of 0.00017 — so which of them a query ranks first is decided by a residual that small. Chapter 10's tokeniser compounds it, because a rare code is cut into pieces that are mostly common fragments, so little of what survives is specific to this code. The second good answer is the negation case: for *was my refund approved*, the score cannot tell you whether you are holding the answer or its negation, because *not* is one token among five. And finish with the check, because a sketch is not a plan: write down twenty identifier queries with the documents that actually answer them, and see where each one ranked.
2. Because they are different kinds of object, and the structural argument is better than any list of cases. Similarity is a symmetric number computed from two texts — $\cos(a, b) = \cos(b, a)$ falls straight out of Chapter 8's formula, since the dot product is symmetric and the denominator is the two lengths multiplied together, which multiplication leaves unchanged. Relevance is a *directed* judgement about whether this document answers that question, and it depends on who is asking and what for: a glossary entry is relevant to *what does idempotent mean*, and the question is not relevant to the entry; a summary answers a request for a summary, and the document it summarises does not. Dense retrieval uses the first as an estimate of the second, which is a useful approximation and never an identity. The consequence worth adding is the one that shows you have used this, and it is a statement about the *arrangement* rather than about retrieval: one shared encoder compared by a cosine gives you a single symmetric number, so no better model makes that arrangement able to tell *x answers y* from *y answers x*. Changing the arrangement does. Score the query and the document together, or encode the two sides with different encoders, and the score is free to be asymmetric.
3. That the results are plausible, which is almost what you knew before. Plausible is what a good list looks like and also what a badly broken one looks like, because a missing document leaves no trace in a result list — no gap, no warning, no low score. The chap-

ter's own example has a top three that is entirely reasonable while the one document answering the question sits sixth, and deleting that document leaves the top three unchanged. So what you know is that nothing *present* is obviously wrong, which is worth something: inspection is a sound way to find errors that are there. What you do not know is anything about what is missing — not whether, not how much, not which queries. And if the queries were chosen by whoever is showing you, you have a demonstration in Chapter 6's sense and nothing more.

4. Partly, and the word is doing a lot of work. A better model will narrow the gap, because some models place identifiers better than others and the difference is measurable — Chapter 24 is how you would find out. But the sketch of the problem is about the arrangement rather than the model's quality: the vector has a fixed width and must describe the whole text, so a string appearing once competes for that room with everything else, and a better model still has the same room to allocate. The thing that actually fixes identifier search is a method that matches the words and strings as written, which is lexical retrieval, combined with the dense signal rather than replacing it — Chapter 26. So the honest answer is: a better model is worth measuring and should not be expected to fix this; what fixes it is not a model.
5. Write an answer key. Take thirty or forty real queries — from your logs if you have them — and, for each, have somebody who knows the corpus write down which documents actually answer it, before anybody runs a search. That is the cheapest thing because it is the only missing input: everything else, the retrieval and the scoring, you already have. With it you can say how often the right document was found and where it ranked; without it every claim about your search being good is a claim about how the results looked to whoever last looked. Chapter 26 is the measures you then compute, and Chapter 20 is how you tell whether a difference between two configurations is real rather than luck on forty queries.

> **You understand this when** you can explain why a plausible top three tells you almost nothing, in one sentence that does not use the word *bad* — and say what has to change about the arrangement, not the model, for a symmetric score to answer a directed question.

CHAPTER 26

Lexical, Dense and Hybrid Retrieval

Chapter 25 ended with a list of limits and handed almost all of them here. This chapter pays that debt. What you get for it is a second kind of search built from zero, a way to combine two searches whose scores cannot be compared, the arrangement that lets a score depend on which text is the question, and three measures that turn an answer key into a number — together with what each measure does not see.

The problem. I have two kinds of search that each fail differently, and no way to say which is better.

The question. How does matching words actually work, how do you combine it with matching meaning, and how do you measure whether any of it improved?

Everything in this chapter ranks. Nothing in it thresholds, which is the promise chapter 24 made.

Matching the Words As Written

Lexical retrieval scores a document by the query words it contains. The name is worth keeping separate from "keyword search", because the method is not a filter: it does not ask *which documents contain these words*, it gives every document a number and sorts.

Start with the units. Chapter 10's tokens are the *model's* pieces, chosen by byte-pair encoding, and they are not what a lexical index counts. An index counts terms, and a **term** is whatever the indexer decided to treat as one unit — usually a whitespace-and-punctuation split, lower-cased, sometimes reduced to a stem so that *servicing* and *serviced* collapse together. The eight documents below are split on whitespace and lower-cased, and that is the whole of it.

Three quantities decide a score. The first is the obvious one, and the other two are each there to answer an objection to it.

Term frequency is how often a term appears in one document, written $f_{t,d}$. A document that says *refund* four times is plausibly more about refunds than one that says it once.

Document frequency is how many documents in the collection hold a term at all, written n_t. It exists because term frequency alone rewards the wrong words: *the* appears constantly and tells you nothing. **Inverse document frequency** turns that around, so a term in few documents is worth more than a term in many:

$$\text{IDF}(t) = \ln\left(\frac{N - n_t + 0.5}{n_t + 0.5} + 1\right)$$

N is the number of documents. Read it with chapter 8's method: inside the logarithm is a ratio of *documents without the term* to *documents with it*, so the fraction grows as the term gets rarer, and chapter 8's logarithm compresses the result.

Document length is the third, because term frequency also rewards long documents: a page mentions everything more often than a sentence does, without being more about any of it.

BM25, and the Two Knobs In It

BM25 combines those three. It is the standard lexical scoring function, it has been for decades, and this is the form you will meet:

$$\text{score}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f_{t,d}\,(k_1 + 1)}{f_{t,d} + k_1\left(1 - b + b\,\dfrac{|d|}{\text{avgdl}}\right)}$$

Work outward from the brackets, as chapter 8 says. The sum runs over the query's terms, so each term contributes independently and a document matching two query terms gets both contributions. Each term's contribution is its IDF multiplied by a fraction built from its term frequency. In the denominator, $|d|$ is this document's length and avgdl is the average length across the collection, so the ratio is *how long this document is compared with a typical one.*

k_1 and b are the two knobs, and they are constants somebody chose — conventionally about 1.5 and 0.75. They are not derived from anything, and that matters for how you talk about this formula.

Here it is from zero, on eight documents.

```
import math

# Eight short documents. Lower-cased and split on whitespace - that is the
# whole of the "tokenising" here, and it is not the model's tokeniser.
CORPUS = {
    "d1": "the refund policy for every order",
    "d2": "how the refund on a cancelled order is calculated",
    "d3": "refund refund refund refund refund refund",
    "d4": "replacing the seal on pump X-4471-D before servicing the unit",
    "d5": "replacing the seal on pump X-4471-B before servicing the unit",
    "d6": "shipping and returns for damaged goods",
    "d7": "getting your money back on an order you already cancelled",
    "d8": "the payment methods we accept and when we charge the card",
}

def terms(text):
    return text.lower().split()

DOCS = {d: terms(t) for d, t in CORPUS.items()}
N = len(DOCS)
AVGDL = sum(len(t) for t in DOCS.values()) / N

def df(term):
    return sum(1 for t in DOCS.values() if term in t)

def idf_classic(term):
    n = df(term)
    return math.log((N - n + 0.5) / (n + 0.5))

def idf(term):
    # The +1 inside the logarithm is what stops it going negative.
    n = df(term)
    return math.log((N - n + 0.5) / (n + 0.5) + 1)

def bm25(query, doc, k1=1.5, b=0.75):
    dl = len(DOCS[doc])
    total = 0.0
    for term in terms(query):
        tf = DOCS[doc].count(term)
        if tf == 0:
            continue
        norm = tf + k1 * (1 - b + b * dl / AVGDL)
        total += idf(term) * tf * (k1 + 1) / norm
    return total

if __name__ == "__main__":
```

```
  print(f"  {N} documents, average length {AVGDL:.3f} terms\n")

  print("  what one term is worth, by how many documents hold it:")
  for term in ("the", "order", "refund", "pump", "x-4471-d"):
      print(f"    {term:9s} in {df(term)} of {N}   idf {idf(term):.4f}"
            f"   classic {idf_classic(term):+.4f}")

  print("\n  term frequency saturates: 'refund' repeated, b=0 so length")
  print("  cannot interfere. k1=1.5, so the ceiling is k1+1 = 2.5 idf:")
  k1 = 1.5
  for tf in (1, 2, 4, 8, 100):
      factor = tf * (k1 + 1) / (tf + k1)
      print(f"    tf {tf:3d}  ->  {factor:.4f} idf   "
            f"({factor / (k1 + 1) * 100:.1f}% of the ceiling)")

  print("\n  'refund policy' ranked:")
  ranked = sorted(DOCS, key=lambda d: bm25("refund policy", d), reverse=True)
  for place, d in enumerate(ranked[:4], 1):
      print(f"    {place}  {bm25('refund policy', d):.4f}  {d}"
            f"  {CORPUS[d][:34]}")
```

```
8 documents, average length 8.500 terms

what one term is worth, by how many documents hold it:
  the       in 5 of 8   idf 0.4925   classic -0.4520
  order     in 3 of 8   idf 0.9445   classic +0.4520
  refund    in 3 of 8   idf 0.9445   classic +0.4520
  pump      in 2 of 8   idf 1.2809   classic +0.9555
  x-4471-d  in 1 of 8   idf 1.7918   classic +1.6094

term frequency saturates: 'refund' repeated, b=0 so length
cannot interfere. k1=1.5, so the ceiling is k1+1 = 2.5 idf:
  tf   1  ->  1.0000 idf   (40.0% of the ceiling)
  tf   2  ->  1.4286 idf   (57.1% of the ceiling)
  tf   4  ->  1.8182 idf   (72.7% of the ceiling)
  tf   8  ->  2.1053 idf   (84.2% of the ceiling)
  tf 100  ->  2.4631 idf   (98.5% of the ceiling)

'refund policy' ranked:
  1  3.1536  d1  the refund policy for every order
  2  1.9761  d3  refund refund refund refund refund
  3  0.9201  d2  how the refund on a cancelled orde
  4  0.0000  d4  replacing the seal on pump X-4471-
```

Read the IDF column first, because it is where the rarity is. `x-4471-d` is in one document

of eight and is worth 1.7918. `the` is in five of eight and is worth 0.4925. A query term that nearly every document holds barely moves a score, and that is the whole job of the inverse.

The classic column is a trap worth knowing. The original IDF has no `+1` inside the logarithm, and when a term is in more than half the collection the fraction drops below one, so its logarithm goes *negative*: `the` scores −0.4520. A negative contribution means containing a query term actively *lowers* a document's score, which no one wants. The modern form adds one inside the logarithm, which keeps the fraction above one and the result above zero for every n_t. Adding one is not the only fix — some engines simply floor the IDF at zero instead — and the two differ in what they do to a near-universal term: the floor makes it worth exactly nothing, while the `+1` leaves it worth a little. If you ever see a lexical engine rank a document lower for matching more words, this is the first thing to check.

Now the saturation, which is the part of BM25 people miss. Look at the second block. Going from one mention to two adds 0.43. Going from eight mentions to a hundred adds 0.36 in total. The fraction climbs towards a ceiling and stops: as $f_{t,d}$ grows without limit the term's contribution approaches $(k_1 + 1)$ multiplied by its IDF, which here is 2.5 IDF, and a hundred mentions reach 98.5% of it.

That is why `d3` — the document that is nothing but the word *refund* six times — comes second and not first. It has the highest term frequency in the collection and it still loses to `d1`, which matches both query terms once each.

Both of these are design choices with stated properties, not derivations. Nothing forces a saturating curve of that particular shape, and nothing derives 1.5 and 0.75; they were tuned on test collections, and other shapes exist. What is worth carrying into an interview is not the constants but the three objections the formula answers in order, and the fact that term frequency saturates — because the property is what makes BM25 hard to fool, and the constants are details that will differ between engines.



k_1 and b are tunable, and the defaults above are what most engines ship. They are a dated detail in chapter 6's sense, not a lasting principle. The lasting parts are the three quantities and the saturation.

Why the Two Scores Cannot Simply Be Added

You now have two searches. Chapter 24's dense retrieval scores by cosine, and BM25 scores by matched terms. The obvious move is to add the two numbers, and it is wrong for a reason chapter 24 already established: **a raw cosine has no meaning on its own**, and neither does a BM25 score.

```
from bm25 import CORPUS, bm25

# Two rankings of the same eight documents for one query. The lexical one is
# BM25 from the first program. The dense one is a stipulated cosine, as in
# chapters 24 and 25 - nothing here shows what a real model would return.
QUERY = "refund on a cancelled order"
DENSE = {"d7": 0.9891, "d2": 0.9874, "d1": 0.9702, "d6": 0.9312,
         "d8": 0.9145, "d3": 0.8871, "d4": 0.6203, "d5": 0.6198}

def ranks(scores):
    order = sorted(scores, key=lambda d: scores[d], reverse=True)
    return {d: place for place, d in enumerate(order, 1)}

def rrf(rank_lists, k=60):
    out = {}
    for rl in rank_lists:
        for doc, place in rl.items():
            out[doc] = out.get(doc, 0.0) + 1 / (k + place)
    return out

if __name__ == "__main__":
    lex = {d: bm25(QUERY, d) for d in CORPUS}
    lex_r, dense_r = ranks(lex), ranks(DENSE)

    print(f"  query: {QUERY}\n")
    print("  doc    BM25      rank     cosine    rank")
    for d in sorted(CORPUS):
        print(f"  {d}    {lex[d]:6.4f}     {lex_r[d]}      "
              f"{DENSE[d]:.4f}     {dense_r[d]}")

    print("\n  not one scale. BM25 here runs"
          f" {min(lex.values()):.4f} to {max(lex.values()):.4f},")
    print(f"  the cosines {min(DENSE.values()):.4f} to"
          f" {max(DENSE.values()):.4f}. Adding them lets one")
    print("  drown the other, and neither range means anything.")

    fused = rrf([lex_r, dense_r])
    print("\n  reciprocal rank fusion, k=60. Ranks only, no scores:")
    for place, d in enumerate(sorted(fused, key=lambda x: fused[x],
                                     reverse=True), 1):
        print(f"    {place}  {fused[d]:.6f}  {d}    "
              f"(lexical {lex_r[d]}, dense {dense_r[d]})")

    print("\n  one document's arithmetic, to check by hand:")
    d = "d7"
    print(f"    {d}: 1/(60+{lex_r[d]}) + 1/(60+{dense_r[d]})")
```

```
    print(f"        = {1/(60+lex_r[d]):.6f} + {1/(60+dense_r[d]):.6f}"
          f" = {fused[d]:.6f}")

    print("\n  what k does - the gap between rank 1 and rank 2:")
    for k in (0, 1, 10, 60, 1000):
        gap = 1 / (k + 1) - 1 / (k + 2)
        print(f"    k {k:4d}   1/(k+1) - 1/(k+2) = {gap:.6f}")

query: refund on a cancelled order

doc   BM25     rank    cosine   rank
d1   2.1771    3     0.9702    3
d2   5.5089    1     0.9874    2
d3   1.9761    4     0.8871    6
d4   0.6422    5     0.6203    7
d5   0.6422    6     0.6198    8
d6   0.0000    7     0.9312    4
d7   2.7038    2     0.9891    1
d8   0.0000    8     0.9145    5

not one scale. BM25 here runs 0.0000 to 5.5089,
the cosines 0.6198 to 0.9891. Adding them lets one
drown the other, and neither range means anything.

reciprocal rank fusion, k=60. Ranks only, no scores:
  1  0.032522  d2   (lexical 1, dense 2)
  2  0.032522  d7   (lexical 2, dense 1)
  3  0.031746  d1   (lexical 3, dense 3)
  4  0.030777  d3   (lexical 4, dense 6)
  5  0.030550  d6   (lexical 7, dense 4)
  6  0.030310  d4   (lexical 5, dense 7)
  7  0.030090  d8   (lexical 8, dense 5)
  8  0.029857  d5   (lexical 6, dense 8)

one document's arithmetic, to check by hand:
  d7: 1/(60+2) + 1/(60+1)
      = 0.016129 + 0.016393 = 0.032522

what k does - the gap between rank 1 and rank 2:
  k    0   1/(k+1) - 1/(k+2) = 0.500000
  k    1   1/(k+1) - 1/(k+2) = 0.166667
  k   10   1/(k+1) - 1/(k+2) = 0.007576
  k   60   1/(k+1) - 1/(k+2) = 0.000264
  k 1000   1/(k+1) - 1/(k+2) = 0.000001
```

Look at the two ranges. BM25 runs from 0 to 5.5089 and the stipulated cosines run from

0.6198 to 0.9891 — stipulated, so the *range* is one I chose, and a real model's would differ; what does not depend on my choosing is that the two have no common unit. Added together, the BM25 column decides everything and the cosine column is noise on top. You could rescale, but into what? There is no unit either number is measured in, so there is no principled scale to put them on. Any rescaling is a weight you chose.

Reciprocal rank fusion sidesteps the problem by throwing the scores away and keeping only the positions:

$$\mathrm{RRF}(d) = \sum_{r} \frac{1}{k + \mathrm{rank}_r(d)}$$

A document's fused score is the sum, over the rankings it appears in, of one divided by \(k\) plus its position in that ranking. The sum is over rankings, so appearing high in two lists beats appearing high in one. **It needs nothing from the two systems except an ordering**, which is exactly what makes it the standard way to combine them: ranks are comparable when scores are not.

And it discards real information, which you can see in the output. `d2` and `d7` tie *exactly* at 0.032522. One is lexical-first and dense-second, the other is the reverse, and since the formula sees only `(1, 2)` against `(2, 1)` it cannot tell them apart. The scores that would break the tie have been thrown away on purpose. That is the price.

The constant k controls how much a top position is worth. The last block prints the gap between rank 1 and rank 2 as k grows. At $k = 0$ being first rather than second is worth 0.5, which is enormous: one system's confident first pick would dominate. At $k = 60$ it is worth 0.000264, so the top few positions are treated as roughly equally good and agreement across lists matters more than any single list's ordering. The conventional 60 is a choice, not a derivation.



Fuse the *top of each list*, not the whole collection. Look at `d6` and `d8` in the table: both score BM25 0.0000, because they share no term with the query, and yet the ranking hands them positions 7 and 8 as though it had an opinion. Feeding that arbitrary tail into the fusion gives it votes it has not earned. In practice each system returns its top few hundred and only those are fused.

Writing Down How Much a Document Answers

Chapter 25 established the relevance judgement — one recorded decision that a document does or does not answer a query — and said an answer key is made of them. Those are binary: yes or no. For ranking measures that is usually not enough, because documents are not equally good answers.

Graded relevance replaces the yes-or-no with a small scale. Three *non-zero* points is the common choice, and the labels have to mean something a second person would apply the

same way:

Grade	What it means
3	Answers the question on its own
2	Useful, but the reader has to do some work
1	Related, and not an answer
0	Not relevant

The grades are the expensive part of everything that follows, and they are the only part that cannot be computed. Two things make them worth the cost. They let a measure distinguish a ranking that put a partial answer first from one that put the full answer first, which a binary key cannot see. And they are reusable: the key outlives every change to the retrieval, so the cost is paid once and the measurements are free afterwards.



Write the grade definitions down before grading anything, and have a second person grade a sample against them. If two people disagree a lot, the definitions are the problem and every number built on the key inherits it. This is the cheapest quality check in retrieval and it is almost always skipped.

Three Measures, and What Each One Misses

Now the measures. All three take a ranking and an answer key and return a number, and they disagree with each other in ways worth knowing.

```
import math

# One query, and a human's graded judgement of eight documents.
# 3 = answers it fully, 2 = useful, 1 = related, 0 = no.
LABELS = {"d7": 3, "d2": 3, "d1": 2, "d6": 1, "d3": 0,
          "d4": 0, "d5": 0, "d8": 0}

# What one system actually returned, best first.
RANKING = ["d1", "d3", "d2", "d6", "d8", "d7", "d4", "d5"]

def recall_at(ranking, k):
    total = sum(1 for g in LABELS.values() if g > 0)
    found = sum(1 for d in ranking[:k] if LABELS[d] > 0)
    return found / total

def reciprocal_rank(ranking):
    for place, d in enumerate(ranking, 1):
        if LABELS[d] > 0:
            return 1 / place
```

```
    return 0.0

def dcg(ranking, k, base=2, exp_gain=False):
    total = 0.0
    for i, d in enumerate(ranking[:k], 1):
        gain = (2 ** LABELS[d] - 1) if exp_gain else LABELS[d]
        total += gain / (math.log(i + 1) / math.log(base))
    return total

def ndcg(ranking, k, base=2, exp_gain=False):
    ideal = sorted(LABELS, key=lambda d: LABELS[d], reverse=True)
    best = dcg(ideal, k, base, exp_gain)
    return dcg(ranking, k, base, exp_gain) / best if best else 0.0

if __name__ == "__main__":
    print("  what came back, with the grade a human gave each one:")
    for i, d in enumerate(RANKING, 1):
        print(f"    {i}  {d}  grade {LABELS[d]}")

    print("\n  recall@k - did the relevant ones arrive at all:")
    for k in (1, 3, 5, 8):
        print(f"    recall@{k}  {recall_at(RANKING, k):.3f}")

    print(f"\n  reciprocal rank: first relevant is at 1, so"
          f" {reciprocal_rank(RANKING):.3f}")

    print("\n  DCG@5, term by term. The discount is log2(i+1):")
    run = 0.0
    for i, d in enumerate(RANKING[:5], 1):
        disc = math.log(i + 1, 2)
        run += LABELS[d] / disc
        print(f"    {i}  grade {LABELS[d]}  / log2({i+1}) = {disc:.4f}"
              f"  -> {LABELS[d]/disc:.4f}   running {run:.4f}")

    ideal = sorted(LABELS, key=lambda d: LABELS[d], reverse=True)
    print(f"\n  the ideal ordering by grade: {' '.join(ideal[:5])}")
    run = 0.0
    for i, d in enumerate(ideal[:5], 1):
        disc = math.log(i + 1, 2)
        run += LABELS[d] / disc
        print(f"    {i}  grade {LABELS[d]}  / log2({i+1}) = {disc:.4f}"
              f"  -> {LABELS[d]/disc:.4f}   running {run:.4f}")

    print(f"\n  nDCG@5 = {dcg(RANKING,5):.4f} / {dcg(ideal,5):.4f}"
          f" = {ndcg(RANKING,5):.4f}")
```

```
  print("\n  the log base cancels, because it divides both halves:")
  for base in (2, 10, math.e):
      name = "e" if base == math.e else str(base)
      print(f"    base {name:>2s}  DCG {dcg(RANKING,5,base):7.4f}"
            f"  ideal {dcg(ideal,5,base):7.4f}"
            f"  nDCG {ndcg(RANKING,5,base):.6f}")

  print("\n  the GAIN convention does not cancel:")
  print(f"    gain = grade        nDCG@5 {ndcg(RANKING,5):.4f}")
  print(f"    gain = 2^grade - 1  nDCG@5 "
        f"{ndcg(RANKING,5,exp_gain=True):.4f}")

what came back, with the grade a human gave each one:
  1  d1  grade 2
  2  d3  grade 0
  3  d2  grade 3
  4  d6  grade 1
  5  d8  grade 0
  6  d7  grade 3
  7  d4  grade 0
  8  d5  grade 0

recall@k - did the relevant ones arrive at all:
  recall@1  0.250
  recall@3  0.500
  recall@5  0.750
  recall@8  1.000

reciprocal rank: first relevant is at 1, so 1.000

DCG@5, term by term. The discount is log2(i+1):
  1  grade 2  / log2(2) = 1.0000  -> 2.0000   running 2.0000
  2  grade 0  / log2(3) = 1.5850  -> 0.0000   running 2.0000
  3  grade 3  / log2(4) = 2.0000  -> 1.5000   running 3.5000
  4  grade 1  / log2(5) = 2.3219  -> 0.4307   running 3.9307
  5  grade 0  / log2(6) = 2.5850  -> 0.0000   running 3.9307

the ideal ordering by grade: d7 d2 d1 d6 d3
  1  grade 3  / log2(2) = 1.0000  -> 3.0000   running 3.0000
  2  grade 3  / log2(3) = 1.5850  -> 1.8928   running 4.8928
  3  grade 2  / log2(4) = 2.0000  -> 1.0000   running 5.8928
  4  grade 1  / log2(5) = 2.3219  -> 0.4307   running 6.3235
  5  grade 0  / log2(6) = 2.5850  -> 0.0000   running 6.3235

nDCG@5 = 3.9307 / 6.3235 = 0.6216
```

```
the log base cancels, because it divides both halves:
  base  2  DCG  3.9307  ideal  6.3235  nDCG 0.621602
  base 10  DCG 13.0574  ideal 21.0061  nDCG 0.621602
  base  e  DCG  5.6708  ideal  9.1228  nDCG 0.621602

the GAIN convention does not cancel:
  gain = grade        nDCG@5 0.6216
  gain = 2^grade - 1  nDCG@5 0.5193
```

Recall@k is the fraction of all the relevant documents that appear in the top k. It asks *did they arrive*, and nothing else. Here recall@5 is 0.750: three of the four *relevant* documents are in the top five. All eight are graded — zero is a grade — and what counts for recall is a grade above zero. **It is blind to order.** The same three documents in any arrangement inside the top five give 0.750. That makes it the wrong measure for what a user sees, and the right one for a first pass whose only job is to collect candidates for something else to sort — which a later section builds.

And recall has a problem the other two do not, which is worth knowing because it is the usual follow-up question. Its denominator is *all* the relevant documents, and on a real collection nobody knows that number: finding it means judging every document against every query. Eight documents make it look free. A hundred thousand make it impossible, so in practice the denominator is whatever the key happens to contain — which means a recall figure is only ever recall against your key, and a document nobody ever judged is invisible to it. That is chapter 25's invisible miss, still here, one level further up.

Reciprocal rank is one divided by the position of the first relevant result; averaged over many queries it is the **mean reciprocal rank**, or MRR. It asks *how far did the reader have to look*, which is the right question when there is one answer and finding it ends the task.

Both of these are binary measures, and they need a convention stated too. Recall and reciprocal rank take *relevant* or *not*, so a graded key has to be cut somewhere, and the programs here cut it at a grade above zero. That means a grade-1 document — *related, and not an answer* — counts as found, and a ranking that puts one first has a reciprocal rank of 1.000. If your key grades, say where you cut it, for the same reason the chapter insists an nDCG is meaningless without its gain convention: a reader cannot interpret the number otherwise.

And here it is worth 1.000, which should stop you. The first result is `d1`, graded 2, so the first relevant document is at position one and the reciprocal rank is perfect. Meanwhile a grade-3 document sits at position six. **MRR says this ranking is flawless, and nDCG says it is 0.6216.** Neither is wrong: MRR measures the distance to the first relevant thing, and this ranking is genuinely excellent at that. It simply cannot see the difference between a partial answer and a complete one, because it stops looking after the first non-zero grade.

Discounted cumulative gain is the one that sees both grade and position. Each result contributes its grade divided by a discount that grows with position, and the contributions are

added up:

$$\mathrm{DCG@}k = \sum_{i=1}^{k} \frac{g_i}{\log_2(i+1)}$$

g_i is the grade of the result at position i, and $\log_2$ is the first logarithm in this book to a base other than chapter 8's e: $\log_2 x$ answers "what power of 2 gives x?", so $\log_2 8 = 3$. It turns out not to matter which base you use here, for a reason worked out three paragraphs below. Follow the printed table: position 1 is divided by $\log_2 2 = 1$, so it is undiscounted; position 3 is divided by 2, so a grade-3 document there contributes 1.5; position 4 is divided by 2.3219. The running column adds up to 3.9307.

The logarithm is a convention and not a derivation, and it is worth being exact about which part is which. What the discount *must* do is decrease with position. Why a logarithm rather than any other decreasing function is a choice: it decays slowly, so the difference between positions 1 and 2 is large while the difference between 10 and 11 is small, which matches a rough intuition about how people read result lists. That intuition is not a theorem, and other discounts are in use. Do not claim the log is forced.

What *is* derivable is that the base does not matter. Look at the third block: base 2, base 10 and base e give DCGs of 3.9307, 13.0574 and 5.6708, and all three give an nDCG of 0.621602.

Chapter 8 gave you the natural logarithm ln and one property of it. A logarithm to another base is that one rescaled, and the relation is worth having in front of you because the rest of this paragraph is just arithmetic on it:

$$\log_c x = \frac{\ln x}{\ln c}$$

So a term $g_i / \log_c(i+1)$ is the same as $g_i \cdot \ln c / \ln(i+1)$: changing the base multiplies every term by the *same* constant $\ln c$. Check it against the printed numbers rather than taking it: base 2 should be $\ln 2 = 0.6931$ of base e, and $3.9307 / 5.6708 = 0.6931$. A constant multiplying the DCG multiplies the ideal DCG by exactly the same amount, so it cancels in the ratio. The normalisation is what buys that, and the cancelling is exact rather than approximate.

Normalised discounted cumulative gain, written nDCG, is the ratio: the DCG of your ranking divided by the DCG of the best possible ranking of the same documents. Here 3.9307 divided by 6.3235 gives 0.6216. The ideal is built by sorting the graded documents by grade, which the program prints: `d7 d2 d1 d6 d3`. Dividing makes the number comparable across queries, because a query with four relevant documents and a query with one no longer live on different scales.

But the gain convention does not cancel, and this is the one that bites. The last block computes the same nDCG two ways. Using the grade directly gives 0.6216. Using $2^g - 1$ — the other standard choice, which weights a grade-3 document far above a grade-2 one — gives 0.5193. Same ranking, same key, same k, two numbers fourteen points apart. **So an**

nDCG is not a number you can compare against one computed by a different tool without knowing its gain convention, its k, and how it treats a query whose ideal DCG is zero.

Two Encoders, or One Model That Sees Both

Chapter 25 proved that a single shared encoder compared by a cosine cannot distinguish *x answers y* from *y answers x*, and named two escapes: score the two texts together, or encode the two sides with *different* encoders. Both are arrangements, and the arrangements have names.

A **bi-encoder** encodes the query and each document separately and compares the two vectors with a cosine. The decisive property is that the document side does not depend on the query, so **every document vector can be computed once, before any query arrives**.

Be careful here, because a bi-encoder is not by itself an escape. Chapter 24's dense retrieval is a bi-encoder in which both sides go through the *same* model — chapter 24 says so in as many words, because otherwise the numbers are not comparable — and that is exactly the arrangement chapter 25 derived cannot answer a directed question. The score is symmetric and no amount of training changes it.

What escapes is any arrangement that makes the two sides go through *different* functions. The **two-tower** form is the obvious one: a bi-encoder with a separate query encoder and document encoder, giving $\cos(f(q), g(d))$ with $f \neq g$. It keeps the precomputation, because g still does not depend on the query, and it gives up the symmetry.

And there is a cheaper way to the same place, which chapter 25 also named and which is how much of current practice does it: keep one shared model and tell it which side it is encoding, by prefixing the text with a marker such as `query:` or `passage:`. The weights are shared, so it still looks like a shared encoder — but the *function* applied to a query differs from the one applied to a document, which is what $f \neq g$ asks for. So **"shared weights" and "symmetric" are not the same claim**: what forces symmetry is putting both sides through the same function, marker and all. Either way the point stands: you can have a directed score *and* precomputed documents at the same time.

A **cross-encoder** puts the query and one document through a model *together*, as a single input, and the model emits one score for the pair. Nothing is precomputed, because the input does not exist until the query does. In exchange, the model can relate the two texts directly, and the score is a function of the ordered pair rather than of two independently made vectors — so it is free to be asymmetric, which is the shape chapter 25 said a directed question needs.

The cost difference is the whole reason for what follows.

```
from measures import LABELS, dcg, recall_at

# What stage one returned, best first, for the same query as before.
STAGE_ONE = ["d1", "d3", "d2", "d6", "d8", "d7", "d4", "d5"]

def ideal_of(candidates, k):
    # The best any reordering of this candidate set can do: sort it by grade.
    best = sorted(candidates, key=lambda d: LABELS[d], reverse=True)
    return dcg(best, k)

def whole_corpus_ideal(k):
    best = sorted(LABELS, key=lambda d: LABELS[d], reverse=True)
    return dcg(best, k)

if __name__ == "__main__":
    print("  a PERFECT reranker, given the top c of stage one.")
    print("  It reorders by the human's own grades, so nothing can beat it")
    print("  on that candidate set. nDCG@5 against the whole corpus:\n")
    print("  c    candidates                  recall    nDCG@5")
    ideal = whole_corpus_ideal(5)
    for c in (3, 5, 6, 8):
        cands = STAGE_ONE[:c]
        got = ideal_of(cands, 5) / ideal
        rec = sum(1 for d in cands if LABELS[d] > 0) / \
              sum(1 for g in LABELS.values() if g > 0)
        print(f"  {c}    {' '.join(cands):26s}  {rec:.3f}     {got:.4f}")

    print("\n  so the reranker is not the thing that is broken at c=3:")
    print("  d7 has grade 3 and sits sixth in stage one, so no reordering")
    print("  of the first three can put it anywhere.")

    print("\n  the cost, as a count of forward passes through a model:")
    print("    corpus       bi-encoder   cross-encoder")
    for n in (1_000, 100_000, 1_000_000):
        print(f"    {n:9,d}             1   {n:13,d}")
    print("\n    two stages: 1 embedding + 1 search + 100 cross-encoder")
    print("    passes. The MIDDLE one is the only one that grows")
    print("    with n, and that is chapter 29's problem.")
```

```
  a PERFECT reranker, given the top c of stage one.
  It reorders by the human's own grades, so nothing can beat it
  on that candidate set. nDCG@5 against the whole corpus:

  c    candidates                  recall    nDCG@5
  3    d1 d3 d2                   0.500     0.6740
  5    d1 d3 d2 d6 d8             0.750     0.7530
```

```
6    d1 d3 d2 d6 d8 d7           1.000     1.0000
8    d1 d3 d2 d6 d8 d7 d4 d5     1.000     1.0000

so the reranker is not the thing that is broken at c=3:
d7 has grade 3 and sits sixth in stage one, so no reordering
of the first three can put it anywhere.

the cost, as a count of forward passes through a model:
  corpus       bi-encoder   cross-encoder
      1,000             1           1,000
    100,000             1         100,000
  1,000,000             1       1,000,000

  two stages: 1 embedding + 1 search + 100 cross-encoder
  passes. The MIDDLE one is the only one that grows
  with n, and that is chapter 29's problem.
```

Read the cost block first. A bi-encoder needs one forward pass at query time, whatever the collection size, because the documents were encoded in advance. A cross-encoder needs one pass *per document*, so scoring a million documents means a million passes for one query. That is not a tuning problem; it is the arrangement.

So the standard arrangement is two stages. **Reranking** means scoring a small set of candidates with a more expensive method than the one that produced them, and the model that does it is a **reranker**. The first stage is cheap and runs over everything — BM25, dense retrieval, or both fused — and returns a **candidate set**, say the top hundred. The second stage cross-encodes those hundred and reorders them. The cost is one embedding, one search, and a hundred cross-encoder passes — and **the number of cross-encoder passes** does not grow with the collection. The search does, which is chapter 29's subject: at a million documents, comparing the query against every vector stops being free, and that is a different problem from this one.



Two-stage retrieval with a cross-encoder is a current convention, in chapter 6's sense, and not settled practice: it is how most systems are built today, the arrangement is what to understand, and the specific models change constantly. Chapter 28 owns reranking as a step in assembling a context — how many candidates to keep, where the passages go, and how to cite them. This chapter owns only what the two encoder arrangements are and why there are two stages.

What a Reranker Cannot Do, and Why That Is a Derivation

Now the top block, which is the more important half.

The reranker in that program is *perfect*. It does not estimate relevance; it reorders the candidates by the human's own grades, so no real reranker can beat it. Watch what it achieves as the candidate set grows: with three candidates it reaches nDCG@5 of 0.6740, with five 0.7530, and with six 1.0000.

The reason is not about rerankers at all. `d7` has the top grade and sits sixth in stage one's output. A reordering of a set cannot introduce an element that is not in the set, so no function whatever of the first three candidates can place `d7` anywhere. This is a derivation and not an observation — it holds for every reranker, every model and every collection, because it is a property of reordering.

So the candidate set is a ceiling. Whatever the second stage is, it cannot retrieve; it can only permute. If your answers are not in the hundred, buying a better reranker is spending money on the wrong stage.

Two quantities, and they are not the same one. The `recall` column counts how many of the relevant documents the candidate set contains, and reranking cannot raise it — permuting a list does not add to it. The `nDCG@5` column is the best score any reordering of that set could reach. At `c=5` these read 0.750 and 0.7530, which are nearly equal here and have no reason to be: one counts documents and the other weighs grades by position. Do not read the near-match as a law.

This is chapter 25's invisible miss, arriving with a number on it. The chapter said a missing document leaves no trace in a result list; here the trace is in the gap between the reranked score and 1.0, and you can only see it because somebody wrote the grades down.

Measuring Hybrid Against Dense on Your Own Data

The chapter's problem said *no way to say which is better*. You now have the measures to try. Here are both methods and their fusion, on two queries with grades — and the result is not the one this section was expected to produce.

```
import math
from bm25 import CORPUS, bm25
from fuse import rrf

# Two queries, and a human's grades for each. Query A is answered by meaning,
# query B by an exact string. The cosines are stipulated, as in chapters 24
# and 25: nothing here shows what a real model returns.
QUERIES = {
    "refund on a cancelled order": {
        "grades": {"d7": 3, "d2": 3, "d1": 2, "d6": 1,
                   "d3": 0, "d4": 0, "d5": 0, "d8": 0},
        "cosine": {"d7": 0.9891, "d2": 0.9874, "d1": 0.9702, "d6": 0.9312,
                   "d8": 0.9145, "d3": 0.8871, "d4": 0.6203, "d5": 0.6198},
```

```
    },
    # Chapter 25's residual, fallen the other way: the two notes are 0.0005
    # apart and the wrong one is on top. Nothing predicts which you get.
    "x-4471-d seal": {
        "grades": {"d4": 3, "d1": 0, "d2": 0, "d3": 0,
                   "d5": 0, "d6": 0, "d7": 0, "d8": 0},
        "cosine": {"d5": 0.99978, "d4": 0.99928, "d6": 0.9712, "d1": 0.8310,
                   "d2": 0.8265, "d7": 0.8102, "d8": 0.8044, "d3": 0.7701},
    },
}

def matched_ranks(scores):
    # Only documents that actually matched. An inverted index never returns a
    # document holding none of the query terms, and fusing a tail of zeros
    # would hand arbitrary positions a vote - the Warning earlier in the
    # chapter. fuse.py ranked all eight to show what that warning is about.
    hit = {d: s for d, s in scores.items() if s > 0}
    order = sorted(hit, key=lambda d: hit[d], reverse=True)
    return {d: place for place, d in enumerate(order, 1)}

def dense_ranks(scores):
    order = sorted(scores, key=lambda d: scores[d], reverse=True)
    return {d: place for place, d in enumerate(order, 1)}

def dcg(order, grades, k):
    return sum(grades[d] / math.log(i + 1, 2)
               for i, d in enumerate(order[:k], 1))

def ndcg(order, grades, k):
    ideal = sorted(grades, key=lambda d: grades[d], reverse=True)
    best = dcg(ideal, grades, k)
    return dcg(order, grades, k) / best if best else 0.0

def best_and_worst(fused, grades, k=5):
    # A tie has no ordering, so report the range it spans rather than
    # whichever one sorted() happened to emit.
    hi = sorted(fused, key=lambda d: (fused[d], grades[d]), reverse=True)
    lo = sorted(fused, key=lambda d: (fused[d], -grades[d]), reverse=True)
    return ndcg(hi, grades, k), ndcg(lo, grades, k)

if __name__ == "__main__":
    rows = {}
    for query, spec in QUERIES.items():
        grades, cos = spec["grades"], spec["cosine"]
        lex = {d: bm25(query, d) for d in CORPUS}
        lr, dr = matched_ranks(lex), dense_ranks(cos)
```

```
    fused = rrf([lr, dr])
    print(f"  query: {query}")
    print(f"    lexical matched {len(lr)} of {len(CORPUS)} documents")
    got = {}
    for name, order in (("lexical", sorted(lr, key=lambda d: lr[d])),
                        ("dense", sorted(dr, key=lambda d: dr[d]))):
        got[name] = ndcg(order, grades, 5)
        print(f"    {name:8s} {' '.join(order[:4])}"
              f"   nDCG@5 {got[name]:.4f}")
    hi, lo = best_and_worst(fused, grades)
    got["fused"] = (hi, lo)
    tied = len({round(v, 12) for v in fused.values()}) < len(fused)
    print(f"    fused    nDCG@5 {lo:.4f} to {hi:.4f}"
          f"   ties present: {tied}")
    rows[query] = got
    print()

print("  the tie on the identifier query, in full:")
q = "x-4471-d seal"
lex = {d: bm25(q, d) for d in CORPUS}
lr = matched_ranks(lex)
dr = dense_ranks(QUERIES[q]["cosine"])
fused = rrf([lr, dr])
for d in ("d4", "d5"):
    print(f"    {d}: lexical {lr[d]}, dense {dr[d]}"
          f"  ->  {fused[d]:.6f}")
print("    the right note and the wrong one score the SAME.")

print("\n  the same two notes, under each method. Both forms of the")
print("  margin are given, because one of each invites a comparison")
print("  across two scales that the chapter has just forbidden:")
print(f"    lexical  d4 {lex['d4']:.4f}  d5 {lex['d5']:.4f}"
      f"   ratio {lex['d4']/lex['d5']:.2f}  gap {lex['d4']-lex['d5']:.4f}")
cos = QUERIES[q]["cosine"]
print(f"    dense    d5 {cos['d5']:.5f}  d4 {cos['d4']:.5f}"
      f"   ratio {cos['d5']/cos['d4']:.4f}  gap {cos['d5']-cos['d4']:.5f}")
print("    what matters is not either number: d4 holds the term and")
print("    d5 does not, so the lexical signal is categorical.")

print("\n  nDCG@5 across both queries:")
print("    method    meaning        identifier     mean")
a_name = "refund on a cancelled order"
for name in ("lexical", "dense"):
    a, b = rows[a_name][name], rows[q][name]
    print(f"    {name:8s}  {a:.4f}         {b:.4f}         {(a+b)/2:.4f}")
fa_lo, fa_hi = rows[a_name]["fused"][1], rows[a_name]["fused"][0]
```

```
  fb_lo, fb_hi = rows[q]["fused"][1], rows[q]["fused"][0]
  print(f"    fused     {fa_lo:.4f}-{fa_hi:.4f}  {fb_lo:.4f}-{fb_hi:.4f}"
        f"  {(fa_lo+fb_lo)/2:.4f}-{(fa_hi+fb_hi)/2:.4f}")
  print(f"\n    dense's mean {(rows[a_name]['dense']+rows[q]['dense'])/2:.4f}"
        f" sits INSIDE the fused range.")
  print("    So this evidence does not say fusion beat dense.")

query: refund on a cancelled order
  lexical matched 6 of 8 documents
  lexical  d2 d7 d1 d3   nDCG@5 0.9319
  dense    d7 d2 d1 d6   nDCG@5 1.0000
  fused    nDCG@5 0.9319 to 0.9319   ties present: True

query: x-4471-d seal
  lexical matched 2 of 8 documents
  lexical  d4 d5   nDCG@5 1.0000
  dense    d5 d4 d6 d1   nDCG@5 0.6309
  fused    nDCG@5 0.6309 to 1.0000   ties present: True

the tie on the identifier query, in full:
  d4: lexical 1, dense 2  ->  0.032522
  d5: lexical 2, dense 1  ->  0.032522
  the right note and the wrong one score the SAME.

the same two notes, under each method. Both forms of the
margin are given, because one of each invites a comparison
across two scales that the chapter has just forbidden:
  lexical  d4 2.8466  d5 1.1867   ratio 2.40  gap 1.6599
  dense    d5 0.99978  d4 0.99928   ratio 1.0005  gap 0.00050
  what matters is not either number: d4 holds the term and
  d5 does not, so the lexical signal is categorical.

nDCG@5 across both queries:
  method    meaning        identifier     mean
  lexical   0.9319         1.0000         0.9659
  dense     1.0000         0.6309         0.8155
  fused     0.9319-0.9319  0.6309-1.0000  0.7814-0.9659

  dense's mean 0.8155 sits INSIDE the fused range.
  So this evidence does not say fusion beat dense.
```

The margin block is chapter 25's complaint, quantified. On the identifier query, the two near-identical maintenance notes sit 1.6599 apart under BM25 and 0.00050 apart under the cosine. Both forms of each margin are printed on purpose, because quoting one as a ratio and the other as a gap is the cross-scale comparison this chapter spent a section for-

bidding. The thing that actually matters is not a number at all: `d4` holds the term `x-4471-d` and `d5` does not, so one gets that term's IDF and the other gets nothing. The lexical signal is categorical. The dense signal is a residual, it fell the wrong way here, and the dense ranking put the wrong note first — which cost it 0.6309 against lexical's 1.0000.

Now read the fused row, because this is where the section turns. On the identifier query `d4` and `d5` score *exactly* the same fused value, 0.032522. It is the tie from two sections ago, arriving where it costs something: `d4` is lexical-first and dense-second, `d5` is the reverse, and reciprocal rank fusion cannot tell `(1, 2)` from `(2, 1)`. **So the right note and the wrong note are tied, and which one a user sees is decided by nothing.**

That is why the fused column is a range rather than a number. If the tie breaks towards `d4` the fused nDCG@5 is 1.0000; if it breaks towards `d5` it is 0.6309. The same structural tie appears on the meaning query and costs nothing there, because the two tied documents are both grade 3 — which is the clearest possible illustration that a tie is harmless until it is not.

And now the honest reading of the means. Lexical scored 0.9659, dense 0.8155, and fusion somewhere between 0.7814 and 0.9659. **Dense's 0.8155 sits inside the fused range**, so this measurement does not establish that fusion beat dense. Fusion never beat lexical on either query, and its best case only ties it.

Hybrid retrieval runs lexical and dense retrieval and combines the rankings. What this chapter has shown about it is narrower than the usual claim and worth saying exactly: on two queries with stipulated cosines, fusion put nothing in the top five that lexical had not already found there, and one of its two results was a coin flip. (It did surface `d6`, a grade-1 document, that lexical missed entirely — at rank 7, where nDCG@5 cannot see it.) That is not a verdict on hybrid retrieval. It is a demonstration that you cannot get a verdict this way.



Do not take "fusion is a hedge that trades the best case for a better floor" from this. It is a popular formulation and it is false as a general claim, because fusion can beat *both* of its inputs outright: a document at rank 2 in both lists scores 0.032258, which wins against documents at `(1, 8)` and `(8, 1)` on 0.031099 each, so if that middling document is the relevant one, fusion scores 1.0000 where each input scores 0.6309. It runs the other way too: that same `(2, 2)` document is beaten by documents at `(1, 3)` and `(3, 1)`, which score 0.032266 against its 0.032258, so it falls to third and fusion scores 0.5000 where both inputs score 0.6309. The reason is that $1/x$ is convex, so spreading two ranks apart beats keeping them together — $1/61 + 1/63 > 2/62$. Fusion is not bounded by its inputs in either direction. Whether it helps *you* is a measurement.



Two queries measure nothing, and the range above is the proof rather than a caveat: a result that moves 0.19 depending on a tie-break has no business being compared with anything. Chapter 20 is how this is done properly — the bootstrap for the interval, and a paired comparison, because both methods run on the identical queries and that is exactly the case where pairing buys the most.

That is the method, then. Write the key once. Run each configuration over the same queries, enough of them that one tie cannot move the answer. Compare paired, with an interval. Keep the configuration that wins and say by how much, with the uncertainty attached.

Lab: Build Both Searches and Score Them

Four programs, pure Python, nothing to install. About an hour, and there is a key for every step. Work in a new empty folder.

Step 1. Save the first program as `bm25.py` and run it.

Expected result: eight documents, average length 8.500 terms; `x-4471-d` worth 1.7918 against `the` at 0.4925; the classic column negative for `the` at −0.4520; and `d1` first on *refund policy* with `d3` second. `d3` is six repetitions of *refund*; the printed line truncates the document text at 34 characters, so only five of them show.

Step 2. In `bm25.py`, change the `k1 = 1.5` inside `__main__` to `0.0` and run it again. There are two `k1`s — that one, and the default in `def bm25(...)` — and this step wants the local one, because it is what the saturation block uses. (Changing the function default instead leaves that block untouched and silently ties `d3` with `d2` in the ranking below it, which is a worthwhile thing to try afterwards.)

Expected result: the saturation block collapses. Every term frequency gives exactly 1.0000 IDF, because with $k_1 = 0$ the fraction becomes $f_{t,d} \cdot 1/f_{t,d}$, which is 1 whatever the count. So $k_1 = 0$ means *ignore how often a term appears and only ask whether it appears at all*. The knob interpolates between presence and count.

Step 3. Put the local `k1` back to `1.5`, then set `b` to `0.0` in the `def bm25(...)` signature — that one is unambiguous, since `b` appears nowhere else — and run it. Watch the `refund policy` ranking at the bottom.

Expected result: both scores *fall* — `d1` from 3.1536 to 2.7362 and `d3` from 1.9761 to 1.8889 — and the reason is worth working out, because the obvious prediction is wrong. `d1` and `d3` are *both six terms long*, against a collection average of 8.500, so neither was being penalised for its length; both were being *rewarded* for being shorter than average, and $b = 0$ removes that reward. `d1` falls further because it matches two terms once each, and the length constant competes with $f_{t,d}$ in the denominator — so the smaller the term frequency, the more of the score that constant decides. The two knobs are independent: k_1 controls saturation, b controls length.

Step 4. Save the second program as `fuse.py` and run it. Check the `d7` arithmetic by hand.

Expected result: `d2` and `d7` tied at 0.032522, and `1/62 + 1/61 = 0.016129 + 0.016393`. If you get a different total, you have added the ranks rather than their reciprocals.

Step 5. In `fuse.py`, call `rrf([lex_r, dense_r], k=0)` and run it again.

Expected result: **the ordering does not change at all** — it is `d2 d7 d1 d3 d6 d4 d8 d5` at $k = 0$ just as it was at 60, and the same at 1, 10 and 1000. That is worth sitting with, because it is easy to assume a constant that big must reorder something. What k changes is the *size* of the gaps, not their direction: at $k = 0$ first place is worth 0.5 more than second rather than 0.000264 more, so one list's top pick would dominate a fusion of *many* lists — with two lists of eight documents, the agreement between them already decides the order. And `d2` and `d7` still tie, at every k: the tie is structural, since $1/(k+1) + 1/(k+2)$ is the same sum whichever way round it is added.

Step 6. Save the third program as `measures.py` and run it. Then answer this before reading on: MRR says 1.000 and nDCG@5 says 0.6216 for the same ranking. Which one is wrong?

Expected result: neither. They measure different things, and the ranking really is perfect at the thing MRR measures — the first result has a non-zero grade, so the reader looked no further than position one. nDCG also weighs *how good* each result was and *where* it sat, and it can see that a grade-3 document is at position six. If you find yourself wanting one number, you have to say which question you are asking first.

Step 7. In `measures.py`, change `RANKING` to put `d7` first and `d1` sixth.

Expected result: MRR stays at 1.000 — it was already at its maximum and cannot report the improvement — while nDCG@5 rises. This is the clearest demonstration in the chapter that a measure at its ceiling has stopped being able to tell you anything.

Step 8. Save the fourth program as `stages.py` and run it.

Expected result: a perfect reranker reaching 0.6740 on three candidates, 0.7530 on five and 1.0000 on six. Then say in one sentence why buying a better reranker cannot fix the `c=3` row.

Step 9. Save the fifth program as `hybrid.py` and run it.

Expected result: lexical 0.9659 mean, dense 0.8155, and the fused mean reported as a *range*, 0.7814 to 0.9659 — with dense's 0.8155 inside it. The reason the fused figure is a range is the tie the program prints just above: on the identifier query the right note and the wrong one both score 0.032522. Say what you would need before claiming fusion is better than either method here. The answer is in chapter 20: an interval, from enough queries that a single tie cannot move it, computed paired.

Step 10. Add a third query to `hybrid.py` whose answer needs meaning *and* an identifier — something like *the seal on X-4471-D*. Grade the eight documents yourself first, writing the grade definitions down before you look at any score.

Expected result: your own. The point of the step is the order of operations: the grades come before the scores, or they are not a key, they are a rationalisation.

Summary, Key Terms, and Review Questions

Summary

Lexical retrieval scores a document by the query terms it contains, and **BM25** is the standard way to do it. It answers three objections in order: **term frequency** says a document mentioning a term often is more about it; **inverse document frequency** says a term in few documents is worth more than a term in many, since otherwise *the* would dominate; and document length says a long document mentions everything more often without being more about any of it. Two properties are worth carrying and the constants are not. Term frequency **saturates** — a hundred mentions reached 98.5% of a ceiling of (k_1+1) IDF, which is why a document consisting of *refund* six times lost to one matching two query terms once each — and the classic IDF goes **negative** for a term in more than half the collection, which is why the modern form adds one inside the logarithm. k_1 and b are tuned constants, not derived ones.

Two searches cannot be added, because neither score has a unit: BM25 ran 0 to 5.5089 and the stipulated cosines 0.6198 to 0.9891, so a sum is whatever weight you implied. **Reciprocal rank fusion** discards the scores and sums $1/(k + \text{rank})$ across rankings, which needs nothing from either system but an ordering — and pays for it by tying two documents at exactly 0.032522 because `(1, 2)` and `(2, 1)` are indistinguishable to it. Its k, usually 60, sets how much a first place is worth: the gap between rank 1 and rank 2 is 0.5 at $k = 0$ and 0.000264 at $k = 60$.

A **bi-encoder** encodes query and documents separately, so every document vector is computed before any query arrives and query time costs one forward pass. **A bi-encoder is not by itself an escape from chapter 25's symmetry limit**: chapter 24's dense retrieval puts both sides through the *same* model, which is precisely the arrangement that cannot answer a directed question. The escape is any arrangement that puts the two sides through different *functions*: the **two-tower** form, with separate query and document encoders giving $\cos(f(q), g(d))$ and $f \neq g$, or — chapter 25's other sub-variant, and how much of current practice does it — one shared model told which side it is encoding by a `query:` or `passage:` marker. Both keep the precomputation and give up the symmetry, so **shared weights and symmetry are different claims**: what forces symmetry is the same function on both sides. A **cross-encoder** is the first one: it puts the pair through one model together, so nothing is precomputable and a million documents cost a million forward passes, and the score is free to be asymmetric. Hence two stages: retrieve cheaply over everything, then **rerank** a **candidate set** of about a hundred. **The candidate set is a ceiling, and that is a derivation**: reordering a set cannot introduce an element not in it, so a *perfect* reranker reached only 0.6740 on three candidates and 1.0000 on six, and the grade-3 document sitting sixth in stage one could not be rescued by any reordering of the first three.

Measuring needs **graded relevance** — chapter 25's binary judgement extended to a small scale, with written definitions two people would apply the same way. **Recall@k** asks only

whether the relevant documents arrived and is blind to their order, which makes it right for a first stage and wrong for what a user sees. **Reciprocal rank** is one over the position of the first relevant result, and on the chapter's ranking it is 1.000 while **nDCG@5** is 0.6216: MRR cannot see a grade-3 document at position six because it stops at the first non-zero grade. **DCG** divides each grade by $\log_2(i + 1)$ and sums; the logarithm is a convention, chosen because it decays slowly, and *not* forced. What is derivable is that the base cancels — base 2, 10 and e gave DCGs of 3.9307, 13.0574 and 5.6708 and an identical nDCG of 0.621602, because changing base scales DCG and ideal DCG by the same constant. The **gain** convention does not cancel: grade directly gave 0.6216 and $2^g - 1$ gave 0.5193, so two tools' nDCGs are not comparable without knowing both conventions.

Hybrid retrieval combines the two rankings, and the attempt to measure it is the chapter's most useful failure. Lexical scored 0.9659 across the two queries and dense 0.8155, but the fused score on the identifier query is not a number: `d4` and `d5` tie at *exactly* 0.032522, being `(1, 2)` and `(2, 1)`, so the right note and the wrong one score the same and the fused nDCG@5 is 1.0000 or 0.6309 according to a tie-break. The fused mean is therefore a range, 0.7814 to 0.9659, **and dense's 0.8155 falls inside it** — so the measurement does not establish that fusion beat dense, and fusion never beat lexical at all. The same tie occurs on the meaning query and costs nothing, because both tied documents are grade 3. **Do not carry away "fusion is a hedge that trades the best case for a better floor"**: fusion can beat both inputs outright, as a document at rank 2 in both lists does on 0.032258 against `(1, 8)` and `(8, 1)` on 0.031099. Fusion is not bounded by its inputs in either direction, and two queries with stipulated cosines settle nothing — chapter 20 is the interval and the paired comparison.

Key Terms

Term	Meaning
Lexical retrieval	Scoring a document by the query terms it contains. Not a filter: every document gets a number and the list is sorted
Term	The unit a lexical index counts — usually a lower-cased word, sometimes stemmed. Not the model's token from chapter 10
Term frequency	How often a term appears in one document
Document frequency	How many documents in the collection contain a term at all
Document length	How many terms a document holds, compared with the collection average. BM25's third input, and the reason a long document is not automatically more relevant
Inverse document frequency	A weight that rises as a term gets rarer, so a term almost every document holds barely moves a score

Term	Meaning
BM25	The standard lexical scoring function, combining term frequency, inverse document frequency and document length, with a saturating term-frequency curve
Reciprocal rank fusion	Combining rankings by summing one over a constant plus each rank, using positions only and never the scores
Hybrid retrieval	Running lexical and dense retrieval and combining their rankings
Bi-encoder	Query and document encoded separately and compared by a cosine, so document vectors are precomputable. With one shared model the score is symmetric
Two-tower	A bi-encoder with separate query and document encoders, so the score is $\cos(f(q), g(d))$ with $f \neq g$: still precomputable, no longer symmetric
Cross-encoder	Query and document passed through one model together, giving one score for the ordered pair. Not precomputable; free to be asymmetric
Reranking	Rescoring a small candidate set with a more expensive method than the one that produced it
Reranker	The model that does the rescoring in the second stage — in current practice usually a cross-encoder
Candidate set	The documents the first stage passes to the second. A reranker can only permute it, so it is a ceiling
Graded relevance	A relevance judgement on a small scale rather than yes-or-no, so a partial answer and a full one can be told apart
Recall@k	The fraction of all relevant documents appearing in the top k. Blind to their order
Reciprocal rank	One divided by the position of the first relevant result, for a single query
Mean reciprocal rank	The average of the reciprocal rank over a set of queries
Discounted cumulative gain	The sum of each result's grade divided by a discount that grows with position
Normalised discounted cumulative gain	A DCG divided by the DCG of the best possible ranking of the same documents, making it comparable across queries

Worth Memorising

Item	Value or formula
BM25	‘(\sum_{t \in q} \mathrm{IDF}(t) \cdot \dfrac{f_{t,d}(k_1+1)}{f_{t,d} + k_1(1 - b + b
Its two knobs	$k_1 \approx 1.5$ controls saturation, $b \approx 0.75$ controls length normalisation. Tuned constants, not derived
What $k_1 = 0$ does	Makes term frequency irrelevant: the factor is 1 whatever the count, so only presence matters
Term-frequency ceiling	A term’s contribution approaches $(k_1 + 1)$ IDF. At $k_1 = 1.5$, 100 mentions reach 98.5% of 2.5 IDF
Inverse document frequency	$\ln\left(\frac{N - n_t + 0.5}{n_t + 0.5} + 1\right)$. Without the +1 it goes negative for a term in over half the collection
Reciprocal rank fusion	$\sum_r 1/(k + \text{rank}_r(d))$, conventionally $k = 60$. Uses ranks only, so no score comparability is needed
DCG	$\sum_{i=1}^{k} g_i / \log_2(i + 1)$
nDCG	DCG divided by the DCG of the ideal ordering of the same documents
Change of base	$\log_c x = \ln x / \ln c$, so a base-c DCG is $\ln c$ times the natural-log one
The log base cancels	A derivation: changing base scales DCG and ideal DCG by the same $\ln c$. Base 2, 10 and e all gave nDCG 0.621602
The gain convention does not	Grade directly gave 0.6216; $2^g - 1$ gave 0.5193 on the same ranking
Why a logarithm at all	Convention, not derivation. It must decrease with position; a log decays slowly, which roughly matches how lists are read
Recall@k against MRR	Recall ignores order, so it suits a first stage. MRR stops at the first non-zero grade, so it cannot see a better answer further down
The reranking ceiling	A derivation: reordering a set cannot add to it, so the candidate set bounds everything the second stage can achieve
Cross-encoder cost	One forward pass per document per query, so a million documents cost a million passes. Hence two stages
What forces symmetry	Putting both sides through the *same function*, as chapter 24’s dense retrieval does. Shared weights alone do not: one model given a `query:` or `passage:` marker is not symmetric
What merely permits asymmetry	A two-tower bi-encoder and a cross-encoder are *free* to be asymmetric, not guaranteed to be

Review Questions

1. What is BM25 doing, roughly?
2. What is the difference between a bi-encoder and a cross-encoder, and where does each belong?

3. What is nDCG, and why is there a logarithm in it?
4. You have two rankings of the same documents. How do you combine them?
5. Your reranker is not helping. What do you check first?
6. Is hybrid retrieval better than dense retrieval?

Review Answers

1. It scores a document by the query terms it contains, and the way to say it is as three objections answered in order. Term frequency: a document that mentions *refund* four times is plausibly more about refunds than one that mentions it once. Inverse document frequency, because term frequency alone rewards the wrong words — *the* appears everywhere and means nothing — so a term is weighted by how rare it is across the collection. Document length, because long documents mention everything more often without being more about any of it. Then the two properties worth knowing above the formula. **Term frequency saturates**: the contribution climbs to a ceiling of $(k_1 + 1)$ IDF, so a hundred mentions are worth about 2.5 IDF and not a hundred times one mention, which is what stops keyword stuffing from working — in the chapter's corpus a document of six *refund*s lost to one matching two terms once each. And the classic IDF goes negative for a term in more than half the collection, which is why the modern form has a +1 inside the logarithm. Finish by saying k_1 and b are tuned constants rather than derived ones, because knowing which parts of a formula are chosen is the thing that separates understanding it from reciting it.
2. A bi-encoder puts the query through a model and each document through a model separately, then compares the two vectors with a cosine. A cross-encoder puts the query and one document through a model *together* and gets one score for the pair. Two consequences follow and they are what the question is really about. **Cost:** a bi-encoder's document vectors do not depend on the query, so they are computed once in advance and query time is one forward pass however large the collection; a cross-encoder cannot precompute anything, so scoring a million documents is a million forward passes for one query. **Shape:** a cross-encoder's score is a function of the ordered pair, so it can be asymmetric and can relate the two texts directly, which is exactly what chapter 25 showed a *shared* encoder with a cosine cannot do for a directed question. Then the correction that most answers to this question get wrong: a bi-encoder is not automatically symmetric either. What forces symmetry is putting both sides through the *same function*, which chapter 24's dense retrieval does. A two-tower bi-encoder uses separate query and document encoders, so its score is $\cos(f(q), g(d))$ with $f \neq g$ — precomputable *and* directed. So does a single shared model told which side it is encoding by a `query:` or `passage:` marker, which is how much of current practice does it, and which shows that *shared weights* and *symmetric* are not the same claim. So each belongs where its cost allows: the bi-encoder over the whole collection, the cross-encoder over a candidate set of about a hundred that the first stage produced. And the limit worth adding: the second stage can only permute what the first stage found, so a cross-encoder can-

not rescue an answer that never made the candidate set.

3. nDCG is a ranking score that sees both how good each result is and where it sat. Build it in three steps. DCG adds up each result's grade divided by a discount that grows with its position, $\sum g_i / \log_2(i+1)$, so a good answer low down counts for less than the same answer at the top. The *normalised* part divides that by the DCG of the best possible ordering of the same documents, which makes the number comparable across queries that have different numbers of relevant documents. On the chapter's ranking that is 3.9307 over 6.3235, or 0.6216. As for the logarithm, the honest answer has two halves and the second is the one that impresses. **Why a decreasing discount at all** is obvious: position matters. **Why specifically a logarithm is a convention and not a derivation** — it decays slowly, so the gap between positions 1 and 2 is large and the gap between 10 and 11 is small, which roughly matches how people read a list; that is a design intuition, not a theorem, and other discounts exist. What *is* derivable is that the base does not matter: since $\log_c x = \ln x / \ln c$, changing base multiplies every term by the same $\ln c$, so it scales the DCG and the ideal DCG equally and the ratio is unchanged — base 2, 10 and e all give 0.621602, and 3.9307/5.6708 is $\ln 2$. What is exact is the cancelling; those two printed figures are rounded, so their quotient matches $\ln 2$ only to six decimals. The thing that genuinely does not cancel is the gain convention: using the grade directly gives 0.6216 where $2^g - 1$ gives 0.5193, so two nDCGs from different tools are not comparable until you know both conventions and both k.
4. Not by adding the scores, and the reason is the first thing to say: neither score has a unit. BM25 ran from 0 to 5.5089 on the chapter's corpus while its stipulated cosines ran 0.6198 to 0.9891 — and the ranges are illustrative, the absence of a common unit is not — so a sum is dominated by whichever happens to have the wider range, and any rescaling that fixes it is a weight you chose rather than one the data gave you. Chapter 24 already established that a raw cosine means nothing on its own. The standard answer is reciprocal rank fusion: throw the scores away, keep the positions, and give each document $\sum_r 1/(k + \text{rank}_r(d))$ with k usually 60. It needs nothing from the two systems but an ordering, which is precisely why it works where addition does not. Then give the two caveats, because they are what show you have used it. It discards information, so a document ranked first-then-second ties *exactly* with one ranked second-then-first — 0.032522 in the chapter. And fuse only the top of each list: a lexical ranking's tail is all zero scores whose order is arbitrary, and feeding that into the fusion gives it votes it has not earned.
5. Whether the answers are in the candidate set, before anything else. The second stage can only reorder what the first stage handed it, so if the right document is not in the hundred, no reranker can place it anywhere — and that is a derivation, not an observation: reordering a set cannot introduce an element not in the set. The measurement to make is the first stage's recall at the candidate-set size. The chapter's own numbers show it: a *perfect* reranker, reordering by the human's own grades, scored 0.6740 on three candidates and 1.0000 on six, and the difference is entirely whether a grade-3

document was in the set. So check recall@c first; if it is low, the money belongs in the first stage — a better embedding model, or lexical retrieval alongside it — and not in the reranker. How many candidates to keep is its own trade and chapter 28's to make. Only once recall@c is high does the reranker's quality become the binding constraint.

6. Nobody knows from two queries, and saying so is the answer. The chapter tried it: lexical scored 0.9659 across its two queries and dense 0.8155, and the fused score could not be reported as a number at all. On the identifier query the right note and the wrong note **tie exactly** at 0.032522 under reciprocal rank fusion, because one is `(1, 2)` and the other is `(2, 1)` and the formula cannot tell those apart — so the fused nDCG@5 is 1.0000 or 0.6309 depending on a tie-break, the fused mean is a range of 0.7814 to 0.9659, and dense's 0.8155 sits inside that range. A measurement that moves 0.19 on a tie-break is not evidence about anything.

Then refuse the formulation most people offer, because it is the part an interviewer is listening for. **Fusion is not a hedge that trades the best case for a better floor.** It is not bounded by its inputs in either direction, and both halves of that are worth being able to show. *Above:* a document ranked 2 in both lists scores 0.032258 and beats documents at `(1, 8)` and `(8, 1)` on 0.031099, so fusion can score 1.0000 where both of its inputs score 0.6309. *Below:* the same `(2, 2)` document scores 0.032258 while documents at `(1, 3)` and `(3, 1)` each score 0.032266, so it falls to third — fusion 0.5000 against 0.6309 in both inputs. The reason is that $1/x$ is convex, so for a fixed pair of positions, spreading the two ranks apart beats keeping them together: $1/61 + 1/63 > 2/62$. So the answer is a method rather than a verdict: write an answer key, run both configurations over the same queries — enough of them that one tie cannot move the result — and compare them paired with an interval, which is chapter 20. Whether hybrid helps depends on the mix of queries your users actually send, and nothing deduces that.

> **You understand this when** you can say why two retrieval scores cannot be added and what reciprocal rank fusion gives up instead — and name the one check to run before blaming a reranker, with the one-sentence reason it is a derivation rather than a guess.

PART III

Apply



What to ignore for now: scale and deployment. Everything here runs on one machine.

PART IV

What Next

APPENDIX A

Cheat Sheet

The numbers, defaults and formulas worth carrying in your head, in the order the book introduces them. Every line came from the chapter that teaches it, so nothing here was written from memory.

Part I · Mental Model

Item	Value or formula	Ch
What cost scales with	The amount of text moved, in and out. Not the difficulty of the question	1
The two rates are not equal	Text you get back costs several times text you send — about 4× in the book's worked shape	1
Cost of a feature, per call	(words in × rate in) + (words out × rate out), then × calls × days	1
Whole-run success over n steps	per-step reliability to the power n	4
Ten steps at nine-in-ten	0.9^10 = 0.349 — about a third of runs finish	4
Ten steps at ninety-nine in a hundred	0.99^10 = 0.904 — the same loop, now usable	4
What that means for a demonstration	A system succeeding a third of the time shows you a good run roughly every third try	4

Part I · Inside the Model

Item	Value or formula	Ch
Dot product	$a \cdot b = \sum_i a_i b_i$	8
Cosine similarity	$a \cdot b / (\|a\|\|b\|)$, between −1 and 1	8

Item	Value or formula	Ch
Softmax	$p_i = \exp(z_i) / \sum_j \exp(z_j)$	8
Matrix shape rule	(m × k) times (k × n) gives (m × n); the inner numbers must match	8
Logarithms turn products into sums	$\ln(xy) = \ln x + \ln y$	8
e	About 2.718. e^x is positive for every x	8
Quadratic growth	$O(n^2)$: ten times the input, a hundred times the work	8
Embedding table size	vocabulary × width numbers. 128,000 × 4,096 = 524,288,000	9
The output step	One score per vocabulary entry, so the final shape is (1 × d) times (d × vocabulary)	9
What one forward pass yields	Exactly one next-token distribution	9
Passes for a reply	One per token produced, each over an input one token longer	9
The three top-level shape numbers	Vocabulary, width, depth. There are more inside a layer, which is Chapter 12's	9
Rule of thumb, English prose	1 token ≈ 4 characters ≈ ¾ of a word; 1,000 tokens ≈ 750 words	10
What a token really measures	Frequency in the corpus the vocabulary was built from, not meaning or length	10
The merge step	Merge the commonest adjacent pair; recount; repeat	10
Why a word is one token	It was common enough for a merge to be spent on it	10
Which half is dear	Output, at several times input, and one forward pass per token	10
What a token count is a fact about	One particular model. A count does not carry across vocabularies	10
The worst case	One token per byte — the most byte-pair encoding can ever charge you	10
The attention formula	$\text{softmax}(QK^\top / \sqrt{d_k})\,V$	11
What the three views are for	Query = what I want, key = what I offer, value = what I give	11
Why divide by $\sqrt{d_k}$	A dot product grows with width; without it softmax saturates and attention stops blending. The *square root*, because the slot products partly cancel, so the sum grows like $\sqrt{d_k}$	11
The score grid	$n \times n$, one entry per pair of positions	11
Attention cost	$O(n^2 d)$ — double the tokens, four times the work	11
What a row of weights is	Positive, adds to exactly 1, one number per visible position	11

Item	Value or formula	Ch
Why positional encoding exists	The formula's output is a sum, so *the formula* is blind to the order of its inputs. Masked attention is not — the mask is positional	11
What a layer does	$x \leftarrow x + \text{attention}(x)$, then $x \leftarrow x + \text{feedforward}(x)$	12
Attention block parameters	$4d^2$ — four matrices of $d \times d$	12
Feed-forward block parameters	$2 \times d \times d_{ff}$, with the middle width d_{ff} between about $2.5d$ and $4d$	12
Which half is bigger	Feed-forward, at roughly 57 per cent of a layer	12
Where the parameters are	In the layers — about nine in ten, at a common shape	12
Matrices compose	$W_2(W_1x) = (W_2W_1)x$, which is why a block without its activation is one matrix	12
Temperature	$p_i = \exp(z_i/T) / \sum_j \exp(z_j/T)$	13
What temperature does	Below 1 sharpens, above 1 flattens, never reorders	13
Temperature and greedy	No effect whatsoever — greedy takes the largest logit either way	13
Top-p against top-k	Top-p keeps the smallest set reaching p; top-k keeps a fixed count	13
The order of operations	Temperature, then the cut, then draw	13
Why $T = 0$ is not a guarantee	Float order, batching, expert routing, provider change	13
The objective	$\mathscr{L} = -\frac{1}{N} \sum_i \log p(t_i \mid t_1 \ldots t_{i-1})$	14
Perplexity	$\exp(\mathscr{L})$; a uniform choice among V options gives V	14
The update rule	$\theta \leftarrow \theta - \eta \nabla_\theta \mathscr{L}$ — and that is the edge of this book	14
The three stages	Pretraining, supervised fine-tuning, preference optimisation	14
Where knowledge comes from	Pretraining, overwhelmingly	14
What the loss ignores	Everything except the probability of the token that actually came next	14
Key–value cache	$2 \times$ layers $\times d \times$ tokens $\times$ bytes	15
At a common shape	32 layers, d 4,096, 2 bytes: 512 KiB per token	15
An 8,000-token conversation	3.91 GiB of cache, on top of the weights	15
Weights at 7 billion parameters	14 GB at 16 bits, 7 GB at 8, 3.5 GB at 4	15
What limits each phase	Prefill: arithmetic. Decode: reading the parameters the pass uses	15

What batching shares, and what it does not	The parameters are read once and shared; each request's own cache is not. So the saving is large at short contexts and small at long ones — about 4x at 8,000 tokens each	15

Part II · Use

Item	Value or formula	Ch
The four parts	Instructions, examples, output format, supplied information	18
The order to put them in	Specification first, data last, so the unchanging part is at the front	18
Cost of a conversation	Every turn re-sends the whole list; the total over n turns grows like n^2	18
Worked: ten turns	14,250 input tokens against 3,000 for ten separate questions	18
What never gets dropped	Your specification: the system message, and the example turns if that is where they live	18
What to change per experiment	One block	18
Backoff with jitter	`wait = random.uniform(0, min(cap, base * 2 ** attempt))`	19
Attempts and waits	n attempts make $n - 1$ waits. Five attempts at base 0.5 is 7.5 s at worst	19
How many at once	rate × duration — 120 a minute at 2 seconds each is 4. It bounds concurrency, not rate, so leave a margin	19
Timeouts	Three: connect, read between chunks, total. Derive the *total* from the token cap	19
Retryable	429, 5xx, timeouts. *Not* a malformed request, a bad key, an over-length one, or a 429 from an exhausted quota	19
What to stream	To a person, yes. One answer you must validate whole, no. Independent units, yes	19
`asyncio.gather`	One failure loses every result unless you pass `return_exceptions=True`	19
The resume file	One flushed line per item, or per `(item, stage)` for a pipeline. Keyed by input, holding the result	19

Item	Value or formula	Ch
Spread of a score	$\sqrt{np(1-p)}/n$ — one standard deviation over *which cases you drew*. 40 cases at 0.70 is 0.072, so two of them is the ±0.14 that spans 95 per cent. Run-to-run spread is a different number and is smaller unless the model is uncertain on every case	20
The bootstrap	Resample your cases with replacement, score each, take the 2.5th and 97.5th percentiles	20
What to put an interval on	The *difference*, paired. Never two separate intervals and a look	20
Cost of resolution	Width shrinks as $1/\sqrt{n}$, so halving it costs four times the cases	20
What carries the comparison	Only the cases where the two versions disagree. Count them and say them out loud	20
What pairing is worth	Here, 0.400 wide down to 0.250. It cancels the variation the two versions share, so the more they agree, the more it buys	20
Cases against trials	Cases for the range of inputs, trials for the model's variation. Neither fixes the other	20
How many trials	Averaging k judgements divides a case's own variance by k. Two or three, then stop	20
Typical interval vs detection	A table built from a fixed mix shows the usual interval. Whether a real draw clears zero is a separate number, and lower	20
Why not prose	No contract, so your code guesses. Right 95 times in 100 is a defect 5 times in 100	21
The four layers	Parses · right shape · values possible · values true. Only the first three are free	21
Rule of three	0 failures in n → the rate may be $3/n$. Exactly, $1 - 0.05^{1/n}$	21
0 in 40	Consistent with a 7.5 per cent failure rate	21
The repair line	Repair the envelope, never the contents. Needing to know what the model meant makes it a retry	21
Failure options	Reject · retry · repair · record. Never a silent default	21
The mechanism	Score → $-\infty$ for forbidden tokens → softmax gives 0 → pick. One line, and only while one token survives	22
What a constraint costs	The mass it deletes, which costs you only when that was the answer rather than the envelope	22
Why engines differ	Schemas are over characters, generation is over tokens; the legal-token set is precomputed per state	22
Both wrong checks	First character only is too loose (admits `{}`); a short window is too strict (rejects `{"s`)	22

Item	Value or formula	Ch
Why strict modes can hurt	No room for intermediate tokens, and the model's only scratchpad is its own output. Also: a schema too narrow to hold the right answer	22
The fix, and its order	A `reasoning` field *before* the answer field — and only if the engine fixes field order	22
What it never buys	Correctness, or the parts of your schema the engine did not compile	22
The five steps	You send · model proposes · **you validate and decide** · you run · you return	23
Who decides	The model decides what to *propose*. Your code decides what *runs*	23
Why not in the instructions	Nothing in the loop checks a proposal's *values* against them except your code. A strict mode holds a shape, and an enum; not a range, and never a fact about the world	23
What the model never sees	The function, and every constraint you kept on your side	23
Description cost	Input on every call. A real tool schema is around 200 characters, so 30 tools is of the order of 1,500 tokens	23
The refusal to send back	The specific reason, so the next proposal can be better	23
What you get back	One vector, fixed width, per text. A few hundred to a few thousand numbers	24
The comparison	Chapter 8's cosine. Scale-free: multiplying a vector by 10 does not move it	24
What a high cosine is	A fact about two vectors. **Not** evidence that two texts mean the same	24
Dense retrieval	Embed the documents once, embed the query per search with the same model, sort by cosine	24
What decides the model	Your language, your domain, your document length. Length first: it is the one that gets forgotten	24
The hard limit	Every embedding model has a maximum input length. Past it, text is cut off, usually with no error	24
A raw cosine	Has no meaning on its own. Not comparable across models, queries or corpora — rank, do not threshold	24
What a benchmark is	One study, somebody else's corpus. A shortlist of three, not a choice of one	24
The input only you have	The answer key: which document each query is about	24
Why identifiers fail	A sketch, not a calculation: the vector holds the whole text at a fixed width, so a string appearing once competes for that room with everything else	25

Item	Value or formula	Ch
What kind of claim that is	An observation on Chapter 6's ladder. The mechanism gives a reason to expect it, not a derivation	25
The illustration	Two notes differing in one character, on stipulated vectors: 0.99983 apart, so a distance of 0.00017	25
Similarity against relevance	$\cos(a, b) = \cos(b, a)$ always. Relevance depends on which is the question	25
The negation case	On stipulated vectors, a true answer and its negation sat closer to each other than either did to the query	25
What the negation case proves	Nothing about which one ranks first. It is a claim about the score: you get no handle on that ordering, so you cannot tune or threshold it	25
What inspection finds	A wrong result that is present. Not a right result that is missing from the list	25
What a five-query demo is	A demonstration on Chapter 6's ladder: it can work once	25
BM25	'(\sum_{t \in q} \mathrm{IDF}(t) \cdot \dfrac{f_{t,d}(k_1+1)}{f_{t,d} + k_1(1 - b + b	d
Its two knobs	$k_1 \approx 1.5$ controls saturation, $b \approx 0.75$ controls length normalisation. Tuned constants, not derived	26
What $k_1 = 0$ does	Makes term frequency irrelevant: the factor is 1 whatever the count, so only presence matters	26
Term-frequency ceiling	A term's contribution approaches $(k_1 + 1)$ IDF. At $k_1 = 1.5$, 100 mentions reach 98.5% of 2.5 IDF	26
Inverse document frequency	$\ln\left(\frac{N - n_t + 0.5}{n_t + 0.5} + 1\right)$. Without the +1 it goes negative for a term in over half the collection	26
Reciprocal rank fusion	$\sum_r 1/(k + \text{rank}_r(d))$, conventionally $k = 60$. Uses ranks only, so no score comparability is needed	26
DCG	$\sum_{i=1}^{k} g_i / \log_2(i + 1)$	26
nDCG	DCG divided by the DCG of the ideal ordering of the same documents	26
Change of base	$\log_c x = \ln x / \ln c$, so a base-c DCG is $\ln c$ times the natural-log one	26
The log base cancels	A derivation: changing base scales DCG and ideal DCG by the same $\ln c$. Base 2, 10 and e all gave nDCG 0.621602	26
The gain convention does not	Grade directly gave 0.6216; $2^g - 1$ gave 0.5193 on the same ranking	26
Why a logarithm at all	Convention, not derivation. It must decrease with position; a log decays slowly, which roughly matches how lists are read	26

Item	Value or formula	Ch
Recall@k against MRR	Recall ignores order, so it suits a first stage. MRR stops at the first non-zero grade, so it cannot see a better answer further down	26
The reranking ceiling	A derivation: reordering a set cannot add to it, so the candidate set bounds everything the second stage can achieve	26
Cross-encoder cost	One forward pass per document per query, so a million documents cost a million passes. Hence two stages	26
What forces symmetry	Putting both sides through the *same function*, as chapter 24's dense retrieval does. Shared weights alone do not: one model given a `query:` or `passage:` marker is not symmetric	26
What merely permits asymmetry	A two-tower bi-encoder and a cross-encoder are *free* to be asymmetric, not guaranteed to be	26

APPENDIX B

Interview Bank

A hundred questions of the kind this book prepares you to answer, each keyed to the chapter that holds the answer.

The questions come first, and they came before the chapters — from question 14 onward. They were written from the contents, so they are not a summary of what got written: they are the standard it has to meet. A chapter is not finished until its questions can be answered from it, and if one cannot, the chapter is short of something rather than the question being unfair.

Questions 1 to 13 are the exception, and they are weaker for it. They cover chapters 1 to 7, which were finished on 23 October; this bank was written on 27 October, by somebody who had read them. Questions 3, 7, 9, 11 and 13 land close to a review question their chapter already asks. **A question reverse-engineered from an answer cannot test whether the answer is there.** Part I was checked against these thirteen and came through — twelve answerable, one needing a second chapter in its key — but that is not a clean pass and should not be read as one. From chapter 8 on, the bank genuinely precedes the chapter, and the test means what it says.

The answers arrive as the chapters do. Each will be 150 to 180 words, which is about what sixty seconds of speech holds: the mechanism, the trade-off, and how you would check it. An answer written before its chapter would be written from memory, which is the one way this book is not allowed to be written. Where the *Ch* column points at a chapter that does not exist yet, the question is still the right question.

Eight chapters carry no question, on purpose. Chapter 7 is this book's own map, and chapter 57 is advice about what to leave out — neither is something anybody will ask you. Labs 40, 42, 43, 44, 46 and 48 are tested through the chapters they rest on: lab 44 is covered by the retrieval questions on chapter 26, lab 46 by those on chapters 30 and 31, and so on. Every other chapter in the book has at least one question, and the six labs that do appear here are the ones you would be asked to describe rather than explain.

Part I · Mental Model

#	Question	Ch
1	What is a language model, described as a component in a system rather than as a product?	1
2	What can a model not know, and why does a better model not fix it?	1
3	What do you choose when you use a model, and what do you not choose?	1
4	Name the parts a contemporary AI application combines, say what each is for, and say what each one costs you.	2–3
5	What is in the context of a single request, and what decides what goes in it?	2
6	Why does retrieval exist? Name the failure it removes.	3
7	Every part you add removes one failure and introduces another. Give two examples.	3
8	What is an agent, and what is it not?	4
9	Each step of your agent is right nine times in ten and the task needs ten steps. What is your success rate, and what does that tell you about a demonstration that worked?	4
10	Where does the time actually go on a project like this?	5
11	A user reports a wrong answer. Name the things that could have caused it, and say what you would need in order to tell them apart.	5
12	Of everything you have learned about this field, what would you bother memorising and what would you look up? Why that split?	6
13	A provider announces a model thirty per cent better at coding. What do you want to know before you repeat that?	6

Part I · Inside the Model

#	Question	Ch
14	What does a dot product measure, and why does that make it the basis of similarity?	8
15	What does one forward pass of a model actually produce?	9
16	What is in the embedding table, and what does a single dimension of it mean?	9
17	Why is text turned into tokens rather than words or characters?	10
18	Why does code cost more tokens than ordinary English?	10
19	Why does a model miscount the letters in a word?	10
20	Walk me through attention on a short sentence.	11

#	Question	Ch
21	Why is the attention score divided by the square root of the key dimension?	11
22	Why does attention cost grow with the square of the input length, and what follows from that in practice?	11
23	What do several attention heads give you that one does not?	11
24	What is the residual stream, and what does a layer do to it?	12
25	Why does “where is that fact stored in the model” have no clean answer?	12
26	What does temperature do to the numbers?	13
27	Why is temperature zero still not a guarantee of the same output twice?	13
28	Name the training stages and say what each one changes.	14
29	Why is the first token slow and every token after it fast?	15
30	Your prompt doubles in length. Why does the work more than double?	15
31	What is the key–value cache, and how big is it?	15
32	Why can a very large model be cheap per token?	15
33	Why is hallucination the default behaviour rather than a defect?	16
34	Why does asking a model to work step by step improve accuracy, and why is the visible reasoning not a reliable account of how it answered?	16
35	An interviewer asks why depth helps. You have not studied that. What do you say?	17

Part II · Use

#	Question	Ch
36	What are the separable parts of a prompt, and why does separating them matter?	18
37	Your conversation history will not fit in the context. What do you drop, and how do you decide?	18
38	Why does current practice weight what the model receives over how the instruction is worded?	18
39	Your script works on ten documents and dies on a 429 at document four thousand of ten thousand. What do you do?	19
40	Why can you not stream a response that you have to validate?	19
41	What is idempotency, and where does it matter in a pipeline like this?	19
42	A multi-step pipeline fails half way. How do you resume it?	19
43	How do you decide whether prompt B is better than prompt A?	20
44	What is a bootstrap interval, and why use one here rather than a named distribution?	20

#	Question	Ch
45	How many cases and how many trials are enough?	20
46	Why must a program never trust prose?	21
47	You are asking for JSON. How do you measure how often you do not get it, and what do you do with the failures?	21
48	How is a format constraint actually enforced while the model generates?	22
49	When can a strict output format make the answer worse?	22
50	Walk me through a tool call. Who decides what runs?	23
51	Where does validation of a tool call belong, and why not in the instructions?	23
52	How would you choose an embedding model for your own data?	24
53	Why is a public benchmark score a starting point rather than a verdict?	24
54	Give me a query that meaning-based search gets wrong, and say why.	25
55	Why is similarity not the same thing as relevance?	25
56	What is BM25 doing, roughly?	26
57	What is the difference between a bi-encoder and a cross-encoder, and where does each belong?	26
58	What is nDCG, and why is there a logarithm in it?	26
59	You have two rankings of the same documents. How do you combine them?	26
60	Why is document parsing the upstream bottleneck for everything downstream?	27
61	What does a chunk boundary decide?	27
62	Long context or retrieval?	28
63	What must a citation actually support, and how would you check that it does?	28
64	What does an approximate index buy you, and what does it cost?	29
65	Each user may see only their own documents. How does that change your retrieval?	29
66	You have forty bad outputs and no metrics. What do you do first?	30
67	Why let the failure modes choose the metrics rather than measuring what is conventional?	30
68	When do you use a model as a judge rather than code?	31
69	How do you establish that a judge can be trusted?	31
70	What is position bias, and what does it do to your numbers?	31
71	Why measure retrieval and generation separately?	32
72	What is a claim-level check on a grounded answer?	32
73	Where do your labelled evaluation cases come from?	32
74	What goes into a regression set, and when does it run?	33
75	What is criteria drift, and how do you notice it?	33

#	Question	Ch
76	What do you record in a trace, and what do you deliberately not record?	33
77	When is an agent warranted instead of a path your code controls?	34
78	Name the fixed-path patterns and say what each is for.	34
79	What is compaction, and how do you prove nothing important was dropped?	35
80	Why treat an agent's own memory as untrusted input?	35
81	What makes a tool good for a model to use rather than good for a programmer to call?	36
82	What is the Model Context Protocol, and what problem does it solve?	36
83	When do several agents help, and how do multi-agent systems fail?	37
84	What is pass@k, how does it differ from pass^k, and which one should worry you?	38
85	Prompt, retrieve or tune? How do you choose?	39
86	What does owning a fine-tuned model cost you that prompting does not?	39

Part III · Apply

#	Question	Ch
87	Walk me through something you built end to end, and tell me the numbers you got.	45
88	How did you compare two prompts in a way you could defend?	41
89	You measured a fixed path against an agent on the same task. What did you find?	47
90	How did you reduce cost per request, and what did it cost you in quality?	49
91	You attacked your own system. What did you try, and what happened?	50

Part IV · What Next

#	Question	Ch
92	Why can a model not reliably tell an instruction from data?	51
93	A web page your agent reads tells it to send your customer list somewhere. What happens, and what would have stopped it?	51
94	What is the rule about combining untrusted input, sensitive access and external action?	51
95	Where does model output reach a shell, a query or a page, and what do you do about it?	51
96	Your system is correct and nobody will wait for it. What do you measure, and which lever do you pull?	52
97	The bill arrived and you cannot account for it. How do you find out where it went?	53
98	It worked last week and nothing in your code changed. What do you investigate, and what should have been in place?	54
99	What do you not know yet, and how would you go about learning it?	56
100	Design a support assistant that answers from a company's own documents. Talk me through it.	55

APPENDIX C

Glossary

Every term the book defines, with the chapter that owns it. A term is defined in exactly one chapter; every other chapter refers to it rather than redefining it.

Term	Meaning	Ch
429	*Too many requests.* The provider's rate limit, on requests per minute and on tokens per minute. Retryable when you are going too fast; the same status comes back when the account is out of credit, which no wait fixes	19
Accountability	Somebody who can be asked to answer for a decision, and who bears a consequence for it. A property of people and organisations, never of a component	3
Activation function	The step in the feed-forward block that is not a matrix multiplication, such as keeping positives and zeroing negatives. Without it the block's two matrices would collapse into one	12
Active parameters	The parameters that actually run for a given token, as against the total the model holds. Cost follows the active figure; the name usually quotes the total	15
Agent	A system in which the model decides what to do next, repeatedly, rather than your code deciding the sequence	4
Answer key	Which document each of your queries is actually about. The only input in the procedure that has to come from you	24
Approval point	A step the loop cannot pass without a person, enforced in your code rather than requested in the instructions	4
Assistant message	What the model said earlier in the conversation, sent back to it by your code. The model does not remember saying it	18
Attention	The step in which each position takes a weighted blend of information from the others, in amounts computed from the content rather than fixed in advance	11

Term	Meaning	Ch
Attention block	The half of a layer that moves information between positions — Chapter 11's formula in full. The only part of a model that does	12
Attention head	One complete set of query, key and value matrices and the weights they produce. A layer runs several in parallel over slices of the width, and a final matrix mixes their outputs	11
Attention weights	One row of the grid after scaling, masking and softmax: positive numbers adding to 1, saying how much of each position to take	11
Autoregressive	Producing output one token at a time, with each token appended to the input before the next pass	9
Backpropagation	The procedure that computes a derivative for every parameter in one backward sweep through the model. How the gradient of Chapter 14's update rule is obtained. Not taught in this book	17
Base model	A model that has had only pretraining. It continues text rather than answering, which is what the later stages change	14
Batch endpoint	Upload requests, collect results hours later, pay substantially less. For work nobody is waiting on	19
Batching	Running many requests through the same forward pass, so one read of the parameters serves all of them. Close to free up to a point — each request still reads its own cache and does its own arithmetic — which is why the price per token is low and why your neighbours affect your output	15
Bi-encoder	Query and document encoded separately and compared by a cosine, so document vectors are precomputable. With one shared model the score is symmetric	26
Big-O	How a cost grows as an input grows, with constants and smaller terms deliberately discarded. Not a measurement of time	8
BM25	The standard lexical scoring function, combining term frequency, inverse document frequency and document length, with a saturating term-frequency curve	26
Bootstrap	Estimating how much a statistic would move by resampling your own cases with replacement. One procedure for every statistic you score, where a closed form is a different formula each time	20

Term	Meaning	Ch
Byte-pair encoding	Building a vocabulary by starting from the 256 single bytes and repeatedly merging the commonest adjacent pair in a corpus until the vocabulary is the size wanted. Starting from bytes is what makes any text at all representable	10
Candidate set	The documents the first stage passes to the second. A reranker can only permute it, so it is a ceiling	26
Case	One input plus whatever marks the answer right or wrong. The unit the statistics count — and they count rows, so forty rephrasings of one question are worth nearer one case than forty	20
Causal mask	Setting every score for a later position to minus infinity before softmax, so it gets exactly zero weight. It makes a position's output depend only on itself and what precedes it, so the arithmetic is the same whether or not the rest of the sentence exists	11
Check	Anything that verifies the output before it is used, from a required shape through to a second model's opinion	2
Compound system	A system of several parts with the model as one of them, rather than as the whole product	2
Compounding error	Two things at once across a run: each step's chance of being right multiplies against the others, so a high per-step figure becomes a low whole-run one; and a wrong step's result is carried into the next step as though it were true	4
Compute-bound	A step whose speed is set by the arithmetic it has to do rather than by how fast its numbers arrive. Prefill is; a large enough batch becomes so	15
Concept owner	The single chapter that teaches a given concept. Every other chapter refers to it by number rather than re-teaching it, so the reader never meets three partial explanations of one thing	7
Confidence interval	The range a measurement would fall in most of the time if you repeated it. Here, the 2.5th to 97.5th percentiles of the resampled scores	20
Constrained decoding	Restricting each token choice to what the format allows, by editing the scores before the pick. The model is unchanged	22
Context	Everything sent to the model on one call: the instructions, any added material, and the request	2

Term	Meaning	Ch
Context window	The maximum number of tokens a model will accept at once. A cost limit rather than a memory, counting the reply as well as the request. The whole conversation is re-sent every call, and something is dropped — or the call refused — when it fills	16
Cosine similarity	The dot product divided by both lengths, which removes magnitude and keeps direction. Always between –1 and 1	8
Cross-encoder	Query and document passed through one model together, giving one score for the ordered pair. Not precomputable; free to be asymmetric	26
Cross-entropy	The training objective: the average of $-\log p$ over every position, where p is the probability given to the token that actually came next. Zero for a perfect prediction, unbounded for a confident mistake	14
Current convention	How competent engineers work at the moment: true, load-bearing and replaceable. It usually names a technique rather than a reason, and usually has a predecessor you can remember	6
Dated detail	A fact with a date on it — a name, a number, a version. It stops being true when somebody ships something or changes a price	6
Dead state	A grammar state with no legal token. It does not raise — the mask stops meaning anything and the decoder emits whatever was first	22
Decode	One forward pass per output token. Does almost no arithmetic and is limited by reading every parameter the pass uses out of memory once per token	15
Delta	One streamed piece: what to add to what you have, not the whole answer again	19
Dense retrieval	Embedding every document once, embedding the query at search time with the *same* model, and ranking by cosine	24
Depth	The number of layers in the stack. The last of the three numbers — vocabulary, width, depth — that describe most of a model's shape	9
Dimension	One slot in a vector. A position in a list, not a direction in space, and usually not nameable on its own	8

Term	Meaning	Ch
Direct preference optimisation (DPO)	The preference objective written as one formula: raise the preferred reply's probability relative to the rejected one, both measured against the model as it was before this stage. Stated in this book, not derived	14
Discounted cumulative gain	The sum of each result's grade divided by a discount that grows with position	26
Dispatcher	The code that takes a proposal, validates it, decides, and runs the function or refuses. The only participant that can refuse	23
Document frequency	How many documents in the collection contain a term at all	26
Document length	How many terms a document holds, compared with the collection average. BM25's third input, and the reason a long document is not automatically more relevant	26
Dot product	Multiply two vectors slot by slot and add the results. Large when both are large in the same slots — and also larger when either is simply longer	8
Embedding	The vector an embedding model returns for a given text. Its width is the model's, and it is the same for a word, a sentence and a page	24
Embedding model	A model whose whole job is to turn one piece of text into one vector of fixed width. No output tokens, nothing to continue	24
Embedding table	A matrix with one row per vocabulary entry. Looking a token up in it is how that token enters the model; the rows are called embeddings	9
Error analysis	The part of evaluation that means reading real outputs, sorting the failures into kinds, and working on the largest kind	5
Evaluation	Building and using something that tells you whether a system works: a set of cases, a way to judge each answer, and a number you can compare over time	5
Evidence strength	How much a claim is supported: announcement, demonstration, one study, practitioner report, settled practice — or a derivation, which stands outside the ladder because you can check it yourself	6
Examples	What a good answer looks like, shown rather than described. Part of the specification	18
Exponential backoff	Doubling the wait after each failed attempt, up to a cap, so a struggling system is not hammered	19

Term	Meaning	Ch
Feed-forward block	The half of a layer that works on each position alone: two matrices with an activation between them, the middle between about two and a half and four times the width of the ends. Holds more of a layer's parameters than attention does	12
Finish reason	The response's own statement of why generation stopped. What distinguishes a truncated reply from a malformed one	21
Forward pass	One journey through the model — lookup, the stack once, one distribution out. It produces exactly one distribution	9
Foundation model	A model trained once on general material, then used for many tasks it was not specifically built for	1
Generalisation theory	Why a model fitted to one body of text works on text it never saw — the bias–variance trade-off, and what learning theory proves about it	17
Graded relevance	A relevance judgement on a small scale rather than yes-or-no, so a partial answer and a full one can be told apart	26
Gradient	For each parameter, how much the loss would rise if that parameter rose. The update rule moves every parameter a small step against it. How it is computed is outside this book	14
Greedy decoding	Always taking the highest-scoring token. What "temperature zero" does in practice, since dividing by zero is not defined	13
Hallucination	Output that is fluent, plausible and not true. Not a malfunction: no step in the computation checks anything, so a plausible continuation is all the mechanism can produce	16
Hybrid retrieval	Running lexical and dense retrieval and combining their rankings	26
Idempotency	Doing an operation twice has the same effect as doing it once. What makes a retry after a timeout safe	19
Inductive bias	What a design assumes about the world before it sees any data	17
Inference	Running a finished model to get a reply. This is what "using a model" means	1
Initialisation	What the parameters are set to before training begins, and why the choice matters	17

Term	Meaning	Ch
Input tokens	Everything you send on a call — instructions, attached material and the request. Countable exactly before you send	10
Instructions	What to do, what not to do and who to be. Part of the specification, so it changes rarely and is paid for on every call	18
Intermediate reasoning	Emitting the steps of a problem as tokens so that later passes can condition on them. The only way to carry partial work forward, because one pass runs its layers once however hard the question is, and nothing persists between passes	16
Interview depth	Knowing something well enough to give the mechanism, the trade-off, and how you would check it, in about a minute of speech. Not exhaustive, and more than a summary	7
Inverse document frequency	A weight that rises as a term gets rarer, so a term almost every document holds barely moves a score	26
Irreversible action	Something a tool does that cannot be undone once it has happened — a refund, a message, a deletion. The reason a tool's reach is decided before it is offered	3
Jitter	Waiting a random amount up to the backoff rather than exactly it, so many workers do not retry in step	19
Key	The vector saying what a position offers to others, made with W_K. Scored against every query	11
Key–value cache	The stored keys and values for every position in every layer, so decode attends to earlier positions without recomputing them. Costs 2 × layers × d × tokens × bytes	15
Lasting principle	A reason for doing something, or a shape that does not move, which outlives the technique it was first attached to. It stops being true only if the reason itself is wrong	6
Latency	The delay between sending a request and receiving the reply, felt as a wait before the first word and a pace after it	1
Layer	A block of arithmetic taking one vector per position and returning one per position, same width in as out. Every layer has the same shape and its own numbers	9
Length	What a ruler would measure if the slots were coordinates: square every slot, add them, take the square root	8

Term	Meaning	Ch
Lexical retrieval	Scoring a document by the query terms it contains. Not a filter: every document gets a number and the list is sorted	26
Logarithm	The inverse of raising e to a power. It turns multiplication into addition, which is why it appears wherever many probabilities are combined	8
Logits	The raw scores from the final matrix multiplication, one per vocabulary entry, before softmax. Can be negative; only the differences between them matter	13
Masked mass	How much probability the constraint deleted at a step. Small means the constraint was free; large means it overruled the model	22
Masking	Setting a forbidden token's score to $-\infty$, which makes its probability exactly zero after the softmax — as long as one token survives. Mask them all and you get `nan`, not zero	22
Matrix	A grid of numbers, read as rows or as columns depending on what you are doing	8
Mean reciprocal rank	The average of the reciprocal rank over a set of queries	26
Memory-bound	A step whose speed is set by how fast numbers arrive from memory rather than by the arithmetic done with them. At two bytes per parameter, decode does about one operation for every byte it reads, on hardware built for hundreds	15
Merge list	The merges, in the order they were made. Kept with the model and replayed in that order to tokenise any later text	10
Mixture of experts	Replacing one feed-forward block per layer with several and routing each token through a few of them. The model holds all of them and runs some	15
Model	A service that takes text and returns generated text, charged by the amount of text	1
Next-token distribution	One probability for every entry in the vocabulary, adding to 1, saying how likely each is to come next. The entire output of one call	9
Normalisation	Rescaling the stream to a predictable range before each block reads it, so the numbers do not grow out of hand as layer after layer adds to them	12

Term	Meaning	Ch
Normalised discounted cumulative gain	A DCG divided by the DCG of the best possible ranking of the same documents, making it comparable across queries	26
Numerical stability	How quantities behave through long chains of arithmetic, and what keeps them in a usable range. The subject behind Chapter 12's one sentence on normalisation. Not taught in this book	17
Objective design	Why one expression rather than another is the right thing for a model to minimise. The question Chapter 14 states its preference objective without answering. Not taught in this book	17
Optimisation theory	The study of when a procedure like the update rule converges, to what, and how fast. Not taught in this book	17
Optimiser design	Choosing and adapting the step size and the form of the update. Not taught in this book	17
Output format	The shape the answer must take, stated rather than hoped for. Part of the specification	18
Output tokens	Everything that comes back. Priced several times higher than input, one forward pass each, and not knowable in advance	10
Paired comparison	Running both versions on the same cases and keeping the results together, so the part of case-to-case variation the two versions share cancels when you subtract	20
Parameter	One number inside the model, set by training and unchanged by inference. The count is what "a 7B model" reports	9
People	The reviewers and approvers who handle the consequential cases and carry responsibility for them	2
Perplexity	The exponential of the cross-entropy. Reads as the number of equally likely options the model is effectively choosing between; a uniform choice among V options has perplexity V	14
Positional encoding	How a token's position is put into the arithmetic, since the attention formula carries no notion of order. Added to the embedding before the stack, or applied as a rotation to queries and keys, or added to the scores themselves	11

Term	Meaning	Ch
Practitioner report	A survey or a set of accounts of what people say they do. It is a practitioner report however it was published, and it supports what people *do* far better than whether it works	6
Preference optimisation	Training on pairs of replies with a preference between them, to make the preferred kind more likely relative to the other. Changes which of its existing behaviours come out; nothing in its objective rewards adding knowledge	14
Prefill	The pass that processes the whole prompt, every position at once. Limited by arithmetic, whose attention part grows with the square of the prompt length and overtakes the rest on a long one. What you wait for before the first word	15
Pretraining	Running the loop on an enormous quantity of general text. Where essentially all of a model's knowledge comes from, and nearly all of the computation	14
Proposal	What the model emits — tokens your code reads as a request to run something. Nothing has happened yet	23
Quantisation	Storing each parameter in fewer bits. Halving the bits halves the memory and roughly halves the time spent reading, at a cost in precision that has to be measured	15
Query	The vector saying what a position is looking for, made by multiplying its vector by W_Q	11
Recall@k	The fraction of all relevant documents appearing in the top k. Blind to their order	26
Reciprocal rank	One divided by the position of the first relevant result, for a single query	26
Reciprocal rank fusion	Combining rankings by summing one over a constant plus each rank, using positions only and never the scores	26
Registry	The map from a name to a function your code is willing to run. The lookup that refuses a name the model invented	23
Regularisation	The techniques that stop a model fitting its training data too closely	17
Relevance	Whether this document answers that question, for this asker. A directed judgement, not a distance	25

Term	Meaning	Ch
Relevance judgement	One recorded decision that a given document does or does not answer a given query. The unit an answer key is made of	25
Repair	Editing a reply so it parses, restricted to things that were never part of the data: a code fence, text outside the braces, a comma JSON disallows	21
Representation learning	Why fitting a model to predict text arranges its vectors so that the arrangement is useful for other things. The question Chapter 9 refused. Not taught in this book	17
Reranker	The model that does the rescoring in the second stage — in current practice usually a cross-encoder	26
Reranking	Rescoring a small candidate set with a more expensive method than the one that produced it	26
Residual connection	Adding a block's output to the vector that went into it, rather than replacing it. What lets a layer contribute nothing and pass its input through	12
Residual stream	The running vector passing up through the stack, which every block adds into. By the end it is a sum of every contribution made along the way, with nothing marking which is which	12
Retrieval	Searching your own collection for the passages that bear on a request, so they can be put into the context	2
Rubber-stamping	Approval given without real attention, usually because the volume is too high for any. It costs what review costs and delivers none of its value	3
Rule of three	With no failures in n tries, the rate can still be about $3/n$. What to use when a bootstrap has no failure to resample	21
Scaling laws	The relationships between model size, data, compute and loss, and their derivation	17
Schema	The list of claims your code makes about a reply — fields, types, allowed values, ranges — and enforces on every one. A contract you check, not a request you hope was honoured	21
Semantic search	What dense retrieval is usually called. The same thing, in the words people use when they complain about it	25
Semaphore	A counter that bounds how many jobs run at once. Three words of code, and the difference between concurrency and attacking your own account	19

Term	Meaning	Ch
Server-sent events	A long-lived HTTP response carrying one small event per line as the server produces them. How a streamed reply arrives	19
Softmax	Turns a list of scores into positive numbers that sum to 1, preserving their order and turning each gap between scores into a multiple between shares	8
State machine	What a schema compiles into: a set of states, each with the tokens allowed next. Engines call it an automaton	22
Step	One pass through the loop: the model decides, something runs, the result comes back	4
Stop condition	A string you nominate that halts generation when it appears. Providers call it a stop sequence; it is not Chapter 4's stopping condition. One of the ordinary ways a reply ends, alongside the model's own stop token and your maximum	13
Stop token	An entry in the vocabulary meaning *finished*, scored and sampled like any other token. Often called an end-of-sequence token. A model stops talking by predicting that it has stopped	13
Stopping condition	The set of rules that end a run — the model finishing, a step limit, or your code refusing	4
Strict mode	A provider's facility for enforcing a schema during generation, as against asking for it in the prompt	22
Subword	A token that is part of a word rather than a whole one. What makes it possible to represent a word the vocabulary has never seen	10
Supervised fine-tuning	Running the same loop with the same objective on curated request–response pairs. Turns a model that continues text into one that answers	14
Supplied information	The part of a prompt that is different on every call. The only one of the four parts that is data rather than specification	18
System message	The message holding your instructions and usually the output format. Your specification, set once per conversation and never the thing to drop when the history will not fit — nor are the example turns, if that is where your examples live	18
Temperature	A number the logits are divided by before softmax. Below 1 sharpens the distribution, above 1 flattens it, and it never changes the order	13

Term	Meaning	Ch
Term	The unit a lexical index counts — usually a lower-cased word, sometimes stemmed. Not the model's token from chapter 10	26
Term frequency	How often a term appears in one document	26
Token	One entry in a model's vocabulary — usually a piece text is cut into. Beyond the single bytes, which are always there, which pieces earned an entry is a matter of frequency in the corpus the vocabulary was built from — not of meaning or length	10
Token bucket	A limiter on the *rate* rather than on how many run at once, for when the requests-per-minute limit is the one that hurts	19
Tokeniser	The program that cuts text into tokens and maps them to indexes, using one model's vocabulary and merge list. A count does not carry from one model's tokeniser to another's	10
Tool	An action the model may ask for. The model proposes it; your code decides whether to run it	2
Tool description	The text your code sends about one function: a name, a sentence and the parameter names and types. Not the function, and not your constraints	23
Tool message	A result your code returns after the model asked for something. Chapter 23	18
Top-k	Keeping only the k highest-scoring tokens and renormalising. Keeps the same number of options however confident the model is	13
Top-p	Also called nucleus sampling. Keeping the smallest set of tokens whose probabilities add to at least p, then renormalising. Narrow where the model is confident, wide where it is not, at the cost of not knowing in advance how many survive	13
Total timeout	The cap on a whole call, as distinct from the gap between chunks that most libraries call the read timeout. Derived from the token cap	19
Training	The one-time process that produced the model's numbers. Not something you do on a request	1
Training cut-off	The point in time after which no text reached the model's training. It knows nothing later, and cannot tell that it does not	1

Term	Meaning	Ch
Trials	Repeated runs of the same case, which measure the model's own variation. Averaged within the case, never counted as extra cases	20
Turn	One exchange — the user's message and the model's reply to it. A conversation is a list of turns, and the whole list is re-sent on each one	18
Two-tower	A bi-encoder with separate query and document encoders, so the score is $\cos(f(q), g(d))$ with $f \neq g$: still precomputable, no longer symmetric	26
User message	The request and the material it is about, from the person or from your code on their behalf	18
Validation layers	Parses / right shape / values possible / values true. The first three are mechanical; the fourth needs a judgement and belongs elsewhere	21
Value	The vector a position hands over when it is attended to, made with W_V. What the weights are spent on	11
Vector	An ordered list of numbers. Two vectors can be compared only if they have the same number of slots	8
Vocabulary	The fixed, numbered list of every entry a model can read or produce — nearly all pieces of text, and a few that are not. Built once, before training, and fixed while you use the model	9
Width	The number of slots in the vectors moving from layer to layer, the same all the way through. Written d in formulas	9

Index

A

A derivation, 57
accountability, 27
action, 25
activation function, 135
active parameters, 178
add these up, 77
agent, 33
announcement, 57, 62
answer key, 305
approval point, 38
assistant, 206
attention, 115
attention block, 133
attention head, 123
attention weights, 119
autoregressive, 92

B

backpropagation, 196
base model, 162
batch endpoint, 223
batching, 177
bi-encoder, 339
big-O, 81
BM25, 327
byte-pair encoding, 99

C

C, 61, 62
candidate set, 341
case, 235
causal mask, 119
change, 132
check, 16
code, 110
compound system, 18
compounding error, 35
compute-bound, 173
concept owner, 70
connect, 220
constrained decoding, 268
context, 14, 15, 204
context window, 186
controlling what it costs, 50
cosine similarity, 77
cross-encoder, 339
cross-entropy, 159
current convention, 54, 55, 58, 60, 210

D

D, 61, 62
dated detail, 54, 55, 60, 69
dead state, 277
decode, 172
delta, 216
demonstration, 57
dense retrieval, 302
depth, 88
designed, 29
dimensions, 75
direct preference optimisation, 164

discounted cumulative gain, 337
dispatcher, 286, 288
do not retry, 218
document frequency, 327
document length, 327
dot product, 75

E

eleven new ones, 127
embedding, 299
embedding model, 299
embedding table, 87
error analysis, 45
evaluation, 45
examples, 205, 211
exponential backoff, 219

F

feed-forward block, 133
finish reason, 257, 291
forward pass, 92
foundation model, 4
four activities, 43
four tokens, 109

G

GB, 175
generalisation theory, 197
GiB, 175
graded relevance, 333
gradient, 161
greedy decoding, 146

H

hallucination, 184
how much, 58
HTTP 429, 218
hybrid retrieval, 346

I

I, 70
I · Understand, 65
idempotent, 223
II, 70
II · Use, 65
III, 70
III · Apply, 65
in general, 17
inductive bias, 197
inference, 4
initialisation, 197
input tokens, 103
inside the Model, 66
instructions, 205, 211
interview depth, 68
invented, 48
inverse document frequency, 327
irreversible action, 25
IV, 70
IV · What Next, 65

J

jitter, 219
JSON Schema, 254
judgement, 67

K

key, 115
key–value cache, 174

L

lasting principle, 54, 55
latency, 8
layer, 88
length, 76
lexical retrieval, 326
logarithm, 79
logits, 145

M

masked mass, 270
masking, 270
matrix, 78
mean reciprocal rank, 337
memory-bound, 173
mental Model, 66

merge list, 100
merge the commonest pair, 101
mixture-of-experts, 178
model, 3
multiply these together, 77

N
next-token distribution, 89
nine, 180
none of the five, 51
none of these, 49, 50
normalised, 136
normalised discounted cumulative gain, 338
not, 363
numerical stability, 196

O
objective design, 196
one operation per byte, 173
one small system, 66
one study, 57, 58
optimisation theory, 196
Optimiser design, 196
output format, 205, 211, 254
output tokens, 103

P
P, 61, 62
paired difference, 238
paragraph 10, 50
paragraph 12, 50
paragraph 3, 50
parameter, 91
people, 17
perplexity, 160
positional encoding, 122
practitioner report, 57, 58, 62, 210
preference optimisation, 162, 163
prefill, 172
pretraining, 162, 163
probability distribution, 86
proposal, 286

Q
quantisation, 177
query, 115

R
read, 220
recall@k, 337
reciprocal rank, 337
reciprocal rank fusion, 333
registry, 288
regularisation, 197
relevance, 316
relevance judgement, 319
reliability, 37
repair, 261
representation learning, 196
reranker, 341
reranking, 341
residual connection, 132
residual stream, 132
retrieval, 15
rubber-stamping, 28
rule of three, 259

S
scaling laws, 197
schema, 255
second, 44
semantic search, 312
semaphore, 222
server-sent events, 216
settled practice, 57
seventy-five million books, 163
six, 179
softmax, 80
state machine, 269
step, 34
stop condition, 149
stop token, 149
stopping condition, 37
stops and waits, 38
strict mode, 268

subword, 99
supervised fine-tuning, 162, 163
supplied information, 205, 211
system, 206

T

temperature, 146
term, 326
term frequency, 327
the instructions, 50
the model decides, 34
the scratchpad is gone, 274
the tests themselves, 19
thirty-seven, 180
three timeouts, not one, 220
token, 98
tokeniser, 106
tool, 15, 206
tool description, 284
top-k, 148
top-p, 148
total timeout, 220
training, 4
training cut-off, 6
turn, 207
two-tower, 339

U

user, 206

V

validation layers, 257
value, 115
vector, 75
vocabulary, 87

W

what to put in, 44
who decides the order, 33
width, 88

Y

you validate and decide, 363
your code decides, 34
your company is answerable, 27

PROGRAMMING / ARTIFICIAL INTELLIGENCE

Modern AI Engineering from Zero

A pretrained model is a service you send text to. Everything that makes it into a product you can trust — the context you assemble, the documents you retrieve, the tools you allow, the checks you run, the bill you pay — is engineering, and this book teaches all of it from zero.

You'll learn how to:

- Reason about tokens, cost, latency and why the same prompt answers differently
- Build retrieval that you can measure instead of admire
- Find out whether your application works, and whether a change helped
- Design tools and agents that fail safely, and test them by attacking them
- Account for every millisecond and every cent a request costs